JAVASE核心

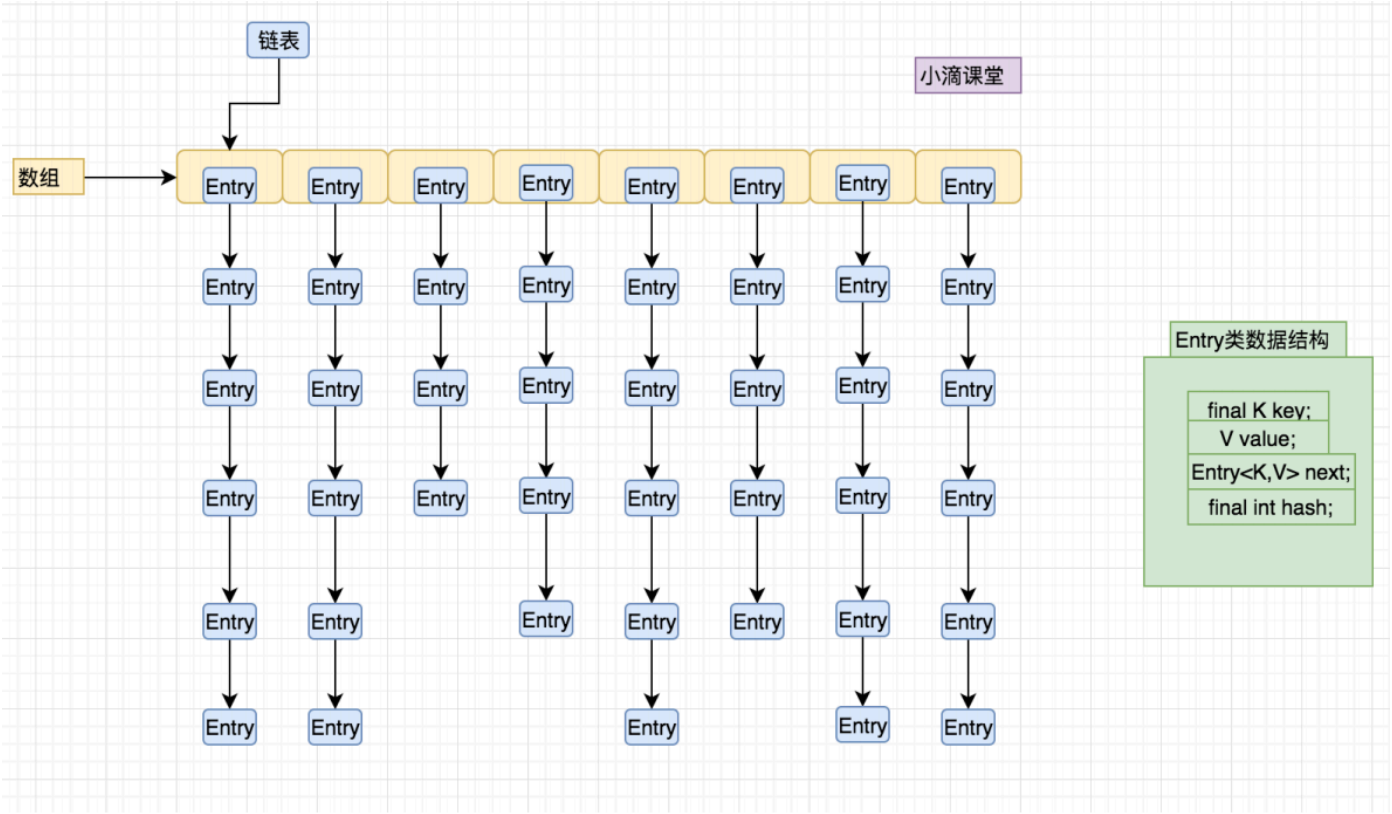
1-String/StringBuffer与StringBuilder的区别和使用场景?

- 题目分析和考点
  - 面试官想知道是否有关注线程安全的字符串操作类和底层区别
  - 应聘者是否知道什么场景下使用哪个类
- 难度【* * 】
- 答案
  - 三者都是final， 不允许被继承，在本质都是char[]字符数组实现
  - String、StringBuffer与StringBuilder中，String是不可变对象，另外两个是可变的
  - StringBuilder 效率更快，因为它不需要加锁，不具备多线程安全
  - StringBuffer里面操作方法用synchronized ，效率相对更低,是线程安全的；
  - 使用场景
    - 操作少量的数据用String，但是常改变内容且操作数据多情况下最好不要用 String ，因为每次生成中间对象性能会降低
    - 单线程下操作大量的字符串用StringBuilder，虽然线程不安全但是不影响
    - 多线程下操作大量的字符串，且需要保证线程安全 则用StringBuffer

2-HashMap底层是 数组+链表+红黑树，为什么用这几类结构呢?

- 题目分析和考点
  - 应聘者是否掌握HashMap的底层实现，对新旧版本的HashMap数据结构是否有关注
- 难度【* * *】
- 答案
  - 数组 Node<K,V>[] table ，根据对象的key的hash值进行在数组里面是哪个节点
  - 链表的作用是解决hash冲突，将hash值一样的对象存在一个链表放在hash值对应的槽位
  - JDK8使用红黑树来替代超过8个节点的链表，主要是查询性能的提升，从原来的O(n)到O(logn)
  - 因为hash碰撞，让HashMap不断产生碰撞，那么相同的key的位置的链表就会不断增长

- 当对这个Hashmap的相应位置进行查询的时候，就会循环遍历这个超级大的链表，性能就会下降，所以改用红黑树



3-java接口是否支持多继承？接口里面是否可以有方法实现

- **题目分析和考点**
  - 面试官想了解你对Java接口的基本概念和特性的了解程度，即接口是一种抽象的行为规范，可以用于定义一组方法的签名
  - 在Java 8之前，接口只能定义抽象方法，即纯粹的方法签名
  - 从Java 8开始，接口可以使用default关键字定义默认方法，即具有方法体的方法实现
- **难度【* *】**
- **答案**
  - 接口相关
    - 接口里可以有静态方法和方法体
    - 接口中所有的方法必须是抽象方法（JDK8之后就不是）
    - 接口不是被类继承了，而是要被类实现，接口支持多继承, 类不支持多个类继承
    - 一个类只能继承一个类，但是能实现多个接口,接口能继承另一个接口，接口的继承使用extends关键字，和类继承一样
  - JDK8里面接口新特性
    - interface中可以有static方法，但必须有方法实现体，该方法只属于该接口，接口名直接调用 该方法

        - 接口中新增default关键字修饰的方法，default方法只能定义在接口中，可以在子类或子接口 中被重写default定义的方法必须有方法体
        - 父接口的default方法如果在子接口或子类被重写，那么子接口实现对象、子类对象，调用该 方法，以重写为准
        - 本类、接口如果没有重写父类（即接口）的default方法，则在调用default方法时，使用父类（接口） 定义的default方法逻辑

4-说下Vector和ArrayList、LinkedList联系和区别？使用场景

- **题目分析和考点**
    - 面试官想了解你Vector、ArrayList和LinkedList在具体实现和性能上有一些差异。
    - 正确选择需要根据具体场景的线程安全性、随机访问、插入删除操作的频率来进行评估和选择
- **难度【* *】**
- **答案**
    - 共同点
        - 都是可变大小的动态数组实现。
        - 都实现了List接口，支持按索引访问、添加、删除和修改集合中的元素
    - 区别：
        - 线程安全性
            - Vector是线程安全的，而ArrayList和LinkedList不是
            - 在并发环境中，如果需要保证线程安全，可以使用Vector
            - 如果不需要线程安全，使用ArrayList或LinkedList会更高效。
        - 扩容机制
            - 当元素数量超过容量时，Vector和ArrayList的扩容机制是自动增长一倍，而LinkedList则没有固定容量，可根据需要动态扩容。
        - 访问速度
            - ArrayList通过索引访问元素的速度较快，因为基于数组实现；LinkedList则因为使用链表结构，访问速度相对较慢。
        - 插入、删除操作
            - ArrayList在尾部添加删除元素的速度较快
            - 但在需要频繁插入和删除元素的场景下，LinkedList的效率更高，因为它只需要调整节点的指针。
    - 使用场景：
        - Vector：由于线程安全，适用于多线程环境下的操作，但在单线程环境下推荐使用ArrayList，因为ArrayList效率更高。
        - ArrayList：适用于读取操作频繁，对于随机访问和索引操作较多的场景。

        - LinkedList：适用于需要频繁插入、删除元素的场景，因为插入和删除只需调整节点指针，而无需移动其他元素。在实现栈、队列等数据结构时，LinkedList也具有优势。

5-如果需要保证线程安全，ArrayList应该怎么做，用有几种方式

- **题目分析和考点**
    - 面试官想了解你ArrayList的底层了解，如何写保证线程安全的ArrayList
    - 常规有多种方式可以确保ArrayList的线程安全性，但在高并发场景下，会带来一定的性能开销
    - 在单线程环境或者不要求强制线程安全的情况下，使用ArrayList的默认实现会更加高效
- **难度【* *】**
- **答案**
    - 方式一：自己写个包装类，根据业务一般是add/update/remove加锁
        - 可以使用同步锁（synchronized）或显式锁（ReentrantLock）来保护ArrayList的访问。
        - 例如，可以在对ArrayList进行读写操作时使用同步锁来确保线程安全。
        - 注意，在使用锁机制时，需要确保所有对ArrayList的访问都在锁的保护下进行，以避免产生不一致或死锁等问题
    - 方式二：Collections.synchronizedList(new ArrayList<>()); 使用synchronized加锁

```
List<String> synchronizedList = Collections.synchronizedList(new ArrayList<>());
```

    - 方式三：CopyOnWriteArrayList<>() 使用ReentrantLock加锁

```
List<String> concurrentList = new CopyOnWriteArrayList<>();
```

6-介绍下对象的 hashCode()和equals()，使用场景

- **题目分析和考点**
    - 面试官想了解你对hashCode()和equals()方法的基本概念和作用的理解程度
    - hashCode()方法返回对象的哈希码，equals()方法用于比较两个对象是否相等
- **难度【* *】**
- **答案**
    - hashcode
        - 顶级类Object里面的方法，所有的类都是继承Object,返回是一个int类型的数
        - 根据一定的hash规则(存储地址，字段，长度等)，映射成一个数组，即散列值
    - equals
        - 顶级类Object里面的方法，所有的类都是继承Object,返回是一个boolean类型

        - 根据自定义的匹配规则，用于匹配两个对象是否一样，一般逻辑如下

```
//判断地址是否一样
//非空判断和Class类型判断
//强转
//对象里面的字段一一匹配
```

    - 使用场景：对象比较、或者集合容器里面排重、比较、排序

7-说下HashMap、TreeMap底层结构和使用场景

- **题目分析和考点**
    - 面试官想了解你对HashMap、TreeMap底层结构掌握情况
    - 根据具体的需求来选择HashMap或TreeMap
    - HashMap适用于需要高效的插入、查找和删除操作的场景，而TreeMap适用于需要按顺序遍历、范围查找和排序操作的场景
- **难度【* * *】**
- **答案**
    - HashMap
        - 使用哈希表实现，通过键值对存储数据。
        - 无序的，不保证元素的插入和遍历顺序一致。
        - 查询、插入和删除操作的时间复杂度为O(1)（平均情况下），具有较高的性能。
        - 允许键和值为null。
        - 不保证元素的顺序，适用于需要高效的查找和插入的场景，例如缓存、快速查找等。
    - TreeMap
        - 使用红黑树（自平衡二叉查找树）实现，根据键进行排序存储数据。
        - 基于键的自然排序或自定义排序规则进行排序。
        - 按照键的顺序遍历，元素有序，支持根据键的范围查找数据。
        - 插入和查询操作的时间复杂度为O(log N)，具有较高的查询性能。
        - 不允许键为null，但值可以为null。
        - 适用于需要对数据进行排序、按序遍历和范围查找的场景，例如按照键的排序输出、统计分析等。
        - 写过微信支付签名工具类就用这个类
    - 使用场景：
        - 如果需要高效的插入、查找和删除操作，并且对元素的顺序没有要求，可以选择HashMap。
        - 如果需要对元素进行排序或按顺序遍历，可以根据键的排序规则选择使用TreeMap。
        - 如果需要根据键的范围进行查找操作，也可以选择TreeMap。

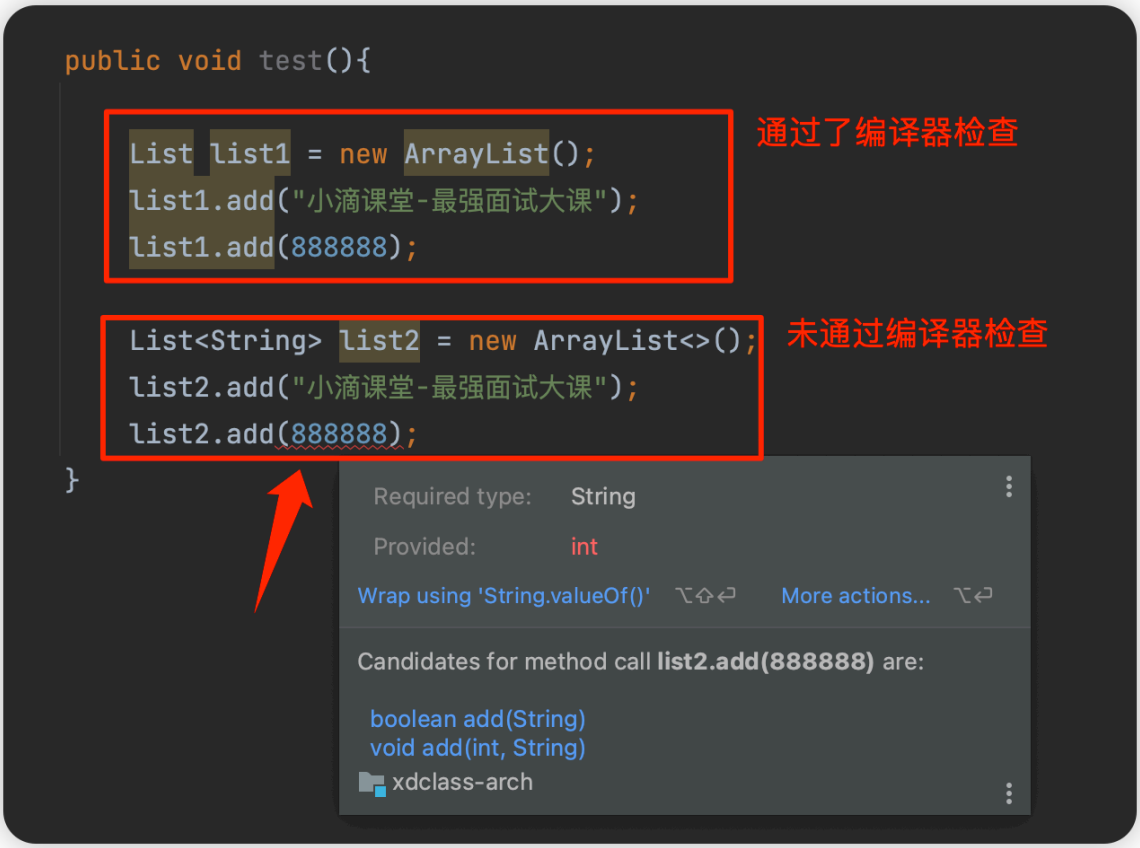
        - 如果对性能要求较高，并且不需要保证元素的顺序，或者不需要排序操作，通常更倾向于选择HashMap。

8-了解ConcurrentHashMap吗？为什么性能比hashtable高

- **题目分析和考点**
    - 面试官想了解你对 ConcurrentHashMap底层结构掌握情况
    - ConcurrentHashMap具有分段锁和组合数据结构的优势，以及更好的并发性能
    - 相比于Hashtable在多线程并发访问时具有更高的性能。
    - ConcurrentHashMap它是一种高效的并发容器，适用于高并发的读写场景
- **难度【* * *】**
- **答案**
    - ConcurrentHashMap是Java并发包中线程安全的哈希表实现，它比Hashtable在性能上更高效。
    - 主要体现在两个方面：
        - 分段锁
            - ConcurrentHashMap内部使用了分段锁的机制，将整个哈希表分成多个小的段，每个段都可以独立地加锁
            - 不同线程可以同时访问不同的段，这样在多线程并发访问时，不同的线程可以同时对不同的段进行读写操作
            - 避免了对整个容器的锁竞争，提高了并发性能。
        - 数组+链表/红黑树的组合结构
            - ConcurrentHashMap的数据结构由数组和链表/红黑树的组合构成。
            - 当一个段中的链表长度超过一定阈值，链表将会转换为红黑树，从而提高了查找、插入和删除等操作的性能。
            - 对于读操作，在不加锁的情况下可以并发执行，而写操作只需要锁定对应的段。
    - 而Hashtable则是使用了全局的锁，当有线程进行操作时，其他线程需要等待全局锁的释放，导致并发性能受限。
    - jdk1.7和jdk1.8里面ConcurrentHashMap实现的区别
        - JDK8之前
            - ConcurrentHashMap使用锁分段技术，将数据分成一段段存储，每个数据段配置一把锁，
            - 即segment类，这个类继承ReentrantLock来保证线程安全
            - 技术点：Segment+HashEntry
        - JKD8的版本后
            - 取消Segment这个分段锁数据结构，底层也是使用Node数组+链表+红黑树
            - 从而实现对每一段数据就行加锁，也减少了并发冲突的概率，CAS(读)+Synchronized(写)
            - 技术点：Node+Cas+Synchronized

9-介绍下什么是java泛型技术和使用场景

- **题目分析和考点**
  - 面试官通过提问Java泛型技术和使用场景，考察候选人对泛型概念的理解程度以及在实际开发中对泛型的应用能力
  - 泛型可以提供类型安全的编程，并且可以提高代码的重用性、可读性和可维护性
  - 其主要用途包括集合框架、自定义数据结构以及API设计等方面
- **难度【* *】**
- **答案**
  - 什么是泛型（generics）
    - 是在定义类、接口和方法时，可以在声明时通过一定的格式指定其参数类型
    - 使用时再指定具体的类型，从而使得类、接口和方法可以被多种类型的数据所实例化或调用
    - 这种可以在编译时进行参数类型检查的技术被称为泛型，是 JDK 5 中引入的一个新特性
    - 本质是**参数化类型，**给类型指定一个参数，在使用时再指定此参数具体的值，那这个**类型就可以在使用时决定**
    - 优点
      - 把运行时的错误，提前到编译时，这样就可以在编译时把错误提示出来，避免了运行时出现错误
      - 使用泛型可以提高代码的复用性，因为它可以支持多种类型的数据。
  - 为啥要用
    - 在没有泛型之前，从集合中读取到的每一个对象都必须进行类型转换
    - 如果插入了错误的类型对象，在运行时的转换处理就会出错
    - 集合容器里面如果没指定类型，默认都是Object类型，那什么到可以插入
    - 减少了源代码中的强制类型转换，代码更加可读
    - 案例



10-说下是java泛型常见的通配符有哪些

- **题目分析和考点**
  - 面试官想了解你对Java泛型通配符的基本概念和语法的了解程度，包括通配符的符号以及在泛型中的使用方式
  - 通配符可以用于灵活接收不同类型的泛型参数，实现更通用的代码编写
- **难度【* * *】**
- **答案**
  - 什么是泛型通配符
    - Java泛型的通配符是用于解决泛型之间引用传递问题的特殊语法

```
表示类型参数可以是任何类型
public class CustomCollection<?>{}

//表示类型参数必须是A或者是A的子类
public class CustomCollection<T extends A>{}

//表示类型参数必须是A或者是A的超类型
public class CustomCollection<T supers A>{}
```

- 分类
  - 通用类型通配符 < ? >，如List < ? >
    - 主要作用就是让泛型能够接受未知类型的数据
    - 可以把 ？ 看成所有泛型类型的父类，是一种真实的类型，类型通配符是实参，不是形参
  - 固定上边界的通配符 采用<? extends E>的形式
    - 使用固定上边界的通配符的泛型，只能够接受指定类及其子类类型的数据。
    - 采用<? extends E>的形式, 这里的E就是该泛型的上边界
    - 注意: 这里虽然用的是extends关键字, 却不仅限于继承了父类E的子类, 也可以代指显现了接口E的类
  - 固定下边界的通配符，采用<? super E>的形式
    - 使用固定下边界的通配符的泛型, 只能够接受指定类及其父类类型的数据。
    - 采用<? super E>的形式, 这里的E就是该泛型的下边界.
    - 可以为一个泛型指定上边界或下边界, 但是不能同时指定上下边界

11-介绍java反射技术和常见的一些API

- **题目分析和考点**
  - 面试官想了解你对Java反射技术的基本概念和原理的了解程度，通过反射机制在运行时动态地获取和操作类、对象、方法、字段
  - 对Java反射常用的API的熟悉程度。例如，Class类、Constructor类、Method类、Field类等反射相关的类和方法
- **难度【* * *】**
- **答案**
  - 反射技术
    - Java的反射（reflection）机制是指在程序的运行状态中
      - 可以构造任意一个类的对象
      - 可以了解任意一个对象所属的类
      - 可以了解任意一个类的成员变量和方法
      - 可以调用任意一个对象的属性和方法。
    - 这种动态获取程序信息以及动态调用对象的功能称为Java语言的反射机制
    - 核心：**得到编译以后得class文件对象，提供了一个Class类型，就是编译后的class类对象**

```
HelloWorld.java -> javac -> HelloWorld.class

Class clz = HelloWorld.class
```

- Class方法api（有很多API，只掌握常见的）

方法名称	作用
`getName( )`	获取 包名+类名
`getSimpleName( )`	获取类名
`getDeclaredConstructor(Class<?>... parameterTypes)`	根据参数，获取构造器，修饰符不影响
`getDeclaredConstructors()`	获取全部声明的构造器，返回数组，修饰符不影响

- 对象创建

方法名	说明
`T newInstance( )`	根据类的空参的构造器创建对象，**类必须提供空参的构造器和public权限**
`T newInstance(Object...initargs)`	根据指定的构造方法创建对象

- 方法获取

方法	说明
`getMethods()`	获取当前运行类和 父类中声明的方法，需要是public访问权限的方法
`getDeclaredMethods()`	获取当前运行时类中声明的全部方法，不包含父类中声明的方法

- 方法元信息

方法	说明
`getReturnType()`	获取全部的返回值
`getParameterTypes()`	获取全部的参数
`getModifiers()`	获取修饰符
`getExceptionTypes()`	获取异常信息

- 通过class对象获取属性

方法名	说明
`getFields( )`	获取当前运行类和 父类中声明的属性，需要是public访问权限的属性
`getDeclaredFields( )`	获取当前运行时类中声明的全部属性，不包含父类中声明的属性

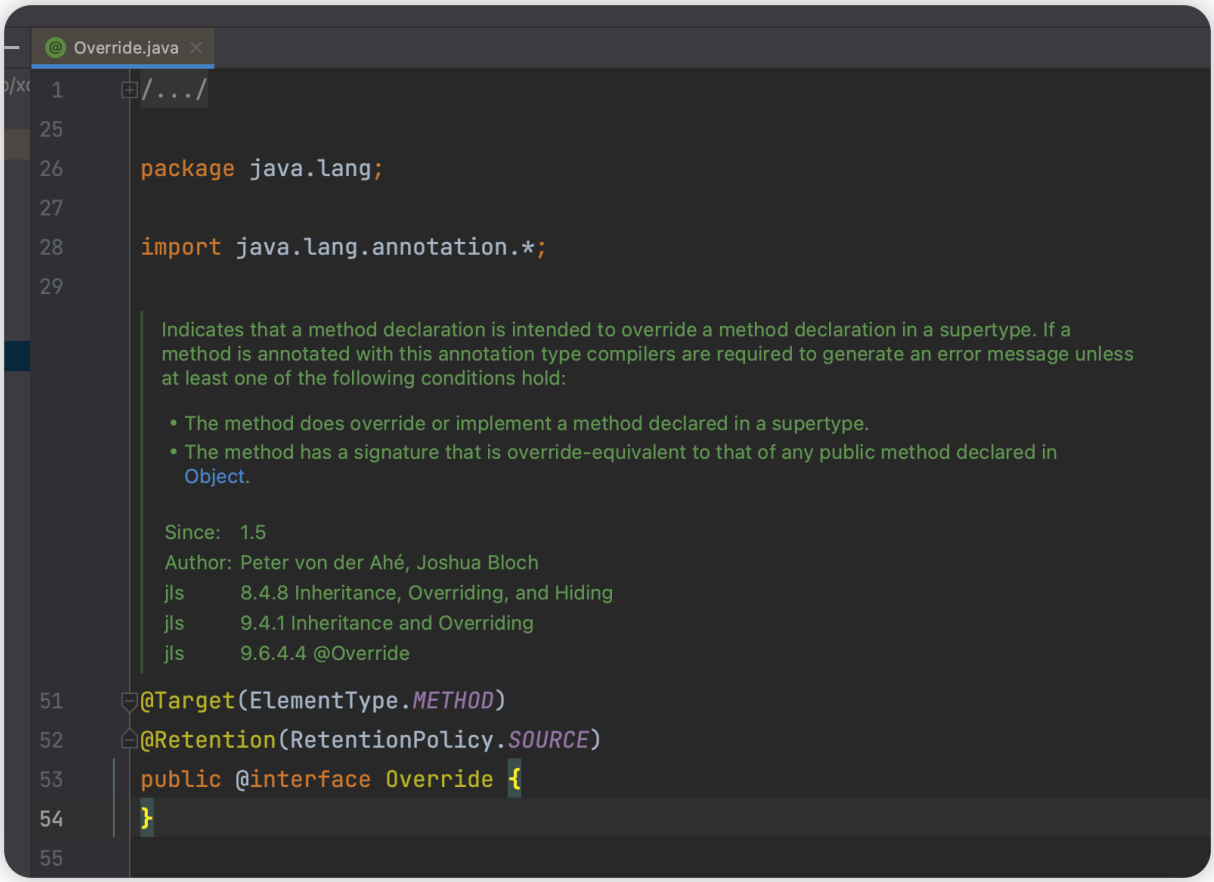
- 属性Field的方法

方法	说明
`getModifiers()`	整数形式返回此`Field`的修饰符，整数对应在`java.lang.reflect.Modifier`里面
`getType()`	返回 Field的属性类型
`getName()`	返回 Field的名称

12-有了解过哪些java自带的注解，什么是元注解

- **题目分析和考点**
  - 面试官想了解你对Java自带的注解的了解程度
  - Java自带了一些常见的注解，如 @Override、@Deprecated、@SuppressWarnings等，在Java编程中具有特殊的含义和作用
  - 自定义注解基本都会用到元注解，在实际开发中对注解的应用能力
- **难度【* * *】**
- **答案**
  - java注解
    - 是在 JDK5 时引入的新特性，其实就是代码里的特殊标记
    - 注解也被称为元数据, 是附加在代码中的一些元信息，用于一些工具在编译、运行时进行解析和使用
    - 允许开发人员在不改变源代码的情况下，在源代码中添加一些元数据
    - 以便让编译器或者其他工具可以读取这些元数据，从而实现更高级的功能
    - 应用场景丰富
      - 生成文档
      - 编译时进行格式检查，如@Override
      - 替代配置文件功能，比如spring的注解
      - 和反射组合应用，自己封装框架组件

- 知识点
  - Java自带的标准注解（这些注解后编译器就会进行检查）
    - @Override 标记覆写父类的方法
    - @Deprecated 标记被修饰的类或类成员、类方法已废弃、过时
    - @SuppressWarnings 用于关闭对类、方法、成员编译时产生的特定警告。
  - 元注解 (用于定义注解的注解)
    - 元注解也是Java自带的标准注解，只不过用于修饰注解，比较特殊。
    - @Retention、@Target、@Inherited、@Documented、@Repeatable
  - 自定义注解
    - 用户可以根据自己的需求定义注解
- 什么是**元注解**
  - 注解的注解，比如当我们需要自定义注解时，会需要一些**元注解**（meta-annotation），如**@Target**和**@Retention**
  - java内置4种元注解
    - **@Target** 表示该注解用于什么地方
      - ElementType.CONSTRUCTOR 用在构造器
      - ElementType.FIELD 用于描述域-属性上
      - ElementType.METHOD 用在方法上
      - ElementType.TYPE 用在类或接口上
      - ElementType.PACKAGE 用于描述包
    - **@Retention** 表示在什么级别保存该注解信息
      - RetentionPolicy.SOURCE 保留到源码上
      - RetentionPolicy.CLASS 保留到字节码上
      - RetentionPolicy.RUNTIME 保留到虚拟机运行时（最多，可通过反射获取）
    - **@Documented** 将此注解包含在 javadoc 中
    - **@Inherited** 是否允许子类继承父类中的注解



13-日常开发中有没用过代理模式，说下动态和静态代理

- **题目分析和考点**
  - 面试官想了解你对代理模式的基本概念和原理的了解程度，以及在日常开发中是否有使用代理模式的经验
  - 静态代理是在编译时期生成代理类，而动态代理是在运行时期通过反射机制动态生成代理类
  - 代理模式可以实现对目标对象的访问控制、增加额外的逻辑和功能、实现懒加载和缓存等
- **难度【* * *】**
- **答案**
  - 代理设计模式（Proxy Pattern）
    - 为其他对象提供一种代理以控制对这个对象的访问，属于结构型模式。
    - 客户端并不直接调用实际的对象，而是通过调用代理，来间接的调用实际的对象
    - 优点
      - 可以在访问一个类时做一些控制，或增加功能
      - 操作代理类无须修改原本的源代码，符合开闭原则，系统具有较好的灵活性和可扩展性
    - 缺点

        - 增加系统复杂性和调用链路
- 什么是静态代理
    - 由程序创建或特定工具自动生成源代码，在程序运行前，代理类的.class文件就已经存在
    - 通过将目标类与代理类实现同一个接口，让代理类持有真实类对象，然后在代理类方法中调用真实类方法，在调用真实类方法的前后添加我们所需要的功能扩展代码来达到增强的目的
    - A -> B -> C
    - 优点
        - 代理使客户端不需要知道实现类是什么，怎么做的，而客户端只需知道代理即可
        - 方便增加功能，拓展业务逻辑
    - 缺点
        - 代理类中出现大量冗余的代码，非常不利于扩展和维护
        - 如果接口增加一个方法，除了所有实现类需要实现这个方法外，所有代理类也需要实现此方法。增加了代码维护的复杂度
- 什么是动态代理
    - 在程序运行时，运用反射机制动态创建而成，无需手动编写代码
    - JDK动态代理与静态代理一样，目标类需要实现一个代理接口,再通过代理对象调用目标方法
    - 常见实现方式
        - JDK动态代理：实现了被代理对象的接口，要求目标对象实现一个接口
        - CGLIB动态代理
    - 核心方法
        - 定义一个java.lang.reflect.InvocationHandler接口的实现类，重写invoke方法

```
/Object proxy:被代理的对象
//Method method:要调用的方法
//Object[] args:方法调用时所需要参数
public interface InvocationHandler {
    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable;
}
```

JUC并发编程

1-你知道多少种并发编程常见的锁

- **题目分析和考点**
  - 面试官想知道你对并发编程的了解程度，是否了解并发编程的常见问题，如死锁、活锁、并发安全等。
  - 是否能根据具体场景选择合适的锁机制，了解锁的性能和开销，以及是否能够合理地进行锁的选择和使用
- **难度【 * * * 】**
- **答案**
  - Java开发里面常见的锁种类
    - 种类一
      - 悲观锁
        - 当线程去操作数据的时候，总认为别的线程会去修改数据，所以它每次拿数据的时候都会上锁，别的线程去拿数据的时候就会阻塞，比如synchronized、ReentrantLock
      - 乐观锁
        - 每次去拿数据的时候都认为别人不会修改，更新的时候会判断是别人是否回去更新数据，通过版本来判断
        - 如果数据被修改了就拒绝更新，比如CAS是乐观锁，但严格来说并不是锁，通过原子性来保证数据的同步
        - 比如说数据库的乐观锁，通过版本控制来实现，乐观的认为在数据更新期间没有其他线程影响
      - 小结：悲观锁适合写操作多的场景，乐观锁适合读操作多的场景，乐观锁的吞吐量会比悲观锁多
    - 种类二
      - 公平锁
        - 指多个线程按照申请锁的顺序来获取锁，简单来说 如果一个线程组里，能保证每个线程都能拿到锁
        - 比如ReentrantLock (底层是同步队列FIFO:First Input First Output来实现)
      - 非公平锁
        - 获取锁的方式是随机获取的，保证不了每个线程都能拿到锁，也就是存在有线程饿死,一直拿不到锁
        - 比如synchronized、ReentrantLock
    - 种类三
      - 独享锁（互斥锁）
        - 也叫排它锁/写锁/独占锁/独享锁/ 该锁每一次只能被一个线程所持有,加锁后任何线程试图再次加锁的线程会被阻塞
        - 直到当前线程解锁。例子：如果 线程A 对 data1 加上排他锁后，则其他线程不能再对 data1 加任何类型的锁
        - 获得互斥锁的线程即能读数据又能修改数据

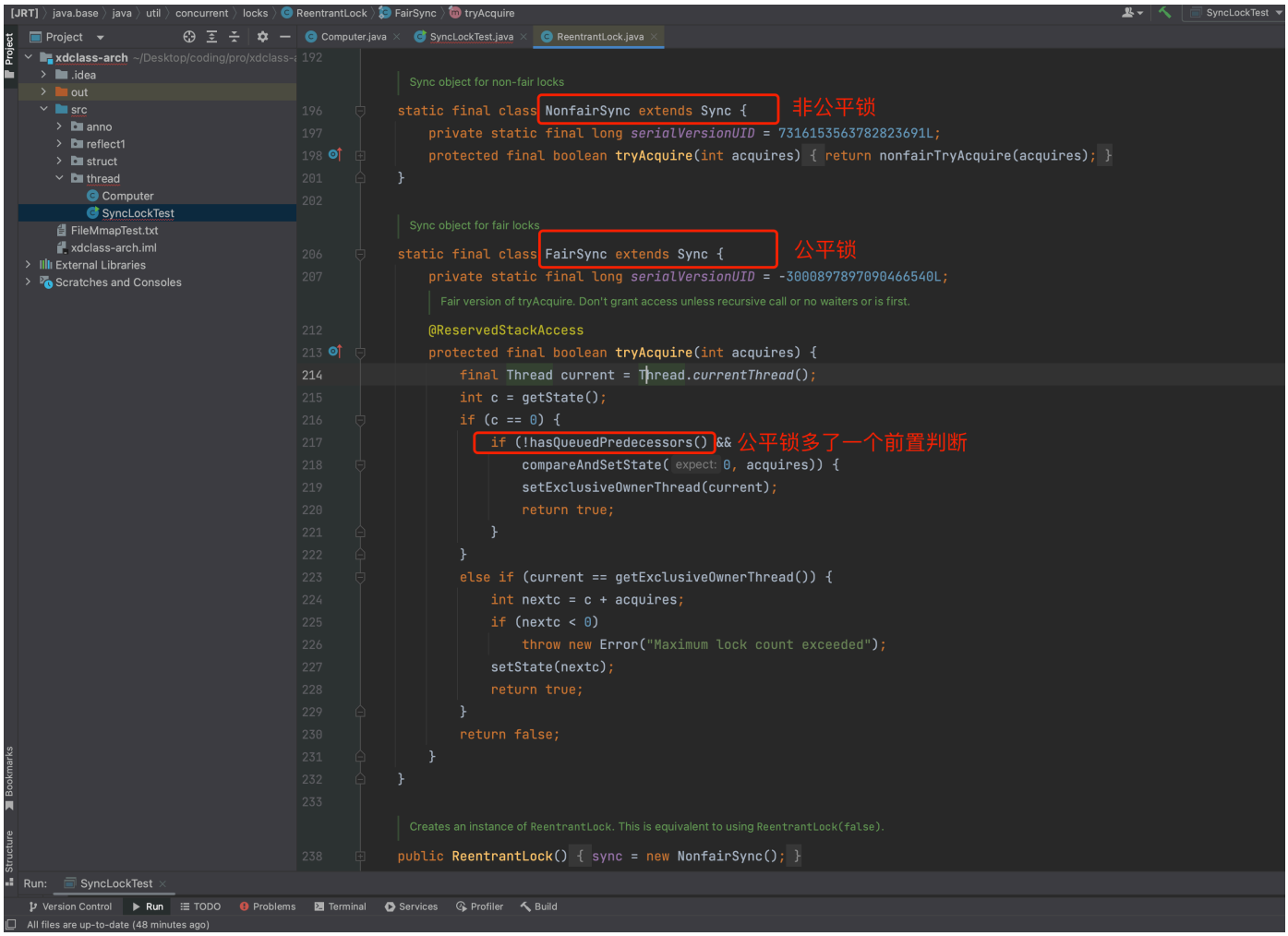
        - 共享锁
            - 也叫S锁/读锁，能查看但无法修改和删除的一种数据锁，加锁后其它用户可以并发读取、查询数据
            - 但不能修改，增加，删除数据，该锁可被多个线程所持有，用于资源数据共享
    - 种类四
        - 可重入锁
            - 也叫递归锁，在外层使用锁之后，在内层仍然可以使用，并且不发生死锁
        - 不可重入锁
            - 若当前线程执行某个方法已经获取了该锁，那么在方法中尝试再次获取锁时，就会获取不到被阻塞
        - 小结：可重入锁能一定程度的避免死锁， synchronized、ReentrantLock 重入锁

    - 种类五
        - 自旋锁
            - 一个线程在获取锁的时候，如果锁已经被其它线程获取，那么该线程将循环等待
            - 然后不断的判断锁是否能够被成功获取，直到获取到锁才会退出循环,任何时刻最多只能有一个执行单元获得锁
            - 小结：不会发生线程状态的切换，一直处于用户态，减少了线程上下文切换的消耗，缺点是循环会消耗CPU
        - 分段锁
            - 并不是具体的一种锁，只是一种锁的设计，将数据分段上锁，把锁进一步细粒度化，可以提升并发量
            - 当操作不需要更新整个数组的时候，就仅针对数组中的一项进行加锁操作
            - CurrentHashMap底层就用了分段锁
        - 死锁
            - 两个或两个以上的线程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法让程序进行下去

2-ReentrantLock是公平锁还是非公平锁，为什么这样设计

- **题目分析和考点**
    - 面试官想知道你对于 ReentrantLock 的特性和用法的掌握程度，以及在实际场景中如何进行合理的锁选择。
    - 对并发编程中锁的公平性和性能的权衡的理解，明确区分公平锁和非公平锁的区别，并能够根据具体场景合理选择使用

- **难度【* * *】**
- **答案**
  - ReentrantLock 通过参数控制，可以是公平也可以是非公平锁
    - 公平锁
      - 指多个线程按照申请锁的顺序来获取锁，简单来说 如果一个线程组里，能保证每个线程都能拿到锁
      - 比如ReentrantLock (底层是同步队列FIFO:First Input First Output来实现)
    - 非公平锁
      - 获取锁的方式是随机获取的，保证不了每个线程都能拿到锁，也就是存在有线程饿死,一直拿不到锁
      - 比如synchronized、ReentrantLock
  - ReentrantLock里面有一个内部类Sync，Sync继承AQS `AbstractQueuedSynchronizer`
  - 有公平锁FairSync和非公平锁NonfairSync两个子类，ReentrantLock默认使用非公平锁
  - 公平锁与非公平锁的lock()方法唯一的区别是多了一个限制条件：`hasQueuedPredecessors()`
  - 公平锁获取锁和释放锁后的相关操作，相关线程也会从休眠和恢复之间变化，这个就涉及到用户态内核态互相转变
  - 非公平锁获取锁不用遵循先到先得的规则，没有了阻塞和恢复执行的步骤，避免了线程休眠和恢复的操作
  - 所以非公平锁性能高于公平锁，更能重复利用CPU的时间
  - **如果是为了各个线程都可以获取锁资源，则推荐采用公平锁，等待锁的线程不会饿死**
  - **如果是为了业务有更大的吞吐量，则推荐采用非公平锁**



3-说下什么是对象锁和类锁，使用场景有什么差异点

- **题目分析和考点**
  - 面试官想知道你对于对象锁和类锁的概念的理解程度，以及在并发编程中如何选择合适的锁的能力
  - 还可以考察你对于锁的粒度和范围、同步控制的方式的理解，在实际场景中如何选择使用对象锁和类锁以及注意事项的能力
- **难度【* * *】**
- **答案**
  - 什么是对象锁（synchronized method{}）
    - 也叫实例锁，对应synchronized关键字，当多个线程访问多个实例时，它们互不干扰，每个对象都拥有自己的锁
    - 但如果是多个线程访问同个对象的sychronized块，是同步的加锁，访问不同对象的话就是不同步的
    - synchronized(object){}的 效果和在实例方法上加锁一样，不同的是可以在（）里添加不同的对象

- 什么是类锁（static sychronized method{}）
  - 关键字是 static sychronized，是一个全局锁，不管创建多少个对象都共享同一个锁
  - 保障同一个时刻多个线程同时访问同一个synchronized块，当一个线程在访问时，其他的线程等待
  - 静态方法使用synchronized关键字后，无论是多线程访问单个对象还是多个对象的sychronized块，都是同步的
  - synchronized(xxx.class){}的 效果和在静态方法方法上加锁一样，不同的是可以在（）里添加不同的类对象

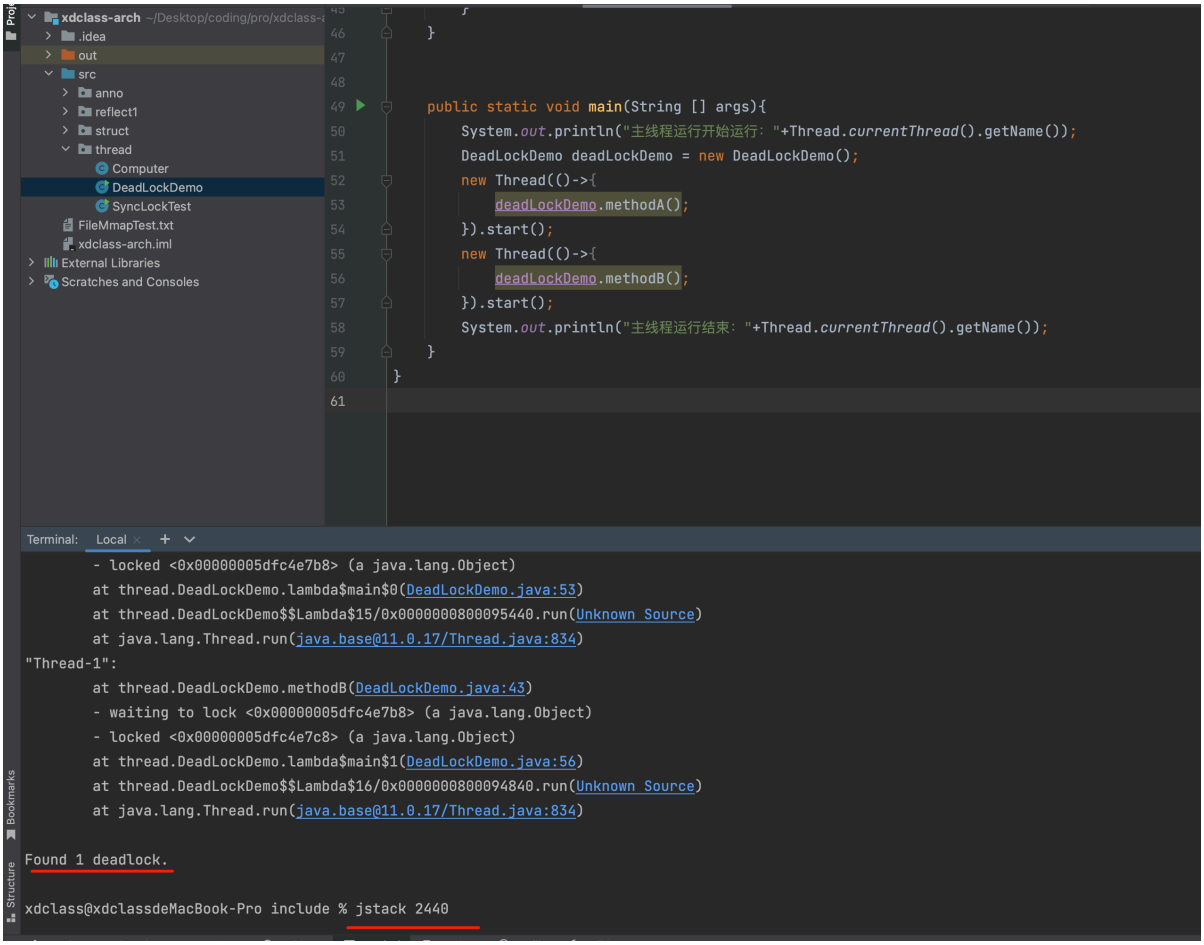
- 差异点
  - 锁的范围：对象锁是针对每个对象实例的，而类锁是针对类的。
  - 获取方式：对象锁可以通过synchronized关键字、wait()、notify()、notifyAll()方法获取，而类锁可以通过在静态方法中使用synchronized关键字或者使用类名.class获取。
  - 作用范围：对象锁只限于同一对象实例的线程获取锁，类锁对该类的所有实例都生效。

- 使用场景
  - 对象锁适用于对实例级别的资源进行同步控制，例如多个线程操作同一对象的实例变量。
  - 类锁适用于对类级别的资源进行同步控制，例如对静态变量或静态方法进行同步控制，或者需要对所有对象实例共享的数据进行同步控制。

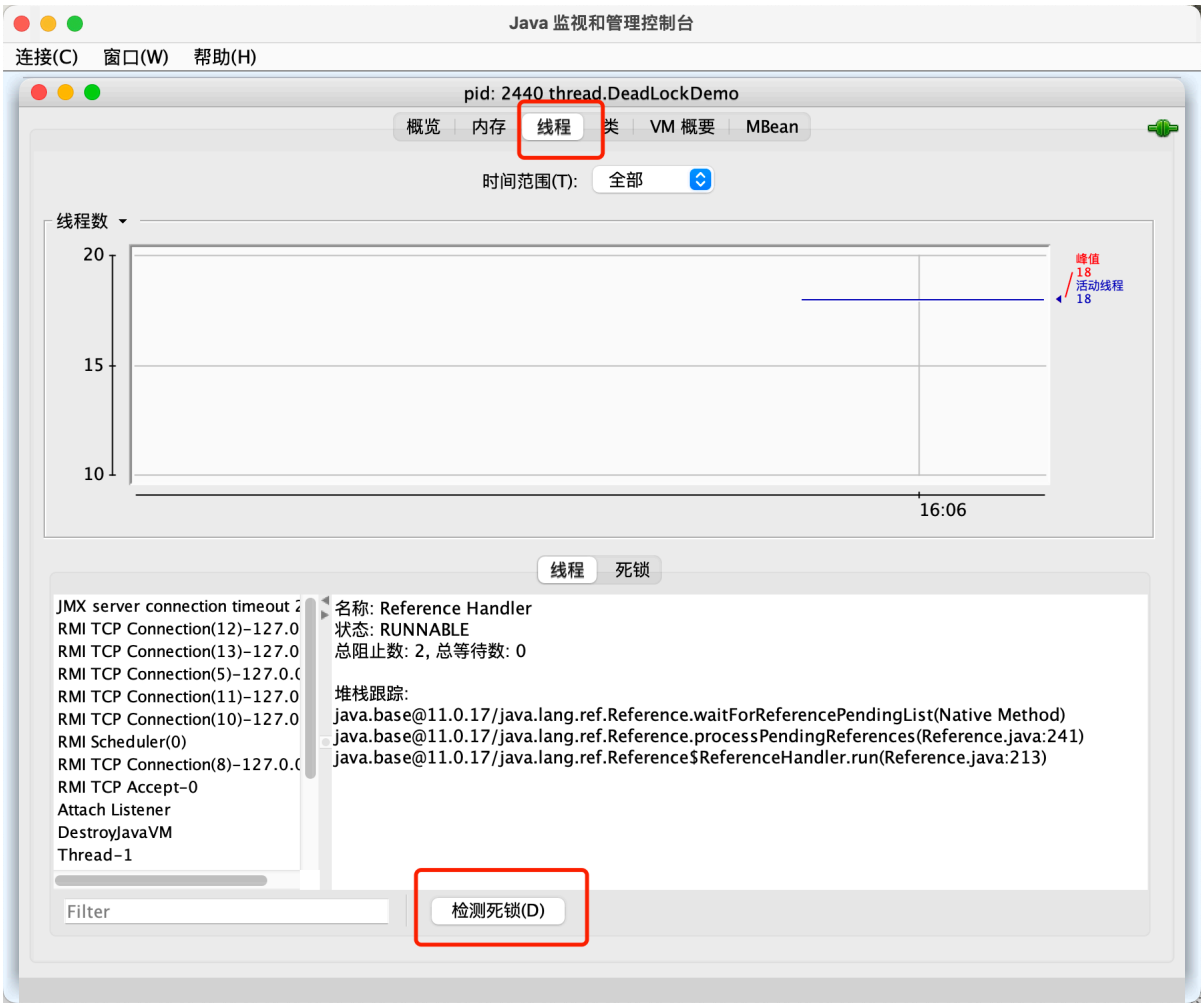
4-线上里面发生了死锁，说下你的排查解决思路

- **题目分析和考点**
  - 面试官想知道对于死锁的理解程度，以及你能否有效地定位和解决死锁问题
  - 还可以考察你对于多线程编程中死锁产生原因，能够通过使用工具和技术解决死锁问题或采取预防措施
- **难度【* * *】**
- **答案**
  - 死锁的4个必要条件
    - 互斥条件：资源不能共享，只能由一个线程使用
    - 请求与保持条件：线程已经获得一些资源，但因请求其他资源发生阻塞，对已经获得的资源保持不释放
    - 不可抢占：有些资源是不可强占的，当某个线程获得这个资源后，系统不能强行回收，只能由线程使用完自己释放
    - 循环等待条件：多个线程形成环形链，每个都占用对方申请的下个资源
  - **只要发生死锁，上面的条件都成立；只要一个不满足，就不会发生死锁**

- 常见的解决办法
  - 调整申请锁的范围
  - 调整申请锁的顺序
  - 做好代码review
- 排查方式
  - 方式一
    - Jps + jstack



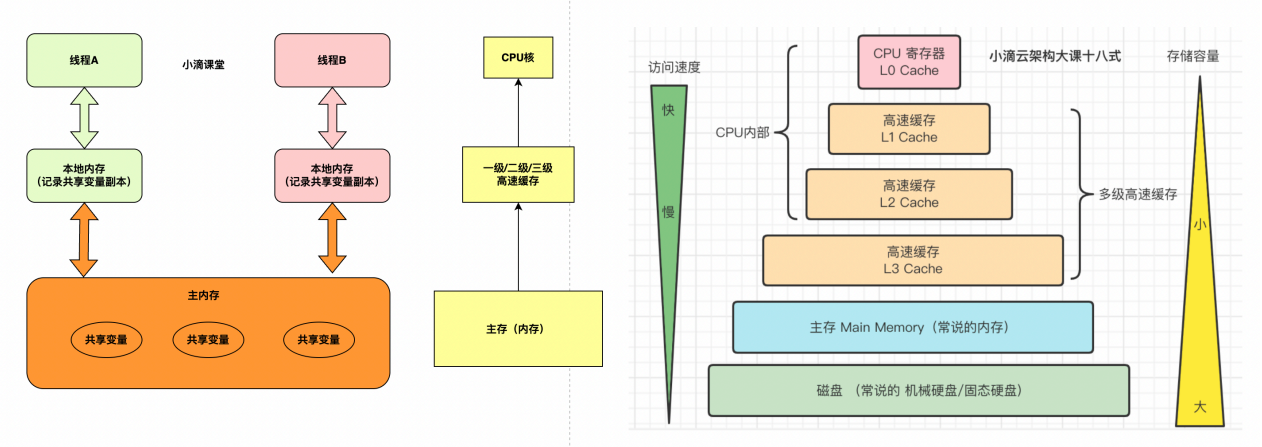
  - 方式二
    - jconsole



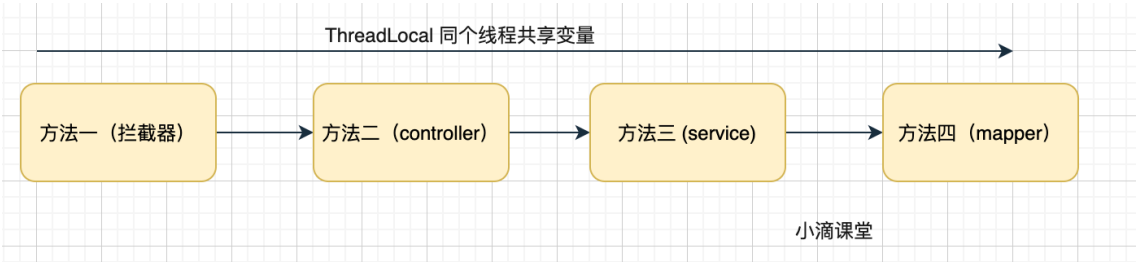
5-介绍下ThreadLocal，一般什么场景下会使用

- **题目分析和考点**
  - 面试官想知道对于ThreadLocal的理解程度，以及你能否准确地分析并应用ThreadLocal的使用场景
  - 还可以考察你对于多线程编程中共享变量和线程隔离的理解，以及ThreadLocal的使用原理
- **难度【***】**
- **答案**
  - 什么是ThreadLocal
    - 全称thread local variable（线程局部变量）功用非常简单，使用场合主要解决多线程中数据因并发产生不一致问题。
    - ThreadLocal为每一个线程都提供了变量的副本，使得每个线程在某时间访问到的并不是同一个对象
    - 这样就隔离了多个线程对数据的数据共享，这样的结果是耗费了内存
    - 但大大减少了线程同步所带来性能消耗，也减少了线程并发控制的复杂度。
    - 总结起来就是：同个线程共享数据

- 注意：ThreadLocal不能使用原子类型，只能使用Object类型



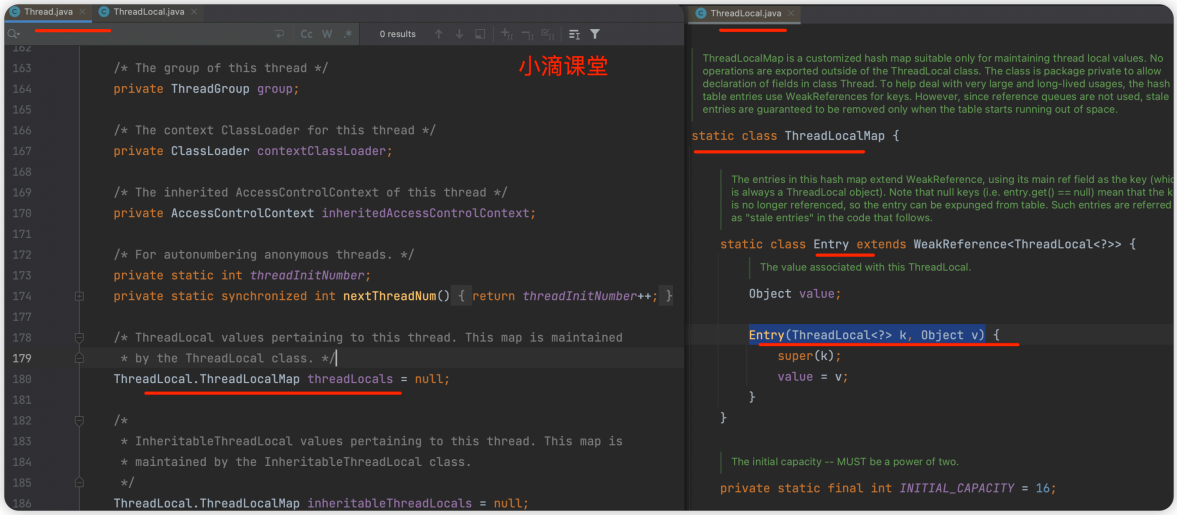
- 应用场景
  - ThreadLocal 用作每个线程内需要独立保存信息，方便同个线程的其他方法获取该信息的场景。
  - 每个线程获取到的信息可能都是不一样的，前面执行的方法保存了信息后，后续方法可以通过 ThreadLocal 直接获取到
  - 避免了传参，类似于全局变量的概念
    - 比如用户登录令牌解密后的信息传递（用户权限信息、从用户系统获取到的用户名、用户ID）



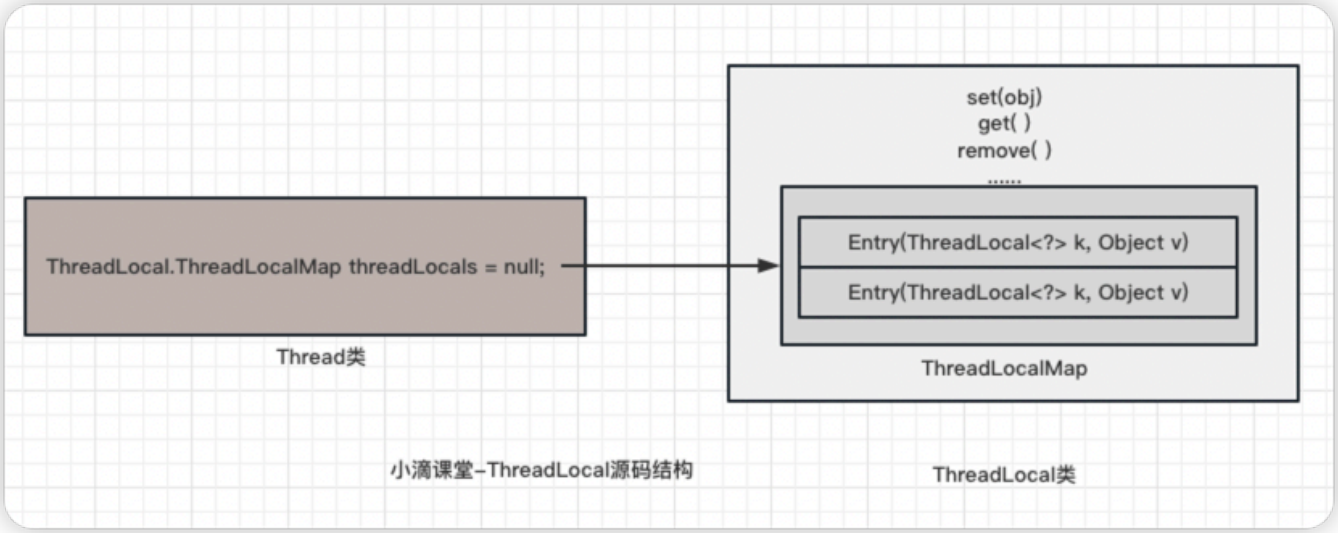
6-说下Thread/ThreadLocal/ThreadLocalMap 三者的关系

- **题目分析和考点**
  - 面试官想知道对于ThreadLocal的理解程度，以及你能否准确地分析并应用ThreadLocal的使用场景
  - 对于Thread/ThreadLocal/ThreadLocalMap的理解程度，能否正确地描述它们之间的关系
  - 还可以考察你对于多线程编程中线程隔离和变量存储的理解，以及对ThreadLocal底层实现原理的了解
- **难度【* * * *】**
- **答案**
  - 关系
    - Thread类，里面有一个ThreadLocalMap类型的变量，**变量名字叫threadLocals，但是ThreadLocalMap类型**
    - ThreadLocal类，里面有一个ThreadLocalMap静态内部类
      - 提供一系列方法操作ThreadLocalMap，比如get/set/remove
      - 隔离Thread和ThreadLocalMap，防止直接创建ThreadLocalMap

        - 自身的get/set内部会判断当前线程是否已经绑定一个ThreadLocalMap，有就继续使用，没有就为其自身绑定
    - ThreadLocalMap 就是保存ThreadLocal的map结构，key就是ThreadLocal本身
        - 所以一个线程只能存储一个值，可以理解为JVM内部维护的一个Map<Thread, Object>
        - 当线程需要用到Object，就用【当前线程】去Map里面获取对应的Object

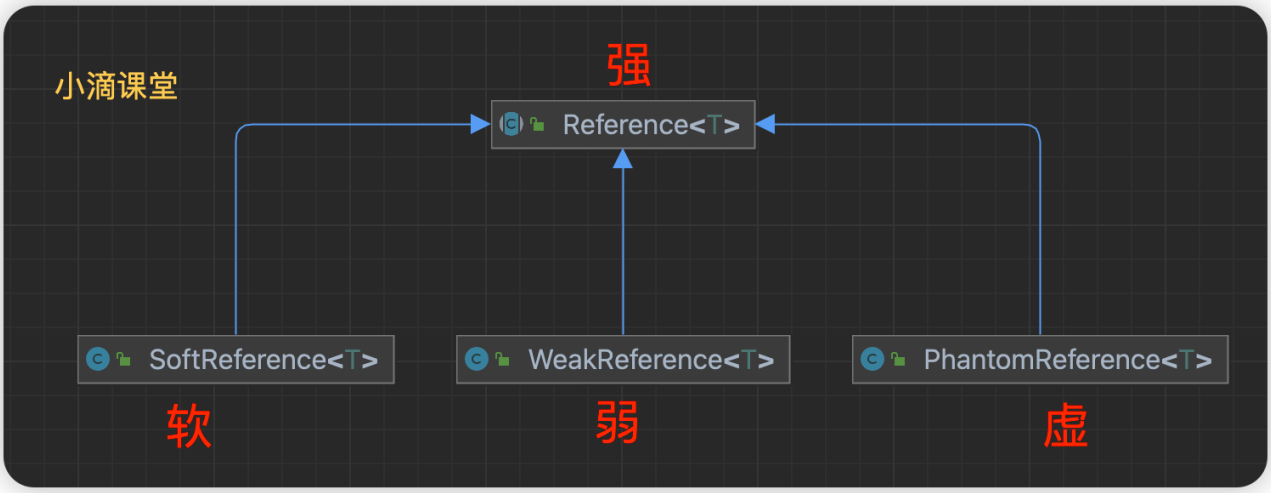


- 操作
    - **Thread类里面有一个ThreadLocalMap类型的变量，不能直接操作这个ThreadLocalMap**
    - **需要通过【工具箱】ThreadLocal才可以操作ThreadLocalMap**
    - **一个Thread只能有一个ThreadLocalMap**
    - **ThreadLocalMap以ThreadLocal为键存储数据**
- 总结
    - ThreadLocal本身并不存储值 (是一个壳子)， 它只是自己作为一个key来让线程从ThreadLocalMap获取value。
    - 因为这个原理，所以ThreadLocal能够实现 “每个线程之间的数据隔离”，获取当前线程的局部变量值，不受其他线程影响



7-JVM里面的四大引用是哪些？使用场景有哪些

- **题目分析和考点**
  - 面试官想知道对于JVM中四大引用的理解程度，以及你能否根据不同引用类型的特点和使用场景进行合理的选择
  - 对于JVM中四大引用的概念和作用的了解：软引用、弱引用、虚引用、强引用
  - 不同引用类型在垃圾回收过程中有不同的生命周期和影响
- **难度【* * * * *】**
- **答案**



  - JVM四大引用类型
    - 强引用
      - 强引用是使用最普遍的引用，当一个对象被强引用关联后，它就不会被垃圾回收器回收
      - 比如String str = “abc”，变量str就是字符串“abc”的强引用
      - 即使在**【内存不足】**的情况下，JVM宁愿抛出OutOfMemoryError，也不会回收这种对象
      - 编码实战
        - 创建User类，覆盖finalize()函数，在finalize()中输出打印信息，方便追踪，来完成“非内存资源”的清理工作
        - finalize()是Object基类的一个方法，在JVM回收内存时执行的，是GC前对待回收的对象进行标记，标记成功后会回调此函数
        - JVM并不保证在回收内存时一定会调用finalize()，可以用这个System.gc()，手动提醒GC，可以进行标记并回收垃圾

```
public class User {
    @Override
    protected void finalize() throws Throwable {
        super.finalize();
        System.out.println("User对象被标记为垃圾啦————————");
    }
}
```

```
public static void main(String[] args) throws InterruptedException {
        testReference();
    }
    /**
     * 强引用测试
     * @throws InterruptedException
     */
    public static void testReference() throws InterruptedException {
        User user  = new User();
        System.out.println("GC前 = "+user);
        user = null;
        System.gc();
        TimeUnit.SECONDS.sleep(1);
        System.out.println("GC后 = "+user);
}
```

- 软引用
  - 软引用是用来描述一些还有用但非必需的对象，当系统内存资源不足时，垃圾回收器会回收这些对象
  - **只有当内存不足时，才会回收软引用关联的对象；当内存资源充足时，不会回收软引用关联的对象，直接调用GC也不回收**
  - 一般在高速缓存中会使用，内存不够时则回收相关对象释放内存
  - 使用 **SoftReference< >** 包装对象就可以转换为软引用
  - 编码实战

```
 /**
     * 只有当内存不足时，才会回收 软引用 关联的对象。当内存资源充足时，不会回收软引用关联的对象，不用手工调用GC
     * -Xms100m -Xmx100m
     */
    public static void testSoftReference() {
        SoftReference<User> softReference = new SoftReference<>(new User());
        System.out.println("内存够用，GC前 = " + softReference.get());
        try {
            TimeUnit.SECONDS.sleep(1);
            consumeMemory();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            System.out.println("内存不够用，GC后 = " + softReference.get());
        }

        try {
```

```
            TimeUnit.SECONDS.sleep(1);}catch (Exception e)
{e.printStackTrace();}
    }


    /**
     * 消耗大量内存, -Xms100m -Xmx100m
     */
    public static void consumeMemory() {
        List<Byte[]> list = new ArrayList<>();
        for (int i = 0; i < 99; i++) {
            //1MB
            Byte[] bytes = new Byte[1 * 1024 * 1024];
            list.add(bytes);
        }
    }


//输出下面
内存够用，GC前 = thread.User@6bf256fa
User对象被标记为垃圾啦————————
内存不够用，GC后 = null
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
  at thread.ReferenceTest.consumeMemory(ReferenceTest.java:63)
  at thread.ReferenceTest.testSoftReference(ReferenceTest.java:47)
  at thread.ReferenceTest.main(ReferenceTest.java:19)
```

- 弱引用
  - 弱引用也是用来描述非必需对象，但是它的强度比软引用更弱一些，只能生存到下一次垃圾收集发生之前
  - **只要垃圾回收器工作时，无论内存是否充足，都会回收被弱引用关联的对象。**
  - 使用了**WeakReference**类来实现弱引用
  - 编码实现

```
 /**
     * 只要垃圾回收器工作时，无论内存是否充足，都会回收被弱引用关联的对象。
     */
    public static void testWeakReference() {
        WeakReference<User> weakReference = new WeakReference<>(new
User());
        System.out.println("内存够用，GC前 = " + weakReference.get());
        try {
            TimeUnit.SECONDS.sleep(1);
            System.gc();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
```

```
            System.out.println("无论内存是否充足，GC后 = " +
weakReference.get());
        }
        try {
            TimeUnit.SECONDS.sleep(1);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

- 虚引用
  - 最弱的一种引用关系，一个对象是否有虚引用的存在，完全不会对其生存时间构成影响，也无法通过虚引用来取得一个对象实例
  - 一个对象设置虚引用关联，目的就是能在这个对象被收集器回收时收到一个系统通知
  - 使用 **PhantomReference**类来实现虚引用，必需要组合使用一个引用队列**ReferenceQueue**
  - **当垃圾回收器要回收一个对象时，如果发现它还有虚引用，会在回收对象的内存之前，把这个虚引用加入到关联的引用队列中**
    - 在虚引用对象传到它的引用队列之前会调用对象的finalize方法
  - 编码实现

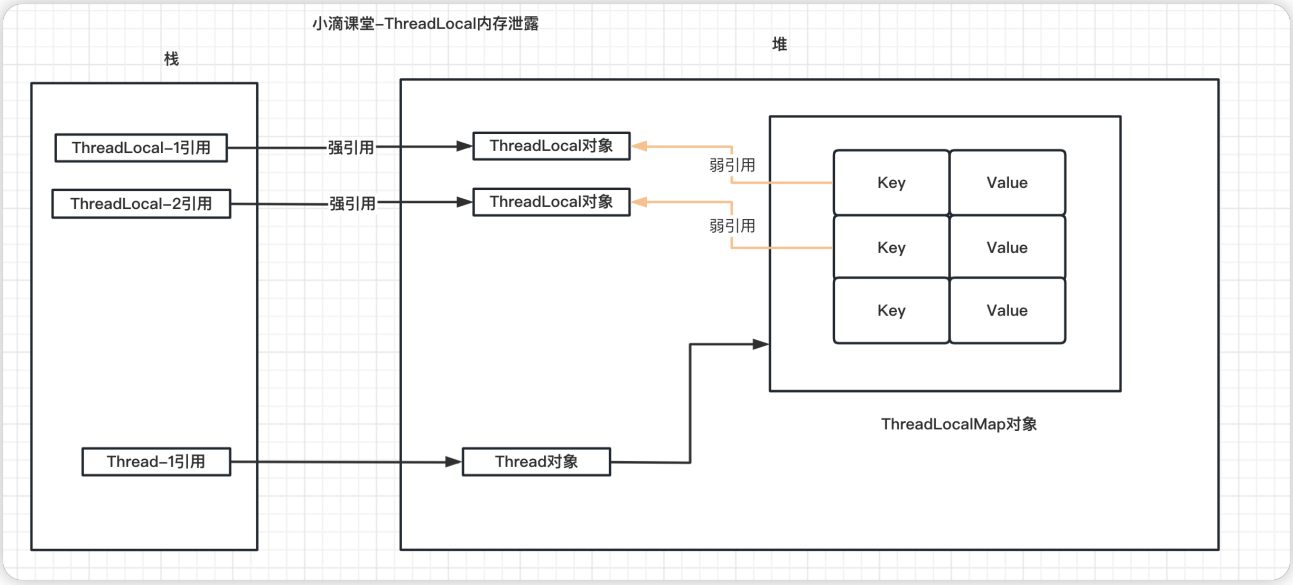
```
   /**
     *  弱引用和虚引用指向的对象在发生GC时一定会被回收，虚引用是得不到引用的对象实例
     */
    public static void testPhantomReference() {
        ReferenceQueue queue = new ReferenceQueue();
        PhantomReference phantomReference = new PhantomReference(new
User(), queue);
        System.out.println("内存够用，GC前 = " + phantomReference.get());
        System.gc();
        consumeMemory();


    }
```

- 总结

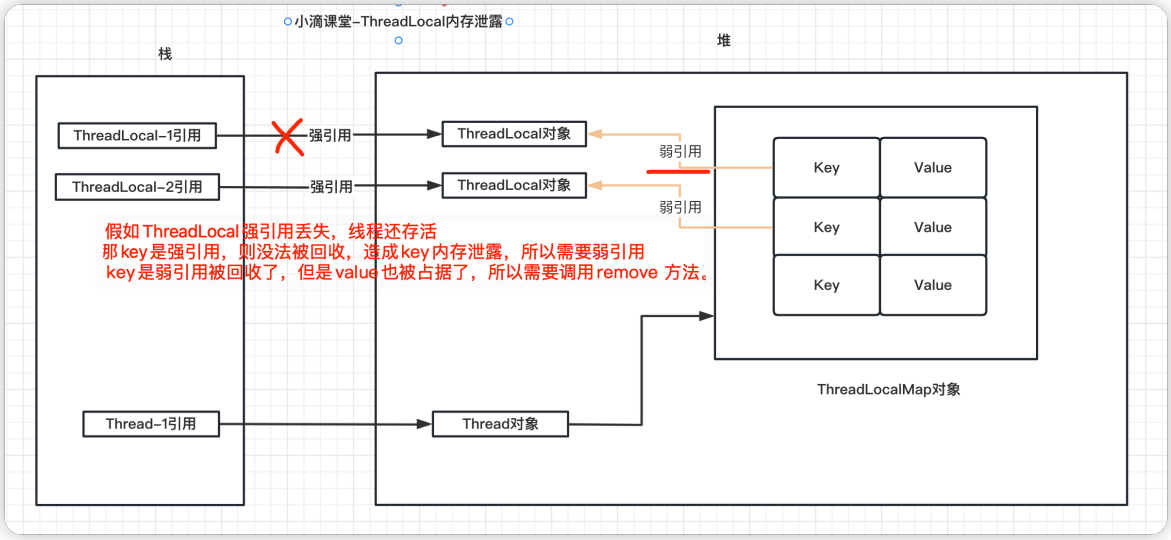
引用类型	被垃圾回收时刻	用途	生存时间
强引用	从来不会	对象的一般状态	JVM停止运行时终止
软引用	在内存不足时	对象简单，缓存，文件缓存，图片缓存	内存不足时终止
弱引用	在gc垃圾回收时	对象简单，缓存，文件缓存，图片缓存	gc运行后终止
虚引用	任何时候都可能被垃圾回收器回收	基本不写，虚拟机使用, 用来跟踪对象被垃圾回收器回收的活动	未知

8-ThreadLocal为啥需要设计成弱引用，用完需要remove呢?

- **题目分析和考点**
  - 面试官想知道对于ThreadLocal的设计原理和内存管理的理解程度，能否意识到ThreadLocal使用弱引用和手动清理的必要性。
  - 考察你对于内存泄漏和内存管理的敏感性和预防意识，能够理解强引用和弱引用的区别
  - 并能够解释为什么ThreadLocal需要使用弱引用来防止内存泄漏，并清理ThreadLocal变量以避免数据错乱的情况
- **难度【*****】**
- **答案**
  - ThreadLocal中的一个内部类ThreadLocalMap，这个类没有实现map接口，是一个普通的Java类，但是实现的类似map的功能
  - 每个数据用Entry保存，继承WeakReference 指向**ThreadLocal（所以是弱引用）键值对存储，键为ThreadLocal的自身引用**
  - 每个线程持有一个ThreadLocalMap对象，每一个新的线程Thread都会实例化一个ThreadLocalMap
  - 并赋值给成员变量threadLocals，使用时若已经存在threadLocals，则直接使用已经存在的对象



- 原因
  - 两个原因分析 ThreadLocalMap 的key和value造成内存泄露解决方案
  - Key问题回收
    - 如果ThreadLocal的引用丢失，ThreadLocalMap的Key如果是强引用，则没法被回收, 造成泄露
    - 所以设计成弱引用，这个时候触发GC时，Key必定会被回收
    - 这个操作是ThreadLocal自身设计进行了解决



  - Value问题回收
    - 由于Key是弱引用被回收了，然后key是null，但是value是强引用对象没法被回收和访问，就导致内存泄露
    - 所以用完需要remove相关的value
    - 这个操作需要开发人员进行操作
  - 坑
    - 常规使用的线程，如果线程对象结束被回收，则上面的key和value都可以被回收

        - 但是业务里面多数是使用线程池，就导致线程不能被回收，从而如果没remove对应的值，则会导致OOM
        - 常规set/get方法里面也会清除key为null的entry对象的方法，但实际开发还是需要直接调用remove方法删除

9-说下CompletableFuture的用途和使用场景

- **题目分析和考点**
    - 面试官想知道对于CompletableFuture的理解程度，以及你能否准确地描述它的用途和适用场景。
    - 考察你对于并发编程和异步任务处理的理解，能够描述它的使用场景
    - 并能够灵活应用CompletableFuture来实现并行计算、异步任务调度和任务组合等功能
- **难度【* * *】**
- **答案**
    - 核心用途：
        - 在项目开发中，由于业务规划逻辑的原因，业务需要从多个不同的地方获取数据，
        - 然后汇总处理为最终的结果，再返回给请求的调用方，就是**聚合信息处理**类的处理逻辑
        - **如果常用串行请求，则接口响应时间长；利用CompletableFuture则可以大大提升性能**
        - 针对多任务，需要进行任务编排调度，也可以使用CompletableFuture进行完成
    - **CompletableFuture类实现了Future和CompletionStage接口，相当于一个Task编排工具**
        - Future
            - 表示异步计算的结果，它提供了检查计算是否完成的方法，以等待计算的完成
            - 计算完成后只能使用 get 方法来获取结果，有cancel、get、isDone、isCancelled等方法
        - CompletionStage
            - 是Java8新增接口，用于异步执行中的阶段处理，CompletableFuture是其中的一个实现类
            - 对任务处理可以构造一条结果传递链，在结果传递过程中任何一个CompletionStage都可以对结果进行处理
                - 包括异常处理、类型转换，可以构造非常简单的传递链也可以构造很复杂的传递链
            - 几个CompletionStage可以串联起来，一个完成的阶段可以触发下一阶段的执行
        - 当前的Task到底由那个Thread执行，使用的不好可能会有性能问题, 根据`CompletableFuture`的方法命名可以掌握
            - xxxx()：表示该方法将继续在当前执行CompletableFuture的方法线程中执行
            - xxxxAsync()：表示异步，在线程池中执行
            - **在没有指定线程池的情况下，使用的是CompletableFuture内部的线程池 ForkJoinPool ,线程数默认是 CPU 的核心数**
                - 一般不要所有业务共用一个线程池，避免有任务执行一些很慢的 I/O 操作，

        - 会导致线程池中所有线程都阻塞在 I/O 操作上，从而造成线程饥饿，影响整个系统的性能

10-说下什么是池化思想，有哪些应用场景

- **题目分析和考点**
    - 面试官想知道你对于高并发场景下性能优化和资源管理的理解程度，以及你能否合理应用池化思想来提高系统性能。
    - 面试者应该能够清晰地解释池化思想的概念和作用，能够描述适用的场景，并举出具体的例子来说明池化思想的应用。
- **难度【* * *】**
- **答案**
    - 什么是池化思想
        - 预先分配申请好相关资源，并维护复用好这些资源
        - 高并发中的池化思想是指事先创建一组对象（例如线程、连接等），并将它们放入一个池中
        - 当需要使用对象时，从池中获取一个可用的对象，使用完毕后再将对象归还给池，以实现对象的重复利用。
        - 提前申请好资源，节省实时分配的时间成本，提升性能，降低相关资源损耗
        - 应用场景
            - 内存池(Memory Pooling)：预先申请内存，提高申请内存的速度，减少内存碎片
            - 连接池(Connection Pooling)：预先申请数据库连接，并保存一定存活的连接，降低系统开销，提高响应速度
            - 对象池(Object Pooling)：循环使用对象，减少资源在初始化和销毁时相关资源损耗
    - 举一个线程池的例子
        - 线程池中会预先创建一定数量的线程，这些线程被放入池中
        - 当有任务需要执行时，会从线程池中获取一个可用的线程来执行任务，执行完毕后将线程归还到线程池中，以便后续的任务继续使用。
        - 好处
            - 可以避免频繁创建和销毁对象带来的开销和性能问题
            - 通过复用已经创建的对象，可以减少对象创建的开销，并通过控制对象的数量，确保系统资源的合理利用。
    - 除了线程池之外，还有连接池、对象池等池化思想的应用。
    - 连接池用于数据库连接的管理，对象池用于缓存、连接、资源等对象的重复利用
    - 这些池化思想都可以提高系统的并发性能和资源利用效率。

11-说下ThreadPoolExecutor设计原理和核心参数

- **题目分析和考点**
  - ThreadPoolExecutor是Java中用于管理线程池的类，是Executor框架的核心实现之一。
  - 它通过预先创建一定数量的线程，并管理这些线程的生命周期，实现了线程的复用和调度。
  - 面试官想知道你对于线程池的理解程度和实际应用能力，以及你对于线程池参数的配置和性能调优的意识。
  - 一个优秀的候选人应该能够清晰地解释ThreadPoolExecutor的设计原理和工作机制
  - 能够描述核心参数的作用和影响，并能够基于业务需求合理配置线程池参数来提高系统的性能和效率。
- **难度【* * * * 】**
- **答案**
  - ThreadPoolExecutor线程池几个重要参数，什么时候会创建线程
    - 查看核心线程池是否已满，不满就创建一条线程执行任务，否则执行第二步。
    - 查看阻塞队列是否已满，不满就将任务存储在阻塞队列中，否则执行第三步。
    - 查看线程池是否已满，即是否达到最大线程池数，不满就创建一条线程执行任务，否则就按照策略处理无法执行的任务
    - 注意
      - 如果队列的任务为空，阻塞队列可以保证任务队列中没有任务时，阻塞获取任务的线程，线程进入wait状态释放cpu资源
      - 阻塞队列自带阻塞和唤醒的功能，无任务执行时线程池利用阻塞队列的take方法挂起
      - 阻塞队列可以维持核心线程的存活且不一直占用cpu资源，而一般的队列只能作为一个有限长度的缓冲没有其他功能
  - 核心参数配置说明

参数	说明
corePoolSize	线程池创建的核心线程数，线程池维护线程的最少数量，即使没有任务需要执行，也会一直存活
maximumPoolSize	最大线程池数量，当线程数>=corePoolSize，且任务队列已满时。线程池会创建新线程来处理任务
keepAliveTime	当线程空闲时间达到keepAliveTime时，线程会退出，直到线程数量=corePoolSize
TimeUnit	时间单位
workQueue	缓存队列（阻塞队列）当核心线程数达到最大时，新任务会放在队列中排队等待执行
threadFactory	线程创建的工厂，一般用默认的 Executors.defaultThreadFactory()
handler	当pool已经达到max size的时候，如何处理新任务

  - 任务队列满，最大线程数也创建满后，新任务进来会根据拒绝策略处理

        - AbortPolicy()：该策略是线程池的默认策略，如果线程池队列满了丢掉这个任务并且抛出RejectedExecutionException异常。
        - CallerRunsPolicy()：交由调用方线程运行，比如 main 线程；如果添加到线程池失败，那么主线程会自己去执行该任务，不会等待线程池中的线程去执行
        - DiscardPolicy()：如果线程池队列满了，会直接丢掉这个任务并且不会有任何异常
        - DiscardOldestPolicy()：丢弃队列中最老的任务，队列满了，会将最早进入队列的任务删掉腾出空间，再尝试加入队列

12-说下java自带的Executors线程池种类和特点

- **题目分析和考点**
    - 面试官想知道你对于Java中提供的不同类型线程池的了解程度，以及你能否根据场景的需求选择合适的线程池类型。
    - 考察你对于性能优化和资源管理的意识，一个优秀的候选人应该能够清晰地描述不同线程池类型的特点和应用场景
    - 并能够根据业务需求合理配置线程池参数，以提高系统的性能和效率。
- **难度【 * * * 】**
- **答案**
    - JUC包下的Executors工具类提供多种线程池

线程池名称	说明
newFixedThreadPool	一个定长线程池，可控制线程最大并发数
newCachedThreadPool	一个可缓存线程池，
newSingleThreadExecutor	一个单线程化的线程池，用唯一的工作线程来执行任务
newScheduledThreadPool	一个定长线程池，支持定时/周期性任务执行

    - **newFixedThreadPool**
        - 通过参数创建固定数量的线程
    - **newCachedThreadPool**
        - 如果线程池长度超过处理需要，可灵活回收空闲线程，若无可回收，则新建线程
        - 该线程池会复用空闲的线程，从而减少创建对象和回收对象带来开销
            - 使用 `SynchronousQueue` 队列，不存储元素的阻塞队列，每put一个必须等待take操作，否则不能添加元素
    - **newSingleThreadExecutor**
        - 一个单线程化的线程池，用唯一的工作线程来执行任务

- **newScheduledThreadPool**
  - 线程池支持定时以及周期性执行任务，创建一个corePoolSize为传入参数
  - 使用 `DelayedWorkQueue` 延迟队列，可以指定多久时间后才可以从队列获取元素

13-生产环境事故分析-线程池使用选择问题

- **题目分析和考点**
  - 面试官想知道你对于Java中提供的不同类型线程池的了解程度，以及你能否根据场景的需求选择合适的线程池类型。
  - 考察你对于性能优化和资源管理的意识，一个优秀的候选人应该能够清晰地描述不同线程池类型的特点和应用场景
  - 并能够根据业务需求合理配置线程池参数，以提高系统的性能和效率。
- **难度【 * * * * 】**
- **问题描述**
  - 微服务架构下，前端请求后端获取商品详情信息，需要把商品的基础、详情信息、库存、优惠活动等聚合起来
  - 一并的返回给用户，使用了`CompletableFuture+自定义线程池`
  - 并行请求多个接口，响应时间更快，用户体验更好，从而可以到达3.75绩效，赢取 冰冰 白富美
  - 基于阿里巴巴编码规范，创建了这样的线程池 （32核64G内存的机器）

```
ThreadPoolExecutor threadPool = new ThreadPoolExecutor(2, 32, 3,
       TimeUnit.SECONDS, new ArrayBlockingQueue<Runnable>(100000),
        new ThreadPoolExecutor.DiscardPolicy());
```

- **事故现象**
  - 大促活动下，多用户进入商品详情页出现超时，页面刷新不出来
- **事故分析**
  - 高并发业务下，应该最大程度的快速响应用户
  - 案例中队列设置过长，maxPoolSize设置失效，导致请求数量增加时大量任务堆积在队列中
  - 应该不设置过大的队列去缓冲并发任务，而是调高corePoolSize和maxPoolSize，快速创建更多线程去处理请求

```
ThreadPoolExecutor threadPool = new ThreadPoolExecutor(32,128, 60,
       TimeUnit.SECONDS, new ArrayBlockingQueue<Runnable>(1000),
        new ThreadPoolExecutor.DiscardPolicy());
```

- 关于线程池的底层参数配置

- 互联网项目可以分**IO密集**还**是CPU密集**项目，不同项目配置不一样
  - CPU密集型项目**主要消耗CPU**，所以建议**设置为跟核心数一样**或者+1
  - IO密集型项目，系统会用大部分的**时间来处理 I/O 交互**，这段时间**不占用CPU**，所以线程可以配置多点，**为2倍CPU核心数**
  - 但**并非万能配置，最好根据压测情况看再进一步优化参数，且不同的模块要做好线程池的隔离**

JVM虚拟机

1-什么是Java虚拟机，常见的版本有哪些?

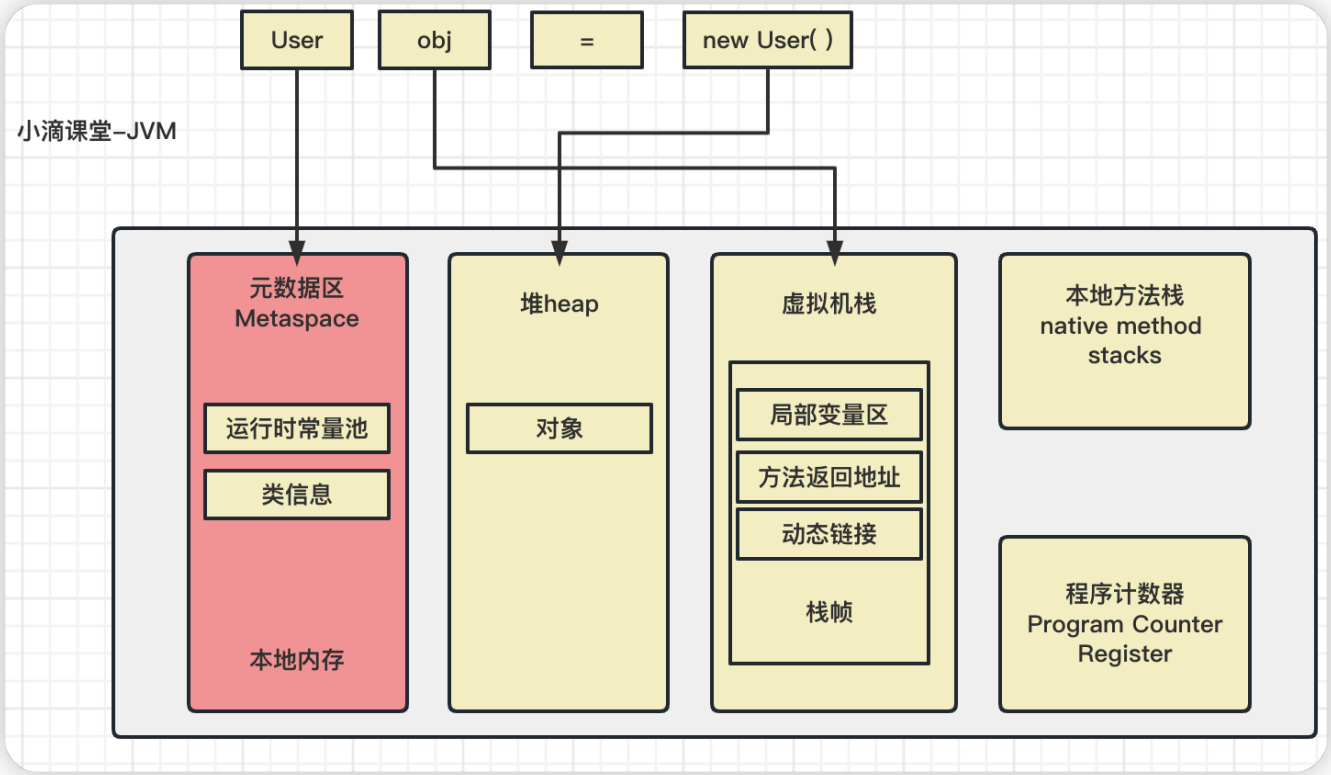
- **题目分析和考点**
  - 面试官想知道你对于Java虚拟机的版本历史了解程度，熟悉主要JVM实现和其特性。
  - 对于不同JVM版本的优劣和适用场景的理解。
  - 回答可以对某个特定的Java虚拟机版本进行更深入的了解，并讨论其在性能、内存管理、多语言支持等方面的优势和应用
- **难度【* *】**
- **答案**
  - JVM虚拟机
    - Java Virtual Machine是一种虚拟机，它是Java程序运行的环境
    - JVM提供了Java程序的运行时环境，包括内存管理、垃圾回收、安全性、类加载等功能
    - 是Java语言的核心，它使Java语言具有跨平台的特性（**一次编译四处运行**）
  - 常见实现
    - 除了以下几种JVM的具体实现，还有一些其他的实现
    - 这些JVM实现在实现上会有一些差异，但是它们的基本功能是相同的
    - JVM实现的差异主要体现在性能、稳定性、兼容性等方面
    - 不同的JVM实现在不同的场景下可能会有不同的表现

虚拟机名称	说明
HotSpot	是Sun JDK和OpenJDK中所带的虚拟机，也是目前使用范围最广的Java虚拟机，主要使用C++实现，JNI接口部分用C实现
JRockit	除 HotSpot 之外另一款比较厉害的 VM， 一开始是BEA 公司的，一度获取运行最快的虚拟机。 Oracle 在 2008 年收购了 BEA 公司，JRockit 与 HotSpot 同属于 Oracle
J9 VM	基于IBM公司开发的JDK，主要应用在IBM公司开发的的软件或服务器端，如：嵌入式、服务端、桌面等，基本上IBM本司出品的产品都是用的J9 VM

2-说下JVM内存的5大组成部分(基于JDK8以上的hotSpot虚拟机)

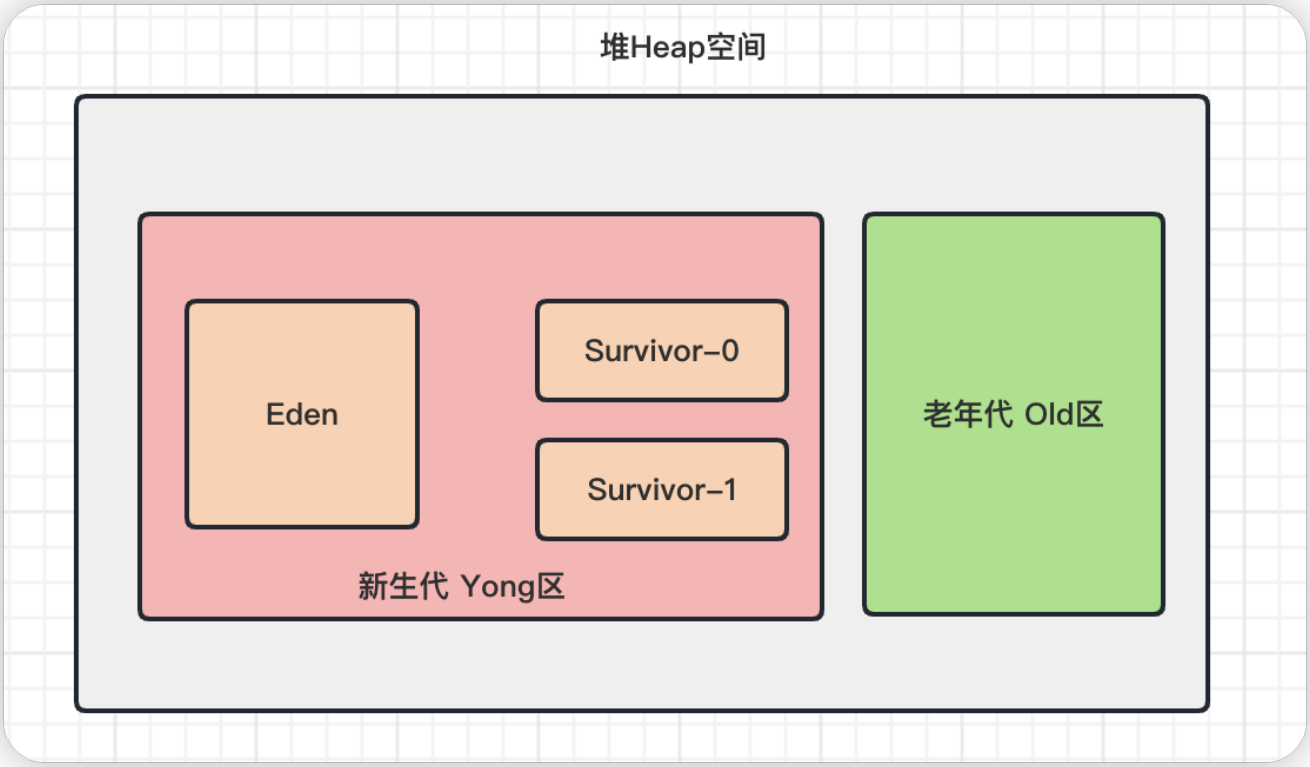
- **题目分析和考点**
  - 面试官想知道你对Java虚拟机内存模型的了解程度和对Java内存管理的熟悉程度
  - 理解这些JVM内存的组成部分，以及每个部分的作用和特点，可以更好地理解Java内存模型并进行内存调优和问题排查
- **难度【* * *】**
- **答案**
  - JVM内存的5大组成（基于JDK8的HotSpot虚拟机，不同虚拟机不同版本会有不一样）

名称	作用	特点
程序计数器	也叫PC寄存器，用于记录当前线程执行的字节码指令位置，以便线程在恢复执行时能够从正确的位置开始	线程私有
Java虚拟机栈	用于存储Java方法执行过程中的局部变量、方法参数和返回值，以及方法执行时的操作数栈	线程私有
本地方法栈	用于存储Java程序调用本地方法的参数和返回值等信息。	线程私有
堆	用于存储Java程序创建的对象，所有线程共享一个堆，堆中的对象可以被垃圾回收器回收，以便为新的对象分配空间	线程共享
元数据区	用于存储类的元数据信息，如类名、方法名、字段名等，以及动态生成的代理类、动态生成的字节码等 元空间是位于本地(直接)内存中的，而不是像JDK8之前方法区位于堆内存中的。	线程共享



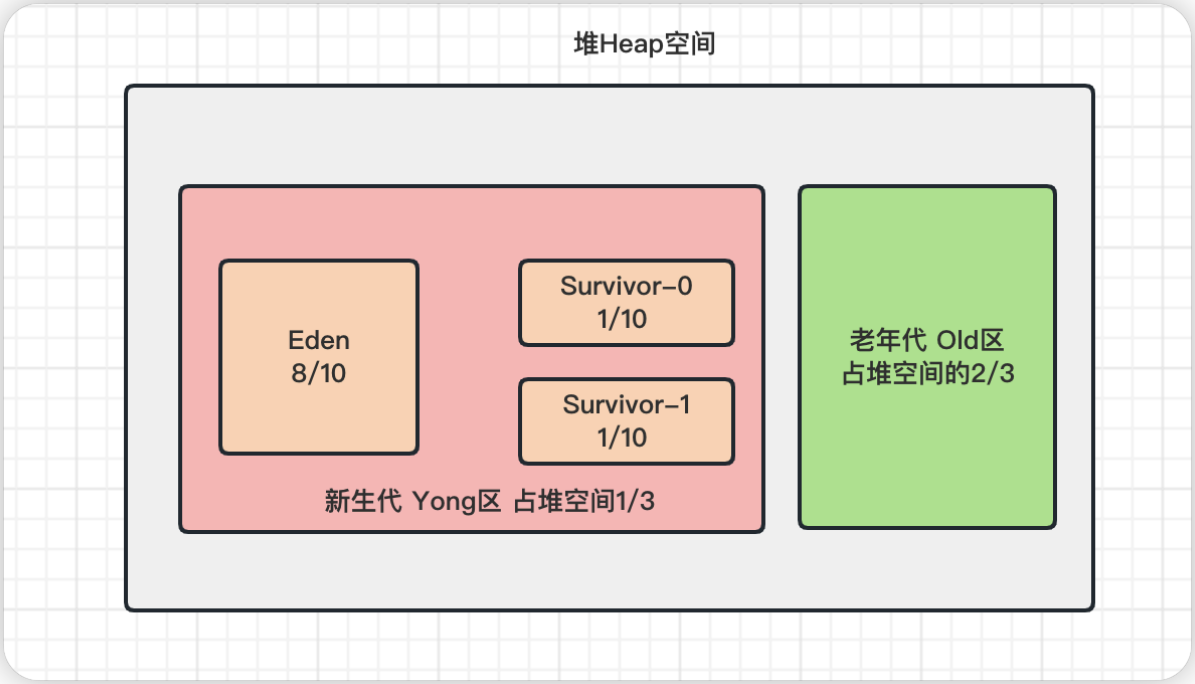
3- JVM的堆内存划分和堆内存垃圾回收流程是怎样的

- **题目分析和考点**
  - 面试官想知道你对Java堆内存结构和垃圾回收机制的了解程度，以及你对内存管理和性能优化的知识
  - 通过了解堆内存划分和垃圾回收流程，内存管理的相关知识，可以更好地理解和解决内存相关的问题，进行性能优化和调优。
- **难度【* * * * 】**
- **答案**
  - 新建对象，放到Eden区，满后触发Minor GC（每次都是由Eden区满触发Minor GC，接连放对象到S0或S1）
  - 存活的对象移动到Survivor的S0区，如果S0满后触发Minor GC
  - S0存活下来的对象移动到S1区，然后S0区空闲
  - S1满后触发Minor GC，再次移动到S0区，然后S1区空闲
  - 反复GC每次对象涨1岁，到达一定次数后(默认15)，进入老年代
  - 当老年代内存不足会触发Full GC，出现STW（Stop-The-World）
  - **堆被垃圾回收，基本都是采用分代收集算法，不通区域的采用不同的垃圾回收算法**
  - **方法结束后，堆中的对象不会马上移除，在垃圾回收的时候才会被移除（不是实时GC的，ThreadLocal里面说过）**



4-新版JDK11以上堆空间里面的划分比例是怎样的?

- **题目分析和考点**
  - 面试官想知道你对Java堆内存划分的了解程度，以及对内存管理和性能调优的知识
  - 理解堆空间的划分比例和不同代的作用，以及对堆空间的动态调整，更好地理解和解决内存相关的问题，并进行性能优化和调优
- **难度【***】**
- **答案**
  - 在JDK8中，堆空间的分配比例如下
    - 年轻代（Young Generation）
      - 年轻代通常被划分为3个区域，分别是Eden区和两个Survivor区（From和To）
      - 年轻代的划分比例为8:1:1，即Eden区占据整个年轻代的80%，而每个Survivor区各占据年轻代的10%
    - 老年代（Old Generation）
      - 老年代在堆空间中通常占据整个堆空间的2/3。
    - 永久代（Permanent Generation）
      - 在JDK8中，永久代已经被元空间（Metaspace）所取代
      - 元空间的占比是不固定的，它使用的是本地内存而非堆内存。



- 注意
  - 具体的堆空间划分比例可能会因为JVM的配置、应用程序的需求以及GC策略的选择而有所不同
  - 不同的JVM实现和版本也可能对堆空间的分配比例进行了优化和调整

5-JVM自带很多参数，列举下有哪些分类和参数案例

- **题目分析和考点**
  - 面试官想知道你对JVM参数的了解程度和对JVM性能优化的知识
  - 了解JVM自带的参数分类以及常见的参数案例，对于理解JVM配置和性能优化非常重要
- **难度【* * *】**
- **答案**
  - JVM参数格式分类

格式	解释	例子
标准参数(-)	所有JVM都实现这些参数的功能	-verbose:gc 打印GC简要信息
非标准参数(-X)	不保证所有JVM实现都满足	-Xmx2048m等价 -XX:MaxHeapSize JVM最大堆内存为2048M
非稳定参数(-XX)	不稳定未来可能取消，但很有用	-XX:+PrintGCDetails每次GC时打印详细信息。
-XX:+	开启对应的参数	-XX:+PrintGCDetails 开启每次GC时打印详细信息。
-XX:-	关闭对应的参数	-XX:-DisableExplicitGC 禁止调用System.gc()
-XX:=	设定数字参数	-XX:NewRatio=2 新生代和老年代内存比例

- JVM堆栈内存配置参数

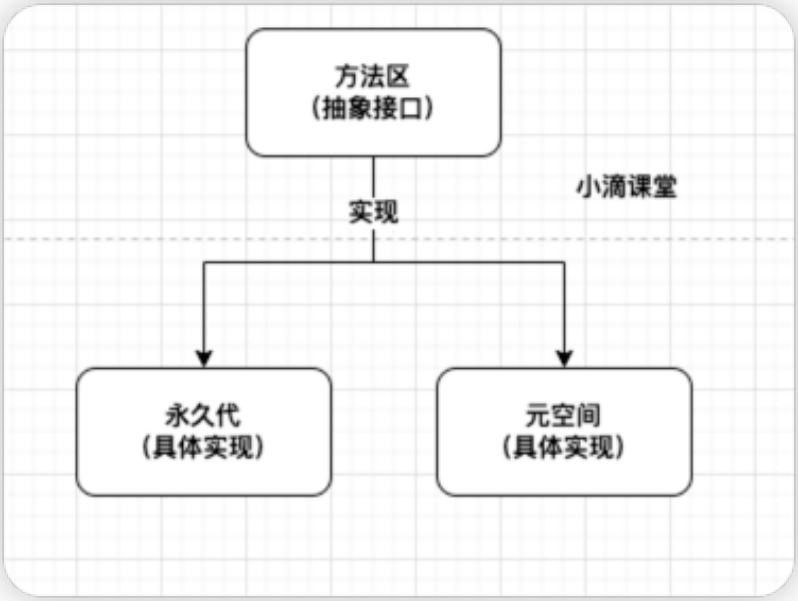
参数	解释
-Xms	初始堆大小，推荐和最大堆一样
-Xmx	最大堆大小，推荐和初始堆一样
-Xmn	年轻代大小
-Xss	每个线程的栈大小

- JVM常见的命令行参数配置

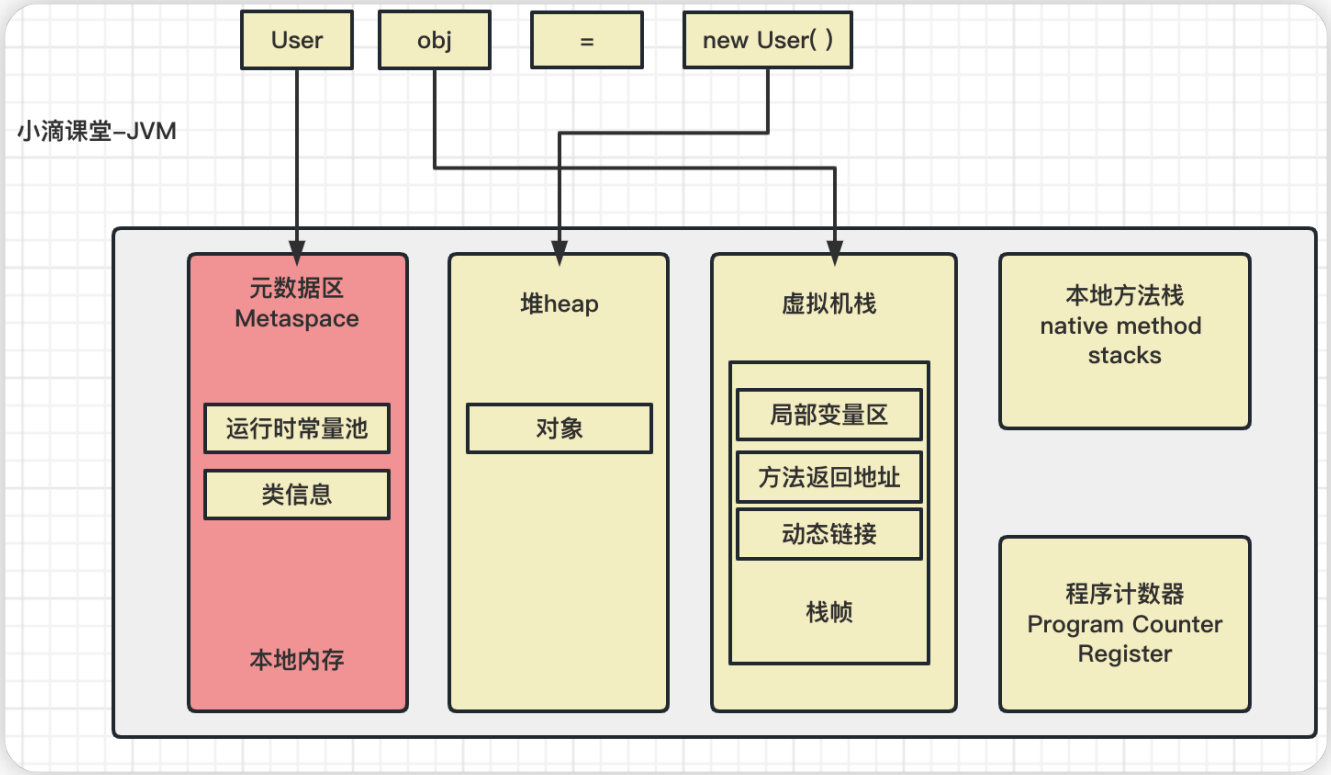
参数	解释
-XX:+PrintGCDetails	打印GC回收信息
-XX:NewRatio	新生代和老年代空间大小的比率，由`-XX:NewRatio`参数控制`-XX:NewRatio`参数的默认值是2，表示新生代和老年代的比例是1:2 如果将`-XX:NewRatio`设置为4，表示新生代和老年代的比例是1:4
-XX:MaxMetaspaceSize	元空间所分配内存的最大值，默认没限制
-XX:+UseConcMarkSweepGC	设置并发收集器

6-介绍下JDK8之后的方法区实现和元空间的联系

- **题目分析和考点**
  - 面试官想知道你对JVM内存模型的了解程度，以及对Java内存管理的熟悉程度
  - 了解JDK8之后的方法区实现和元空间的联系，可以更好理解Java内存模型的变化和进一步的内存管理优化
  - 方法区和元空间都是用于存储类的元数据，它们的作用相同，都是为了提供运行时的类信息。
  - 不同之处在于，方法区基于固定大小的永久代实现，而元空间使用本地内存，可以根据应用程序的需要动态分配和释放内存空间
- **难度【* * *】**
- **答案**
  - 什么是方法区和元空间
    - 【方法区】是JVM中用来存储类的元数据信息的区域，包括**类的结构**、**方法**、**字段信息**等，Java堆类似各个线程共享的内存区域
    - **元空间、永久代是方法区具体的落地实现**
      - java8之前是称为永久代（PermGen），java8后引入的一个新概念【元空间】用于替代旧版JVM中的永久代（PermGen）
        - 方法区和永久代以及元空间的关系很像 Java 中接口和类的关系
        - 类实现了接口，类就可以看作是永久代和元空间，接口可以看作是方法区
        - 永久代是 JDK 1.8 之前的方法区实现，JDK 1.8 及以后方法区的实现便成为元空间



      - 元空间的大小是动态的，可以根据需要进行自动扩展，如果元空间不足，JVM会抛出OutOfMemoryError : Metaspace



7-介绍下元空间配置参数和什么操作会出现元空间不足

- **题目分析和考点**
  - 面试官想知道你对JVM内存模型的了解程度，以及对Java内存管理的熟悉程度
  - 了解JDK8之后的元空间，可以更好理解Java内存模型的变化和进一步的内存管理优化
  - 不同之处在于，方法区基于固定大小的永久代实现，而元空间使用本地内存，可以根据应用程序的需要动态分配和释放内存空间
  - 如果应用程序定义了大量的类，或者使用了动态生成类的技术，会导致元数据占用的空间增加
- **难度【***】**
- **答案**
  - 元空间大小配置
    - -XX:MetaspaceSize
      - 用来设置元空间初始大小的参数，它的默认值是21 MB
    - -XX:MaxMetaspaceSize
      - 用来设置元空间最大大小的参数，它的默认值是-1 即不限制，使用的是本地内存，不像旧版的永久代是堆内存

        - 如果不限制元空间的大小，可能会导致元空间占用过多的内存，从而引起内存溢出
    - 系统参数查看
        - 这两个参数的单位是字节（B），可以使用K、M、G等后缀来表示更大的单位
        - 代码

```
public class HeapDemo {
    public static void main(String[] args) throws InterruptedException {
        System.out.println("小滴课堂-架构大课");
        Thread.sleep(10000000);
    }
}
```

        - 查看命令

```
jps #查看进程号
jinfo -flag MetaspaceSize 进程号  #查看Metaspace分配内存空间
jinfo -flag MaxMetaspaceSize 进程号 #查看Metaspace最大空间
```

        - 调整

```
-XX:MetaspaceSize=126m -XX:MaxMetaspaceSize=524m
```

    - 空间不足异常信息 `“Java.Lang.OutOfMemoryError：Metaspace”`

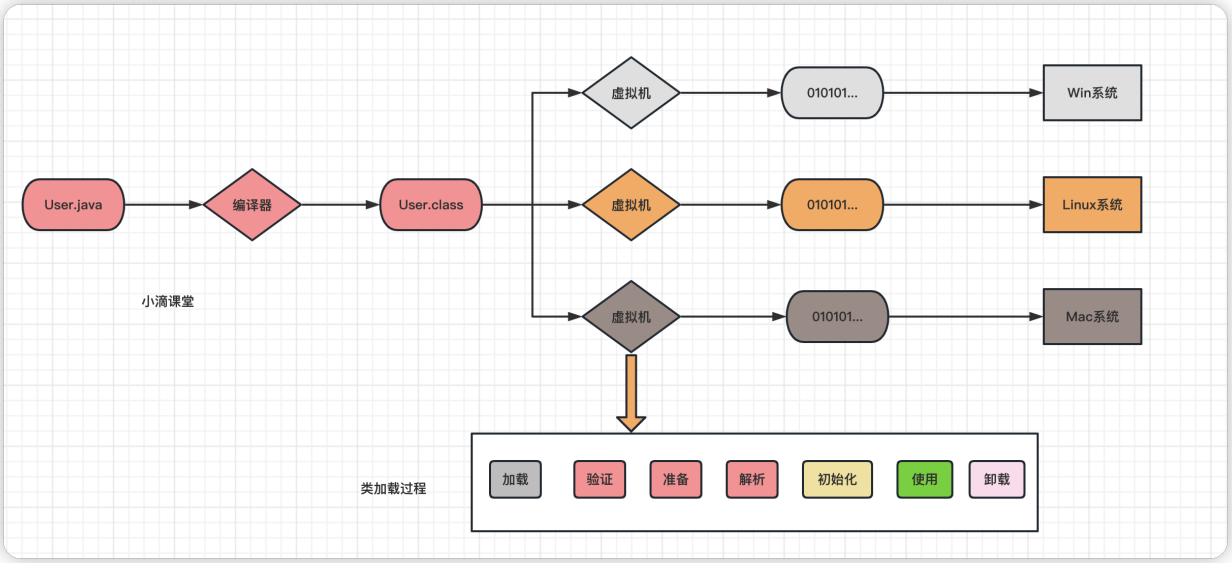
- 什么时候容易出现元空间不足的情况
    - 应用程序使用大量的反射技术，例如使用Class.forName()等方法加载类，或者使用Java Reflection API进行操作
    - 使用大量的动态代理技术，例如使用Java Proxy类等技术
    - 使用大量的注解，例如使用Spring框架的注解等
    - 使用的第三方库过多，这些库可能会在运行时动态生成新的类，导致元空间内存占用过多。
    - 应用程序的业务逻辑比较复杂，需要加载大量的类
    - 应用程序使用大量的JSP页面，其中每个页面都对应一个类文件（现在很少）

8-介绍新版JVM虚拟机的类加载子系统

- **题目分析和考点**
    - 面试官想知道你对类加载机制的了解程度，以及对JVM内部机制的熟悉程度

- 了解类加载机制是JVM的关键部分，负责加载、验证、准备和解析类文件，将类的字节码转换成Java虚拟机可以执行的字节
- 了解新版JVM虚拟机的类加载子系统，可以更好理解JVM内部机制和类的加载过程

- **难度【* * * * *】**
- **答案**
  - 类加载子系统
    - 是Java虚拟机的一个重要子系统，主要负责将类的字节码加载到JVM内存的**方法区**，并将其转换为JVM内部的数据结构



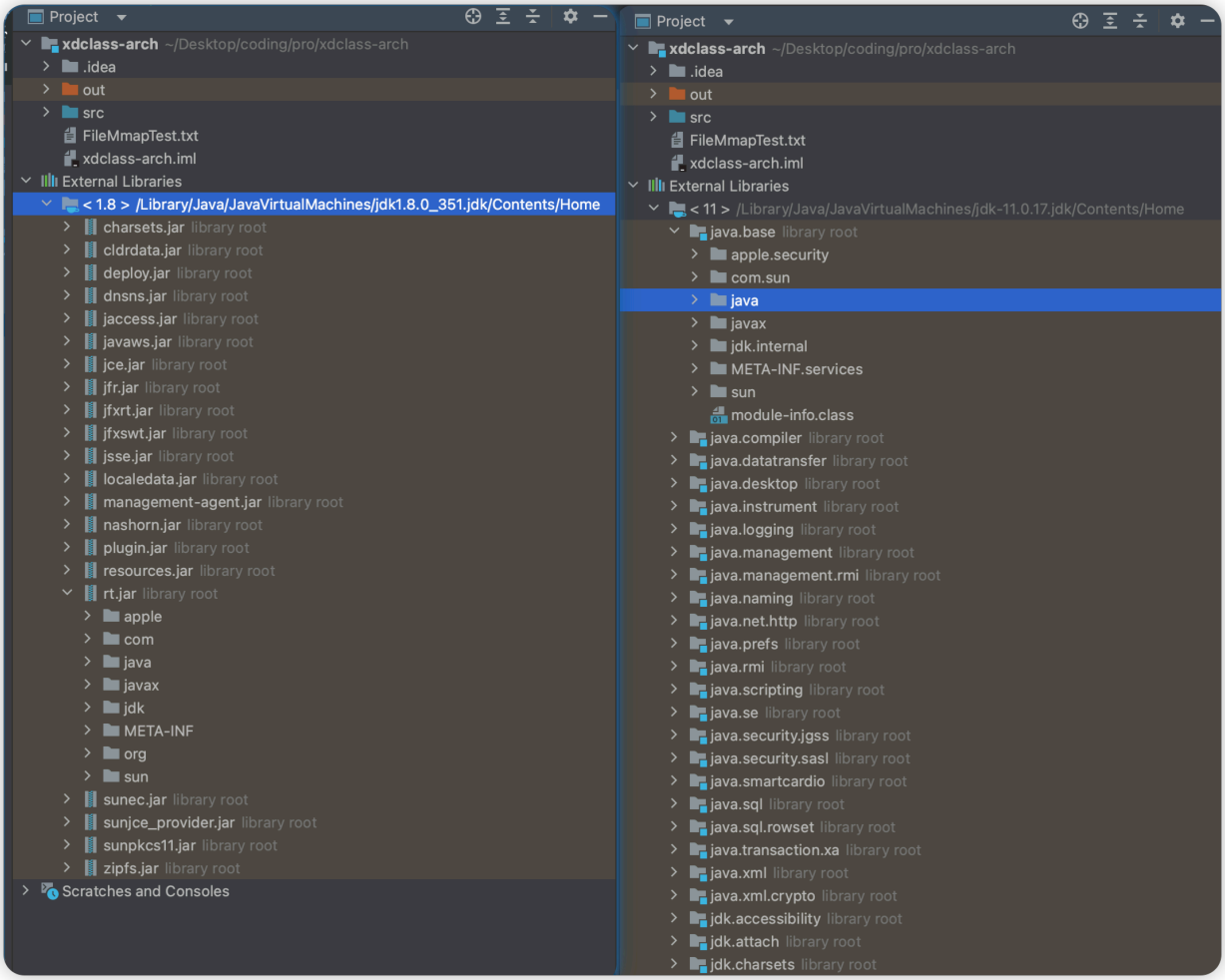
  - **三大特点**
    - **双亲委派模型**
      - Java虚拟机采用双亲委派模型来加载类，即先从父类加载器中查找类，如果找到了就直接返回
      - 否则再由自己的加载器加载，这种模型可以避免类的重复加载，提高系统的安全性。
    - **延迟加载**
      - Java虚拟机采用延迟加载的策略，即只有在需要使用某个类时才进行加载
      - 这种策略可以减少系统启动时间，提高系统的性能
    - **动态加载**
      - Java虚拟机支持动态加载类，即可以在程序运行时动态地加载和卸载类
      - 这种特性可以使Java程序更加灵活和可扩展

  - 三个模块组成类加载子系统
    - 加载器（ClassLoader）
      - 加载器负责将类的字节码加载到JVM中
      - 三种类型的加载器
        - 启动类加载器（Bootstrap ClassLoader）
          - c/c++实现，加载核心类库使用，不继承ClassLoader，没父加载器

        - 平台类加载器（Platform ClassLoader）
            - 注意：JDK9之前是扩展类加载器 Extension ClassLoader
        - 应用程序类加载器（Application ClassLoader）
            - 程序的默认加载器，我们写的代码基本都是由这个加载器负责加载
        - **类加载器用父类加载器、子类加载器这样的名字，虽然看似是继承关系，实际上是组合（Composition）关系**
    - 链接器（Linker）
        - 负责将Java类的二进制代码链接到Java虚拟机中，并生成可执行的Java虚拟机代码，包括 **验证、准备和解析** 等
        - 验证操作主要是验证类的字节码是否符合JVM规范
        - 准备操作主要是为类的静态变量分配内存并设置默认值
        - 解析操作主要是将符号引用转换为直接引用
    - 初始化器（Initializer）
        - 负责执行Java类的静态初始化，包括静态变量的赋值、静态代码块的执行等
        - 初始化器是类加载子系统的最后一个阶段

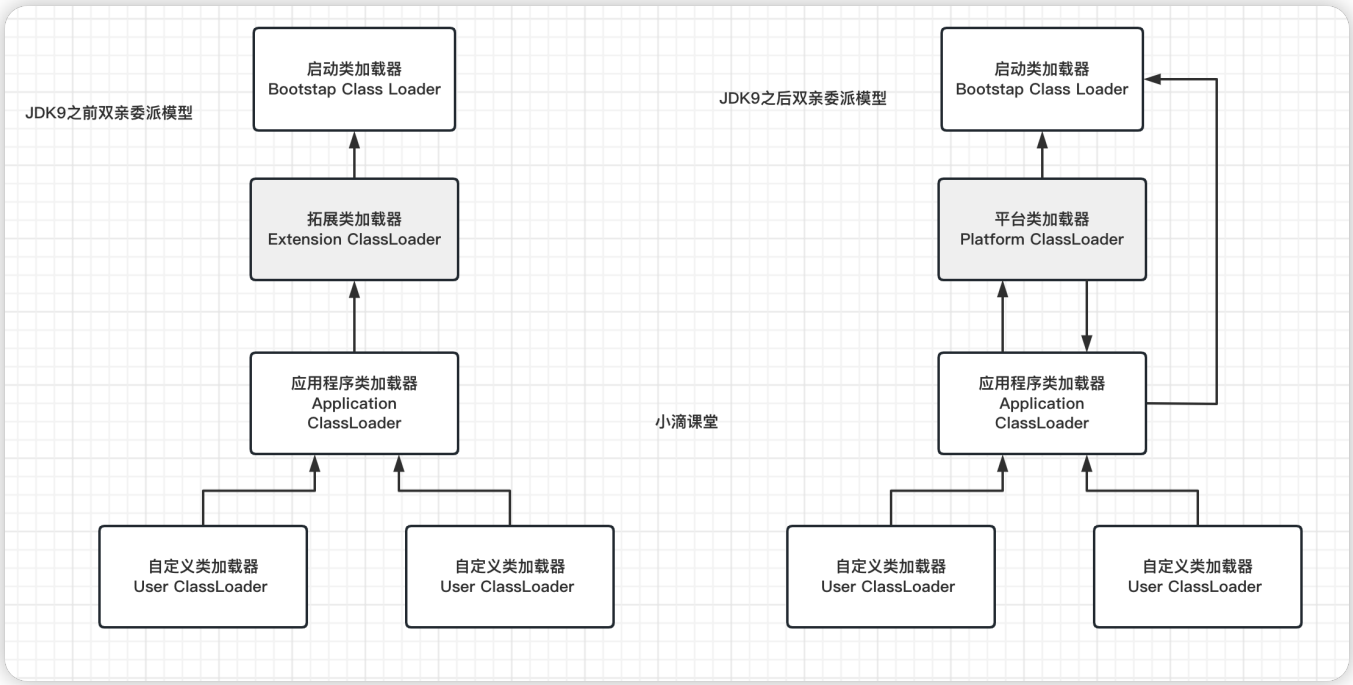
9-说下JDK9的模块化系统和新版双亲委派模型

- **题目分析和考点**
    - 面试官想知道你对Java模块化和类加载机制的了解程度，以及对JDK版本升级的关注程度
    - JDK9引入的模块化系统，即Java Platform Module System允许将代码和依赖关系构建为模块，提高维护性、安全性和性能
    - JDK9中改进的双亲委派模型，即Platform ClassLoader和App ClassLoader之间增加了一个名为PlatformLoader的新类加载器。
- **难度【*****】**
- **答案**
    - JDK9模块化系统
        - 是一种新的Java平台的组织方式，将Java SE分成多个模块，每个模块都有自己的API和实现
        - 每个模块都有一个唯一的标识符和版本号，可以独立地进行开发、测试、部署和维护
        - 模块之间的依赖关系通过模块描述文件（module-info.java）来声明
        - 这个文件包含模块的名称、版本号、导出的包、依赖的模块等信息
        - 在编译和运行时，模块系统会根据模块描述文件来加载和链接模块，确保模块之间的依赖关系正确
    - JDK9之前和JDK89类库的组织结构



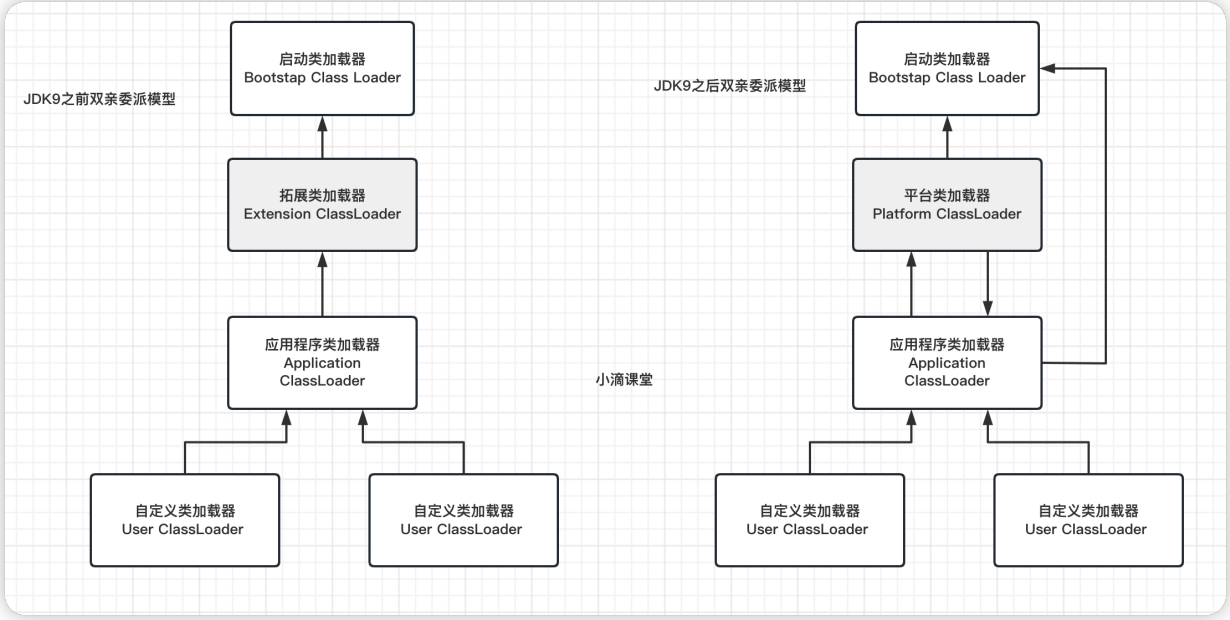
- 双亲委派模型
  - 是一种类加载机制，类加载器之间形成了一条类加载器链，每个类加载器都有一个父类加载器，形成了从下到上的一条继承链
  - 如果所有的类加载器都无法加载该类，则会抛出 ClassNotFoundException 异常
  - 加载流程
    - 一个类的加载请求首先会被委派给其父类加载器进行处理
    - 如果父类加载器无法加载该类，则会将加载请求委派给其自身进行加载
    - 如果自身也无法加载该类，则会将加载请求委派给其子类加载器进行处理，直到找到能够加载该类的类加载器为止
  - 优点
    - 可以保证类的唯一性和安全性。由于每个类加载器都只能加载自己的命名空间中的类
    - 由于类加载器之间形成了一条继承链，因此可以保证类的安全性，防止恶意代码的注入
- 新版本的JDK9后的类加载器
  - 模块化系统中的类加载器可以分为两种类型
    - **应用程序类加载器，加载器用于加载应用程序中的模块； 平台类加载器，加载器用于加载 JDK 中的模块**
    - 当平台及应用程序类加载器收到类加载请求，**在委派给父加载器加载前，要先判断该类是否能够归属到某一个系统模块中**

        - 如果`findLoadedModule`可以找到这样的归属关系，就要优先委派给负责那个模块的加载器完成加载
    - 在模块化系统中，每个模块都有一个类加载器，它根据模块的依赖关系来加载模块中的类和依赖的模块中的类
    - 在模块化系统中，类加载器的原理与传统的类加载器相似，都是采用双亲委派模型
    - 当一个类被加载时，类加载器首先会检查自己是否已经加载过该类，如果没有，则会将该类的加载请求委托给其父加载器
    - 直到达到顶层的 Bootstrap ClassLoader 为止，如果所有的父加载器都无法加载该类，则由当前加载器自己来加载该类



10-不同类加载器是否会重复加载同个class类

- **题目分析和考点**
    - 面试官想知道你对自定义类加载器知识
    - 定义类相同的条件：类加载器相同+ class文件相同
- **难度【* * *】**
- **答案**
    - **不同类加载器会加载同名的 class 类，这些类在 JVM 中是不同的，即它们的 Class 对象是不同的**
    - **关键点，定义类相同的条件：类加载器相同+class文件相同**



- 不同类加载器不会重复加载同一个Class类，因为Java类加载器采用了双亲委派模型
- 一个类加载器在加载类时，会先委派给其父类加载器尝试加载，只有当父类加载器无法加载时，才会由该类加载器加载类。
- 在这种模型下，每个类加载器都有自己的加载命名空间，由它加载的类存放在自己的命名空间中。
- 当一个类加载器需要加载一个类时，它会先搜索自己的命名空间，如果找到该类就直接返回；
- 如果找不到，则委托给父类加载器进行加载，这样一层层的委托会一直到引导类加载器（Bootstrap ClassLoader）
- 如果引导类加载器也找不到该类，则表示该类无法被加载。因此，不同类加载器加载的同一个类会被看作是两个不同的类。
- 它们的实例是不兼容的，无法进行相互转换
- 这就确保了不同类加载器加载的类的隔离性和独立性，使得不同的类加载器可以加载相同名称的类而不会相互干扰。

11-介绍JVM自动垃圾回收机制，是否可以完全解决内存泄露

- **题目分析和考点**
  - 面试官想知道你对垃圾回收和内存管理的理解程度，以及对内存泄漏问题的认识
  - 自动垃圾回收机制是一种内存管理技术，用于自动地释放不再使用的对象的内存空间
  - 通过标记垃圾对象并进行回收，自动垃圾回收机制可以减少程序员对内存管理的负担，提高开发效率。
- **难度【* *】**
- **答案**
  - 垃圾回收机制 （Garbage Collection， 简称GC)

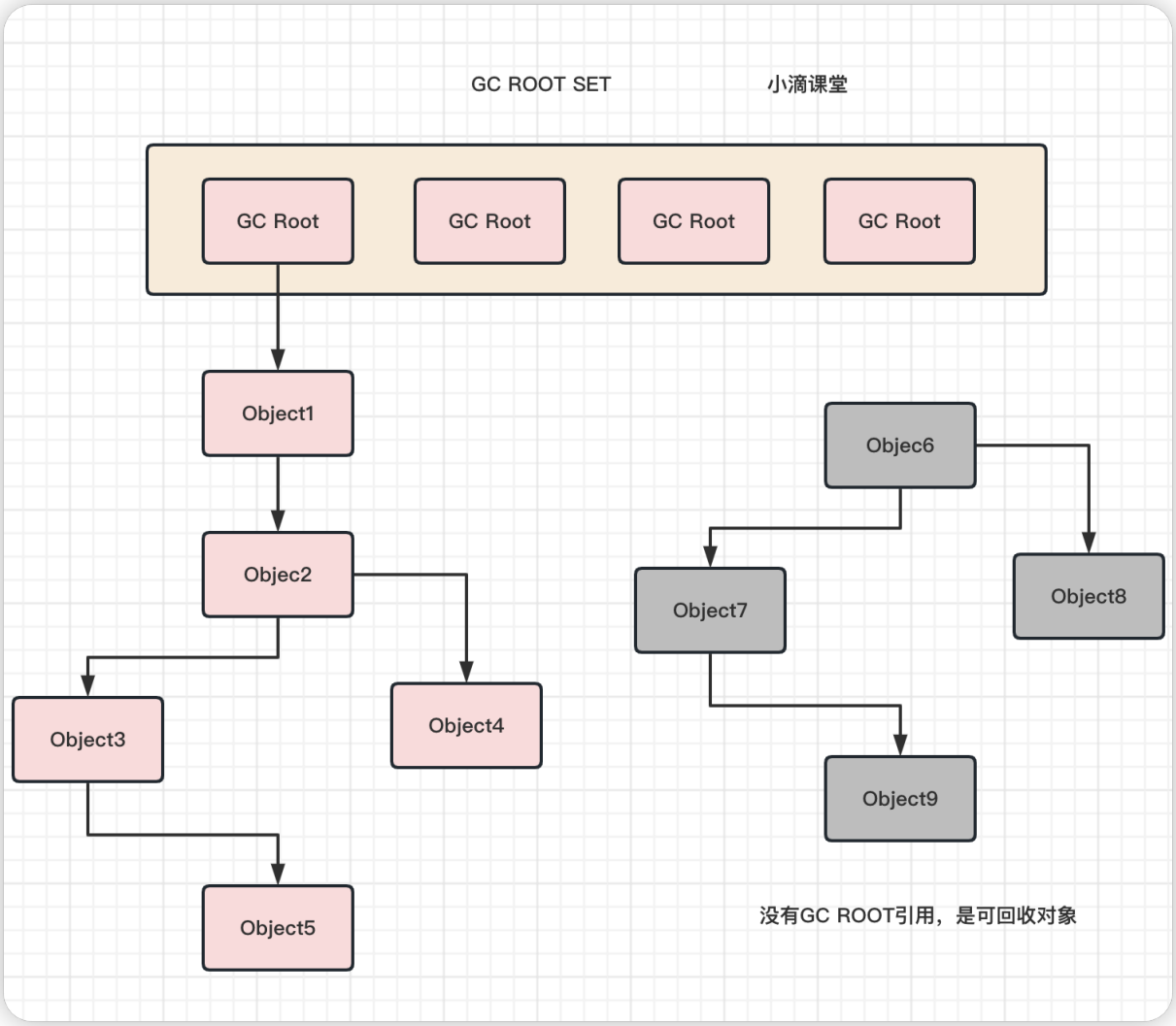
        - 指自动管理动态分配的内存空间的机制，自动回收不再使用的内存，以避免内存泄漏和内存溢出的问题
        - 最早是在1960年代提出的，程序员需要手动管理内存的分配和释放
        - 这往往会导致内存泄漏和内存溢出等问题，同时也增加了程序员的工作量，特别是C++/C语言开发的时候
        - Java语言是最早实现垃圾回收机制的语言之一，其他编程语言，如C#、Python和Ruby等，也都提供了垃圾回收机制
    - JVM自动垃圾回收机制
        - 指Java虚拟机在运行Java程序时，自动回收不再使用的对象所占用的内存空间的过程
        - Java程序中的对象，一旦不再被引用会被标记为垃圾对象，JVM会在适当的时候自动回收这些垃圾对象所占用的内存空间
        - 优点
            - 减少了程序员的工作量，不需要手动管理内存
            - 动态地管理内存，根据应用程序的需要进行分配和回收，提高了内存利用率
            - 避免内存泄漏和野指针等问题，增加程序的稳定性和可靠
        - 缺点
            - 垃圾回收会占用一定的系统资源，可能会影响程序的性能
            - 垃圾回收过程中会停止程序的执行，可能会导致程序出现卡顿等问题
            - **不一定能够完全解决内存泄漏等问题，需要在编写代码时注意内存管理和编码规范**

12-介绍下什么是GCRoot和可达性分析算法

- **题目分析和考点**
    - 面试官想知道你对垃圾回收机制和内存管理的具体实现以及相关算法的了解程度
    - 是否了解GCRoot是垃圾回收器判断对象是否存活的起始点
    - 通过标记垃圾对象并进行回收，自动垃圾回收机制可以减少程序员对内存管理的负担，提高开发效率。
    - 可达性分析算法是一种垃圾回收的算法，用于确定对象的存活状态
    - 这个算法从GCRoot对象开始，通过对象之间的引用关系形成引用链，然后遍历引用链
    - 标记所有可达的对象，将不可达的对象判定为垃圾并进行回收
- **难度【* * *】**
- **答案**
    - 什么是GC Root
        - 指一些被JVM认为是存活的对象，它们是垃圾回收算法的起点
        - **可以理解为由堆外指向堆内的引用**， 本身是没有存储位置，都是字节码加载运行过程中加入 JVM 中的一些普通引用
        - 通俗的例子可以是一个树形结构，树的根节点就是GC Roots

        - 是垃圾回收器的起点，如果一个节点没有任何子节点与根节点相连，那这个节点就被认为是不可达的，可以被回收器回收
        - 通俗的例子
            - 将GC Roots比喻成一座城市，城市中有很多建筑物，这些建筑物就是内存中的对象
            - GC Roots就像城市的卫生局、消防局等，它们直接或间接地与城市中的建筑物相连
            - 从这些机构出发，通过道路、桥梁等连接，最终能够到达所有的建筑物
            - 如果一个建筑物没有与这些机构相连，那么它就被认为是废弃的，可以被清理掉
        - JVM中的GC Roots对象包括以下几种
            - 虚拟机栈（栈帧中的本地变量表）中引用的对象。
            - 方法区中类静态属性引用的对象
                - JDK 1.7 开始静态变量的存储从方法区移动到堆中
                - 比如你定义了一个static 的集合对象，那里面添加的对象就是可以被GC Root可达的
            - 方法区中常量引用的对象
                - 字符串常量池从 JDK 1.7 开始由方法区移动到堆中
            - 本地方法栈中JNI（即一般说的Native方法）引用的对象。
            - 小技巧
                - 由于GC Roots采用栈方式存放变量和指针，如果一个指针它保存了堆内存里面的对象
                - 但是自己又不能存放在堆内存里面，那么它就是一个GC Roots

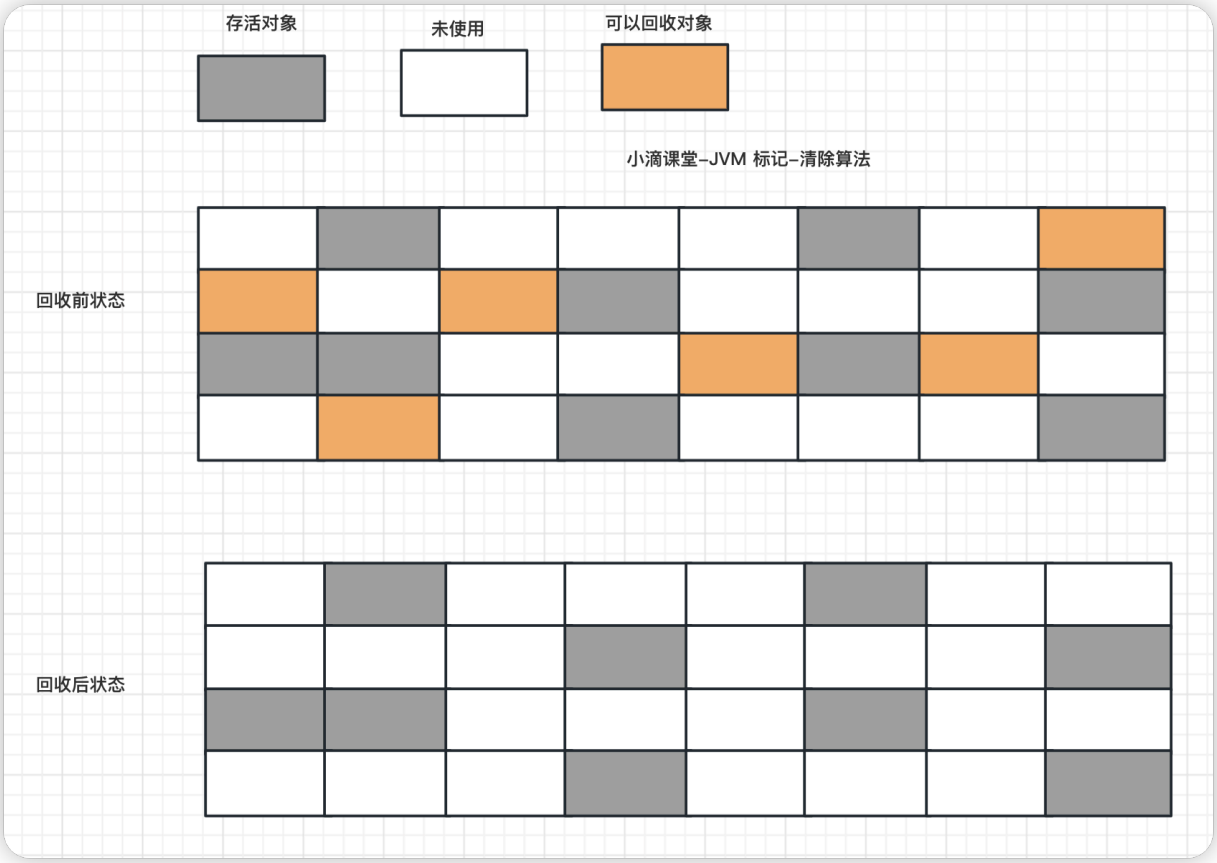
    - 什么是可达性分析算法
        - 基本思路是从一些“GC Roots”对象开始，通过搜索引用链的方式，找到所有可达对象
        - 如果一个对象没有任何引用链与GC Roots相连，那么它就被判定为不可用的，是可以被回收的垃圾对象



13-介绍下JVM的标记-清除算法原理和优缺点

- **题目分析和考点**
  - 面试官想知道你对垃圾回收算法的了解程度，以及对不同算法的优缺点的认识
  - 标记清除算法是一种经典的垃圾回收算法，它的原理是通过两个阶段来进行垃圾回收：标记阶段和清除阶段
- **难度【* * *】**
- **答案**
  - 垃圾回收算法之**标记-清除算法 Mark-Sweep**
    - 是一种常见的垃圾回收算法，它的基本思路是分为两个阶段：标记阶段和清除阶段
    - 标记阶段

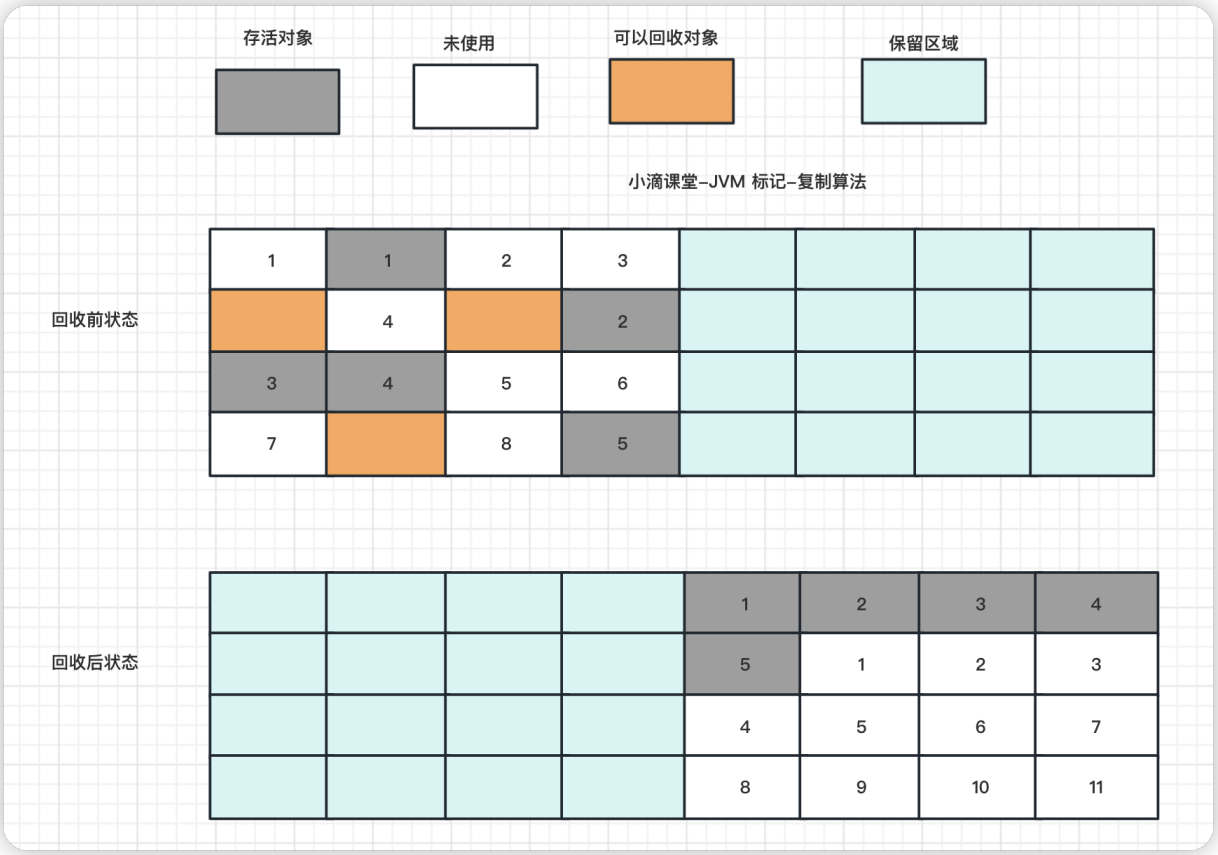
        - 垃圾回收器会从一些GC Roots对象开始，遍历整个对象图，标记所有被引用的对象
        - 被标记的对象会被打上标记，表示这些对象是“活”的对象，需要保留下来，未被标记的对象就是垃圾对象，可以被回收
    - 清除阶段
        - 垃圾回收器会对所有未被标记的对象进行回收



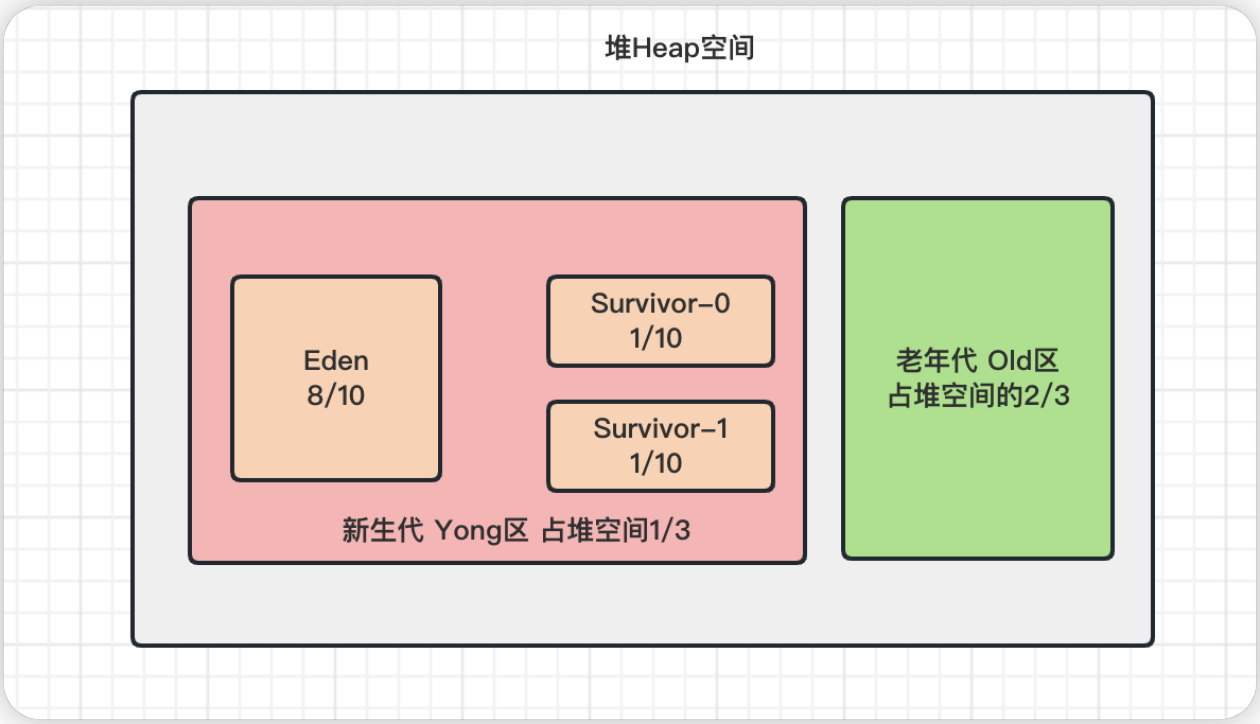
- 优点
    - 能够回收不连续的内存空间
- 缺点
    - 标记和清除两个步骤，都需要垃圾回收器遍历整个对象图，耗费时间较长
    - 会产生内存碎片，当频繁进行垃圾回收时，内存碎片会越来越多导致可用内存空间不足，从而影响程序的性能和稳定性
- 应用场景
    - 在实际应用中，标记清除法一般用于不需要频繁进行垃圾回收的场景，比如在Java堆中大对象的分配和回收
    - 后续的收集算法大多都是以标记-清除算法为基础，对其缺点进行改进

14-介绍下JVM的标记-复制算法原理和优缺点

- **题目分析和考点**
  - 面试官想知道你对垃圾回收算法的了解程度，以及对不同算法的优缺点的认识
  - 标记-复制算法是一种垃圾回收算法，它将内存空间划分为两个区域：一个用于存储对象，另一个用于分配新对象
  - 由于需要将存活对象复制到另一个区域，所以需要两倍于原有内存空间的大小
  - 这可能导致内存的浪费，并且对于大型对象或长时间存活的对象，复制的成本较高
- **难度【* * *】**
- **答案**
  - 什么是标记-复制算法
    - 是一种常见的垃圾回收算法，它的基本思路是将Java堆分为两个区域：一个活动区域和一个空闲区域
    - 在垃圾回收过程中，首先标记所有被引用的对象
    - 然后将所有被标记的对象复制到空闲区域中，最后交换两个区域的角色，完成垃圾回收
    - 标记复制算法的详细实现步骤
      - 将Java堆分为两个区域：一个活动区域和一个空闲区域，初始时，所有对象都分配在活动区域中
      - 从GC Roots对象开始，遍历整个对象图，标记所有被引用的对象
      - 对所有被标记存活的对象进行遍历，将它们复制到空闲区域中，并更新所有指向它们的引用，使它们指向新的地址
      - 对所有未被标记的对象进行回收，将它们所占用的内存空间释放
      - 交换活动区域和空闲区域的角色，空闲区域变为新的活动区域，原来的活动区域变为空闲区域
      - 当空闲区域的内存空间不足时，进行一次垃圾回收，重复以上步骤

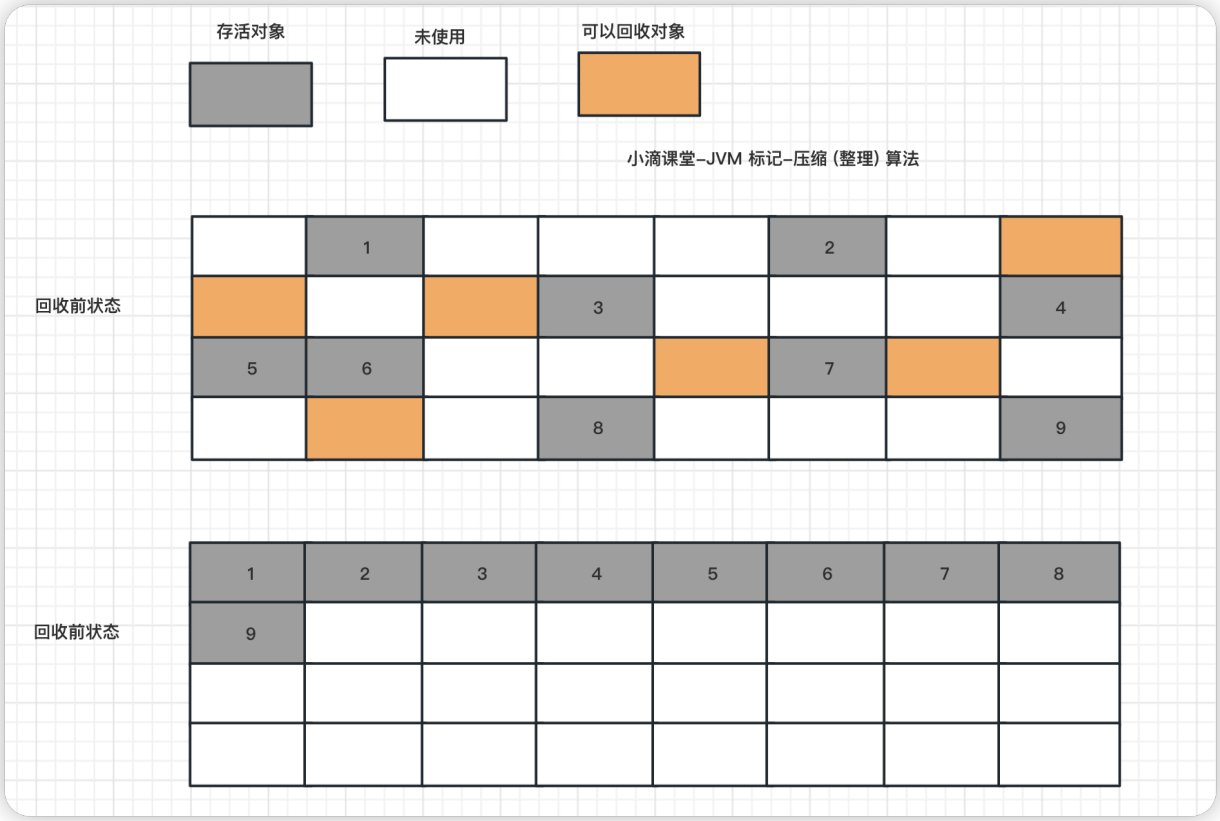


- 优点
  - 如果内存中的垃圾对象较多，需要复制的对象就较少，则效率高
  - 清理后，内存碎片少
- 缺点
  - 标记复制算法的效率较高，但是预留一半的内存区域用来存放存活的对象，占用额外的内存空间
  - 如果出现存活对象数量比较多的时候，需要复制较多的对象 效率低
  - 假如是在老年代区域，99%的对象都是存活的，则性能底，所以老年代不适合这个算法
- 应用场景
  - 标记复制算法一般用于**新生代**的垃圾回收，因此需要对新生代的对象进行分代管理
  - 虚拟机多数采用这个算法，对新生代进行内存管理，因为多数这个新生代区域的存活对象数量少
  - 国外有公司统计过多数业务，98%撑不过一次GC； 所以不用1:1比例分配新生代的空间
  - 原因
    - GC时, 将Eden和Survivor中存活对象一次性复制到另外一块Survivor空间上， 然后清理掉Eden和已用过的那块Survivor空间
    - 每次新生代中可用内存空间为整个新生代容量的90% (Eden的80% + Survivor的 10%) ,
    - 只有一个Survivor空间， 即10%的新生代是会被浪费而已



15-介绍下JVM的标记-整理算法原理和优缺点

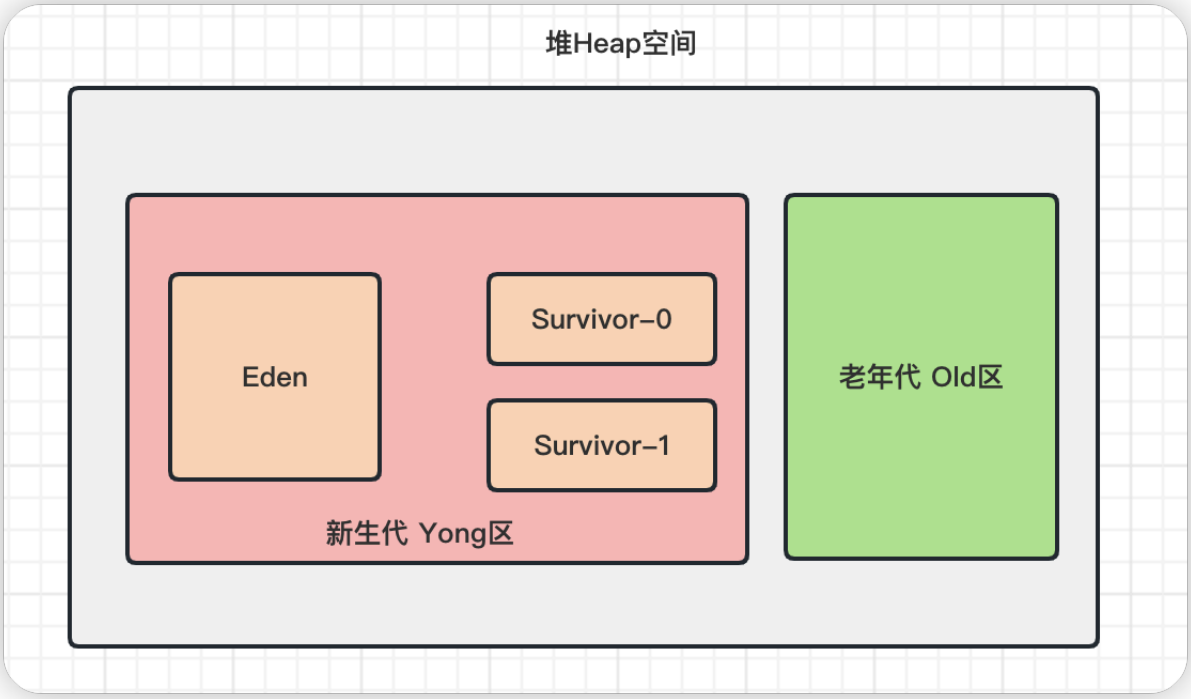
- **题目分析和考点**
  - 面试官想知道你对垃圾回收算法的了解程度，以及对不同算法的优缺点的认识
  - 首先标记所有存活的对象，然后将对象在内存中进行整理，将存活对象向一端移动，然后清理掉末尾的垃圾对象
  - 通过这种方式来进行垃圾回收，使得内存空间得到整理和压缩，减少内存碎片
- **难度【* * *】**
- **答案**
  - 什么是标记-整理（压缩）算法 Mark-Compact
    - 从根节点开始对所有可达对象做一次标记，但之后并不简单地清理未标记的对象，而是将所有的存活对象压缩到内存的一端
    - 然后清理边界外的垃圾，避免了碎片的产生，也不需要两块相同的内存空间，因此性价比比较高



- 优点
  - 解决了标记清除算法的碎片化的问题
  - 对比标记-复制算法，不用浪费额外的空间
  - 对比 标记-清除算法与标记-整理算法，前者是一种非移动式的回收算法， 而后者是移动式的回收，且没有了碎片化内存
- 缺点
  - 效率相比于标记复制算法低一些
  - 在整理存活对象时，因对象位置的变动，需要该调整虚拟机栈中的引用地址
- 应用场景
  - 标记-压缩算法是一种老年代的回收算法，它在标记-清除算法的基础上做了一些优化。
- 总结
  - 标记-复制算法 适合 在存活对象少、垃圾对象多的场景
    - 即新生代空间，朝生夕灭的场景
  - 标记-整理算法 适合 在存活对象多、垃圾对象少的场景
    - 即老年代空间，都是历经多次打击，依旧存活的老油条
    - 如果使用【标记清除】算法则会有碎片化空间、效率低下等缺点
    - 如果使用【标记复制】算法需要复制特别多对象，效率低下等缺点

16-说下JVM垃圾回收的分代收集算法思想

- **题目分析和考点**
  - 面试官想知道你JVM垃圾回收的分代收集算法的了解程度，以及对不同算法的优缺点的认识
  - 针对不同生命周期的对象采用不同的垃圾回收策略，以达到更好的垃圾回收效果
  - **年轻代多数对象存活时间短，高频进行回收；老年代多数对象存活时间久，进行低频回收**
- **难度【* * *】**
- **答案**
  - 什么是分代收集算法
    - 针对不同生命周期的对象采用不同的垃圾回收策略，以达到更好的垃圾回收效果
    - **年轻代多数对象存活时间短，高频进行回收；老年代多数对象存活时间久，进行低频回收**
    - 分代算法是根据回收对象的特点进行选择，年轻代适合标记-复制算法，老年代适合标记清除或标记压缩算法
    - 通过将内存划分为不同的代，可以使得Minor GC的频率更高，更早地回收垃圾对象，减少Full GC的发生频率，提高整体性能



  - JVM将内存分为年轻代和老年代，其中年轻代又分为Eden区和两个Survivor区
  - 年轻代
    - 用于存放新生的对象，其中Eden区是新对象的分配区域
    - 当Eden区满时，会触发Minor GC，将存活的对象移动到Survivor区，同时清空Eden区
    - Survivor区则用于存放经过一次Minor GC后存活的对象
  - 老年代

        - 用于存放长时间存活的对象，当年轻代中的对象经过多次Minor GC后仍然存活，就会被移动到老年代
        - 当老年代满时，会触发Full GC

17-介绍下JVM垃圾回收里面的Partial GC/Full GC/STW概念

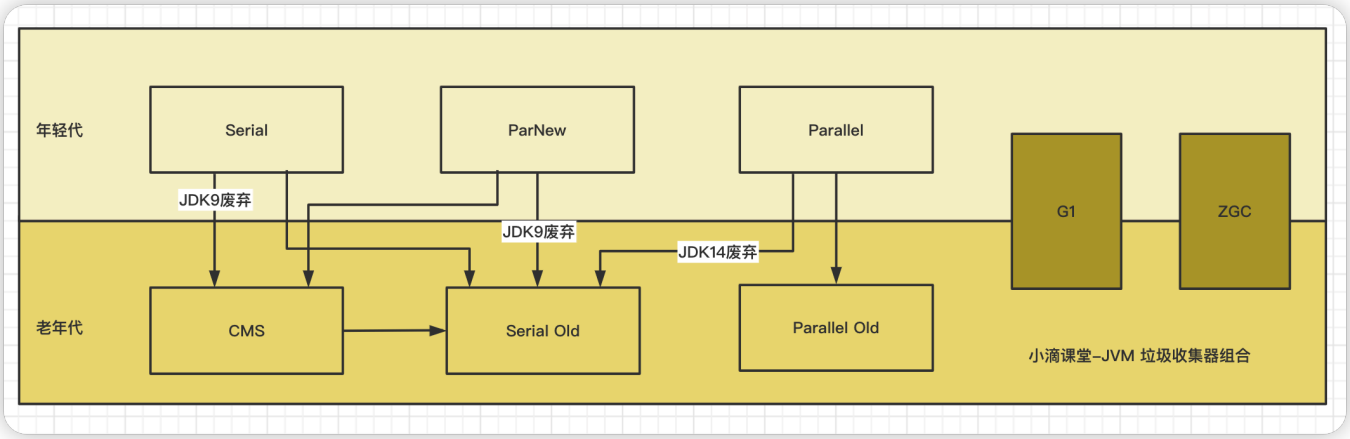
- **题目分析和考点**
    - 面试官想知道你JVM垃圾回收的里面多个概念是否理解，对垃圾回收过程中不同类型和行为的理解程度
    - 了解Partial GC、Full GC和STW的概念，可以帮助开发java应用理解垃圾回收的过程和对应用程序性能的影响
- **难度【* * *】**
- **答案**
    - 部分收集(Partial GC)
        - 新生代收集
            - 对象从Young 区域消失的过程称为”**minor GC / Young GC** “
            - **Eden 的清理，S0\S1的清理都由于MinorGC**
            - YoungGen区内存不足，会触发minorGC
        - 老年代收集
            - 对象从老年代中消失的过程，清理整合OldGen的内存空间，称为**”Major GC/Old GC“**
            - 有些垃圾收集器 针对老年代单独回收，所以比较少用
    - 整堆收集(Full GC)
        - 是清理整个堆空间，包括年轻代和老年代
        - 理解为Major GC+Minor GC组合后进行的一整个过程，是清理JVM整个堆空间
    - 注意
        - 不用去关心是叫 Major GC 还是 Full GC，应该关注当前的 GC 是否停止了所有应用程序的线程
        - 许多Major GC 是由 Minor GC 触发的, 出现 Major GC通常出现至少一次的 Minor GC
        - MajorGC 的速度一般比 Minor GC 慢 10倍以上
    - FullGC触发场景
        - 手工调用System.gc( )，建议执行Full GC，不一定会执行，可通过-XX:+ DisableExplicitGC 参数来禁止调用System.gc()
        - 老年代空间不足 , 通过Minor GC后进入老年代的平均大小大于老年代的可用内存
    - STW（Stop The World）

        - 垃圾回收发生过程中，用户线程在运行至安全点（safe point）后，就自行挂起进入暂停状态，对外的表现就是卡顿
        - **所以应尽量减少Full GC的次数，是Minor GC还是Major GC都会STW，区别只在于STW的时间长短**

18-什么是垃圾收集器和常用的垃圾收集器组合

- **题目分析和考点**
    - 面试官想知道你对垃圾收集器的概念和了解常见的垃圾收集器组合的能力
    - 常用的垃圾收集器组合是指在Java虚拟机中常用的垃圾收集器的组合方式。
    - 在不同的JVM版本和配置下，可能会有不同的组合
    - 常见的组合包括Serial收集器、Parallel收集器、CMS收集器、G1收集器等
- **难度【* * * *】**
- **答案**
    - 什么是垃圾收集器
        - 垃圾回收算法是内存回收的方法论，垃圾收集器则是内存回收的具体实现
        - 目前Java规范中并没有对垃圾收集器的实现有任何规范
        - 不同的厂商、不同的版本的虚拟机提供的垃圾收集器是不同的，主要讨论的是HotSpot虚拟机
        - 是不是有最厉害的收集器?
            - **不存在最厉害的垃圾收集器，只有在对应场景中最合适的垃圾收集器**
        - 为什么要有很多收集器?
            - 因为Java的使用场景很多，移动端，服务器等，然后内存里面对象存活时间不一样
            - 需要针对不同的场景，提供不同的垃圾收集器，提高垃圾收集的性能
    - 垃圾收集器分类
        - 新生代收集器
            - Serial 串行垃圾收集器
            - ParNew 年轻代的并行垃圾回收器
            - Parallel 并行垃圾收集器
        - 老年代收集器
            - Serial Old 串行老年代垃圾器
            - Parallel Old 老年代的并行垃圾回收器
            - CMS （ConcMarkSweep）并发标记清除
        - 整堆收集器：G1、ZGC
    - 图解分配垃圾收集器的组合
        - 两个垃圾收集器之间如果存在连线，则说明它们可以搭配使用

- 图中：Serial old作为CMS出现"Concurrent Mode Failure"失败的后备预案
- 注意
  - JDK8中默认使用: Parallel Scavenge GC + ParallelOld GC
  - JDK14 弃用了: Parallel Scavenge GC + Parallel OldGC
  - JDK9默认是用G1为垃圾收集器
  - JDK14 移除了 CMS GC

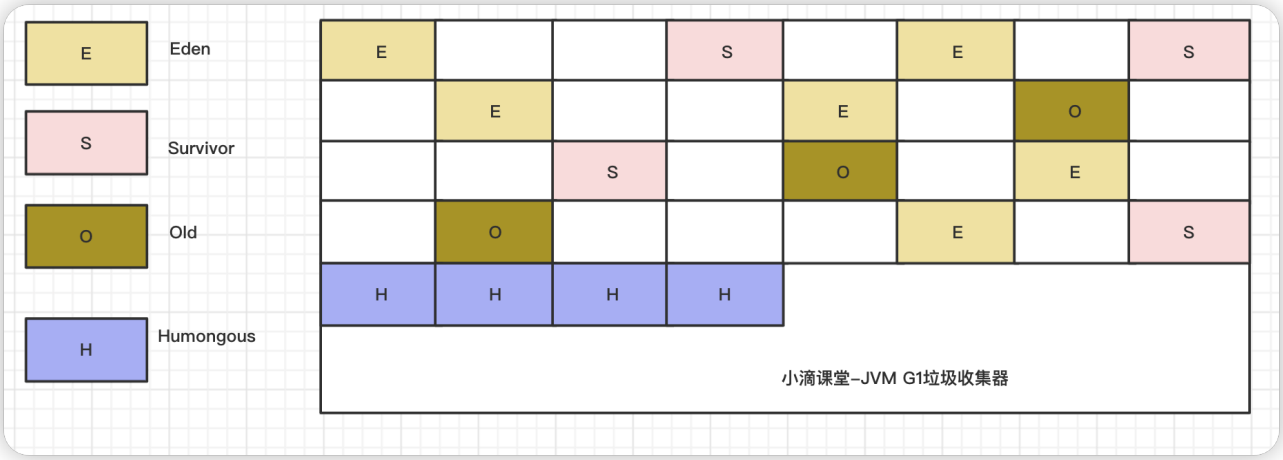


19-介绍下新一代垃圾收集器G1收集器原理

- **题目分析和考点**
  - 面试官想知道你对新一代垃圾收集器的了解程度，以及对垃圾回收算法和性能优化的理解
  - G1（Garbage-First）收集器是一种新一代的并发垃圾收集器，改善长时间停顿和内存碎片问题。
  - G1将整个堆划分为多个大小相等的区域（Region），每个区域可以是Eden、Survivor或Old Generation
  - G1通过增量方式进行垃圾回收，并根据实际需求动态调整每个区域的大小
- **难度【* * * *】**
- **答案**
  - G1垃圾收集器
    - Garbage First 垃圾收集器是JDK7版本之后引入的一种垃圾回收器，jdk9中将G1变成默认的垃圾收集器
    - 可以在不同的内存区域中分配垃圾回收的工作，提高了垃圾回收效率
    - 查看默认垃圾收集器
      - `-XX:+PrintCommandLineFlags` 查看命令行相关参数（包含使用的垃圾收集器）
      - JDK11

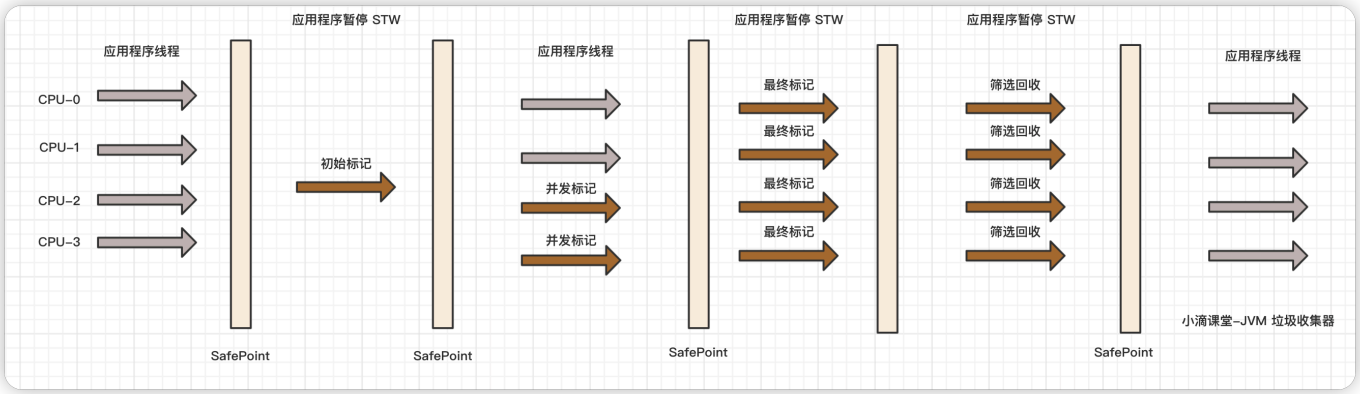
```
-XX:G1ConcRefinementThreads=9 -XX:GCDrainStackTargetSize=64 -
XX:InitialHeapSize=536870912 -XX:MaxHeapSize=8589934592 -
XX:+PrintCommandLineFlags -XX:ReservedCodeCacheSize=251658240 -
XX:+SegmentedCodeCache -XX:+UseCompressedClassPointers -
XX:+UseCompressedOops -XX:+UseG1GC
```

- 核心
  - 保留了分代思想，把内存划分为多个独立的区域Region，区域中包含逻辑上的年轻代、老年代区域
  - 取消了年轻代、老年代的物理划分，不用单独对每个年代空间进行设置
  - **Region的区域类型是动态变化的，可能之前是年轻代，经过了垃圾回收之后就变成了老年代，实现更加精细化的垃圾回收**
  - **整体采用标记整理算法, 局部是采用复制算法,不会产生内存碎片**
  - 把整个Java堆划分成约2048个独立Region块，每个Region块大小根据堆空间的大小而定，为2的N次幂，1MB～32MB
    - 每个Region的大小可通过参数 `-XX:G1HeapRegionSize` 配置
  - **新增加一种叫Humongous内存区域，用于存储大对象**
  - **如果超过1.5个region，就是巨型对象，就放到H区，默认直接会被分配在老年代，一般被视为老年代.图中的H块**
  - **如果一个H区装不下一个巨型对象，G1会寻找连续的H区来存储，为了能找到连续的H区，有时需要启动Full GC**



- G1的MixGC垃圾收集分为下面几个步骤
  - 初始标记（STW）
    - 记录下GC Roots能直接引用的对象，并标记所有存活的对象，会执行一次年轻代GC，需要暂停所有线程，速度很快
  - 并发标记
    - 与应用线程一起工作，进行可达性分析
    - g1收集器会对堆内存进行并发标记，找出所有存活的对象，并记录它们所在的Region
  - 最终标记（STW）

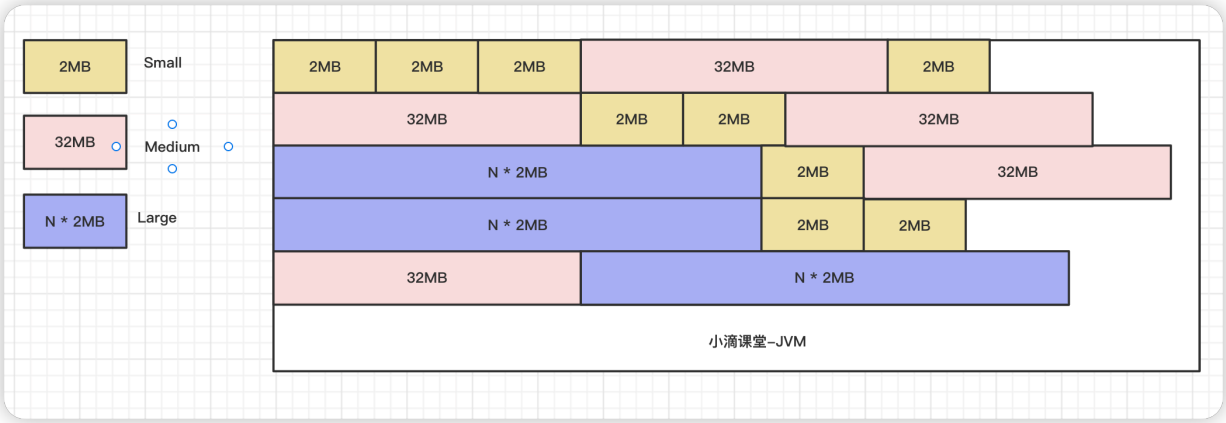
        - 修正并发标记期间, 部分因程序运行导致发生变化的那一部分对象，**根据算法修复一些引用的状态**
    - 筛选回收（STW）
        - 对各个Region的回收价值和成本进行排序，根据用户所期望的GC停顿STW时间，即`-XX:MaxGCPauseMillis`制定计划
        - 成本排序案例
            - 现在有Region1、Region2和Region3三个区域
                - Region1预计可以回收1.5MB内存，预计耗时2MS，投产比ROI=1.5/2
                - Region2预计可以回收1MB内存，预计耗时1MS，投产比ROI=1/1
                - Region3预计可以回收0.5MB内存，预计耗时1MS，投产比ROI=0.5/1
            - 那Region1、Region2和Region3各自的回收价值与成本比值分别是：0.75、1和0.5，
            - 比值越高说明同样的付出收益越高，如果此时只能回收一个Region的内存空间，G1就会选择Region2进行回收
            - 保证了G1收集器在有限的时间内尽可能地提高收集效率



20-介绍下新一代垃圾收集器ZGC原理和特点

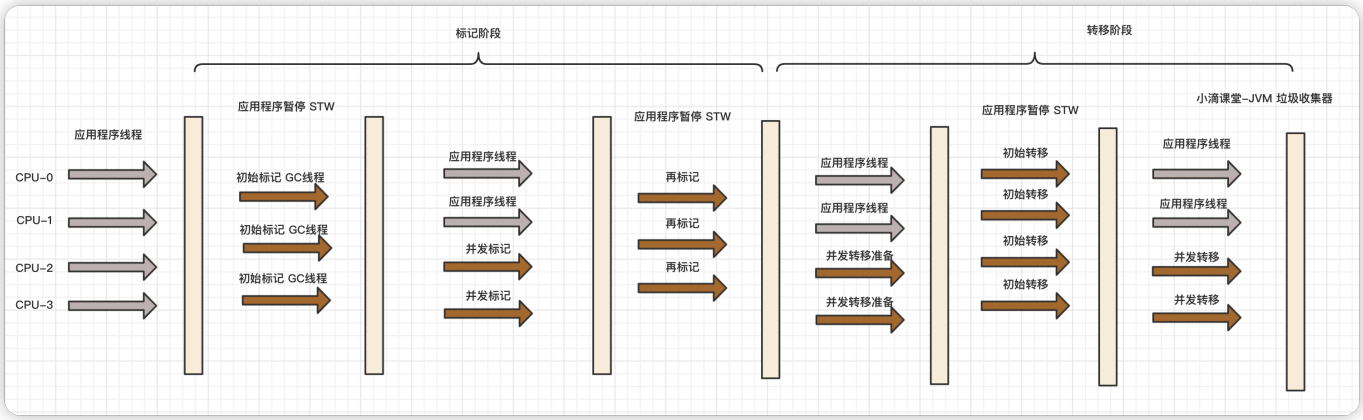
- **题目分析和考点**
    - 面试官想知道你对新一代垃圾收集器的了解程度，ZGC是一种面向低延迟和大堆内存的垃圾收集器。
    - 使用了 并发中央垃圾收集、标记压缩和着色指针策略来减少延迟，通过将垃圾收集的过程与应用程序并发执行来减少停顿时间
    - ZGC不按照传统的分代模型划分为Young Generation和Old Generation，而是将堆划分为连续的内存区域（Region）。
    - 这种设计可避免了分区带来的内存碎片问题并减少了收集器的内部工作
- **难度【* * * *】**
- **答案**
    - 什么是ZGC

- Z Garbage Collector 是Oracle公司开发一种可伸缩、低停顿时间的垃圾收集器，标记-复制算法（进行了改进）
- 垃圾回收过程几乎全部是并发，实际STW停顿时间极短，停顿时间控制10ms内，主要采用的**染色指针和读屏障**技术
- **在 JDK 11 中是实验性的特性引入，在 JDK 15 中 ZGC 可以正式投入生产使用，使用 –XX:+UseZGC 启用**
- ZGC 的堆内存也是基于 Region 来分布，和G1类似，不区分新生代老年代的，Region 支持动态地创建和销毁，大小不是固定
- 三种类型的 Region
  - 小型页面 Small Region：容量固定2MB，主要用于放置小于 256 KB 的小对象。
  - 中型页面Medium Region：容量固定32MB，主要用于放置大于等于 256 KB 小于 4 MB 的对象。
  - 大型页面Large Region
    - 容量不固定 为N * 2MB， Region 是可以动态变化的，但必须是 2MB 的整数倍，最小支持 4 MB。



- 特点
  - 低停顿时间
    - ZGC最大的特点是在不增加延迟的情况下，能够处理非常大的内存数据
    - 可以将停顿时间限制在10ms以内，对于需要快速响应的应用程序来说是非常重要
  - 可伸缩性
    - 可以处理非常大的内存数据，适应不同规模的应用程序，从小型应用程序到大型企业级应用程序
  - 不需要分代
    - 不需要将内存分为新生代和老年代，不需要复杂的内存回收算法
  - 并发处理
    - 采用了并发处理的方式来进行垃圾回收可以在应用程序运行的同时进行垃圾回收
- 工作流程
  - 初始标记（STW）：找 GC Roots 直接引用的对象，处理时间和GC Roots的数量成正比，停顿时间不随着堆的大小而增加。

- 并发标记（没有STW）：扫描剩余的所有对象，处理时间比较长，业务线程与GC线程同时运行，但这个阶段会有漏标问题
- 再标记（STW）：通过算法解决漏标对象，和G1中的解决漏标的算法类似
- 并发转移准备（没有STW）：分析最有回收价值GC分页，即ROI计算
- 初始转移（STW）：转移初始标记的存活对象和做对象重定位，时间和GC Roots的数量成正比，时间不随堆的大小而增加。
- 并发转移（没有STW）：对转移并发标记的存活对象做转移



21-你知道哪些JVM常用的命令，说下使用场景

- **题目分析和考点**
  - 面试官想知道你对JVM调优和性能监控的了解程度，以及对命令行工具的熟悉程度
  - 了解常用的JVM命令及其使用场景可以显示自己对JVM调优和性能监控的了解程度
  - 在面试中，展示对这些常用命令的理解，对面试官表达自己在JVM性能调优和故障排查方面表现出色
- **难度【* *】**
- **答案**
  - 命令一：jps
    - 全称 `java process Status Tool`，Java版的ps命令，查看java进程及其相关的信息的pid则可以用这个命令，和linux的ps类似
    - 最常用示例： jps -l 输出jar包路径，类的全名

```
xdclass@xdclassdeMacBook-Pro reflect1 % jps -l
56017 jdk.jcmd/sun.tools.jps.Jps
49073
55990 org.jetbrains.jps.cmdline.Launcher
55994 jvm.JVMTest
33132 /Users/xdclass/Desktop/coding/apache-jmeter-5.5/bin/ApacheJMeter.jar
```

- 命令二 : jinfo
  - 全称 Configuration Info for Java ， 可以用来查看jvm参数和动态修改部分jvm参数的命令
  - 启动java程序，不会指定所有的Java虚拟机参数，如果开发人员想知道某一个具体的Java虚拟机参数的默认值则可以使用
  - 还可以在运行时修改部分参数，且立即生效，但注意并非所有参数都支持动态修改，被标记manageable的才可以动态修改
  - 最常用示例： jinfo -flags pid 查看曾经赋过值的参数值

```
xdclass@xdclassdeMacBook-Pro reflect1 % jinfo -flags 55994
VM Flags:
-XX:CICompilerCount=4 -XX:ConcGCThreads=2 -XX:G1ConcRefinementThreads=9 -XX:G1HeapRegionSize=1048576 -XX:GCDrainStackTargetSize=64 -XX:InitialHeapSize=553648128 -XX:MarkStackSize=419430
4 -XX:MaxGCPauseMillis=100 -XX:MaxHeapSize=553648128 -XX:MaxNewSize=331350016 -XX:MinHeapDeltaBytes=1048576 -XX:NonNMethodCodeHeapSize=5836492 -XX:NonProfiledCodeHeapSize=122910874 -XX:
+PrintCommandLineFlags -XX:ProfiledCodeHeapSize=122910874 -XX:ReservedCodeCacheSize=251658240 -XX:+SegmentedCodeCache -XX:+UseCompressedClassPointers -XX:+UseCompressedOops -XX:+UseG1GC
```

- 命令三：**jstat**
  - Java Virtual Machine statistics monitoring tool, 对Java应用程序的资源进行实时监控，包括堆和垃圾回收状况的监控
  - 最常用示例：jstat -gc 56353　查看JVM中堆的垃圾收集情况的统计，输出实际的值

```
xdclass@xdclassdeMacBook-Pro reflect1 % jstat -gc 56353
 S0C    S1C    S0U    S1U      EC       EU        OC         OU       MC     MU    CCSC   CCSU   YGC     YGCT    FGC    FGCT    CGC    CGCT     GCT
0.0    0.0    0.0    0.0   28672.0   2048.0   512000.0     0.0      0.0    0.0    0.0    0.0       0    0.000   0      0.000   0      0.000    0.000
```

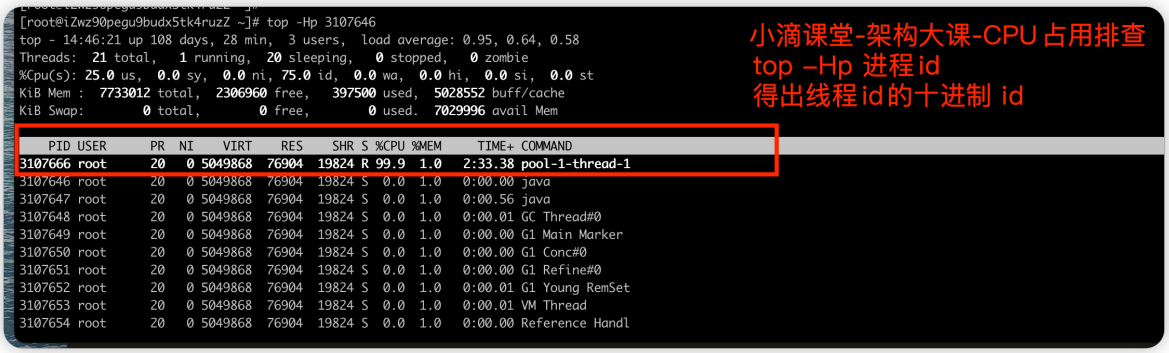
22-生产环境的JVM导致CPU占用过高，说下排查思路

- **题目分析和考点**
  - 面试官想知道你对JVM调优和性能监控的了解程度，以及对命令行工具的熟悉程度
  - 主要是了解你在故障排查和性能优化方面的经验和思考方式，首先需要确认CPU占用高的问题是否由JVM引起
  - 可以通过查看系统监控工具，如top（Linux）、Task Manager（Windows）等来确认CPU占用情况。
  - 展示应聘者在故障排查和性能优化方面的经验和能力
- **难度【* * * *】**
- **答案**
  - 命令 jstack

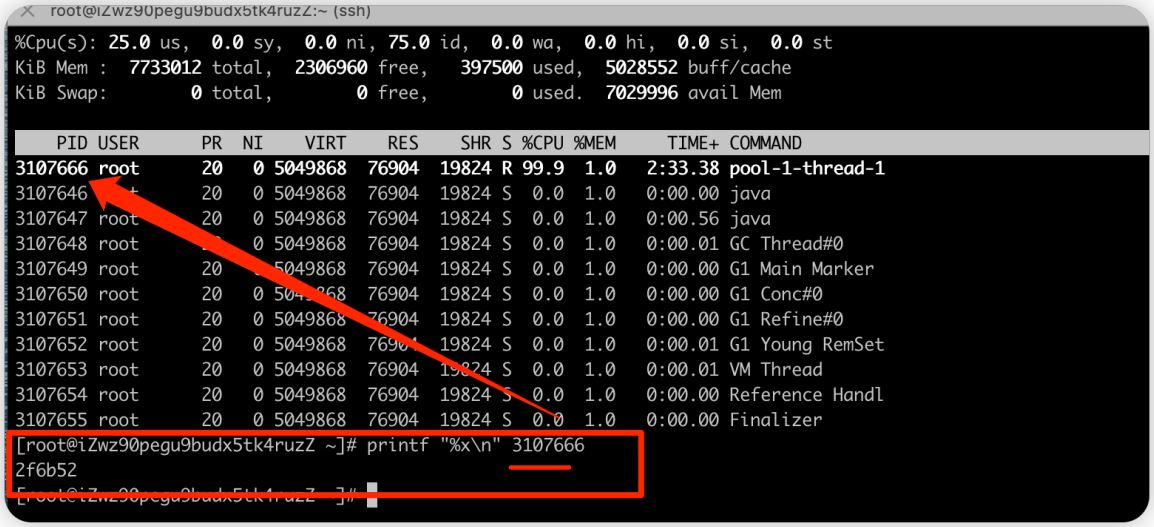
- Java堆栈跟踪工具, 可以打印出Java应用程序中所有线程的堆栈信息，包括线程状态、调用栈信息、锁信息等
- 用于诊断线程死锁、死循环、内存泄漏等问题

- CPU占用高案例分析思路，宏观到细节，结合Linux命令
  - 分析思路
    - 确认问题：是否是 CPU 过高导致的应用程序性能问题

    - 确认进程 ID，使用top命令查找该【进程】下CPU使用最高的【线程】 `top -Hp 进程id`



    - 分析线程堆栈信息：使用 jstack 命令查看 Java 应用程序中所有线程的堆栈信息。
      - 把十进制的线程id转为16进制 `printf "%x\n" 线程id`



- 定位问题线程堆栈信息，一般会生成快照到文本文件里面进行分析 `jstack -l [PID] >/tmp/log.txt`

```
[root@iZwz90pegu9budx5tk4ruzZ ~]# top -Hp 3107646
top - 14:46:21 up 108 days, 28 min,  3 users,  load average: 0.95, 0.64, 0.58
Threads:  21 total,   1 running,  20 sleeping,   0 stopped,   0 zombie
%Cpu(s): 25.0 us,  0.0 sy,  0.0 ni, 75.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  7733012 total,  2306960 free,   397500 used,  5028552 buff/cache
KiB Swap:        0 total,        0 free,        0 used.  7029996 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S %CPU %MEM     TIME+ COMMAND
3107666 root      20   0 5049868  76904  19824 R 99.9  1.0   2:33.38 pool-1-thread-1
3107646 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 java
3107647 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.56 java
3107648 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.01 GC Thread#0
3107649 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 G1 Main Marker
3107650 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 G1 Conc#0
3107651 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 G1 Refine#0
3107652 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.01 G1 Young RemSet
3107653 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.01 VM Thread
3107654 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 Reference Handl
3107655 root      20   0 5049868  76904  19824 S  0.0  1.0   0:00.00 Finalizer
[root@iZwz90pegu9budx5tk4ruzZ ~]# printf "%x\n" 3107666
2f6b52
[root@iZwz90pegu9budx5tk4ruzZ ~]# jstack -l 3107646 > /tmp/log.txt
[root@iZwz90pegu9budx5tk4ruzZ ~]#
```

进程id获取堆栈

- 分析解决问题，根据线程id的十六进制查找所属代码的位置，采取相应的措施解决问题
  - 文本搜索，cat、more、vim方式都行

```
"Sweeper thread" #8 daemon prio=9 os_prio=0 cpu=5.54ms elapsed=506.73s tid=0x00007f1c4c166000 nid=0x2f6b4b runnable  [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

   Locked ownable synchronizers:
        - None

"Common-Cleaner" #9 daemon prio=8 os_prio=0 cpu=0.80ms elapsed=506.68s tid=0x00007f1c4c211000 nid=0x2f6b4c in Object.wait()  [0x00007f1c24a12000]
   java.lang.Thread.State: TIMED_WAITING (on object monitor)
        at java.lang.Object.wait(java.base@11.0.8/Native Method)
        - waiting on <0x000000008a040e80> (a java.lang.ref.ReferenceQueue$Lock)
        at java.lang.ref.ReferenceQueue.remove(java.base@11.0.8/ReferenceQueue.java:155)
        - waiting to re-lock in wait() <0x000000008a040e80> (a java.lang.ref.ReferenceQueue$Lock)
        at jdk.internal.ref.CleanerImpl.run(java.base@11.0.8/CleanerImpl.java:148)
        at java.lang.Thread.run(java.base@11.0.8/Thread.java:834)
        at jdk.internal.misc.InnocuousThread.run(java.base@11.0.8/InnocuousThread.java:134)

   Locked ownable synchronizers:
        - None

"Service Thread" #10 daemon prio=9 os_prio=0 cpu=0.10ms elapsed=506.66s tid=0x00007f1c4c264000 nid=0x2f6b4e runnable  [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

   Locked ownable synchronizers:
        - None

"pool-1-thread-1" #11 prio=5 os_prio=0 cpu=506076.22ms elapsed=506.15s tid=0x00007f1c4c589000 nid=0x2f6b52 runnable  [0x00007f1c053b9000]
   java.lang.Thread.State: RUNNABLE
        at CpuTest$Task.run(CpuTest.java:24)
        - locked <0x0000000090d10dd8> (a java.lang.Object)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(java.base@11.0.8/ThreadPoolExecutor.java:1128)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(java.base@11.0.8/ThreadPoolExecutor.java:628)
        at java.lang.Thread.run(java.base@11.0.8/Thread.java:834)

   Locked ownable synchronizers:
        - <0x0000000090d10ec8> (a java.util.concurrent.ThreadPoolExecutor$Worker)

"pool-1-thread-2" #12 prio=5 os_prio=0 cpu=21.17ms elapsed=506.15s tid=0x00007f1c4c58a800 nid=0x2f6b53 waiting for monitor entry  [0x00007f1c052b9000]
   java.lang.Thread.State: BLOCKED (on object monitor)
        at CpuTest$Task.run(CpuTest.java:22)
        - waiting to lock <0x0000000090d10dd8> (a java.lang.Object)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(java.base@11.0.8/ThreadPoolExecutor.java:1128)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(java.base@11.0.8/ThreadPoolExecutor.java:628)
        at java.lang.Thread.run(java.base@11.0.8/Thread.java:834)

   Locked ownable synchronizers:
        - <0x0000000090d11108> (a java.util.concurrent.ThreadPoolExecutor$Worker)

"DestroyJavaVM" #13 prio=5 os_prio=0 cpu=572.67ms elapsed=506.15s tid=0x00007f1c4c022800 nid=0x2f6b3f waiting on condition  [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

   Locked ownable synchronizers:
        - None

"Attach Listener" #14 daemon prio=9 os_prio=0 cpu=0.24ms elapsed=0.09s tid=0x00007f1c00001000 nid=0x2f6d10 waiting on condition  [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

   Locked ownable synchronizers:
(2f6b52
```

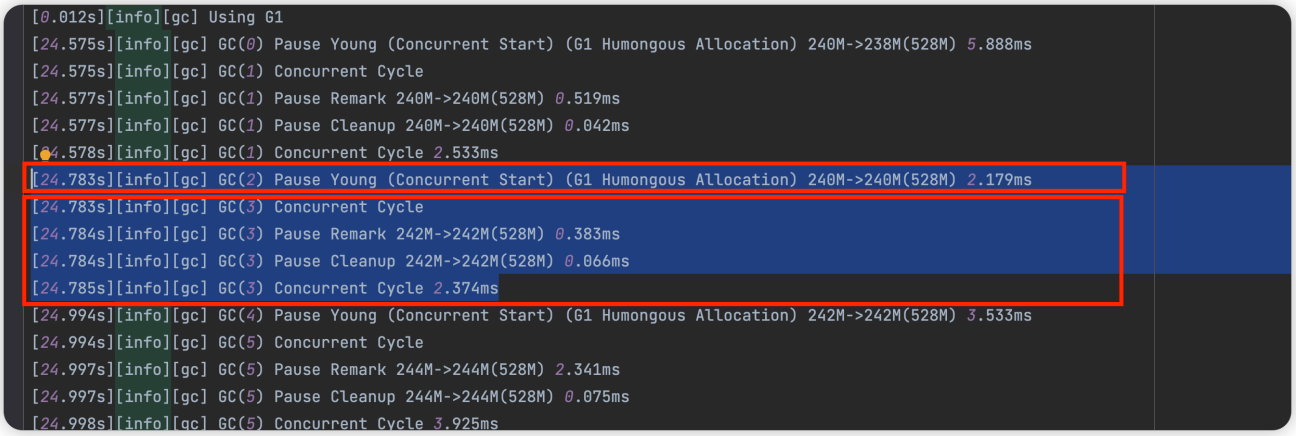
线程id的十六进制

代码所在类和位置

小滴课堂-架构大课-生产环境JVM问题诊断实战

23-解读下面新版JVM垃圾GC日志参数

- **题目分析和考点**
  - 面试官想知道你对JVM日志参数的熟悉程度，以及对垃圾回收算法和性能优化的理解
  - 明确GC日志参数的含义和使用方法可以显示你对JVM调优和性能监控的了解程度
  - 在面试中，展示对GC日志参数的解读和应用，可以使自己在JVM性能调优和故障排查方面表现出色
  - 日志内容



- **难度【****】**
- **答案**
  - 日志内容

```
[24.575s][info][gc] GC(0) Pause Young (Concurrent Start) (G1 Humongous Allocation) 240M->238M(528M) 5.888ms


字段拆解
  [24.575s]：GC发生的时间戳，表示程序运行的时间
  [info]：日志级别，表示这是一条信息级别的日志
  [gc]：日志标签，表示这是一条GC相关的日志
  GC(0)：GC的编号，表示这是第一次GC

  Pause Young (Concurrent Start) (G1 Humongous Allocation)：
    GC的类型，表示这是一次Young GC，同时也是一次 Humongous Allocation的GC，其中Concurrent Start表示并发启动的GC。

  240M->238M(528M)：
    GC前后堆内存的使用情况，其中240M表示GC前的已使用内存，238M表示GC后的已使用内存，528M表示堆内存的总大小

  5.888ms：GC的耗时，表示这次GC的执行时间

总结
  这条GC日志记录了程序运行了24.575秒时发生的一次Young GC，回收了2M的内存空间，耗时5.888毫秒
```

  - 日志内容

```
[24.783s][info][gc] GC(3) Concurrent Cycle
[24.784s][info][gc] GC(3) Pause Remark 242M->242M(528M) 0.383ms
[24.784s][info][gc] GC(3) Pause Cleanup 242M->242M(528M) 0.066ms
[24.785s][info][gc] GC(3) Concurrent Cycle 2.374ms

字段拆解
[24.783s][info][gc] GC(3) Concurrent Cycle：
  Mixed GC的相关信息，表示这是一次Mixed GC的开始。

[24.784s][info][gc] GC(3) Pause Remark 242M->242M(528M) 0.383ms
  Mixed GC的相关信息，表示这是一次Remark阶段的GC，回收了0M的内存空间，耗时0.383毫秒。

[24.784s][info][gc] GC(3) Pause Cleanup 242M->242M(528M) 0.066ms：
  Mixed GC的相关信息，表示这是一次Cleanup阶段的GC，回收了0M的内存空间，耗时0.066毫秒。

[24.785s][info][gc] GC(3) Concurrent Cycle 2.374ms：
  Mixed GC的相关信息，表示这是一次Mixed GC的结束，耗时2.374毫秒。

总结
  这段GC日志记录了程序运行了24.783秒一次Mixed GC，Mixed GC回收了0M的内存空间，耗时2.374毫秒
```

24-电商项目服务器8核16G内存，基于JDK11+OOM给一份JVM配置

- **题目分析和考点**
  - 当面试官提供一个具体的场景，并要求给出适合的JVM配置时，考查点主要是了解你对JVM调优和性能优化的经验和能力
  - 根据服务器的物理内存大小和应用程序的需求，适当设置堆内存的大小
  - 选择合适的垃圾收集器以根据应用程序的特点提供最佳性能
  - 根据应用程序的特点和性能需求，设置其他可调优的JVM参数
  - 如何根据服务器配置和应用程序需求进行JVM配置可以展示自己在JVM调优和性能优化方面的经验和能力
- **难度【* * * *】**
- **答案**
  - 面试题
    - 服务器配置是8核16g内存，需要部署一个springboot写的电商项目，日访问量100万左右的UV
    - 给一份生产环境配置的jvm参数的值，要求基于jdk11+配置oom时的堆栈快照信息
  - 配置OOM时的堆栈快照信息
    - -XX:+HeapDumpOnOutOfMemoryError：当发生OOM时，自动生成堆栈快照文件。
    - -XX:HeapDumpPath=：指定堆栈快照文件的输出路径。

- -XX:OnOutOfMemoryError="；"：当发生OOM时，执行指定的命令
- 案例`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/log/heapdump.hprof`

```
-XX:+UseG1GC -XX:MaxGCPauseMillis=100 -Xms524m -Xmx524m -
XX:+PrintCommandLineFlags -
Xlog:gc*=info:file=portal_gc.log:utctime,level,tags:filecount=5,filesize=1M
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=heapdump.hprof
```

  - 将在发生OOM时生成一个名为heapdump.hprof的堆栈快照文件，并将其保存到/var/log目录下
  - `heapdump.hprof` 文件可以使用多种工具进行分析

- 完整答案

```
-server
-Xms8g
-Xmx8g
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=32M
-XX:ActiveProcessorCount=8
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=heapdump.hprof
-XX:+PrintCommandLineFlags
-Xlog:gc*=info:file=portal_gc.log:utctime,level,tags:filecount=50,filesize=100M

参数说明

  -Xms8g：指定JVM堆内存最小值为8G。
  -Xmx8g：指定JVM堆内存最大值为8G。
  -XX:+UseG1GC：指定使用G1垃圾收集器。
  -XX:MaxGCPauseMillis=200：指定最大垃圾回收暂停时间为200毫秒。
  -XX:G1HeapRegionSize=32M：指定G1垃圾收集器的堆区域大小为32MB。
  -XX:ActiveProcessorCount=8：指定并行垃圾回收器的线程数为8，在JDK 9及之后的版本中，
ParallelGCThreads参数已被替代为-XX:ActiveProcessorCount参数，用于自动计算并行垃圾回收线
程数

  -server：指定JVM使用服务器模式运行，优化性能。
  -XX:+HeapDumpOnOutOfMemoryError：指定在发生内存溢出时生成堆转储文件。
  -XX:HeapDumpPath=/var/log/heapdump.hprof：指定堆转储文件的路径。
  -
Xlog:gc*=info:file=portal_gc.log:utctime,level,tags:filecount=50,filesize=100M
：指定GC日志的输出格式和位置，记录GC相关信息。
   -XX:+PrintCommandLineFlags：打印JVM启动时的命令行参数，可以去除

方便测试OOM，可以调整 -Xms524m -Xmx524m
```

  - 注意：使用G1收集器的时候，不用指定-Xmn

        - 在G1中，堆内存被划分为多个区域，每个区域都可以作为年轻代或老年代的一部分
        - G1的年轻代采用了不同于传统的基于分代的HotSpot垃圾收集器的方式，因此不需要指定-Xmn参数来设置年轻代的大小
        - G1利用自适应的内存分配策略来动态地调整年轻代的大小
            - 根据堆的使用情况来确定哪些区域应该作为年轻代，以及年轻代的大小
            - G1垃圾收集器不需要显式地指定-Xmn参数，通过自适应的方式来优化内存的使用和垃圾收集的效率

25-说你有哪些对生产环境JVM性能优化的方法论

- **题目分析和考点**
    - 面试官想知道你是否有系统的思考和方法来解决JVM性能问题，以及是否有实际的经验和案例来支持
    - 主要在考察作为高级Java工程师在生产环境中解决JVM性能问题的能力
    - 是否了解常见的JVM性能分析和监控工具，如jstat、jstack、VisualVM、Java Mission Control等，
    - 是否能够灵活运用这些工具来分析和解决性能问题
- **难度【* * * *】**
- **答案**
    - JVM性能优化
        - 这是一项复杂的工作，也是大家最喜欢问的问题点，也是面试官最喜欢的
        - **回答这个问题前请记住下面几句话，别问谁说的，就是我 二当家小D说的：**
            - **任何java业务做性能优化，都需要掌握JVM内部的工作机制和应用程序的特性**
            - **某个节点性能优化接近极致的时候，需要从局部跳到宏观层面进行分析，考虑自己和团队的ROI**
            - **缺少业务场景的性能优化都是垃圾**
    - 常见面试回答思路
        - 监控JVM性能
            - 对JVM的运行情况进行监控，以了解应用程序的瓶颈和性能瓶颈
            - 可以使用JVM自带的工具，如jstat、jmap、jstack等，或者第三方工具，如VisualVM、JProfiler等
        - 压测基准指标
            - 对程序进行压测，得出接口对应的吞吐量、响应时间等
            - 外部现象
                - 对用户体验来说，就是响应速度
                - 可以用压测工具jmeter进行压测得出相关性能指标

    - 内部现象：
        - 分析GC情况，是JVM性能调优的重要因素，需要掌握GC的工作机制和GC日志的含义
        - 可以使用JVM自带的GC日志或者第三方工具，如GCEasy等来分析GC情况，了解GC的频率、时间、内存占用等情况
- 调整JVM参数
    - 通过调整堆大小、GC算法、线程池大小等参数来提高应用程序的性能
    - 注意：不同的应用程序和环境可能需要不同的JVM参数配置，比如IO密集型和CPU密集型应用

- 二次压测分析
    - 通过调整jvm参数后，二次压测看性能指标提升还是下降
    - 内部：GC日志，看吞吐量，GC次数，停顿时间变化
    - 外部：接口对应的吞吐量、响应时间是否更优

- 其他优化方式
    - 优化代码
        - 通过避免不必要的对象创建、减少同步操作、使用缓存等方式来优化代码。
        - 注意：代码优化应该遵循“先正确，再优化”的原则，不应该牺牲代码的可读性和可维护性
    - 使用并发编程
        - 使用多线程、线程池等方式来提高并发性能，比如调整线程池的队列长度，存活线程数量等
        - 注意：并发编程需要考虑线程安全和锁竞争等问题，需要进行正确的设计和实现
    - 使用缓存
        - 可以使用本地缓存、分布式缓存等方式来提高数据访问性能
        - 注意：缓存需要考虑缓存一致性和缓存失效等问题，需要进行正确的设计和实现
    - 避免IO阻塞
        - 使用异步IO、NIO等方式来提高IO性能，比如前面学的CompletableFuture异步任务编排
        - 注意：IO编程需要考虑并发性和可靠性等问题，需要进行正确的设计和实现
    - 分布式+集群技术
        - 使用负载均衡+集群技术，提升单节点的处理能力
    - 其他...

MQ消息队列

1-新项目你如何选择 Kafka-RabbitMQ-RocketMQ?

- 题目分析和考点
  - 面试官想知道对业界主流的消息队列是否掌握，知道对应的技术框架特点
  - 什么业务场景和团队能力下，选择哪个技术，需要具备多个思考维度
- 难度【* * *】
- 答案
  - **面试回答（重点）：作为技术负责人，你业务里面使用了MQ，为什么决定用这个MQ, 而不考虑其他MQ产品**
    - 业务内部：业务项目引入MQ能满足业务当下的需求解决和未来的拓展
    - 业务外部：团队对某个MQ熟悉，公司现有运维团队和基础实施有比较完善的支撑
  - **消息队列三驾马车快速认知 Kafka、RabbitMQ、RocketMQ**
    - Kafka：http://kafka.apache.org/
      - Kafka是一种高吞吐量的分布式发布订阅消息系统(严格意义上是不属于队列产品，是一个流处理平台)
      - 它可以处理大规模的网站中的所有动作流数据(网页浏览，搜索和其他用户的行动)，副本集机制，实现数据冗余，
      - 保障数据尽量不丢失；支持多个生产者和消费者，类似MQ，功能较为简单，主要支持常规的MQ功能
      - 它提供了类似于JMS的特性，但是在设计实现上完全不同，它并不是JMS规范的实现
      - 缺点：运维难度大，文档比较少, 需要掌握Scala
    - RocketMQ：http://rocketmq.apache.org/
      - 阿里开源的一款的消息中间件, 纯Java开发，具有高吞吐量、高可用性、适合大规模分布式系统应用的特点,
      - 性能强劲(零拷贝技术)，支持海量堆积, 支持指定次数和时间间隔的失败消息重发,支持consumer端tag过滤、延迟消息等
      - 在阿里内部进行大规模使用，适合在电商，互联网金融等领域，基于JMS Provider的实现
      - 缺点：社区相对不活跃(全球)，更新比较快，纯java支持
    - RabbitMQ：http://www.rabbitmq.com/
      - 是一个开源的AMQP实现，服务器端用Erlang语言编写，支持多种客户端，如：Python、Ruby、.NET、Java、C

        - 用于在分布式系统中存储转发消息，在易用性、扩展性、高可用性等方面表现不错
        - 缺点：使用Erlang开发，阅读和修改源码难度大
    - 区别和联系，作为如何选择
        - 设计目标和重点：
            - Kafka的设计目标是高吞吐量、可持久化和可水平扩展的分布式流处理平台，它专注于日志处理和流式数据处理场景。
            - RabbitMQ的设计目标是实现高可靠性、灵活性和可扩展性的消息代理。它支持丰富的消息传递模式和高级特性。
            - RocketMQ的设计目标是实现低延迟、高吞吐量和可靠性的分布式消息中间件，它专注于大规模分布式系统的消息传递
        - 使用场景
            - Kafka适用于实时流数据处理、日志收集和流式ETL等场景，如实时数据流处理、日志聚合、指标监控等。
            - RabbitMQ适用于任务调度、异步处理、工作队列、RPC等场景，如任务分发、事件驱动、消息传递和系统解耦等。
            - RocketMQ适用于大规模分布式系统的异步通信/消息削峰/顺序消息等场景，如分布式事务、订单支付、实时消息推送等。
        - 共同点
            - 通用消息队列系统
                - Kafka、RabbitMQ和RocketMQ都是用于实现消息队列的中间件系统，提供高效、可靠、异步的消息传递机制。
            - 分布式架构
                - 这三个系统都支持分布式架构，能够实现高可用性、扩展性和容错性。
            - 可靠性保证
                - 它们都提供消息持久化和可靠性传递的机制，确保消息不会丢失，并且能够支持消息的高可靠性传递。
            - 多语言支持
                - Kafka、RabbitMQ和RocketMQ都支持多种语言客户端，只要协议保持一致即可
            - 生态系统
                - Kafka、RabbitMQ和RocketMQ都拥有丰富的生态系统和广泛的应用案例，并且有各自的社区支持和社区贡献。

2-什么时候应该用MQ，什么时候应该用RPC调用?

- 题目分析和考点
    - 面试官想知道对MQ和RPC的理解，服务之间通信方式都可以利用上述方案
    - 什么业务场景和团队能力下，选择哪个技术，需要具备多个思考维度

- 难度【* * *】
- 答案
  - 在选择使用消息队列（MQ）还是远程过程调用（RPC）时，考虑以下几个方面
  - 异步或同步
    - 如果需要异步处理消息或事件，例如处理大量的日志、实时消息通知等，使用MQ是一个不错的选择
    - MQ可以解耦发送者和接收者，发送者将消息发送到消息队列，接收者可以异步地去消费这些消息
    - 而当需要同步请求和响应时，RPC是更合适的选择
    - 比如 用户登录，权限校验 用RPC；用户注册发送短信或者邮件，电商下单生成物流信息用MQ
  - 松耦合或紧耦合
    - 如果是不同的服务之间需要进行松耦合的交互，使用MQ可以实现服务之间的解耦
    - 发送者只需要将消息发送到队列，而不需要知道具体的接收者。这样可以方便地进行服务的扩展和升级
    - 而当服务之间有固定的依赖关系，并需要直接调用对方的方法时，RPC更适合。
  - 可靠性要求
    - 如果对消息的可靠性要求较高，例如需要保证消息不丢失、发送有序等，使用MQ是较好的选择。
    - 大多数的消息队列都提供了消息持久化、消息重试、消息回退等功能，以保证消息传递的可靠性
    - 相比之下，RPC通常是基于网络传输调用的，对网络的稳定性要求相对较高，可靠性稍差。
  - 服务调用频率和性能需求
    - 如果服务之间的调用频率较高，且需要在同一个进程内进行调用，RPC是更适合的选择
    - RPC一般是基于方法调用的，通常性能更高，并且可以在同一个进程内直接调用，无需网络开销。
    - 而MQ的性能受到消息中间件的吞吐量和延迟的影响，相对于RPC而言性能较差。
- 总结
  - 当需要实现异步、松耦合、可靠性要求较高的场景时，使用MQ更适合
  - 而当需要实现同步、紧耦合、性能要求较高的场景时，使用RPC更适合。
  - 具体选择还需要根据具体的业务场景和需求来考虑，有时候也可以结合使用MQ和RPC来满足不同的需求。

3-业务项目的架构里面引入MQ消息队列，可能带来哪些问题?

- 题目分析和考点
  - 项目引入MQ, 则需要考虑会带来哪些问题，任何技术的引入，都需要考虑利弊
  - 如果应聘者没用过MQ，则可能回答不上来
- 难度【* * *】
- 答案

- 消息丢失
  - 由于各种原因，如网络问题、消息队列服务故障、消费者处理失败等，消息可能会丢失
  - 导致数据不一致或丢失重要的业务消息。
- 消息重复
  - 当消息发送失败时，生产者可能会尝试重新发送消息，导致消息在队列中被重复处理
  - 在某些场景下，重复消息可能会引发业务逻辑错误或重复处理的问题。
- 消息堆积
  - 在高并发或处理能力不足的情况下，消息队列可能会出现消息堆积的情况
  - 消息的积压可能导致系统性能下降、延迟增加或服务不可用。
- 延迟消息
  - 某些业务场景下需要处理实时性要求较高的消息，而消息队列的传递和处理过程中可能会引入一定的延迟
  - 这可能会导致某些业务逻辑出现延迟或无法满足实时性需求。
- 系统可用性
  - 消息队列作为关键的分布式系统组件，其可用性和稳定性对整个系统的稳定性和可靠性至关重要
  - 如果消息队列出现故障、性能瓶颈或部分服务不可用，可能会影响整个系统的正常运行。
- 版本兼容性
  - 在消息队列的生产环境中，可能会面临升级版本或者使用不同版本的客户端的问题
  - 版本升级或不兼容可能导致部分功能不可用、数据丢失或影响系统稳定性。

4-说下RabbitMQ消息丢失的场景有哪些和怎么解决?

- 题目分析和考点
  - 消息队列引入项目有很多利弊，比较重要的是避免消息丢失，要多方面考虑丢失的场景
  - 如果应聘者没用过MQ，则可能回答不上来
- 难度【* * * *】
- 答案
  - 场景一：未开启消息持久化机制
    - 生产者已经将消息发送给了队列，但消费者还没以及对消息进行消费
    - 如果队列所属的主机宕机，则存储在队列的消息也会丢失
    - 解决方案
      - 对消息进行持久化操作，如果消息一旦被发送到mq中的某一个队列，那么此时Rabbitmq会立马将消息进行持久化
      - **日常开发中，SpringBoot和Rabbitmq整合默认消息的存储就是持久化方式**
      - 可以将所有的消息都设置为持久化，但持久化会写到磁盘，写磁盘速度肯定不上写内存的速度，影响Rabbitmq的性能。

        - 对于可靠性不是那么高的消息可以不采用持久化处理，以提高整体系统的吞吐量
        - 在RabbitMQ中，deliveryMode是消息的持久化属性，用于指定消息在队列中的持久化方式
            - 非持久化消息，deliveryMode = 1
                - 消息在发送到队列后，会被保存在内存中如果RabbitMQ服务重启或崩溃，这些消息将会丢失。
                - 适用场景：适用于一些临时性消息，对于丢失消息没有严格的要求，例如临时计算结果、通知等。
            - 持久化消息，deliveryMode = 2
                - 消息在发送到队列后，会被保存在磁盘上，即使RabbitMQ服务重启或崩溃，这些消息仍然可以被恢复和投递
                - 适用场景：适用于对消息的可靠性和持久化要求较高的场景，例如重要的业务消息、订单信息、日志记录等。
    - 场景二：生产者到交换机失败
        - 生产者投递消息到交换机，未经确认导致消息丢失
        - 解决方案
            - 发送确认机制可以通过confirmCallback实现，开启该机制后，发送端可以感知消息是否成功到达交换机
            - 生产者投递消息后，如果Broker收到消息后，会给生产者一个ACK
            - 生产者通过ACK，可以确认这条消息是否正常发送到Broker，这种方式是消息可靠性投递的核心
            - 开启消息确认机制以后，保证了消息的准确送达，但由于频繁的确认交互， rabbitmq 整体效率变低
            - 吞吐量下降严重，不是非常重要的消息不建议用消息确认机制
    - 场景三：交换机转发消息到队列失败
        - 交换机需要转发到队列，如果转发失败，也会导致消息丢失
        - 解决方案
            - 通过returnCallback，消息从交换器发送到对应队列失败时触发
            - 两种模式
                - 交换机到队列不成功，则丢弃消息（默认）
                - 交换机到队列不成功，返回给消息生产者，触发returnCallback
    - 消息丢失场景四
        - 消费者从broker中监听消息，需要确保消息被合理处理
        - SpringAMQP则允许配置三种确认模式
            - manual：手动ack，根据业务情况，判断什么时候该ack，调用api发送ack
            - auto：自动ack，由spring自身监测消费者是否出现异常，没有异常则返回ack；抛出异常则返回nack，消息回滚到mq
            - none：关闭ack，消息投递不可靠，可能丢失，认为消费者获取消息后会成功处理，所以消息投递后立即被删除

    - RabbitMQ的ACK介绍
        - 消费者从RabbitMQ收到消息并处理完成后，反馈给RabbitMQ，RabbitMQ收到反馈后才将此消息从队列中删除
        - 消费者在处理消息出现了网络不稳定、服务器异常等现象，那么就不会有ACK反馈
        - RabbitMQ会认为这个消息没有正常消费，会将消息重新放入队列中
        - 只有当消费者正确发送ACK反馈，RabbitMQ确认收到后，消息才会从RabbitMQ服务器的数据中删除。
        - 消息的ACK确认机制默认是打开的，消息如未被进行ACK的消息确认机制，这条消息被锁定Unacked
    - 消息消费的确认方式
        - 自动确认（默认）
        - 手动确认 `spring.rabbitmq.listener.simple.acknowledge-mode=manual`

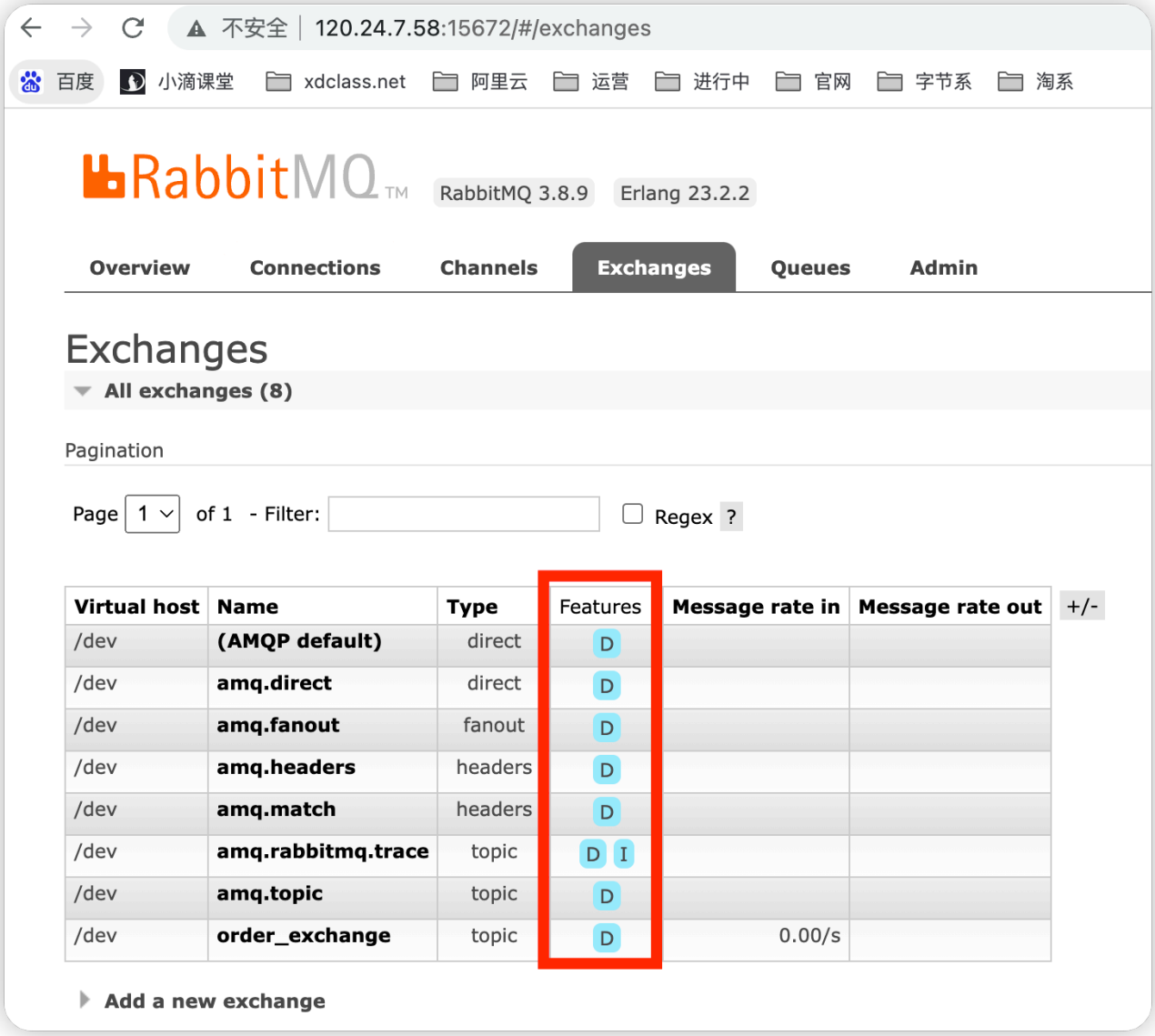
5-如果单机RabbitMQ消息没开启持久化，重启是否一定会丢失?

- 题目分析和考点
    - 本题目考察点不止MQ，还有操作系统和内存，硬盘相关
    - 如果应聘者只是会简单的使用MQ，没掌握操作系统的内存相关的内容，则难回答清楚
- 难度【*****】
- 答案
    - 不一定，因为无论消息是持久化还是非持久化，都可以被写入磁盘
    - RabbitMQ持久化消息会在到达队列时被写入磁盘，同时可能会在内存中保留备份，以提高性能
    - 当内存紧张时，可以从内存中清除这些备份
    - 非持久化消息通常只保存在内存中，但是当内存紧张时，会被写入磁盘，以节省内存空间
        - 即利用操作系统的swap分区
    - 拓展：什么是交换分区Swap
        - 进程占用内存很大，会导致内存消耗完，为解决该问题操作系统中运用Swap技术，拿部分硬盘空间来充当内存使用
        - 操作系统欺骗进程，内存很多你放心大胆去用；好比CPU的超线程，欺骗 操作系统 1个核变成两个核

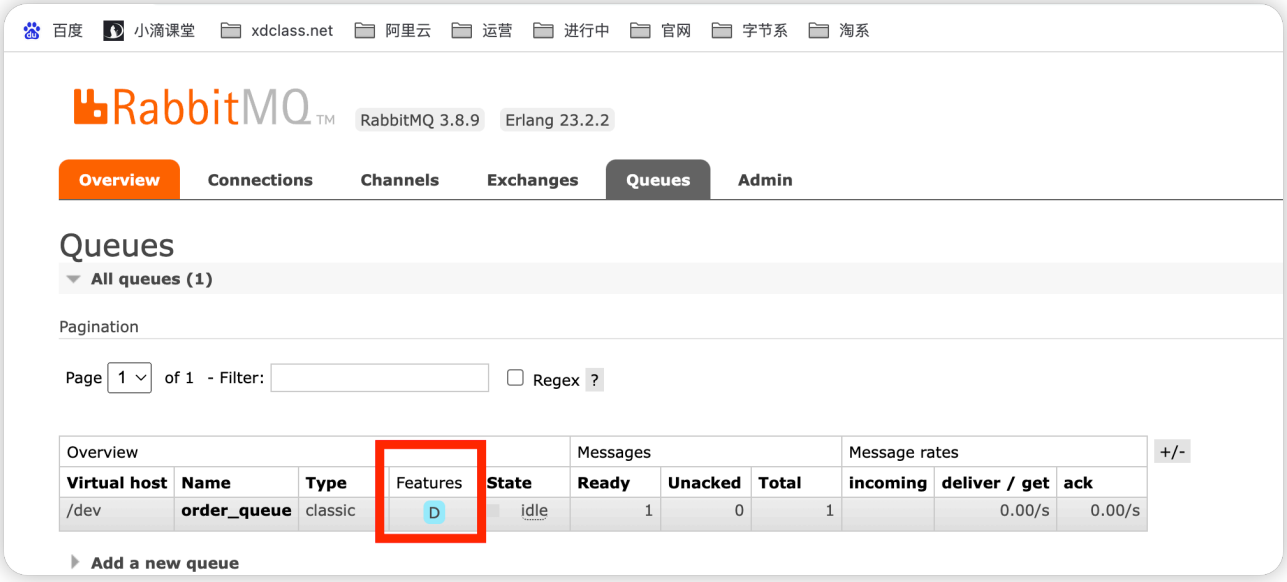
        - 一句话:就是Linux下的虚拟内存分区，作用是在物理内存使用完之后,将磁盘空间(也就是SWAP分区)虚 拟成内存来使用
        - 换页机制
            - 操作系统把物理内存中放不下的数据临时放到磁盘上，等到需要的时候再放回到物理内存中
            - 提供超过物理内存容量的空间

6-基于RabbitMQ框架，如何保证消息的可靠性传输?

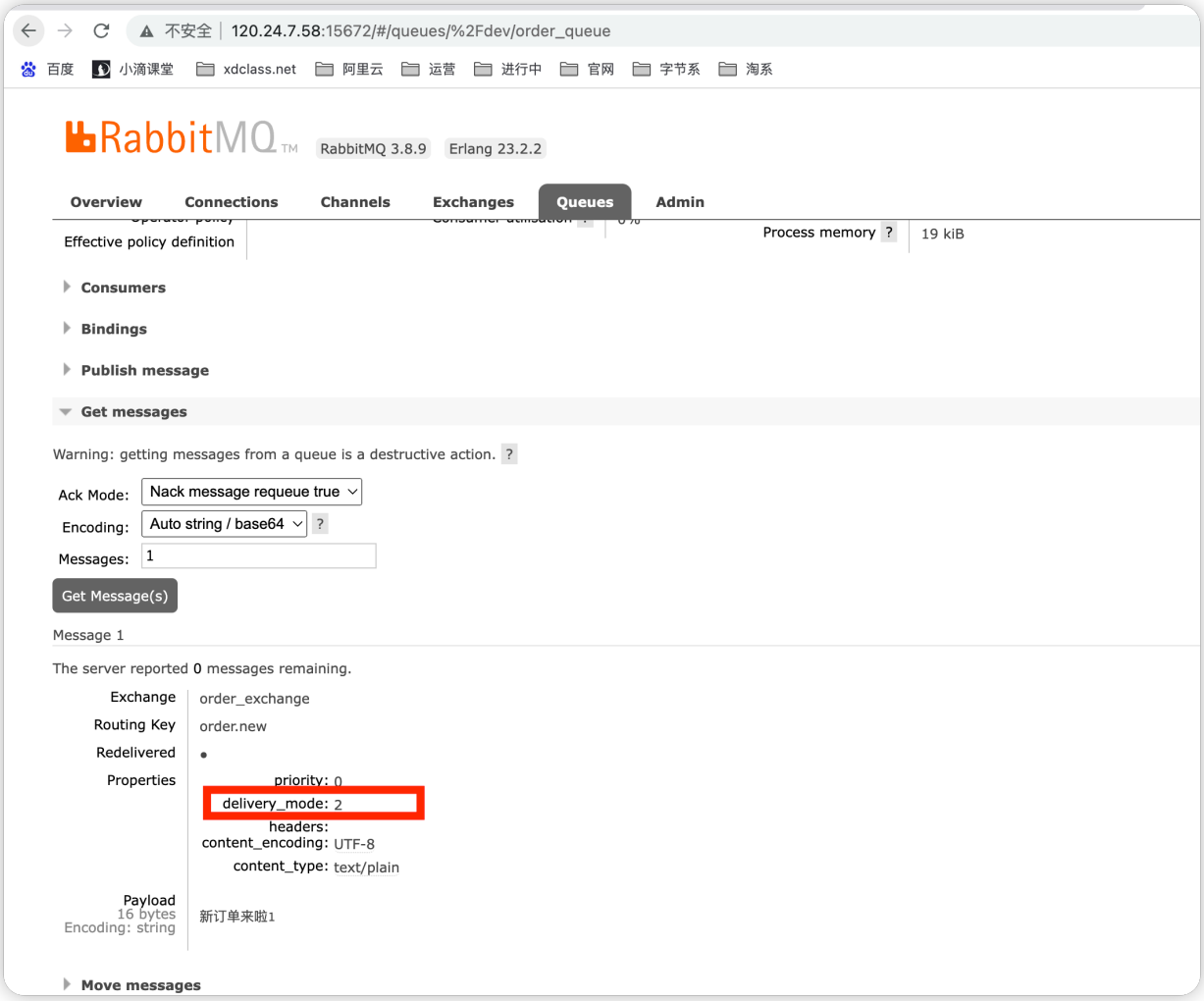
- 题目分析和考点
    - 面试官问及如何保证消息的可靠性传输，想了解你对消息队列框架中消息可靠性的保障机制的理解。
    - 消息队列的可靠性传输，需要从多方面考虑好，避免消息丢失
    - 考察点包括：
        - 对消息队列架构和可靠性传输的了解。
        - 对RabbitMQ框架的特性和机制的熟悉程度。
        - 对消息持久化、队列持久化、Confirms机制的认识
- 难度【* * * * 】
- 答案
    - 需要先明白RabbitMQ消息投递路径，通过多个的点控制消息的可靠性投递
        - **生产者-->交换机->队列->消费者**
    - 交换机持久化：RabbitMQ中交换机默认是非持久化的，mq重启后就丢失； D 就是持久化durable的意思



- 队列持久化：队列默认是非持久化的，mq重启后就丢失



- 消息持久化：deliveryMode消息持久化配置

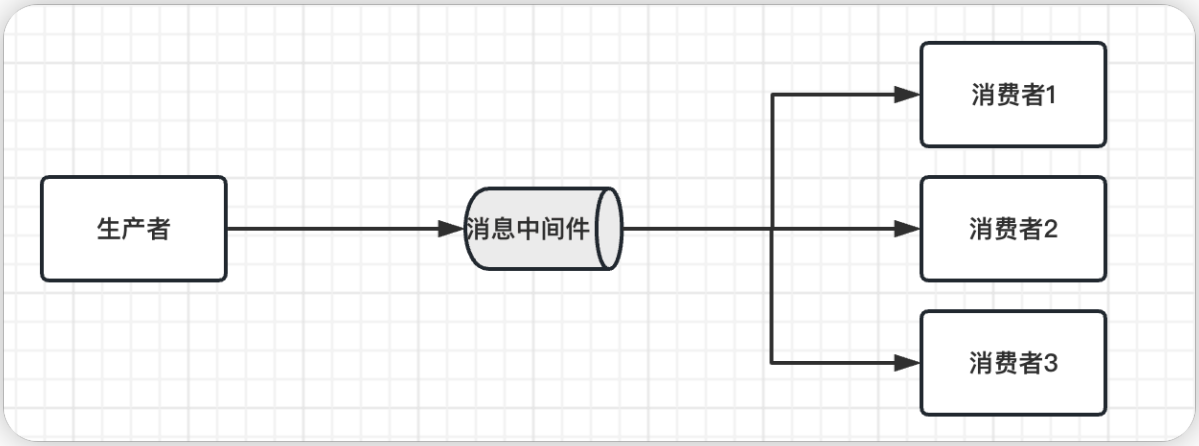


- 生产者投递到交换机，通过confirmCallback 确认
- 交换机到队列，通过returnCallback确认
- 消费者进行手工消费，manual确认

7-什么是消息堆积，什么场景下会发生消息堆积?

- 题目分析和考点
  - 消息堆积是很多MQ都存在的问题，不同MQ消息模型，导致解决方案不一样
  - 面试官想确认应聘者是否对不同MQ的消费模型是否清楚，有没思考过生成环境出现这个问题应该怎么解决?

- 难度【* * * * *】
- 答案
    - 队列中的消息不能被及时消费，导致大量积压在队列里面



    - 发生消息堆积的场景
        - 场景一：消费者处理速度慢
            - 如果消费者的处理能力跟不上消息的产生速度，就会导致消息堆积
            - 这可能是由于消费者的处理逻辑复杂、处理速度低下，或者消费者数量不足。
        - 场景二：消费者故障或延迟
            - 如果消费者发生故障、延迟或阻塞，无法及时处理消息，消息就会在队列中积累。
        - 场景三：消息生产速度过快
            - 如果生产者产生消息的速度过快，而消费者无法及时处理，就会导致消息堆积。
        - 场景四：MQ服务器性能不足
            - 如果MQ服务器的资源（CPU、内存、磁盘）不足或超负载，无法及时处理消息，就会导致消息堆积。
        - 场景五：网络故障或连接问题
            - 如果在消息传递过程中发生网络故障、连接断开或丢失，消息可能无法及时到达消费者，从而导致堆积。
        - 场景六：消息消费失败重试
            - 如果消息消费失败并进行了重试操作，但是重试操作不断失败，可能会导致消息堆积

8-说下RabbitMQ里面的惰性队列和普通队列的区别是哪里?

- 题目分析和考点
    - RabbitMQ的惰性队列是新特性，和普通队列的区别比较大

    - 面试官想确认应聘者是否对RabbitMQ的特殊知识点是否掌握
- 难度【* * *】
- 答案
    - RabbitMQ的3.6.0版本开始，增加 Lazy Queue ，即惰性队列
    - 官方文档地址：https://www.rabbitmq.com/lazy-queues.html
    - 特点
        - 会尽可能的将消息存入磁盘中，在消费者消费到相应的消息时才会被加载到内存中
        - 设计目标是能够支持更长的队列，即支持更多的消息存储
    - 普通队列
        - 生产者将消息发送到RabbitMQ的时候，队列中的消息会尽可能的存储在内存之中
        - 消息开启持久化，在被写入磁盘的同时也会在内存中驻留一份备份
    - 惰性队列
        - Broker接收消息后将消息直接存入磁盘，不管是持久化的或者是非持久化的
        - 消费者需要消费时才会从磁盘中读取，并加载到内存中
        - 惰性队列和普通队列相比，只有很小的内存开销
        - **缺点：基于磁盘存储，写入磁盘有IO损耗，性能和容量受限于磁盘**
        - 官方案例数据
            - 发送100万条消息，每条消息的大小为1KB，那么普通队列会消耗1GB多内存，而惰性队列只能消耗1MB多的内存

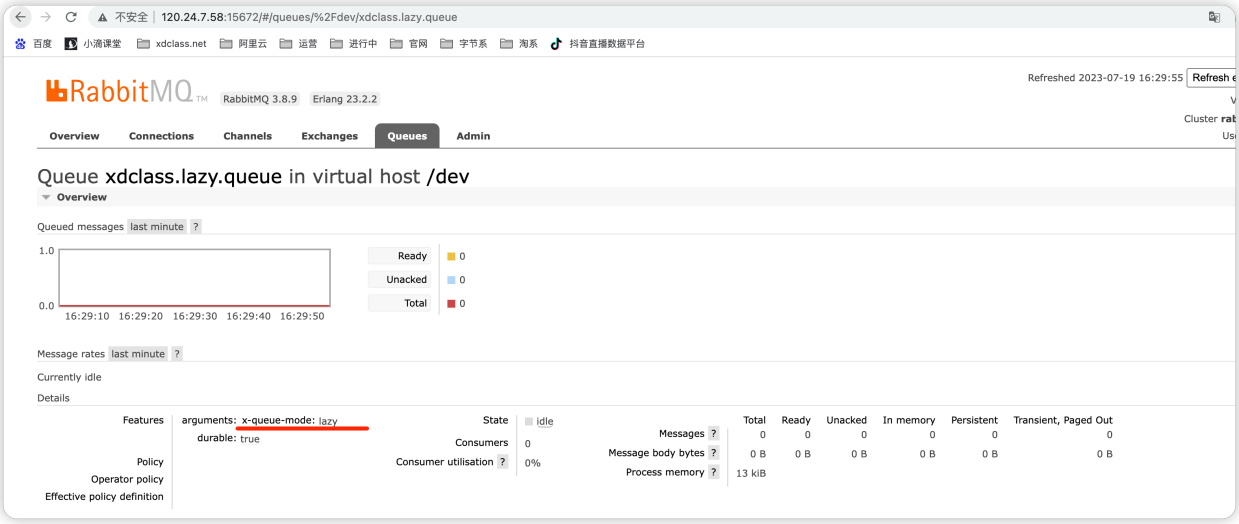
            - 测试中使用了以下参数：
                - 消息数量：1,000,000条
                - 消息体大小：1,000字节
                - 消息类型：持久化（persistent）
                - 生产者数量：1个
                - 消费者数量：0个
            - 测试结果显示了标准队列（default）和惰性队列（lazy）在摄入这些消息后的内存利用情况。
                - 默认队列（default）
                    - 使用257 MB的队列进程内存，存储了386,307条消息，消息占用368 MB的内存，节点总内存占用为734 MB。

        - 惰性队列（lazy）
            - 使用了159 KB的队列进程内存，没有消息存储在内存中，没有内存被消息使用，节点总内存占用为117 MB。
    - 结论
        - 两个队列都持久化了1,000,000条消息，并使用了1.2 GB的磁盘空间
        - 测试表明，在相同的消息数量和大小情况下，惰性队列相对于默认队列节省了大量的内存空间
        - 惰性队列只在需要时分配内存资源，而默认队列会保留所有的消息在内存中。

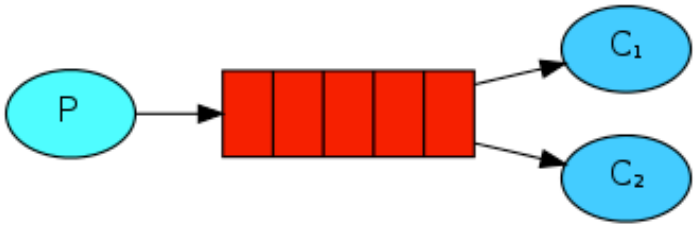
9-说下RabbitMQ发生消息堆积的解决方案有哪些?

- 题目分析和考点
    - 面试官想确认应聘者是否对RabbitMQ的消费模型是否掌握，生产环境中应该怎么应对这个问题
    - RabbitMQ的消费模型比较简单，加上本身的惰性队列，可以比较好解决消息堆积
- 难度【*****】
- 答案
    - RabbitMQ解决消息堆积问题
        - 方案一 惰性队列： 扩大队列容积，提高堆积上限 ，把消息存储从内存中剥离出来，使用了磁盘

```
   @Bean
    public Queue lazyQueue() {
        return QueueBuilder.durable("xdclass.lazy.queue")
                .lazy()
                .build();
    }

    @Bean
    public Binding orderLazyBinding(Queue lazyQueue, Exchange exchange) {
        return
BindingBuilder.bind(lazyQueue).to(exchange).with("lazy.order.#").noargs();
    }
```



- 方案二 增加更多消费者: 提高消费速度即可，队列上绑定更多消费者即可，Work queues模式
  - WorkQueue工作队列介绍
    - 消息生产能力大于消费能力，增加多几个消费节点
    - 让多个消费者绑定到一个队列，共同消费队列中的消息，不同的消费节点，处于竞争关系
    - 当消息处理不过来或者产生积压的时候，可以直接采用这个方案进行



- 消息堆积解决的关键点
  - 优化应用程序的性能，缩短响应时间
  - 增加消费者节点实例 ， 成本增加，**底层数据库操作和其他关联链路也可能是瓶颈** (特别注意)
  - 调整并发消费的线程数，比如程序内部使用线程池和队列临时缓冲，但是要避免宕机丢失内存消息

10-说下RabbitMQ高可用集群方案有哪些和使用场景?

- 题目分析和考点
  - 面试官想确认应聘者是否对RabbitMQ的高可用架构是否掌握
  - RabbitMQ的高可用架构有多种，主要是普通集群和镜像集群
  - 两个集群方案适合不一样的场景，性能也有差异
- 难度【*****】

- 答案
  - 常见的集群方案
    - RabbitMQ普通集群
      - 默认的集群模式, 比如有节点 node1和node2 两个节点是普通集群，但他们**仅有相同的元数据,即交换机、队列的结构**
      - 案例：
        - 消息只存在其中的一个节点里面，假如消息A存储在node1节点，消费者连接node1个节点消费消息时，可以直接取出来；
        - 但如果 费者是连接的是其他节点 那rabbitmq会把 queue 中的消息从存储它的节点中取出，并经过连接节点转发后再发送给消费者
      - 问题
        - 假如node1故障，那node2无法获取node1存储未被消费的消息；
        - 如果node1持久化后故障，那需要等node1恢复后才可以正常消费，如果ndoe1没做持久化后故障，那消息将会丢失
        - 这个情况无法实现高可用性，且节点间会增加通讯获取消息，性能存在瓶颈
        - 项目中springboot+amqp里面需要写多个节点的配置，比如下面

          ```
          spring.rabbitmq.addresses = 192.168.1.1:5672,192.168.1.2:5672
          ```

        - **该模式更适合于消息无需持久化的场景，如日志传输的队列**
    - RabbitMQ镜像集群
      - 队列做成镜像队列，让各队列存在于多个节点中
      - **和普通集群比较大的区别就是【队列queue的消息message 】会在集群各节点之间同步**
      - 且并不是在 consumer 获取数据时临时拉取，而普通集群则是临时从存储的节点里面拉取对应的数据
      - 结论
        - 实现了高可用性，部分节点挂掉后，不影响正常的消费
        - 可以保证100%消息不丢失，推荐3个奇数节点，结合LVS+Keepalive 或 Haproxy进行IP漂移，防止单点故障
      - 缺点
        - 由于镜像队列模式下，消息数量过多，大量的消息同步也会加大网络带宽开销
        - 适合高可用要求比较高的项目
          过多节点的话，性能则更加受影响

11-日常开发中有没用RabbitMQ的visual host，主要的用途是?

- 题目分析和考点
  - 面试官想确认应聘者是否对RabbitMQ的虚拟主机是否掌握
  - 如果是项目里面担任高级工程师，且有设计团队开发规范， 基本都是会用到虚拟主机
- 难度【***】
- 答案
  - 在RabbitMQ中，Virtual Host（虚拟主机）是逻辑上的一个独立环境
  - 提供了一种将RabbitMQ资源（交换机、队列、绑定关系等）进行逻辑隔离的机制
  - 每个虚拟主机都是一个完全独立的消息代理，它们之间互相隔离，拥有独立的配置、权限和资源
  - 作用
    - 逻辑隔离：通过虚拟主机，可以将不同的应用或不同的业务逻辑隔离开来。每个虚拟主机中的资源相互独立，不会受到其他虚拟主机的影响，这样可以实现多个应用或多个团队之间的资源隔离，避免资源冲突和互相影响。
    - 安全性：虚拟主机可以用于实现不同的安全策略和权限控制。每个虚拟主机可以拥有不同的用户和权限设置，这样可以精确控制不同应用或不同用户访问资源的权限，提高系统的安全性。
    - 限流和资源分配：通过虚拟主机，可以为不同的应用或团队分配独立的资源。可以在虚拟主机级别进行限流和配额设置，确保资源的合理分配和利用。
    - 环境隔离：虚拟主机可以用于将开发、测试和生产环境分开。可以创建不同的虚拟主机用于不同环境的部署，以便更好地管理和监控各个环境的消息流。
  - 总结
    - 每个虚拟主机都有一个名称，通过名称来唯一标识，在连接RabbitMQ时，需要选择虚拟主机来进行操作
    - 默认情况下，RabbitMQ已经创建了一个名为"/"的虚拟主机，它是默认的虚拟主机，用于不同业务模块的逻辑隔离
    - 一个Virtual Host里面可以有若干个Exchange和Queue
    - 同一个VirtualHost 里面不能有相同名称的Exchange或Queue

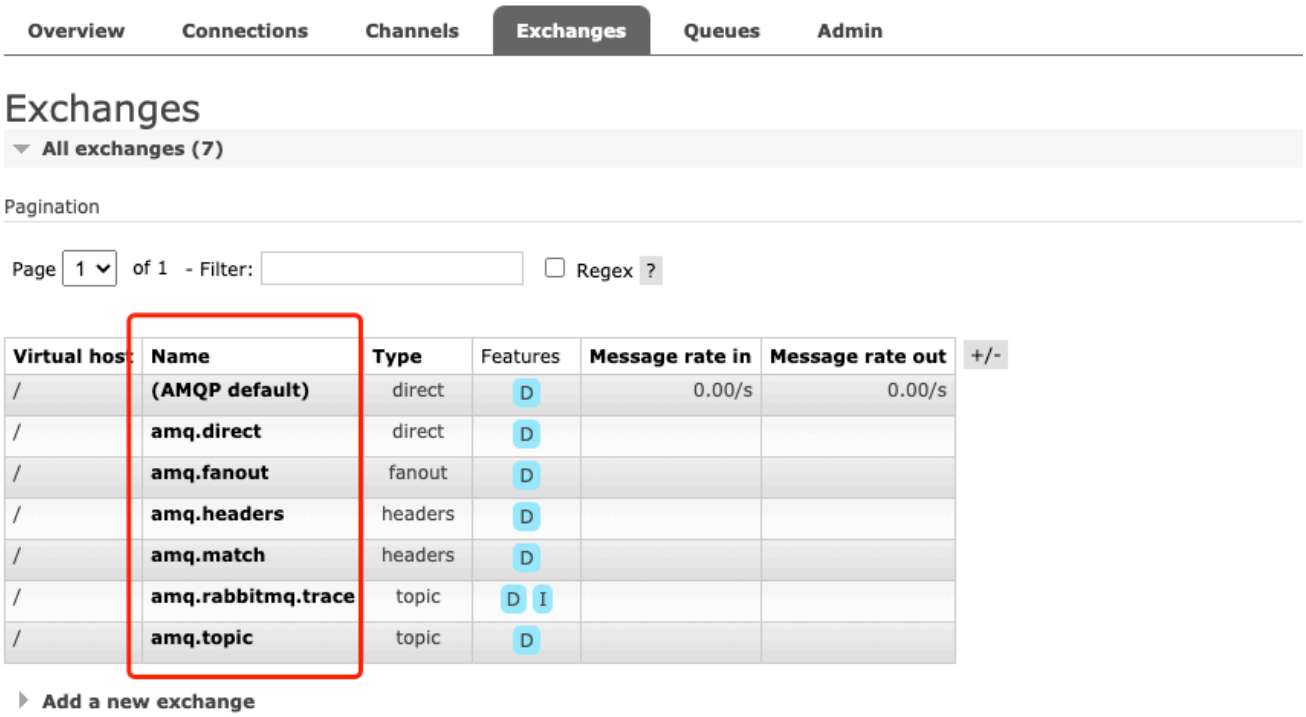
12-什么是顺序消息，RabbitMQ如何确保消息的顺序性?

- 题目分析和考点
  - 面试官想确认应聘者是否顺序消息是否掌握，不管是哪个MQ框架，实现顺序消息的关键思想都是一样的
- 难度【***】
- 答案
  - 什么是顺序消息：消息的生产和消费顺序一致

        - 全局顺序：topic下面全部消息都要有序(少用)
            - 性能要求不高，所有的消息严格按照 FIFO 原则进行消息发布和消费的场景，并行度成为消息系统的瓶颈, 吞吐量不够.
            - 在证券处理中，以人民币兑换美元为例子，在价格相同的情况下，先出价者优先处理，则可以通过全局顺序的方式按照 FIFO 的方式进行发布和消费
        - 局部顺序：只要保证一组消息被顺序消费即可
            - 性能要求高
            - 电商的订单创建，同一个订单相关的创建订单消息、订单支付消息、订单退款消息、订单物流消息、订单交易成功消息 都会按照先后顺序来发布和消费
            - 阿里巴巴集团内部电商系统均使用局部顺序消息，既保证业务的顺序，同时又能保证业务的高性能
    - 注意：
        - 顺序消息暂不支持广播模式
        - 顺序消息不支持异步发送方式，否则将无法严格保证顺序
    - 实现顺序消息的关键点
        - 顺序发布：对于指定的一个 Topic，客户端将按照一定的先后顺序发送消息
        - 顺序消费：对于指定的一个 Topic，按照一定的先后顺序接收消息，即先发送的消息一定会先被客户端接收到。
    - 基于顺序消息的特点，RabbitMQ如果需要实现顺序消息
        - 要保证消息的顺序性，可以使用单一队列或消息分区进行处理，保证消息按照顺序依次处理。

13-RabbitMQ中的Exchange有几种类型？并解释它们的用途

- 题目分析和考点
    - 面试官想确认应聘者是否对RabbitMQ的交换机是否掌握，有多个类型的交换机
    - 不同交换机的特点不一样，需要合理的知道全部交换机种类和适合的业务场景
- 难度【* * *】
- 答案
    - 在RabbitMQ中有四种主要类型的Exchange：Direct、Topic、Fanout和Headers

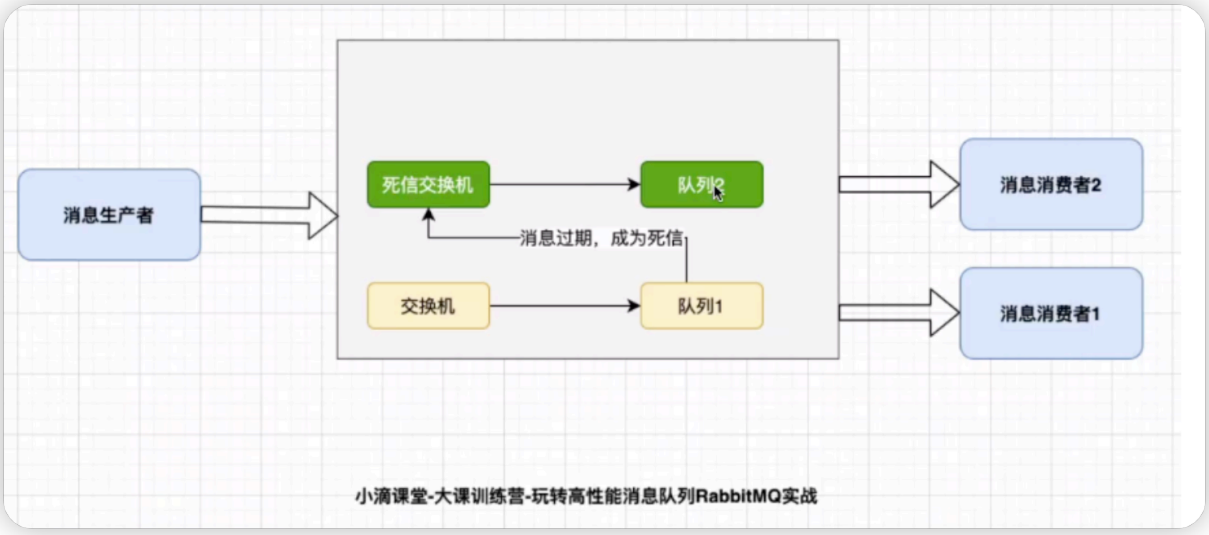


- 用途
  - Direct Exchange（直连交换机）：
    - 是最简单的一种交换机类型。它将消息按照Routing Key精确匹配并发送到具有相同Routing Key的队列
    - 这种类型的Exchange在消息的选择性发布上非常有用，可以根据具体的Routing Key将消息发送到特定队列中
    - 例如，将日志消息根据日志级别（如error、info）发送到对应的队列。
    - 处理路由健
  - Fanout Exchange（扇形交换机）：
    - Fanout Exchange将消息广播到与之绑定的所有队列上，忽略Routing Key
    - 它将消息同时发送到绑定的所有队列，实现了一对多的消息发布。
    - 关键应用场景，例如系统日志广播给多个日志处理器进行分析或通知消息广播到所有在线的订阅用户
    - 不处理路由健
  - Topic Exchange（主题交换机）
    - 主题交换机是一种发布/订阅的模式，结合了直连交换机与扇形交换机的特点
    - 允许使用通配符与Routing Key进行匹配。它可以根据Routing Key的模式进行匹配，并将消息路由到匹配的队列
    - 通配符可以使用"#"（表示零个或多个单词）和"*"（表示一个单词）来匹配Routing Key的各个部分
    - Topic Exchange非常适用于灵活的分发机制，使得消息能够根据主题进行动态路由。
    - 可以根据消息的主题、类型或其他属性进行灵活的订阅和过滤，以满足不同的消费需求
    - 处理路由健

- Headers Exchange（头交换机）
  - Headers Exchange使用消息的Headers属性对消息进行匹配
  - 它通过发送消息头中的键值对来进行匹配，而不考虑Routing Key和绑定键
  - Headers Exchange主要用于一些复杂的匹配规则。根据消息头属性对消息进行选择性地路由到队列
  - 这种Exchange类型在一些特殊的业务需求（如基于消息头的高级路由）中很有用
  - 不处理路由键

14-说下RabbitMQ的死信队列和业务中的应用场景?

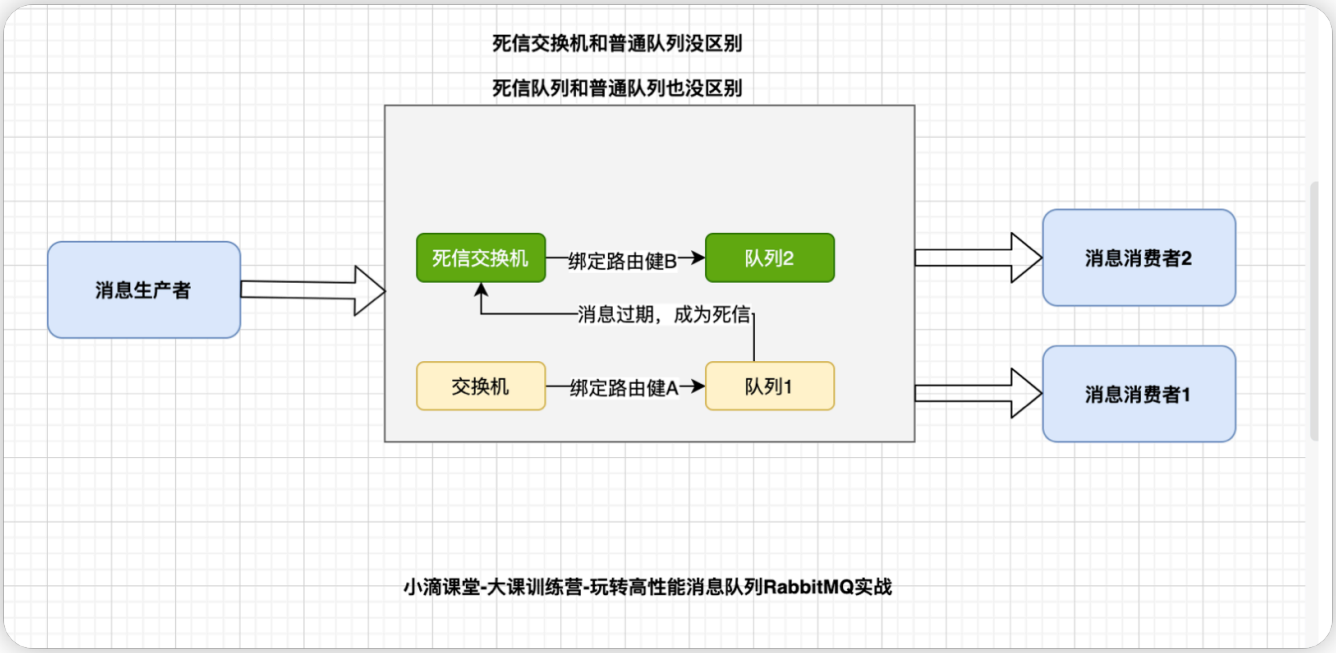
- 题目分析和考点
  - 面试官想确认应聘者是否对死信队列掌握和业务中的使用是否清楚
  - 合理的利用MQ特性，很多业务解决方案可以轻松解决
  - 比如RabbitMQ不支持延迟队列，但是可以利用死信队列的特点实现延迟队列的功能
- 难度【* * *】
- 答案
  - rabbitmq的死信交换机 Dead Letter Exchange（死信交换机，缩写：DLX）
    - 当消息成为死信后，会被重新发送到另一个交换机，这个交换机就是DLX死信交换机。



  - 延迟队列
    - 一种带有延迟功能的消息队列，Producer 将消息发送 到消息队列 服务端，但并不期望这条消息立马投递

- 而是推迟到在当前时间点之后的某一个时间投递到 Consumer 进行消费，该消息即定时消息
- 业界的一些实现方式
  - 定时任务高精度轮训 采用RocketMQ自带延迟消息功能
  - RabbitMQ本身是不支持延迟队列的，怎么办? 结合死信队列的特性，就可以做到延迟消息
  - 操作
    - 消息生产
      - 投递到普通的topic交换机
      - 消息过期，进入死信交换机
    - 消息消费
      - 消费者监听死信交换机的队列

- 使用场景
  - 通过消息触发一些定时任务，比如在某一固定时间点向 用户发送提醒消息
    - 用户登录之后5分钟给用户做分类推送、用户多少天未 登录给用户做召回推送;
    - 消息生产和消费有时间窗口要求
      - 比如在天猫电商交易 中超时未支付关闭订单的场景，在订单创建时会发送一 条 延时消息。
      - 这条消息将会在 30 分钟以后投递给消费者，消费者收到此消息后需要判断对应的订单是否已完 成支付。
      - 如支付未完成，则关闭订单。如已完成支付则忽略



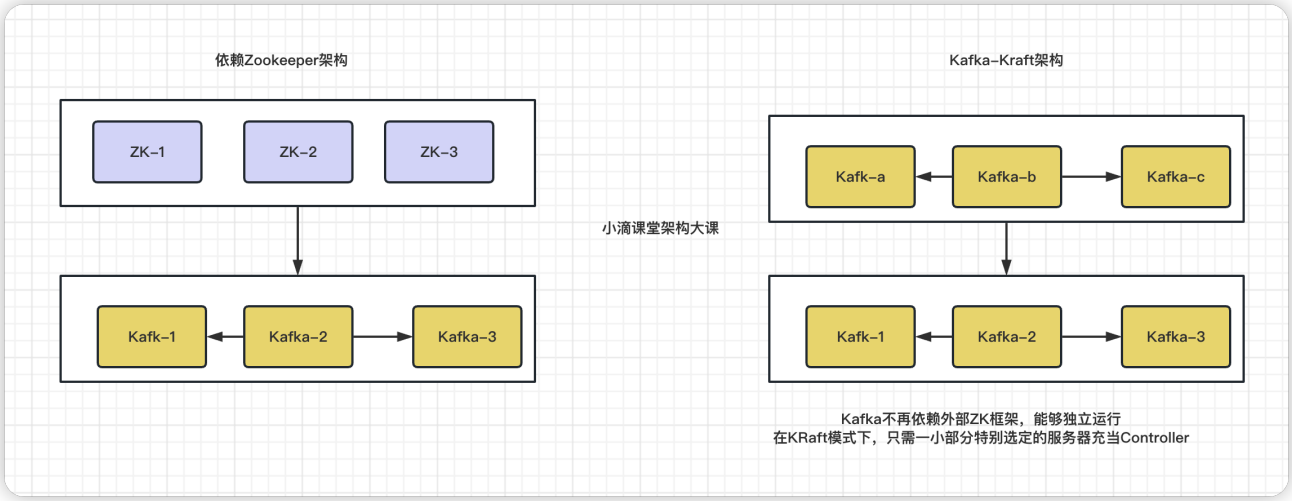
15-说下RabbitMQ的消息有哪几种情况成为死信?

- 题目分析和考点
  - 面试官想确认应聘者是否对死信队列掌握和业务中的使用是否清楚
  - 业务开发中什么情况消息会成为死信消息
- 难度【* * * *】
- 答案
  - 消息被拒绝（Rejected）
    - 当消费者拒绝接收消息，并且设置了requeue参数为false时，消息会变成死信，并被投递到死信队列。
    - 代码里面的 **（basic.reject/ basic.nack）**
  - 消息过期（Expired）
    - 如果消息设置了过期时间（TTL），并且在这个时间段内未被消费者消费，消息会被标记为死信，并被投递到死信队列。
  - 队列达到最大长度（Maximum Queue Length）
    - 当队列中的消息数量达到了队列的最大限制，并且有新的消息要被投递时，之前的旧消息会成为死信，被投递到死信队列。
  - 队列达到最大存储空间（Maximum Queue Storage）
    - 当队列占用的存储空间达到了队列的最大限制，且有新的消息要被投递时，之前的旧消息会成为死信，被投递到死信队列。
  - 消息传递失败（Unroutable Message）
    - 当消息无法被路由到任何队列时，会成为死信。这通常发生在消息的Routing Key无法与现有的Exchange和Binding匹配时
  - 结论
    - 消息成为死信后，如果该队列绑定了死信交换机，则消息会被死信交换机重新路由到死信队列
    - 可以用于分析和处理消息异常、重新投递或进行进一步的错误处理
    - 提供这种机制来处理无法被正常消费的消息，保证消息系统的健壮性和可靠性。

16-有没了解Kafka3.X的Kraft架构，对比之前改进了什么?

- 题目分析和考点
  - 面试官想确认应聘者是否对新版kafak关注
  - 作为一个高级工程师，对新技术有一定的追求和关注，在合适的场景下升级业务架构，也有利用提高性能

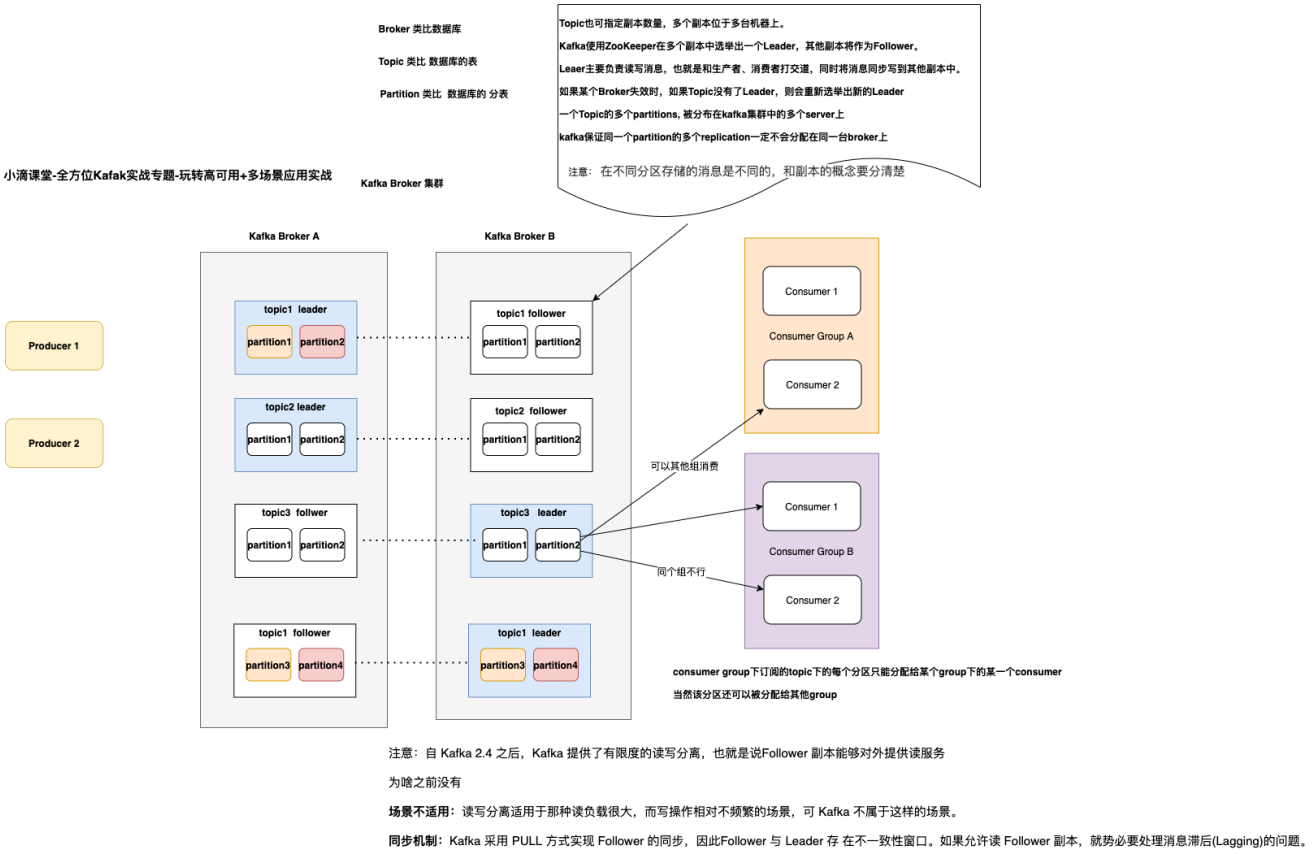
- 难度【* * *】
- 答案
  - 是 Kafka 在 2.8.0 版本中引入的一种新的复制协议和存储系统架构
  - 之前Kafka元数据存储在zk中，运行时动态选举controller，由controller进行Kafka集群管理。
  - 而Kraft模式不依赖ZK集群，用controller节点代替ZK，元数据保存在controller中，由controller直接进行Kafka集群管理



  - 相较于之前的部署方式（传统的ZooKeeper依赖方式），Kafka-Kraft 改进了很多
    - **无 ZooKeeper 依赖**
      - Kafka-Kraft 将 Kafka 的元数据和控制逻辑从 ZooKeeper 中解耦，使得 Kafka 集群不再依赖 ZooKeeper
      - 简化了部署和维护，并减少了对单点故障的依赖。
    - **分布式协调和故障处理**
      - Kafka-Kraft 架构中引入了 Raft 一致性协议，用于分布式协调和处理故障
      - Raft 协议可以更可靠地选举新的 Controller、复制元数据和处理副本故障。这使得 Kafka 集群的状态更加健壮和可靠。
    - **集群扩展和弹性**：
      - Kafka-Kraft 架构允许在不影响现有副本的情况下进行集群扩展，有助于提高 Kafka 的可伸缩性和弹性
      - 新的节点可以简单地加入集群并参与协调和数据复制。

17-对Kafka是否熟悉，有哪些核心概念可以介绍下?

- 题目分析和考点
  - 面试官想确认应聘者是否对kafak是否熟悉，不一定项目中要有到，但是同类解决方案需要知道
  - 关注：Kafka的定义和核心概念，包括生产者、消费者、主题、分区、偏移量等
- 难度【***】
- 答案



注意：自 Kafka 2.4 之后，Kafka 提供了有限度的读写分离，也就是说Follower 副本能够对外提供读服务

为啥之前没有

场景不适用：读写分离适用于那种读负载很大，而写操作相对不频繁的场景，可 Kafka 不属于这样的场景。

同步机制：Kafka 采用 PULL 方式实现 Follower 的同步，因此Follower 与 Leader 存 在不一致性窗口。如果允许读 Follower 副本，就势必要处理消息滞后(Lagging)的问题。

  - Broker
    - Kafka的服务端程序，可以认为一个mq节点就是一个broker
    - broker存储topic的数据
  - Producer生产者
    - 创建消息Message，然后发布到MQ中
    - 该角色将消息发布到Kafka的topic中
  - Consumer消费者:
    - 消费队列里面的消息
  - ConsumerGroup消费者组
    - 同个topic, 广播发送给不同的group，一个group中只有一个consumer可以消费此消息
  - Topic
    - 每条发布到Kafka集群的消息都有一个类别，这个类别被称为Topic，主题的意思
  - Partition分区
    - kafka数据存储的基本单元，topic中的数据分割为一个或多个partition，每个topic至少有一个partition，是有序的

        - 一个Topic的多个partitions, 被分布在kafka集群中的多个server上
        - 消费者数量 <=小于或者等于Partition数量
    - Replication 副本（备胎）
        - 同个Partition会有多个副本replication ，多个副本的数据是一样的，当其他broker挂掉后，系统可以主动用副本提供服务
        - 默认每个topic的副本都是1（默认是没有副本，节省资源），也可以在创建topic的时候指定
        - 如果当前kafka集群只有3个broker节点，则replication-factor最大就是3了，如果创建副本为4，则会报错
    - ReplicationLeader、ReplicationFollower
        - Partition有多个副本，但只有一个replicationLeader负责该Partition和生产者消费者交互
        - ReplicationFollower只是做一个备份，从replicationLeader进行同步
    - ReplicationManager
        - 负责Broker所有分区副本信息，Replication 副本状态切换
    - offset
        - 每个consumer实例需要为他消费的partition维护一个记录自己消费到哪里的偏移offset
        - kafka把offset保存在消费端的消费者组里

18-Kafka的消费者组是什么？如何实现消费者的负载均衡和水平扩展?

- 题目分析和考点
    - 面试官想确认应聘者是否对kafak是否熟悉，对生产环境突发消息过来有没应对策略
    - 消费者群组的概念和工作原理，如何通过添加消费者实现负载均衡和增加吞吐量。
    - 通过创建消费者群组、使用消费者协调机制、负载均衡和增加消费者实例的方式实现 Kafka 消费者的负载均衡和水平扩展
- 难度【* * * * 】
- 答案
    - Kafka消费者组
        - Kafka 的消费者群组是一组消费者实例，它们共同消费 Kafka 主题中的消息，每个消费者群组可以有一个或多个消费者实例
        - 同个topic, 广播发送给不同的group，一个group中只有一个consumer可以消费此消息
    - 基于消费者组可以实现
        - 基于队列的模型：所有消费者都在同一消费者组里，每条消息只会被一个消费者处理
        - 基于发布订阅模型：消费者属于不同的消费者组，假如每个消费者都有自己的消费者组，这样kafka消息就能广播到所有消费者实例上

- 实现消费者的负载均衡和水平扩展主要通过以下几个步骤来完成
  - 创建消费者群组
    - 首先，需要创建一个消费者群组，并为它指定一个名称。这个名称可以在消费者实例中进行配置
  - 消费者协调机制
    - Kafka 使用消费者协调机制来分配分区给消费者实例，确保每个分区只被一个消费者实例消费。
    - 一旦消费者实例加入或离开群组，或者有新的分区创建或分区被重新分配，消费者协调机制将重新进行分区分配。
  - 负载均衡
    - 在消费者群组中，Kafka 会根据分区的数量和消费者实例的数量进行负载均衡
    - Kafka 会尽可能均匀地将分区分配给群组中的每个消费者实例，以确保所有消费者实例都能均衡消费消息
  - 水平扩展
    - 要实现消费者的水平扩展，可以简单地增加消费者实例的数量；
    - 当增加消费者实例时，Kafka 会重新分配分区给新的消费者实例，以确保负载的均衡。
    - 消费者实例通过与协调器进行心跳和重新平衡协议，与 Kafka 保持连接并接收消息。
  - 总结
    - Kafka 是根据分区来进行消息的有序处理。如果要增加消费者并实现更高的吞吐量，需要确保主题有足够多的分区
    - 通过创建消费者群组、使用消费者协调机制、负载均衡和增加消费者实例的方式，实现 Kafka 消费者的负载均衡和水平扩展，以处理大量的消息，并提高系统的可伸缩性和吞吐量

19-Kafka的生产者如何发送消息，消息发送的过程是怎样的?

- 题目分析和考点
  - 面试官想确认应聘者是否对kafak是否熟悉
  - 确定应聘者理解Kafka消息发送的基本工作原理，包括消息发送的到不同分区的机制
  - 生产者发送到broker里面的流程是怎样的呢，一个 topic 有多个 partition分区，每个分区又有多个副本
- 难度【* * * *】
- 答案
  - 生产者到broker发送流程

        - Kafka的客户端发送数据到服务器，不是来一条就发一条，会经过内存缓冲区（默认是16KB）
        - 通过KafkaProducer发送出去的消息都是先进入到客户端本地的内存缓冲里
        - 然后把很多消息收集到的Batch里面，再一次性发送到Broker上去的，这样性能才可能题高
    - 一个 topic 有多个 partition分区，每个分区又有多个副本，具体发送策略是怎样的?
        - 如果指定Partition ID,则PR被发送至指定Partition (ProducerRecord)
        - 如果未指定Partition ID,但指定了Key, PR会按照hash(key)发送至对应Partition
        - 如果未指定Partition ID也没指定Key，PR会按照默认 round-robin轮训模式发送到每个Partition
            - 消费者消费partition分区默认是range模式
        - 如果同时指定了Partition ID和Key, PR只会发送到指定的Partition (Key不起作用，代码逻辑决定)
        - 注意：Partition有多个副本，但只有一个replicationLeader负责该Partition和生产者消费者交互
    - ProducerRecord（简称PR），发送给Kafka Broker的key/value 值对, 封装基础数据信息

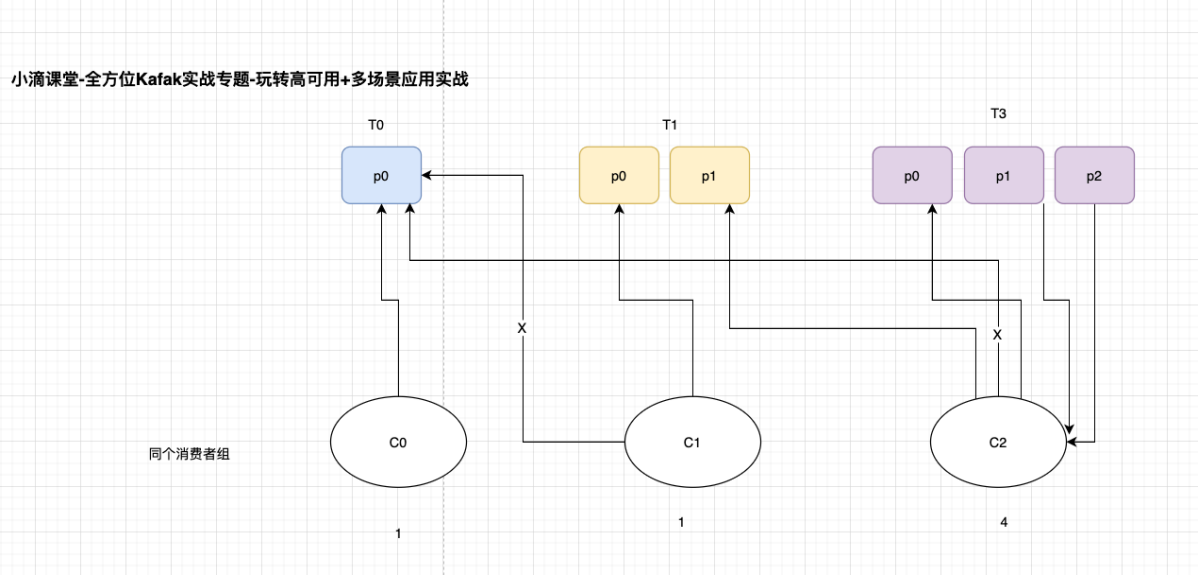
```
/**
 * Creates a record to be sent to a specified topic and partition
 *
 * @param topic The topic the record will be appended to
 * @param partition The partition to which the record should be sent
 * @param key The key that will be included in the record
 * @param value The record contents
 */
public ProducerRecord(String topic, Integer partition, K key, V value) {
    this(topic, partition,  timestamp: null, key, value,  headers: null);
}
```

        - key默认是null，大多数应用程序会用到key
            - 如果key为空，kafka使用默认的partitioner，使用RoundRobin算法将消息均衡地分布在各个partition上
            - 如果key不为空，kafka使用自己实现的hash方法对key进行散列，决定消息该被写到Topic的哪个partition，拥有相同key的消息会被写到同一个partition，实现顺序消息

20-Kafka的消费者是如何接收消息的？消息的消费过程是怎样的?

- 题目分析和考点
    - 面试官想确认应聘者是否对kafak是否熟悉
    - 考察应聘者对消息消费机制的理解，包括消费者组、分区分配和消费的顺序保证。
- 难度【* * * *】
- 答案
    - 消费者根据pull模式从broker获取数据的，为什么是pull模式，而不是broker主动push?

        - 消费者采用 pull 拉取方式，从broker的partition获取数据
        - pull 模式则可以根据 consumer 的消费能力进行自己调整，不同的消费者性能不一样
            - 如果broker没有数据，consumer可以配置 timeout 时间，阻塞等待一段时间之后再返回
        - 如果是broker主动push，优点是可以快速处理消息，但是容易造成消费者处理不过来，消息堆积和延迟。
    - 消费者从哪个分区进行消费?
        - 一个 topic 有多个 partition，一个消费者组里面有多个消费者，那是怎么分配过
            - 一个主题topic可以有多个消费者，因为里面有多个partition分区 (leader分区)
            - 一个partition leader可以由一个消费者组中的一个消费者进行消费
            - 一个 topic 有多个 partition，所以有多个partition leader，给多个消费者消费，那分配策略如何?
    - 消费者从哪个分区进行消费？两个策略
        - round-robin （**RoundRobinAssignor**非默认策略）轮训
            - 【按照消费者组】进行轮训分配，同个消费者组监听不同主题也一样，是把所有的 partition 和所有的 consumer 都列出来， 所以消费者组里面订阅的主题是一样的才行，主题不一样则会出现分配不均问题，例如7个分区，同组内2个消费者
            - topic-p0/topic-p1/topic-p2/topic-p3/topic-p4/topic-p5/topic-p6
            - c-1: topic-p0/topic-p2/topic-p4/topic-p6
            - c-2:topic-p1/topic-p3/topic-p5
            - 弊端
                - 如果同一消费者组内，所订阅的消息是不相同的，在执行分区分配的时候不是轮询分配，可能会导致分区分配的不均匀
                - 有3个消费者C0、C1和C2，他们共订阅了 3 个主题：t0、t1 和 t2
                - t0有1个分区(p0)，t1有2个分区(p0、p1)，t2有3个分区(p0、p1、p2))
                - 消费者C0订阅的是主题t0，消费者C1订阅的是主题t0和t1，消费者C2订阅的是主题t0、t1和t2



- range （**RangeAssignor**默认策略）范围
  - 【按照主题】进行分配，如果不平均分配，则第一个消费者会分配比较多分区， 一个消费者监听不同主题也不影响，例如7个分区，同组内2个消费者
  - topic-p0/topic-p1/topic-p2/topic-p3/topic-p4/topic-p5//topic-p6
  - c-1: topic-p0/topic-p1/topic-p2/topic-p3
  - c-2:topic-p4/topic-p5/topic-p6
  - 弊端
    - 只是针对 1 个 topic 而言，c-1多消费一个分区影响不大
    - 如果有 N 多个 topic，那么针对每个 topic，消费者 C-1 都将多消费 1 个分区，topic越多则消费的分区也越多，则性能有所下降

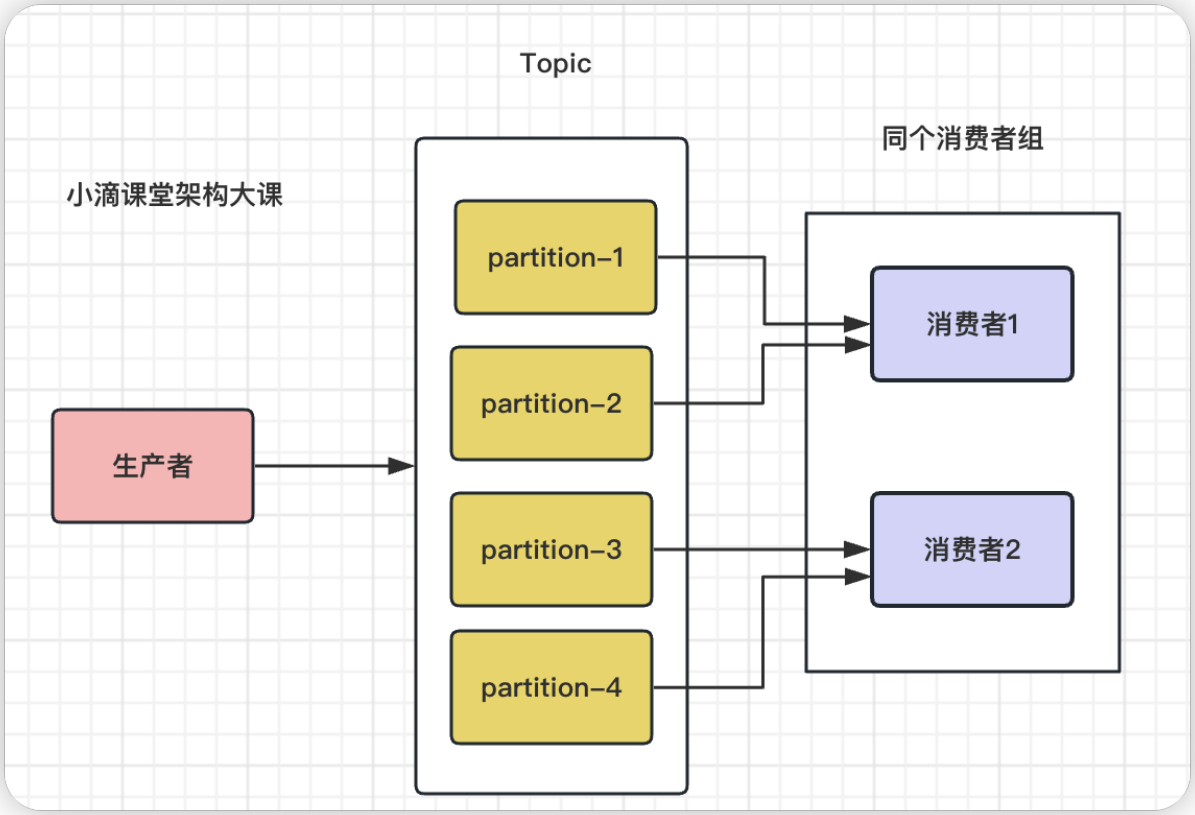
21-Kafka的主题有10个分区2个消费者，再启动一个消费者会发生什么?

- 题目分析和考点
  - 面试官想确认应聘者是否对kafak的Rebalance熟悉，重平衡是 Kafka 消费者群组中一种常见的机制
  - 它确保消费者实例之间的负载均衡，并在消费者实例的加入或离开、或主题分区变化时进行动态调整
  - 开发者应该了解重平衡的工作原理和注意事项，以确保消费者群组的可靠运行和高性能消费
- 难度【* * *】
- 答案
  - 会发生重平衡Rebalance操作
    - 重平衡（Rebalance）是消费者群组中消费者实例在加入或退出群组时重新分配分区的过程
    - 重平衡的主要目的是确保消费者群组中的每个消费者实例都能够平均分配分区，实现负载均衡
    - kafka 会均匀地分配某个 topic 下的所有 partition 到各个消费者，从而使得消息的消费速度达到最快，
    - 这就是重平衡（balance），其实就是重新进行 partition 的分配，从而使得 partition 的分配重新达到平衡状态
  - 下面也都会发生rebalance操作
    - 新的消费者实例加入群组
      - 当有新的消费者实例加入消费者群组时，Kafka 会触发重平衡，重新分配分区给新的消费者实例。
    - 消费者实例离开群组
      - 当消费者实例从消费者群组中退出时，Kafka 会触发重平衡，重新分配其处理的分区给其他消费者实例。
    - 主题分区变化

        - 当 Kafka 集群中的主题的分区发生变化时（增加或减少分区），Kafka 会触发重平衡，重新分配分区给消费者实例
    - 注意：
        - 确保消费者实例的唯一性
            - 为了确保重平衡成功，每个消费者实例在消费者群组中必须具有唯一的 ID
            - 使用相同的消费者组 ID 将被视为相同的消费者实例，这将导致分区的重分配。
        - 限制群组内消费者数量变化
            - 频繁的加入或退出消费者实例可能导致频繁的重平衡，对 Kafka 集群造成额外的开销
            - 因此，在应用程序设计时，最好避免频繁地加入和离开群组。

22-说下生产环境-Kafka海量消息堆积解决方案你会采用哪些?

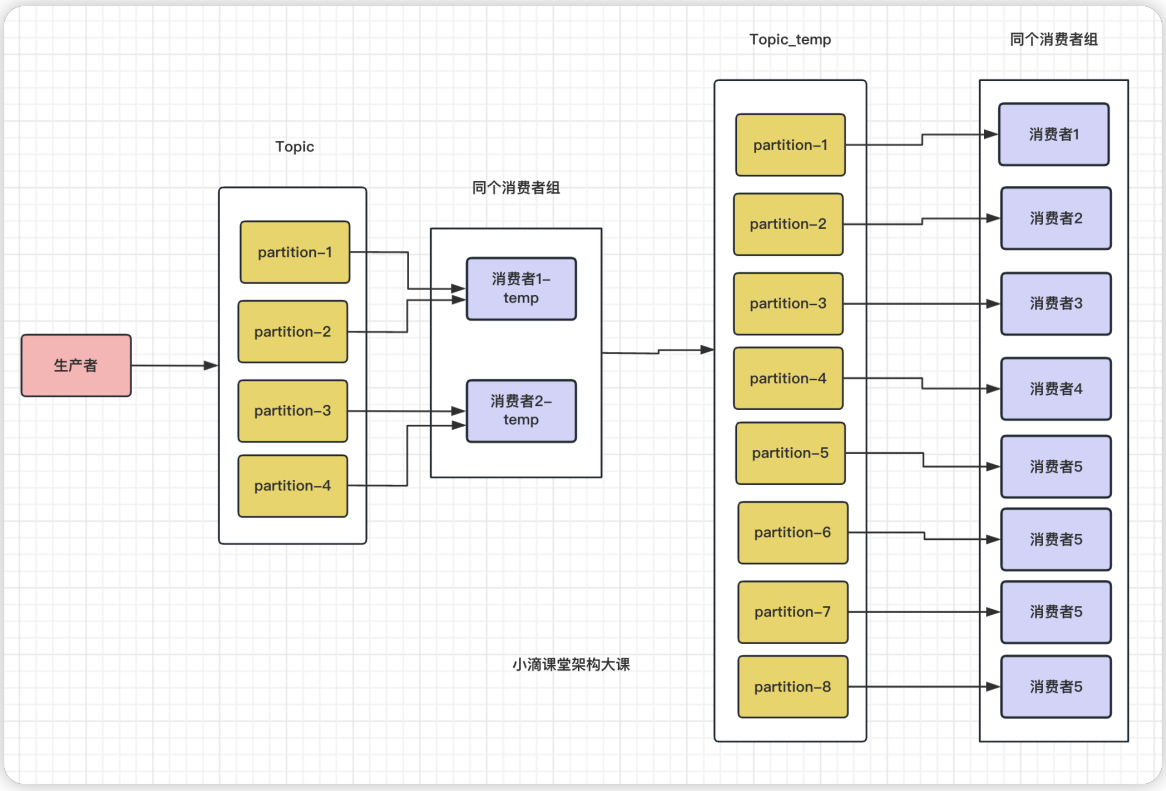
- 题目分析和考点
    - 面试官想确认应聘者是否对kafak的消费模型知道哪些
    - 在业界不同MQ消费模型里面，解决消息堆积的方案是不一样的，有些是通用的，有些是有差异
    - 需要根据具体的场景和需求选择合适的解决方案，通常结合多种方法和策略来应对 Kafka 中的海量消息堆积问题
- 难度【*****】
- 答案
    - Kafka里面发生消息堆积的常见分析思路和解决方案
        - 识别消息堆积的原因和选择合适的解决方案
            - 通过监控Kafka Broker和消费者的指标，确定消息堆积发生在哪个部分，是在生产者端还是消费者端。
            - 根据堆积的原因选择相应的解决方案，如增加消费者、增加分区、调整消费者端配置



- 场景解决方案（**不管哪种方案，都需要避免消费者的重复消费**）
  - 增加消费者数量
    - 通过增加消费者的数量来提高消息的消费速度
    - 可以启动更多的消费者实例，并将它们添加到消费者组中，前提是 **现有Topic的分区数 > 消费者数**
  - 增加分区数
    - 通过增加Kafka主题的分区数来增加消息的并行处理能力。
    - 增加分区数将允许更多的消费者并行消费消息，并减少每个分区上的消息堆积量。
    - 注意，增加分区数可能会导致消息重新分配和消费者组的重新平衡
  - 临时队列分发
    - 通过将现有积压的Topic分发到另外一个支持更多分区的临时Topic
    - 启动更多消费者消费临时的Topic完成消息积压的处理
  - 提高消费者端的处理能力：
    - 检查消费者端的处理逻辑，确保消费者能够高效地处理消息
    - 优化消费者的代码，减少处理逻辑的复杂性，提高消息的处理速度。
    - 例如
      - 调整消费者的最大拉取数据量（fetch.max.bytes）和拉取间隔（fetch.min.bytes）
      - 批量处理的消息数量（max.poll.records）等，以提高消费者的吞吐量。
  - 增加Kafka Broker的数量

        - 如果消息堆积是由于Kafka Broker的性能瓶颈导致的，可以考虑增加Kafka Broker的数量来提高整个集群的处理能力。
    - 扩展硬件资源
        - 如果堆积的原因是由于硬件资源的限制，例如磁盘空间不足或网络带宽受限
        - 可以通过增加硬盘容量、更换高性能磁盘、增加网络带宽等方式扩展硬件资源。

- Kafka里面消息堆积解决方案之一：临时Topic分发方案, 其他方案可以自行验证
    - 问题
        - 由于本身消息队列设计的特性，没法直接增加消费者进行快速提升消费者能力
        - 一个Topic里面有多个partition分区，一个分区只能给一个消费者组内的一个消费者消费
        - 假如有个3个分区，同个消费者组有3个消费者，那刚好每个消费者可以消费一个分区
    - 解决方案（临时分发扩容，适合比较多MQ场景）
        - 如果堆积的消息需要比较久才可以消费完成，则可以建立一个临时Topic，建立多个分区



        - 例子
            - Topic的分区数量是3个，消费者宕机了24小时，里面堆积了60万消息未消费
            - 假如一个消费者的能力是2000/分钟，那消费完成则需要300分钟 即5小时；
            - 5个消费者则需要1小时，10个消费者则30分钟
            - 直接提升消费者数量为10个，但是分区只有3个，则7个消费者是消费不到消息
            - 步骤
                - 需要新建一个临时队列，分区配置为15个

```
    public static final String TOPIC_NAME_TEMP = "xdclass-shop-
order-v10-temp";


    @Bean
    public NewTopic topicTemp(){
        return TopicBuilder.name(TOPIC_NAME_TEMP)
                .partitions(15)
                .compact()
                .build();
    }
```

- 把现有的消费者逻辑修改，直接消费消息投递到新的临时队列里面

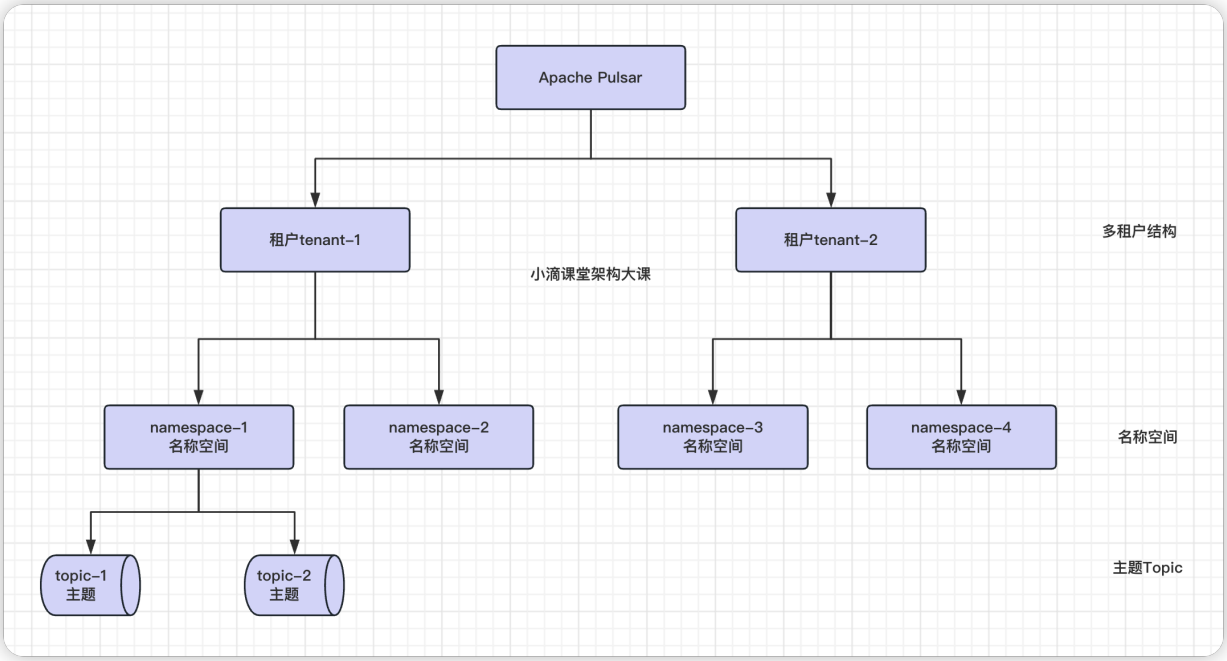
```
@KafkaListener(topics = {KafkaConfig.TOPIC_NAME},groupId =
"xdlcass-test-gp-v1")
public void onMessage(ConsumerRecord<?, ?> record, Acknowledgment
ack, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic){
   // 打印出消息内容
   System.out.println("消费："+record.topic()+"-分区
="+record.partition()+"-内容="+record.value());


   String uuid = UUID.randomUUID().toString().replace("-","");
   //分发临时队列
   kafkaTemplate.send(KafkaConfig.TOPIC_NAME_TEMP, "xdclass-
key"+uuid, record.value());


   ack.acknowledge();
}
```

- 然后启动更多临时的消费者去消费临时Topic

```
@KafkaListener(topics = {KafkaConfig.TOPIC_NAME_TEMP},groupId =
"xdlcass-test-gp-v1")
public void onMessage(ConsumerRecord<?, ?> record, Acknowledgment
ack, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic){
        // 打印出消息内容
        System.out.println("消费：“+record.topic()+"-分区
="+record.partition()+"-内容="+record.value());
        try {
            System.out.println("模拟写数据库 IO慢的操作");
            TimeUnit.SECONDS.sleep(1);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        ack.acknowledge();
    }
```

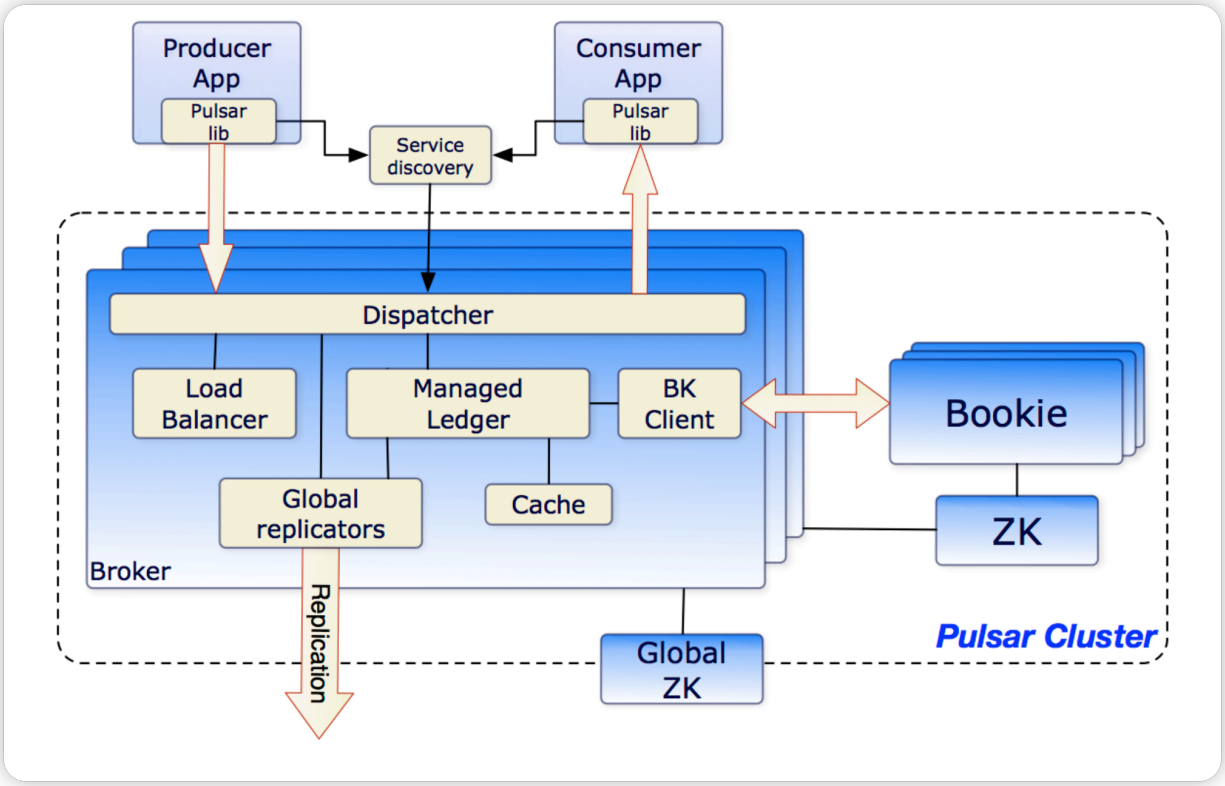
23-有没了解Aapche Pulsar 说下常见的概念和特点

- 题目分析和考点
  - 面试官想确认应聘者是否对新技术是否有掌握
  - Pulsar是一个高性能、可靠性和可扩展性的分布式流处理平台，解决大规模实时数据传输和流式处理的问题
  - 具有多租户支持、Schema管理和持久性等强大的核心特点，适用于各种大规模数据流处理的应用场景
  - 目前也是很多大厂在采用的消息订阅系统
- 难度【***】
- 答案
  - 常见概念

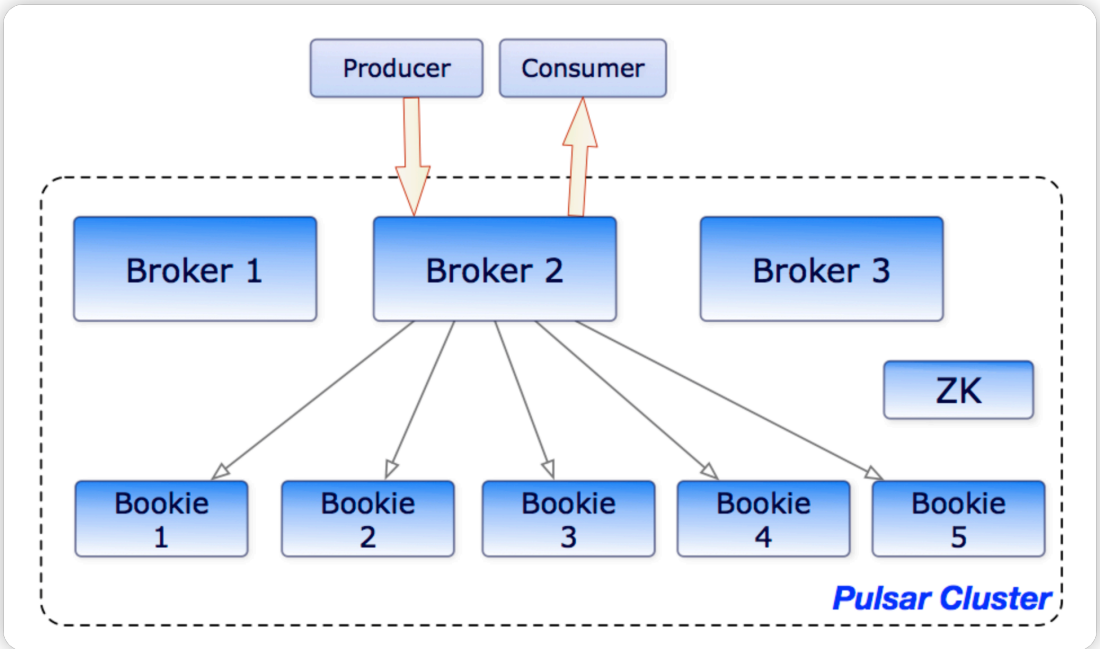


- 主题与命名空间
  - Pulsar使用主题（Topic）来组织和隔离消息。主题具有唯一的名称，可以动态创建并按需进行扩展。
  - 主题按照命名空间（Namespace）进行组织，命名空间提供逻辑上的隔离和资源分配。
- 消息发布和订阅：
  - 生产者（Producer）通过指定主题将消息发布到Pulsar。
  - 消费者（Consumer）订阅感兴趣的主题，并从中接收实时的消息。
- 分区和持久订阅：
  - 主题可以划分为多个分区（Partition），每个分区在Broker上都有自己的副本。
  - 消费者可以选择订阅特定分区的消息，以实现更高的并发处理和负载均衡。
  - Pulsar支持持久订阅，消费者可以持久化保存其消费的位置，以便在重连后继续消费未处理的消息。

- Pulsar包含三个核心组件：Broker、BookKeeper和ZooKeeper



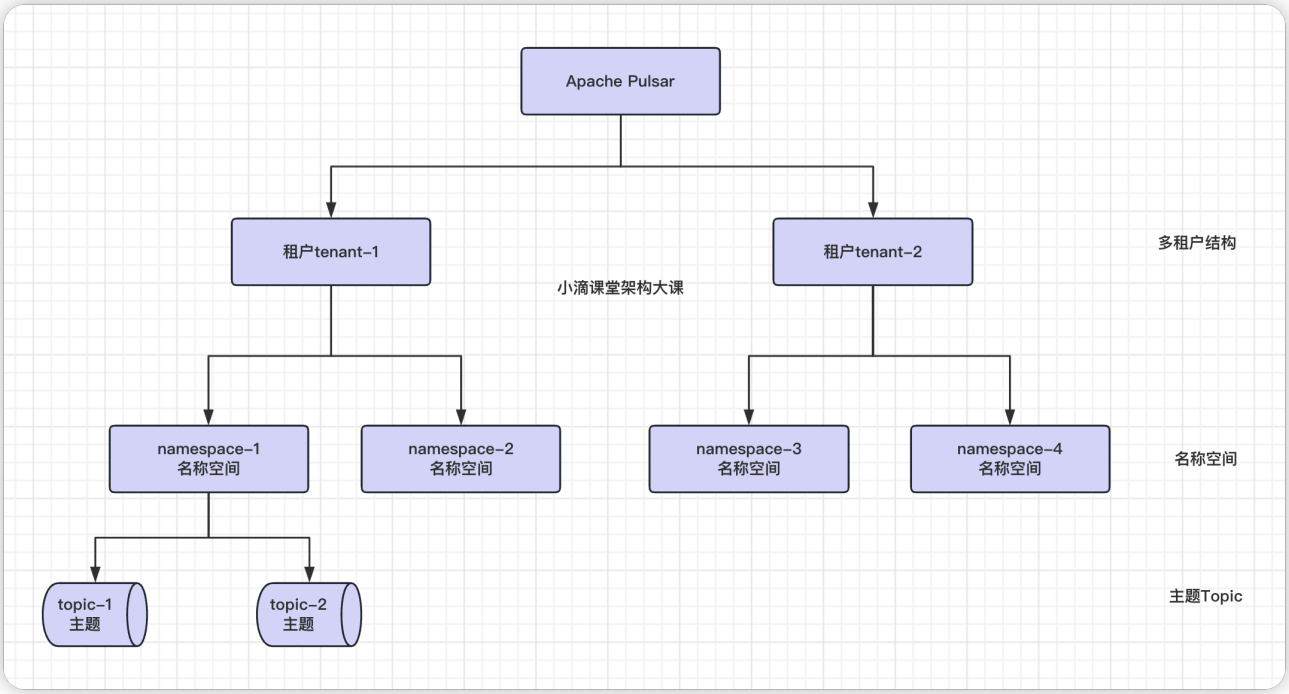
- Broker
  - Broker 是 Pulsar 的核心组件，负责接收、路由和处理来自生产者的消息， 是一个无状态组件，主要负责运行两个组件
    - 一个 HTTP 服务器（Service discovery），暴露REST 系统管理接口以及在生产者和消费者之间 Topic 查找的 API
    - 一个调度分发器（Dispatcher），它是异步的 TCP 服务器，通过自定义二进制协议应用于所有相关的数据传输
  - 还负责维护关于主题的元数据信息，如订阅信息和消费者群组信息。
  - Broker 是可扩展的，可以在多个节点上部署，以实现高可用和负载均衡。
  - 生产者可以将消息发送到特定的 Broker，消费者可以从特定的 Broker 订阅消息
  - Broker 之间可以通过 Pulsar 的自动消息复制机制来实现数据的冗余备份。
- BookKeeper（又被称为bookie）
  - BookKeeper 是 Pulsar 的持久化存储组件，用于保证消息的可靠存储,就是一个分布式预写日志系统WAL
  - 使用 Write-Ahead Log (WAL) 的方式来持久化消息，确保在 Broker 故障时，消息的可靠性和一致性也能得到保证。
  - Broker 使用 BookKeeper 来存储消息数据和元数据，以确保数据的持久性和高性能
  - 每个主题分区本质上都是存储在 Apache BookKeeper 中的分布式日志



- ZooKeeper
  - 是 Apache Pulsar 的协调服务，用于管理和协调 Pulsar 集群的元数据和状态信息。
  - Pulsar 使用 ZooKeeper 来存储和跟踪主题的元数据、Broker 的可用状态和配置信息等。
  - ZooKeeper 提供了分布式锁、选举和协调机制，以确保在集群中的各个节点之间实现一致性和可靠性。

24-说下Pulsar的租户Tenants和命名空间Namespaces使用场景

- 题目分析和考点
  - 面试官想确认应聘者是否对新技术是否有掌握
  - Pulsar是一个高性能、可靠性和可扩展性的分布式流处理平台，解决大规模实时数据传输和流式处理的问题
  - 具有多租户支持和名称空间进行不同业务隔离，目前也是很多大厂在采用的消息订阅系统
- 难度【 * * * 】
- 答案



- **在Apache Pulsar中，租户（Tenants）和命名空间（Namespaces）都用于组织和隔离消息，具有不同的作用和功能**
  - 租户（Tenants）
    - 租户是Pulsar内部多租户架构的一部分，允许多个独立的应用程序或团队在同一个Pulsar集群上运行。
    - 多用户环境下使用同一套程序，不同用户之间 做好数据资源隔离
    - 租户是Pulsar的最高层级的逻辑概念，用于隔离和管理不同的应用程序或团队。
    - 租户可以包含一个或多个命名空间，而且不同租户之间的资源是相互隔离的，如主题、消费者组等。
    - 租户通过分配独立的身份和访问凭证，可以进行身份验证和授权，以限制对资源的访问。
  - 命名空间（Namespaces）
    - 命名空间是逻辑上的容器，用于组织和隔离不同的主题，可以根据应用程序、团队或功能来划分
    - 命名空间是在租户内进行更细粒度的组织和隔离消息的单元。
    - 命名空间在逻辑上用来划分不同的业务场景、功能模块或应用程序。
    - 命名空间允许创建和管理主题，并提供更详细的权限和配额设置。
    - 在命名空间内可以创建分区、生产者和消费者组，并进行消息的发布和订阅
- 总结
  - 租户提供了更高层级的隔离和管理机制，**用于区分和管理不同的应用程序或团队**
  - 而命名空间则在租户内提供了更细粒度的组织和隔离消息的能力，**用于划分不同的业务场景或应用程序**
  - 通过租户和命名空间进行隔离和管理，Pulsar支持多租户共享集群，确保多个应用程序或团队之间的资源隔离和安全性

Spring框架

1-Spring的bean 属性scope作用域有哪些，默认是哪个

- **题目分析和考点**
  - 面试官想确认应聘者对Spring框架的理解和对Bean的生命周期管理的熟悉程度。
  - Spring的Bean作用域的理解以及在实际开发中的应用经验，对不同作用域的了解和适用场景
- **难度【*】**
- **答案**
  - 单例（Singleton）
    - 单例作用域是默认的Bean作用域，表示在整个应用程序中只有一个实例存在
    - 多次获取该Bean时，将返回同一个实例。
  - 原型（Prototype）
    - 原型作用域表示每次获取该Bean时，Spring都会创建一个新的实例
    - 即每次请求都返回一个新的Bean实例。
  - 会话（Session）
    - 会话作用域表示在一个会话（session）期间（通常为Web应用中的用户会话），该Bean实例是唯一的。
    - 不同会话中将有不同的实例。
  - 请求（Request）
    - 请求作用域表示在一个HTTP请求过程中，该Bean实例是唯一的
    - 即每个请求都会有一个新的实例。
  - 全局会话和全局请求（Global session和Global request）
    - 这两个作用域适用于Portlet应用，类似于会话和请求作用域，但是生命周期更长

2-说下Spring事务的传播属性和隔离级别，默认是哪个

- **题目分析和考点**
  - 面试官想确认应聘者对Spring事务管理的理解、熟悉不同事务传播属性和隔离级别的能力以及在实际项目开发中的运用。
  - 在回答时，可以提及使用@Transactional注解或编程式事务管理来配置事务属性
  - 结合具体应用场景说明事务传播属性和隔离级别的选择原则和注意事项
- **难度【* * *】**
- **答案**
  - 事物传播行为
    - 如果在开始当前事务之前，一个事务上下文已经存在，此时有若干选项可以指定一个事务性方法的执行行为
    - @Transactional(propagation=Propagation.REQUIRED)
      - 如果有事务, 那么加入事务, 没有的话新建一个(默认情况下)
    - @Transactional(propagation=Propagation.NOT_SUPPORTED)
      - 不为这个方法开启事务
    - @Transactional(propagation=Propagation.REQUIRES_NEW)
      - 不管是否存在事务,都创建一个新的事务,原来的挂起,新的执行完毕,继续执行老的事务
    - @Transactional(propagation=Propagation.MANDATORY)
      - 必须在一个已有的事务中执行,否则抛出异常
    - @Transactional(propagation=Propagation.NEVER)
      - 必须在一个没有的事务中执行,否则抛出异常(与Propagation.MANDATORY相反)
    - @Transactional(propagation=Propagation.SUPPORTS)
      - 如果其他bean调用这个方法,在其他bean中声明事务,那就用事务.如果其他bean没有声明事务,那就不用事务.
    - @Transactional(propagation=Propagation.NESTED)
      - 如果当前存在事务，则创建一个事务作为当前事务的嵌套事务来运行；
      - 如果当前没有事务，则该取值等价于Propagation.REQUIRED。
  - 事务隔离级别: 是指若干个并发的事务之间的隔离程度
    - @Transactional(isolation = Isolation.READ_UNCOMMITTED) 读取未提交数据(会出现脏读, 不可重复读) 基本不使用
    - @Transactional(isolation = Isolation.READ_COMMITTED) 读取已提交数据(会出现不可重复读和幻读)
    - @Transactional(isolation = Isolation.REPEATABLE_READ) 可重复读(会出现幻读)
    - @Transactional(isolation = Isolation.SERIALIZABLE) 串行化
  - **MYSQL: 默认为REPEATABLE_READ级别**

3-介绍下@Conditional注解的作用和使用场景

- **题目分析和考点**
  - 面试官想确认应聘者对Spring条件装配的理解和在配置和组件装配中的应用经验。
  - @Conditional注解用于根据特定的条件进行Bean的条件装配，即根据一定条件决定是否创建某些Bean。
  - Bean定义上添加@Conditional注解后，Spring会根据所标注的条件类的条件评估结果来决定是否创建该Bean或配置类。
- **难度【***】**
- **答案**
  - 什么是@Conditional注解
    - Spring4.0之后才有的，放在任意类型或者方法上
    - 通过@Conditional配置一些条件判断，当所有条件都满足时，被该@Conditional注解标注的目标才会被Spring处理
      - 例如根据当前环境、系统属性、配置文件等条件来决定是否注册某个Bean或执行某个组件
    - 应用场景
      - 在某个特定的环境下，需要注册一个特定的Bean，常用的是当bean不存在的时候才注册
      - 根据配置文件中的某个属性来决定是否注册Bean
      - 根据环境选择配置类，比如当前系统的操作系统类型、版本等条件来决定是否要执行
  - 案例：当spring容器中不存在某个bean A时，才注册这个bean B到容器里面

```
public class DataSource {
    public String name;
    public String url;
    //set get toString省略
}


@Configuration
public class DataSourceBeanManager {
    @Bean("source-1")
    public DataSource dataSource1(){
        return new DataSource("source-1","xdclass.net");
    }
    @Bean("source-2")
    public DataSource dataSource2(){
        return new DataSource("source-2","baidu.com");
```

```
    }
}


//获取全部某个类型全部对象
public static void test2(AnnotationConfigApplicationContext context){
        Map<String, DataSource> beansOfType =
context.getBeansOfType(DataSource.class);
        System.out.println(beansOfType);
}
```

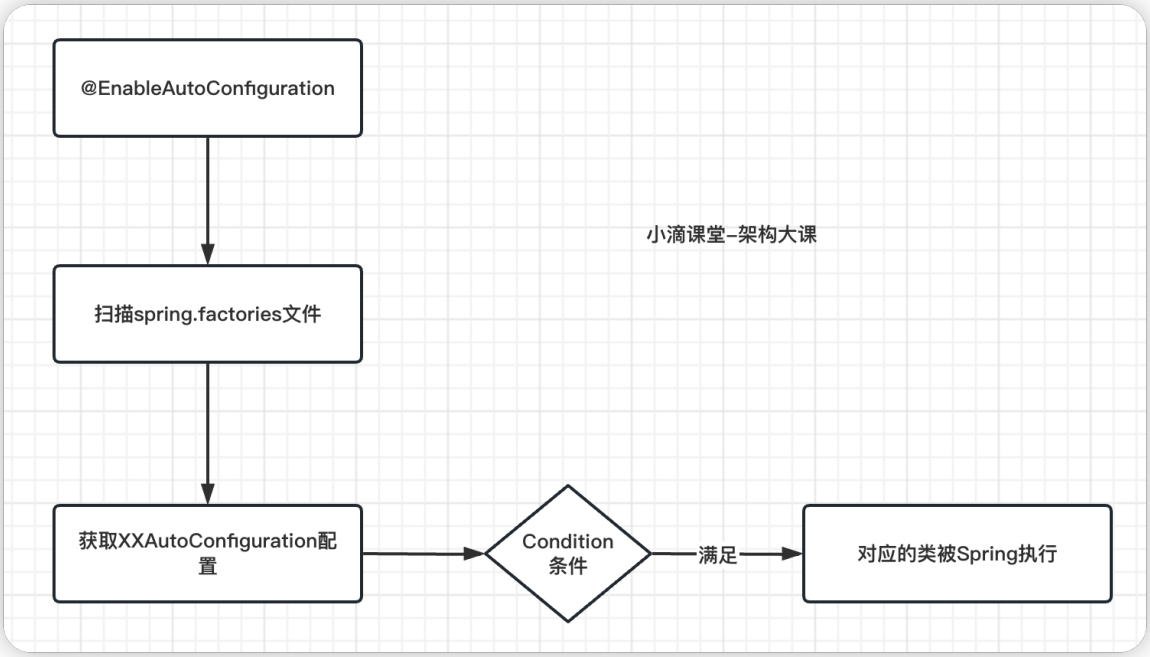
4-列举下你知道的SpringIOC容器加入对象的方式

- **题目分析和考点**
  - 面试官想确认应聘者对了解IOC容器的概念，知道它是Spring框架的核心组件，负责管理和维护对象的生命周期和依赖关系
  - 熟悉Spring IOC容器中加入对象的不同方式，能够基于不同的情况和需求选择合适的方式
  - 回答这个问题时，可以列举对应的方式并简要解释每种方式的适用场景和操作步骤
- **难度【* * *】**
- **答案**
  - Spring的IOC容器里面加入对象的常见方式
    - 使用XML配置文件
      - 在XML配置文件中使用< bean >标签来定义Bean，通过ClassPathXmlApplicationContext等容器来加载并初始化Bean。
    - 使用注解
      - 使用Spring提供的注解，例如@Component、@Service、@Controller、@Repository等注解来标识Bean
      - 然后通过@ComponentScan等注解扫描器来加载并初始化Bean。
    - 使用Java配置
      - 可以使用Java代码来配置Bean，例如使用@Configuration和@Bean注解来定义Bean
      - 然后通过AnnotationConfigApplicationContext等容器来加载并初始化Bean。
    - 使用Import注解

        - 使用@Import注解来引入其他配置类，然后通过容器加载并初始化Bean
        - @Import注解提供了三种用法
            - 直接Import导入
            - ImportSelector接口批量
            - ImportBeanDefinitionRegistrar条件注册

5-介绍下新版SpringBoot3.X如何自定义Starter

- **题目分析和考点**
    - 面试官想确认应聘者对Spring Boot自定义Starter的了解和在实际开发中的应用经验
    - Spring Boot的自定义Starter是一种组件化的方式，用于简化应用开发中的配置和依赖管理
    - 能够解释在项目中为什么需要自定义Starter以及如何使用和扩展现有的自定义Starter
- **难度【* * *】**
- **答案**
    - **关于新版Spring Boot3.X和旧版SpringBoot2.7之前自定义Starter区别**
        - SpringBoot2.7之前
            - META-INF/spring.factories文件里添加`org.springframework.boot.autoconfigure.EnableAutoConfiguration=XXAutoConfiguration`



        - SpringBoot2.7推出新的自动配置
            - 在META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
            - 文件里添加配置类名称，每行包含一个配置类全限定名

        - 兼容META-INF/spring.factories方式
    - SpringBoot3.x 移除spring.factories
        - 移除META-INF/spring.factories方式
        - 只支持META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports 增加自动配置

- SpringBoot3.X自定义Starter步骤
    - 创建项目 xx-spring-boot-starter
    - 添加依赖

```
 <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-autoconfigure</artifactId>
            <version>3.0.2</version>
</dependency>
```

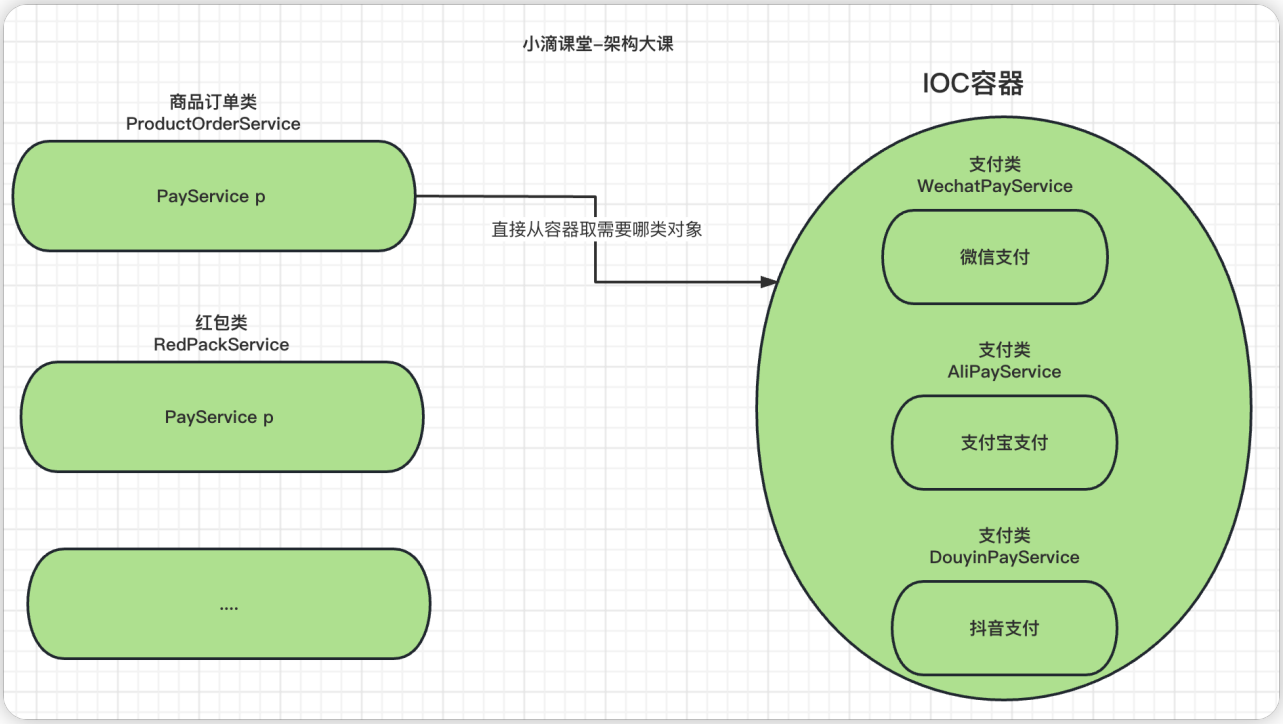
    - 创建配置类 XXAutoConfiguration类
        - 增加注解`@ConfigurationProperties(prefix = "xdclass.sms")`
        - 增加Condition条件注解
    - 创建自动配置文件，配置AutoConfiguration.imports自动配置类

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

6-什么是Spring的IOC和DI，他们之间的联系和区别是怎样的

- **题目分析和考点**
    - 面试官想确认应聘者对Spring核心概念的理解和使用
    - Spring的IOC和DI是Spring框架的核心概念和特性，理解这两个概念对于掌握和应用Spring框架至关重要
    - 回答这个问题时，可以先对IOC和DI的概念进行简要解释，阐述它们之间的关系和区别，结合项目经验展示自己的应用
- **难度【* * *】**
- **答案**

- 什么是IoC
  - **Inversion of Control 控制-反转,属于一个技术思想，不止Spring框架有具体实现，其他框架也有**
  - **是一种解耦方式，系统更容易维护和升级**
  - 对象的创建、管理、销毁等过程不再由开发人员自行控制，而是由容器来帮助我们管理
  - 容器负责从配置文件中读取bean的定义以及对bean的创建和销毁等过程
  - 开发人体只需在容器中配置好Bean的定义，容器便会管理Bean的生命周期，完成相应的操作
  - 为啥叫【控制反转】
    - 控制: 指的对象创建、管理的工作
    - 反转: 想工作交给外部环境，比如 spring框架的IOC容器

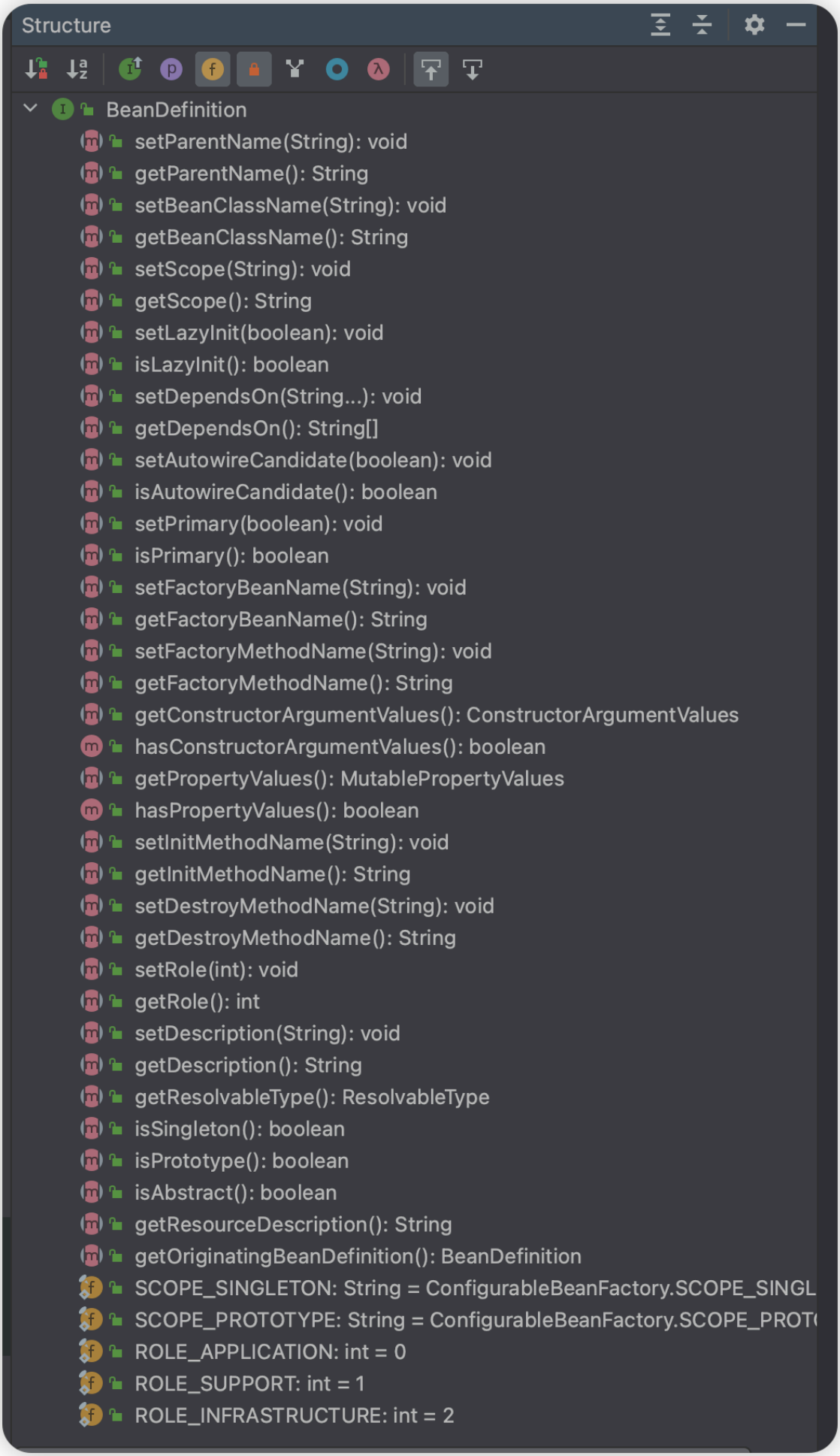


- 什么是DI
  - Dependancy Injection 依赖注入
  - 在IoC的基础上，容器自动将一些对象的依赖关系注入到另一个对象中，也就是一个对象被注入了它所依赖的对象
  - 在一个web应用程序中，控制器类（Controller）依赖于数据访问对象（DAO）和业务逻辑对象（Service）
  - 使用DI技术，可以将DAO和Service注入到Controller中， Controller中就可以轻松的使用DAO和Service中的方法。
  - DI主要有三种注入方式：
    - Setter Injection(Setter注入)
    - Constructor Injection（构造函数注入）
    - Field Injection（字段注入）
- 总结

        - IOC和DI描述的是同一件事情，就是角度不一样
            - IOC是在对象的角度，把对象的管理创建交给了IOC容器
            - DI是在容器的角度，容器把对象里面需要依赖的其他对象注入进去

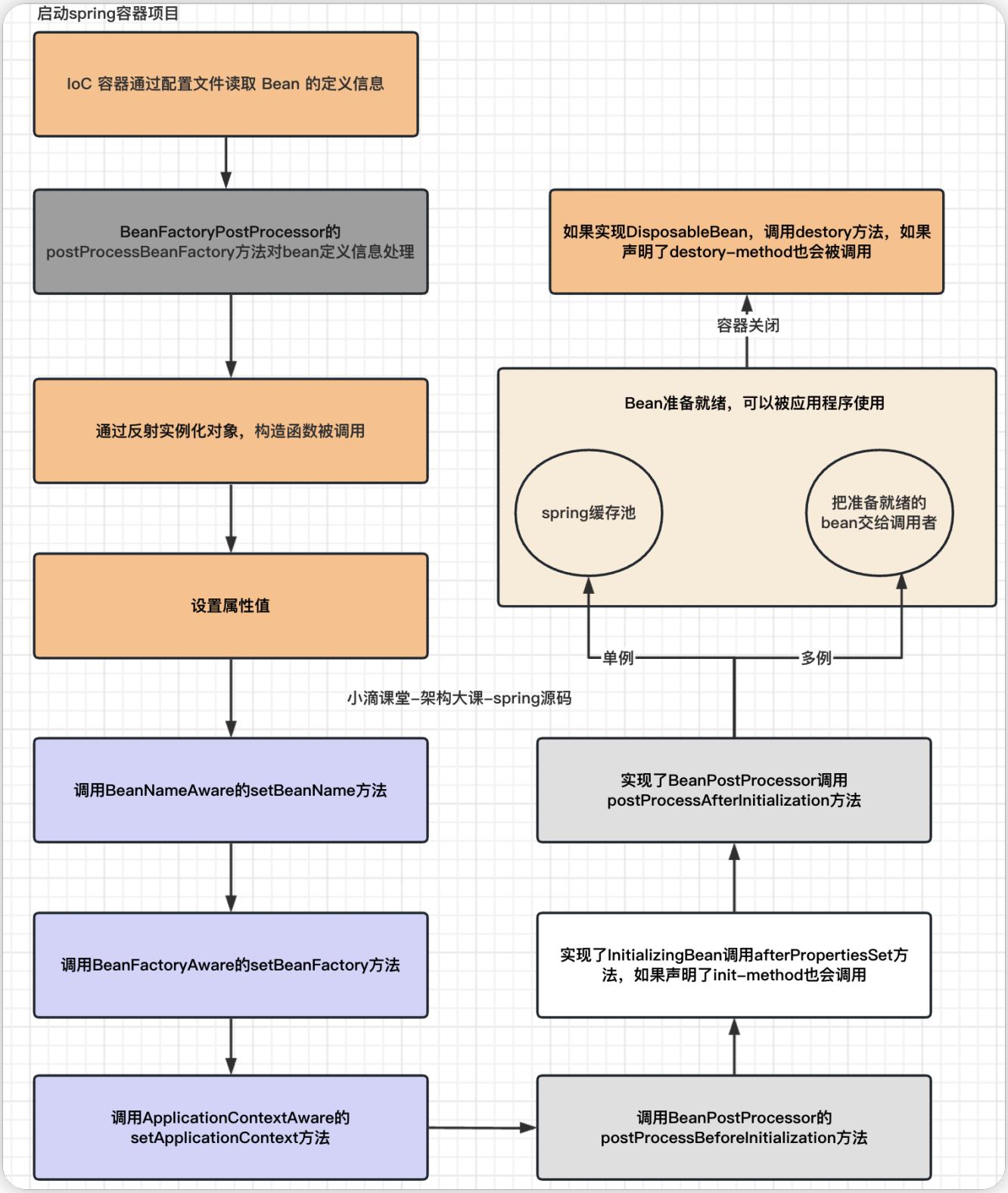
7-介绍下BeanDefinition的作用和使用场景有哪些

- **题目分析和考点**
    - 面试官想确认应聘者对Spring框架的深入理解和在实践中使用BeanDefinition的经验。
    - BeanDefinition作为Spring框架中的重要概念之一，对于理解Spring容器的工作原理以及实现自定义扩展都非常重要
    - 可以通过实现或扩展BeanDefinition接口，定义自己的BeanDefinition实现类，以适应特定的需求和场景
- **难度【* * *】**
- **答案**
    - BeanDefinition 介绍
        - 是是Spring容器中最重要的概念之一，它是容器创建和管理Bean实例的基础，对Bean的定义信息的抽象和封装
        - 描述一个Bean的定义信息，包括Bean的名称、类型、作用域、属性等信息
        - 可以对Bean的创建和管理进行详细的配置和控制，例如可以指定Bean的作用域、是否懒加载、是否自动注入等属性
        - 在 XML 中定义的 bean标签，被Spring 解析成为一个 JavaBean， 这个JavaBean 就是 BeanDefinition


Structure
BeanDefinition
setParentName(String): void
getParentName(): String
setBeanClassName(String): void
getBeanClassName(): String
setScope(String): void
getScope(): String
setLazyInit(boolean): void
isLazyInit(): boolean
setDependsOn(String...): void
getDependsOn(): String[]
setAutowireCandidate(boolean): void
isAutowireCandidate(): boolean
setPrimary(boolean): void
isPrimary(): boolean
setFactoryBeanName(String): void
getFactoryBeanName(): String
setFactoryMethodName(String): void
getFactoryMethodName(): String
getConstructorArgumentValues(): ConstructorArgumentValues
hasConstructorArgumentValues(): boolean
getPropertyValues(): MutablePropertyValues
hasPropertyValues(): boolean
setInitMethodName(String): void
getInitMethodName(): String
setDestroyMethodName(String): void
getDestroyMethodName(): String
setRole(int): void
getRole(): int
setDescription(String): void
getDescription(): String
getResolvableType(): ResolvableType
isSingleton(): boolean
isPrototype(): boolean
isAbstract(): boolean
getResourceDescription(): String
getOriginatingBeanDefinition(): BeanDefinition
SCOPE_SINGLETON: String = ConfigurableBeanFactory.SCOPE_SINGL
SCOPE_PROTOTYPE: String = ConfigurableBeanFactory.SCOPE_PROT
ROLE_APPLICATION: int = 0
ROLE_SUPPORT: int = 1
ROLE_INFRASTRUCTURE: int = 2

8-简单说下Spring Bean的生命周期核心方法调用栈

- **题目分析和考点**
  - 面试官想确认应聘者对Spring Bean生命周期的了解程度，以及是否掌握在实际开发中应用或扩展Bean生命周期的经验
  - Bean的生命周期是Spring框架的重要组成部分，理解Bean生命周期的方法调用顺序对于实现自定义操作、管理等非常重要
  - 介绍Bean生命周期的核心方法调用，强调Bean的实例化、属性注入、初始化和销毁等关键阶段
  - 并结合具体的项目经验说明自己对Bean生命周期的理解和应用
- **难度【*****】**
- **答案**
  - Spring Bean的生命周期由一系列的核心方法组，调用栈可以简化为下面的顺序
    - 实例化：通过构造函数或工厂方法创建Bean实例。
    - 属性注入：通过依赖注入将Bean的属性值或引用注入到Bean实例中。
    - 实现Aware接口：调用Bean实现的Aware接口方法，如实现BeanNameAware的setBeanName()方法，可以获取Bean的名称。
    - 初始化前回调：调用BeanPostProcessor接口的postProcessBeforeInitialization()方法，可以在Bean初始化前进行自定义操作。
    - 初始化：如果Bean实现了InitializingBean接口，调用它的afterPropertiesSet()方法；如果在配置中指定了init-method，调用它指定的初始化方法。
    - 初始化后回调：调用BeanPostProcessor接口的postProcessAfterInitialization()方法，可以在Bean初始化后进行自定义操作。
    - 销毁前回调：如果Bean实现了DisposableBean接口，调用它的destroy()方法；如果在配置中指定了destroy-method，调用这个指定的销毁方法。

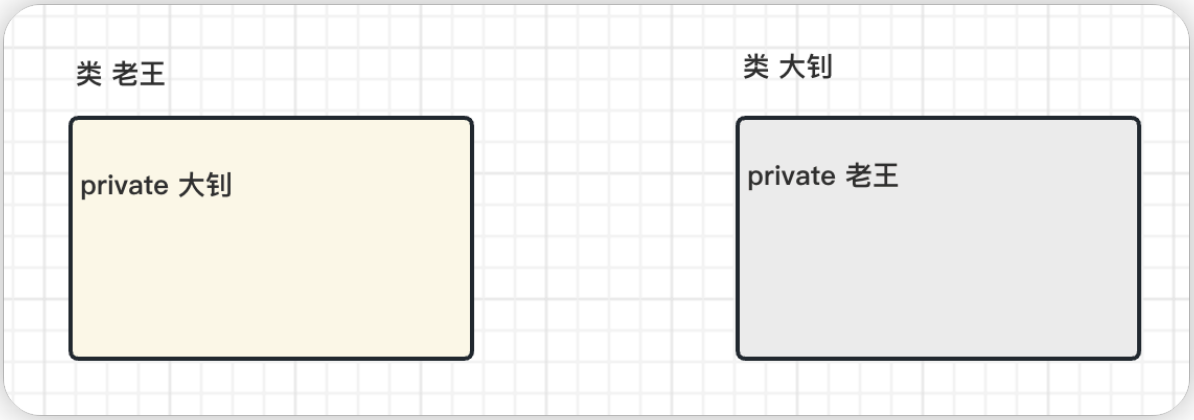


9-详细说下Spring是如何解决循环依赖问题的

- 题目分析和考点

- 面试官想知道应聘者对Spring框架原理的深入理解和解决问题的能力
- 理解Spring是如何解决循环依赖问题能够体现对Spring框架原理和机制的了解
- 循环依赖指的是多个Bean之间相互依赖，形成一个环状依赖关系
- 回答问题可以先简要解释循环依赖的概念，然后描述Spring解决循环依赖的机制和具体步骤，强调三级缓存的作用和重要性
- 同时可以结合具体的项目经验或问题解决经验来说明自己对该问题的理解和实践能力

- **难度【*****】**
- **答案**
  - 循环依赖
    - 指的是两个或多个Bean之间相互依赖，如Bean A依赖于Bean B，而Bean B又依赖于Bean A，则会出现循环依赖的情况
    - Spring默认采用Singleton模式，Bean默认是单例的，在容器启动时就会创建Bean实例，同时也会注入它所依赖的Bean实例
    - 当出现循环依赖的情况时，Spring的Bean实例化策略就会出现问题
    - 大家开发过程中好像对循环依赖这个概念无感知，这种错觉是因为工作在Spring的中，已经帮你解决了

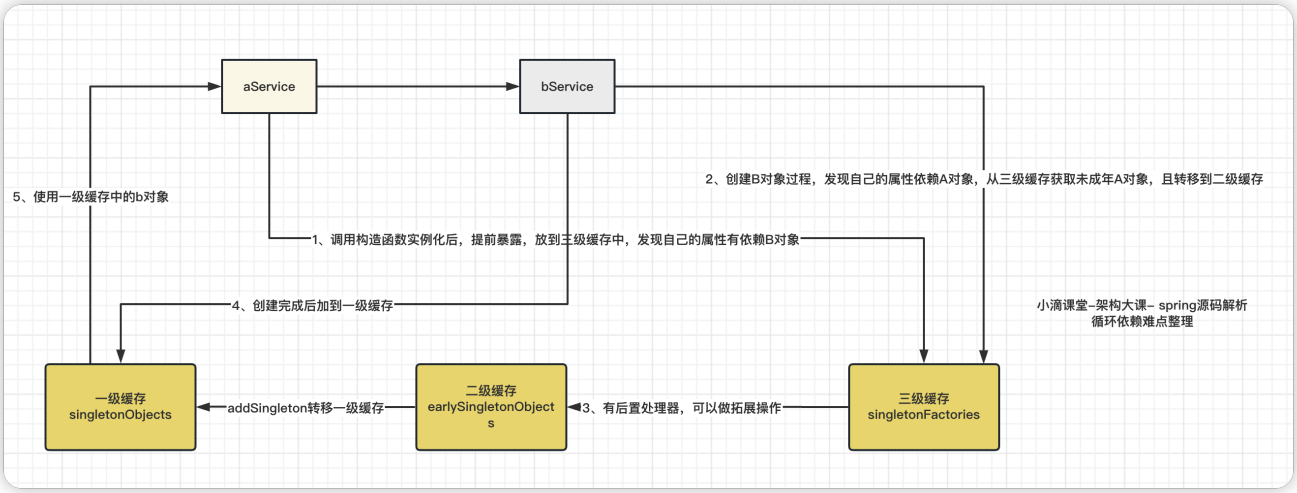


  - 解决方案
    - Spring引入了“提前暴露Bean”的机制，在创建A对象时，会先创建一个A的空对象并将其添加到缓存池中
    - 即“提前暴露Bean”，然后继续创建B对象，将其注入A对象中
    - 在创建B对象时，由于A对象已经在缓存池中，可以直接获取到A对象，接着将B对象注入到A对象中，完成Bean的初始化
    - 注意：循环依赖的场景
      - 单例
        - **构造器循环依赖（构造器注入，循环依赖无法解决，日常开发不推荐构造器注入）**
        - **Field属性循环依赖（set方法注入，循环依赖问题可以解决,通过三级缓存）**
      - 多例
        - **prototype类型bean（循环依赖无法解决，构造函数和属性注入都不能解决循环依赖）**

- `AbstractBeanFactory#doGetBean`

```
// 如果正在创建一个原型实例，则抛出异常，表示当前单例实例与其存在循环依赖
if (isPrototypeCurrentlyInCreation(beanName)) {
    throw new BeanCurrentlyInCreationException(beanName);
}
```

- 循环依赖流程图



- 三级缓存，当Spring创建Bean的过程中，使用3个缓存存储已经创建的Bean实例，Map结构
  - 一级缓存（成熟的bean） Singletons object cache
    - `singletonObjects` 单例池，存储已经创建好的无代理的单例Bean对象，从该缓存中取出的 bean 可以直接使用
  - 二级缓存 Early-singletons object cache（**巧记：early 拼音 二**）
    - `earlySingletonObjects` 提前曝光的单例对象的cache，存放原始的 bean 对象（尚未填充属性），用于解决循环依
  - 三级缓存 Singleton Factory object cache
    - `singletonFactories` 单例对象工厂的cache，存放 bean 工厂对象，解决循环依赖 避免Bean的实例化顺序出现问题。

- 总结
  - Spring的三级缓存机制是为了避免Spring中的循环依赖问题而引入的
  - 也可以提高Bean的创建效率，避免创建重复的对象，从而提高应用程序的性能

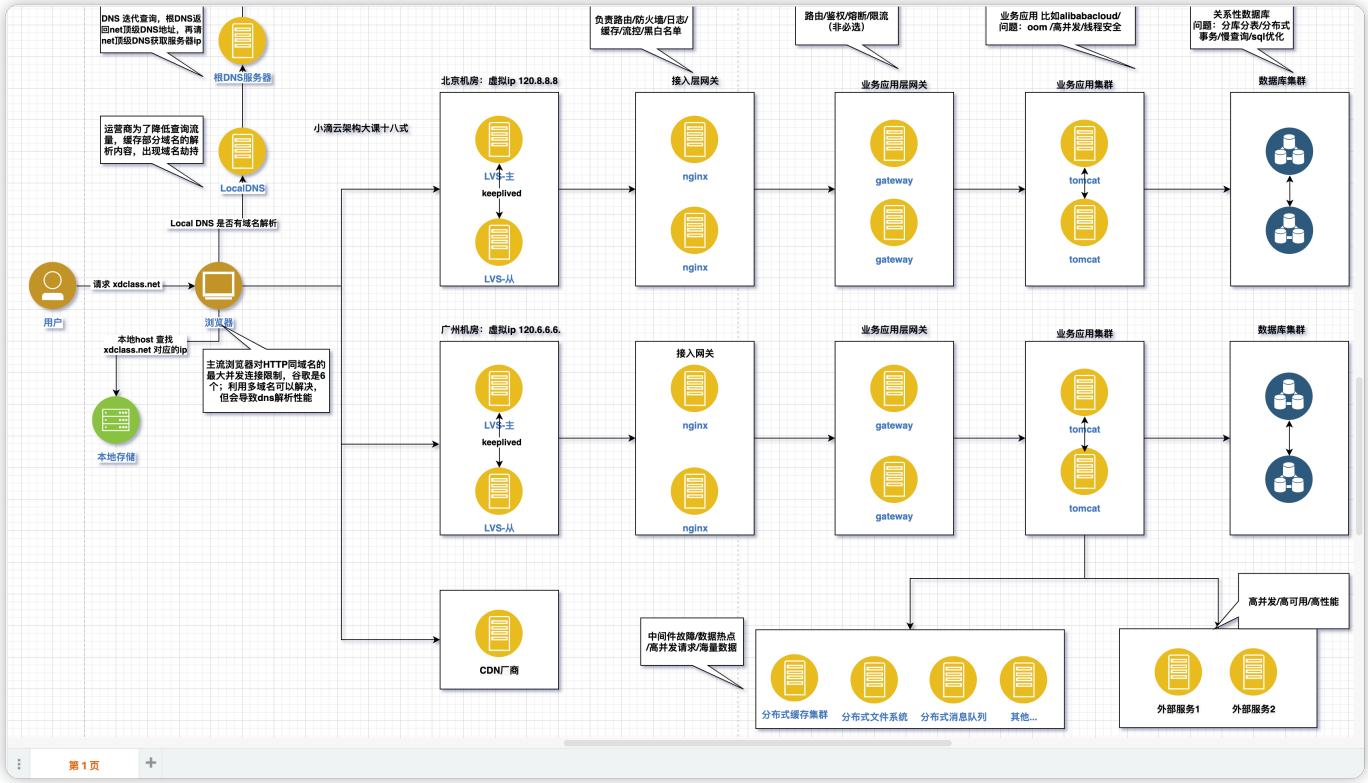
分布式缓存板块

1-什么是缓存，项目中如何选择用本地缓存和分布式缓存

- **题目分析和考点**
  - 面试官想确认应聘者对缓存技术的理解和在不同场景下做出合适选择的能力
  - 可以从性能需求、数据一致性、可靠性和容错性、部署和管理等方面综合考虑，并权衡各种优缺点来选择适合的缓存策略
- **难度【* * *】**
- **答案**
  - 什么是缓存
    - 程序经常要调用的对象存在内存中,方便其使用时可以快 速调用,不必去数据库或者其他持久化设备中查询
    - 主要就是提高性能 ， 常见有： DNS缓存、前端缓存、代理服务器缓存Nginx、应用程序缓存、数据库缓存
  - 缓存分类
    - 分布式缓存
      - 与应用分离的缓存组件或服务，与本地应用隔离一个独 立的应用，多个应用可直接的共享缓存
      - 常见的分布式缓存 Redis、Memcached等
    - 本地缓存
      - 和业务程序一起的缓存，例如myabtis的一级或者二级 缓存，本地缓存自然是最快的，但是不能在多个节点共享
      - 常见的本地缓存
        - myabtis 一级、二级缓存；Guava Cache、Caffeine
  - 本地缓存和分布式缓存如何选择
    - 本地缓存
      - 适用于数据处理稳定的情况，缓存的生命周期较短，而且缓存的数据量比较小
      - 例如
        - 在Web服务器或应用程序中使用本地缓存来提高响应速度
        - 缓存一些不变的静态文件，以避免每次请求都需要重新加载。
    - 分布式缓存
      - 适用于大规模分布式系统，缓存的数据量很大，需要快速查询和更新数据
        - 例如，可以使用分布式缓存来缓存常用的数据库查询结果，从而避免频繁查询数据库，提高响应速度
        - 当需要分别多个服务共享数据时，使用分布式缓存也更为方便
    - 结论：高并发项目里面一般都是有本地缓存和分布式缓存共同存在的

2-描述下常见微服务的一个HTTP请求经过的全链路缓存

- **题目分析和考点**
  - 面试官想确认应聘者对微服务架构和缓存的理解，并查看其对全链路缓存的设计与实现能力
  - 可能问及如何保持缓存与后端数据的一致性，并处理缓存失效情况
- **难度【* * * *】**
- **答案**
  - 系统访问全链路缓存，则是特别多样化的，俗称【多级缓存】,具体项目具体分析，只是列举多数情况
    - 操作系统缓存、本地缓存（浏览器缓存，APP缓存）CDN缓存、反向代理缓存、应用本地缓存、分布式缓存、数据库缓存..
  - 关注
    - 80%性能提升
      - 本地缓存
      - 反向代理缓存
      - 应用本地缓存
      - 分布式缓存
      - 其他...
    - 20%性能提升
      - CDN缓存
      - 操作系统缓存
      - 数据库缓存
      - 其他...



3-举例下Redis里面常见的数据结构和应用场景

- **题目分析和考点**
  - 面试官想确认应聘者对Redis 知识广度和深度，以及他们对数据结构和其应用场景的理解
  - 能够描述每种数据结构的特点和优势，并给出相应的应用场景示例即可
- **难度【* * *】**
- **答案**
  - String(字符串)
    - 计数器，incr/decr，比如商品库存的扣减
    - 业务数据key-value缓存，缓存结果数据，提高网站性能、减少数据库DB压力
    - 分布式session会话解决方案，集群环境下存储token鉴权信息
    - 分布式锁，集群下保证线程安全

  - List（列表）
    - 简易版消息队列或栈结构
      - 通过lpush和rpop命令实现快速插入和删除消息。生产者向左边推送消息，消费者从右边弹出消息
      - 在很多实时性较高、安全性要求不高的场景中，消息队列可以存储在Redis中，以实现较快的读写操作和策略控制
    - 最近联系人列表
      - 社交应用业务领域里面，保留每个用户最近联系的一些人
      - 使用Redis的List来实现，将用户ID按照最近联系的时间逆序插入List中，超出限定的数量则从List中删除
    - 历史记录和操作日志
      - 用户的历史操作或者操作结果，每个用户的历史操作都可以存储在一个List中
      - 使用LPUSH和RPUSH命令在List头部插入历史记录，使用LTRIM命令保留最近的一些历史记录
    - 批量查询数据分页存储
      - 一次查询100条数据，存储list，后续直接查找list

  - Set(集合)
    - 用户画像、产品标签集合存储、文章有多个标签，可以将对应的标签存储在不同的Set中
    - 社交应用里面的共同的好友、共同关注、计算不同用户之间的交集、并集
    - 抽奖小程序
      - **SRANDMEMBER key [count]**，如果命令执行时，只提供了 `key` 参数，那么返回集合中的一个随机元素

        - 注意：**SPOP 随机元素从集合中移除并返回，而 SRANDMEMBER 则仅仅返回随机元素，而不对集合进行任何改动**
    - 网站独立ip统计
- Sorted Set(有序集合)
    - 根据分数，实时排行榜，如果商品热销、视频热播、热门文章
    - 优先级任务队列，将任务的优先级作为Sorted Set中的分值

- hash(哈希)
    - 对象存储
        - Hash结构存储对象的所有属性信息，例如存储一个人的姓名、性别、年龄等信息
        - 这些属性名可以作为Hash表中的键，属性值可以作为Hash表中的值
        - 在存储大量对象时，使用Hash结构可以提高数据读写效率
        - 对比java数据结构 `Map< String , Map< Object, Object> >`
        - 案例
            - 存储用户的基本信息，例如用户名、电子邮件、地址等
            - HSET命令向Hash中添加或修改用户信息，使用HGETALL命令获取用户的所有信息或者使用HGET命令获取单个信息
    - 计数器系统
        - 可以使用Hash结构实现计数器系统
        - 例如网站上的点赞和评论数，每个文本、图片或者视频等支持点赞和评论的内容ID作为Hash表中的键
        - 点赞和评论的次数则作为Hash表中的值
        - key=文章id、field=评论、value=值，field=点赞、value=值
    - 电商里面的购物车存储结构 (自营电商平台，中小电商平台)
        - 第一层Map，Key是用户id
        - 第二层Map，Key是购物车中商品id，值是购物车数据

- Bitmap(位存储)
    - 极小的内存占用下对大量的二进制数据进行存储和处理
    - 布隆过滤器
        - 使用Bitmap来实现布隆过滤器算法，用于判断一个元素是否在一个集合中
        - 将元素通过Hash算法转换为多个二进制位，并将其对应的二进制位设置为1
    - 去重操作
        - 可以使用Bitmap来进行重复数据的去重操作，将需要去重的数据的Hash值作为Bitmap的键
        - 将数据的ID号或标识作为Bitmap中对应的二进制位，1表示已经存在该数据，0表示未存在该数据

        - 延伸：日活/月活**UV**统计、签到统计、用户点赞，用户签到，访问计数，在线用户数等

4-场景题-如何实现基于位置的附近商家推荐系统，类似美团

- **题目分析和考点**
    - 面试官想确认应聘者对Redis 知识广度和深度，以及他们对数据结构和其应用场景的理解
    - 如何使用空间索引结构（如 R 树、Quadtree 或 Geohash）来优化位置查询的性能
- **难度【* * * * *】**
- **答案**
    - 使用Redis的GEO数据结构(地理位置)
        - Redis 3.2版本之后新增的一个数据类型，是一种用于处理地理位置信息的数据结构
        - 存储和查询地理位置数据，并快速地计算距离和位置集合的交集
        - **不是新的底层数据结构，而是直接使用了 Sorted Set 集合类型**
            - Key member score 结构，对应的权重 score 是经纬度信息
            - 对经度和纬度编码，把经纬度各自的编码组合成一个最终编码
        - 应用场景
            - 位置服务
                - 可以使用GEO结构存储用户或商家的位置信息，并计算用户或商家之间的距离
            - 商业分析
                - 可以使用GEO结构在地图上可视化商家的分布状况，以进行市场分析和营销策略的制定
            - 推荐系统
                - 可以使用GEO结构计算用户与商家之间的距离，以实现根据附近商家推荐和推荐商家位置排名等功能
                - 用于实现拼团和卡券等代表性做法，诸如可以实现商家对距离自己较近的用户进行自动优惠券发放等
        - 其他解决方案
            - Mysql+程序计算
            - MongoDB
            - Elastic Search
    - 常见API，官方文档：https://redis.io/commands/?group=geo
        - geoadd：增加某个地理位置的坐标
            - 语法 `GEOADD key longitude latitude member`
        - geopos：获取某个地理位置的坐标
            - 语法 `GEOPOS key [member [member ...]]`
        - geodist：获取两个地理位置的距离

        - 语法 `GEODIST key member1 member2 [M | KM | FT | MI]`
        - 范围单位：m | km | ft | mi --> 米 | 千米 | 英尺 | 英里
    - georadius：根据给定地理位置坐标获取指定范围内的地理位置集合
        - 语法 `GEORADIUS key longitude latitude radius <M | KM | FT | MI>`
    - georadiusbymember：根据给定地理位置获取指定范围内的地理位置集合。
        - 语法 `GEORADIUSBYMEMBER key member radius <M | KM | FT | MI>`
    - geohash：获取某个地理位置的geohash值
        - 语法 `GEOHASH key [member [member ...]]`
- 思路
    - 一个用户想要在他附近找到沐足店，使用Redis的GEO结构
    - 首先将所有沐足店商家的位置信息（经纬度）添加到Redis中
    - 并使用GEOADD命令将沐足店商家的位置信息添加到GEO数据集中
    - 然后使用GEORADIUS命令，以用户当前位置为中心，找出离用户最近的沐足店
    - 这样用户就可以实时查看附近的沐足店商家，选择离自己最近的商家，从而提高用户的满意度和用户消费量

5-Redis性能为啥那么快，常规读写QPS有多高

- **题目分析和考点**
    - 面试官想确认应聘者对Redis 的内部工作原理和线程模型的了解，并查看他们对 Redis 的性能及并发处理能力的认识
    - 能够说明 Redis 的高性能特点，并理解 Redis 内部如何通过内存存储、单线程模型、高效的数据结构和网络模型来实现高速读写
- **难度【* * *】**
- **答案**
    - Redis 将数据保存在内存中，因此可以高效地读取和写入数据，缩短了数据存取中磁盘操作带来的时间成本。
    - 支持多种数据结构，支持对这些数据的原子操作，原子操作避免了多个操作的竞争和数据冲突，减少了锁的使用
    - 采用非阻塞和多路复用IO，服务端一个线程处理多个请求，避免了线程上下文切换和IO阻塞
    - 性能
        - 常规读写 QPS 的性能指标，因具体的部署环境和硬件条件的不同而有所差异
        - 可以轻松处理百万级别的并发读请求，写请求每秒数万到10万+写性能，集群部署可以进一步提升
        - 官方数据

        - Redis单例能处理key:2.5亿个
        - 一个key或value大小最大是512M
    - 常规瓶颈：一般机器的内存和带宽，而不是CPU

6-分布式缓存Redis是单线程还是多线程?

- **题目分析和考点**
    - 面试官想确认应聘者对Redis 的内部工作原理和线程模型的了解，对 Redis 的性能及并发处理能力的认识
    - 回答一定要基于版本，不然都是没意义的，不同版本的redis在不同模块是有多线程和单线程区分
- **难度【* * *】**
- **答案**
    - redis4.X~6.X 工作线程是单线程，但是整个服务来说也是多线程，比如 rdb备份、主从同步、异步删除也是独立线程完成
        - 对于 **Big Key删除 ，不推荐直接del key，会阻塞主线程 ,而应该采用 unlink key，异步删除**
        - redis4.X引入的 unlink key 和 flushall async ，异步删除（交给外包小弟）
    - redis6.X开始多线程只是用来处理网络数据的读写和协议解析上，但底层数据操作还是单线程
    - 执行命令仍然是单线程，这么设计是不想因为多线程而变得复杂 ，需要去控制 key、lua、事务，LPUSH/LPOP 等等的并发问题
    - 默认不开启

      ```
      io-threads-do-reads yes
      io-threads 线程数
      ```

    - 官方建议 (线程数小于机器核数)
        - 4 核的机器建议设置为 2 或 3 个线程
        - 8 核的建议设置为 4或6个线程，
    - 开启多线程后，是否会存在线程并发安全问题?
    - 不会有安全问题，Redis 的多线程部分只是用来处理网络数据的读写和协议解析，执行命令仍然是单线程顺序执行。

7-说下Redis的key过期时间删除策略你知道多少?

- **题目分析和考点**
  - 面试官想确认应聘者对Redis 的过期策略和内部机制的了解，在设计和优化缓存系统时的能力
  - Redis 提供了两种主要的 Key 过期删除策略，分别是`定时删除（定期删除）`和`惰性删除`
  - 定时删除和惰性删除各有优缺点，需要知道对应的优缺点
- **难度【 * * * 】**
- **答案**
  - Redis服务器使用的是惰性删除和定期删除两种策略
  - 通过配合使用这两种删除策略，服务器可以很好地在合理使用CPU时间和避免浪费内存空间之间取得平衡。
    - 定期删除：
      - 隔一段时间，就随机抽取一些设置了过期时间的 key，检查其是否过期，如果过期就删除，
      - 定期删除可能会导致很多过期 key 到了时间并没有被删除掉，那咋整呢，所以就是惰性删除
    - 惰性删除 ：
      - 概念：当一些客户端尝试访问它时，key会被发现并主动的过期
      - 放任键过期不管，但是每次从键空间中获取键时，都检查取得的键是否过期，如果过期的话，就删除该键
  - 问题
    - 如果定期删除漏掉了很多过期 key，然后你也没及时去查，也就没走惰性删除，此时会怎么样?
    - 如果大量过期 key 堆积在内存里，导致 redis 内存块耗尽了，就需要走内存淘汰机制

8-Redis内存只能用10G，往里面写了20G的数据会发生什么?

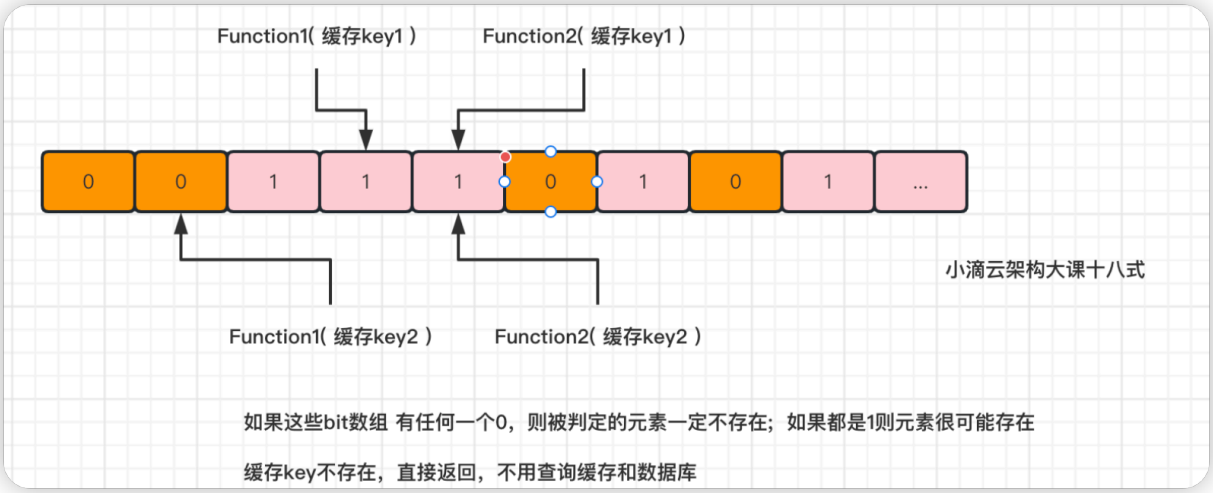
- **题目分析和考点**
  - 面试官想确认应聘者对 Redis 的内存管理和容量限制的了解，以及对 Redis 内存溢出和数据丢失的处理能力
  - Redis 提供了多种策略来处理内存溢出的情况。例如可以配置 LRU策略或淘汰算法来在插入新数据时删除旧数据保持内存使用
  - 如果 Redis 内存已满且没有启用持久化机制，写入新数据时可能会发生数据丢失
  - 因为 Redis 在内存溢出时默认的行为是拒绝写入新数据，而不会自动删除旧数据。
- **难度【 * * * * 】**
- **答案**
  - redis在占用的内存超过指定的maxmemory之后
  - 通过maxmemory_policy确定redis是否释放内存以及如何释放内存
  - 提供多种策略

        - volatile-lru(least recently used)
            - 最近最少使用算法，从设置了过期时间的键中选择空转时间最长的键值对清除掉；
        - volatile-lfu(least frequently used)
            - 最近最不经常使用算法，从设置了过期时间的键中选择某段时间之内使用频次最小的键值对清除掉；
        - volatile-ttl
            - 从设置了过期时间的键中选择过期时间最早的键值对清除 (删除即将过期的)
        - volatile-random
            - 从设置了过期时间的键中，随机选择键进行清除；
        - allkeys-lru
            - 最近最少使用算法，从所有的键中选择空转时间最长的键值对清除；
        - allkeys-lfu
            - 最近最不经常使用算法，从所有的键中选择某段时间之内使用频次最少的键值对清除；
        - allkeys-random
            - 所有的键中，随机选择键进行删除；
        - noeviction
            - 不做任何的清理工作，在redis的内存超过限制之后，所有的写入操作都会返回错误；但是读操作都能正常的进行;

9-说下什么是缓存雪崩/缓存穿透/缓存击穿和解决方案

- **题目分析和考点**
    - 面试官想确认应聘者对 Redis缓存相关的常见问题和解决方案的了解
    - 简要解释每个问题的定义和原因，并列举一些针对每个问题的常见解决方案
- **难度【 * * * * 】**
- **答案**
    - 缓存雪崩
        - 现象
            - 多个热点key都过期或Redis宕机
            - 大量的key设置了相同的过期时间，导致在缓存在同一时刻全部失效，造成瞬时DB请求量大、压力骤增，引起雪崩
        - 解决方案
            - 搭建高可用集群环境 ，分布式缓存组合本地缓存使用
            - 存数据的过期时间设置随机，防止同一时间大量数据过期现象发生
            - 设置热点数据永远不过期，定时任务定时更新

- 缓存穿透(查询不存在数据)
  - 现象
    - 查询一个不存在的数据，由于缓存是不命中的，如发起为id为“-1”不存在的数据
    - 如果从存储层查不到数据则不写入缓存，导致这个不存在的数据每次请求都要到存储层去查询
    - 大量查询不存在的数据，可能DB就挂掉了，是黑客利用不存在的key频繁攻击应用的一种方式
  - 解决方案
    - 接口层增加校验，数据合理性校验
    - 缓存取不到的数据，在数据库中也没有取到，将key-value对写为key-null，设置短点过期时间，防止同个key被一直攻击
    - 布隆过滤器（类似白名单）
      - 将所有要【缓存的数据】经过处理后存储布隆过滤器中，即对应的bit上是1
      - 当外部请求发起时，首先会把请求的参数 通过哈希算法处理，获得相应的哈希值;
      - 根据哈希值计算出位数组中的位置，如果全部计算的hash值对于的bit存储都是1，则表示数据在合理中
      - 再从缓存读出，缓存失效则从数据库中取出，少数误判不影响，可以把误判的value 设置null, 加上短期过期时间
      - 如果计算的hash值对应的bit存储存在一个是0或以上，表示数据不合理，直接返回数据不存在，不查缓存和数据库
      - 如果布隆过滤器认为值不存在，那么值一定是不存在的，无需查询缓存也无需查询数据库



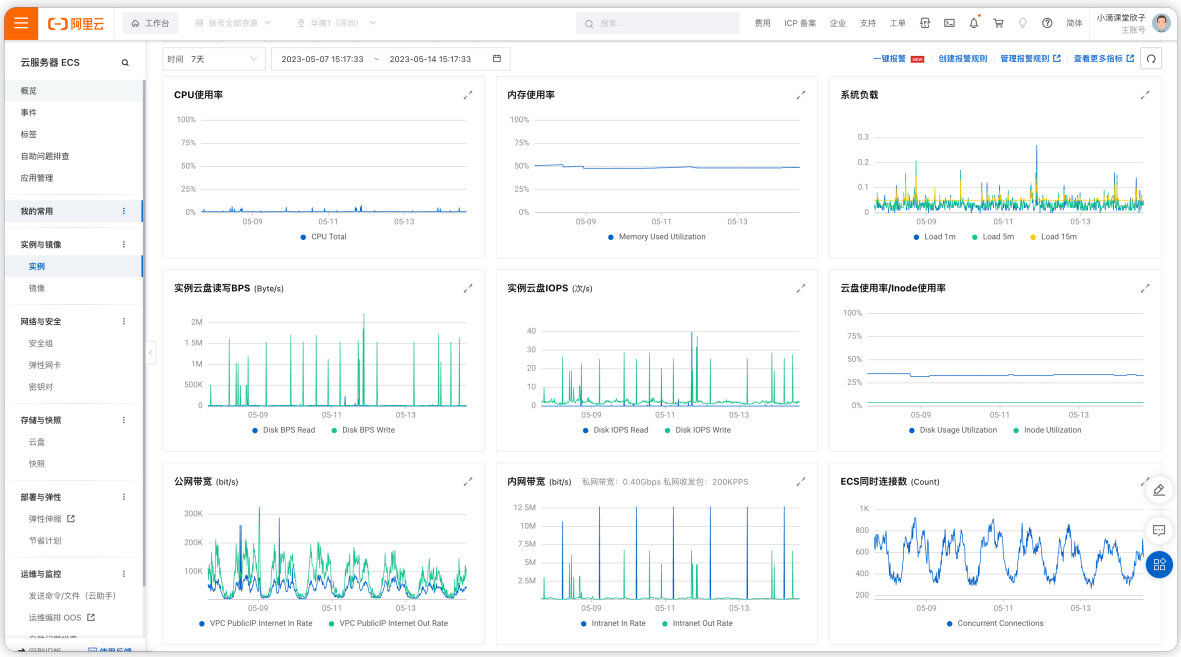
- 缓存击穿
  - 现象
    - 缓存中没有但数据库中有的数据，假如是热点数据，那key在缓存过期的一刻，同时有大量的请求
    - 这些请求都会击穿到DB，造成瞬时DB请求量大、压力增大
    - 和缓存雪崩的区别在于这里针对某一key缓存，后者则是很多key。
  - 解决方案

        - 设置热点数据不过期
        - 定时任务定时更新缓存
        - 多级缓存，nginx前置短期缓存，然后Redis配置不过期
        - 设置互斥锁，案例代码

10-介绍下Redis热点key，如何发现热点key

- **题目分析和考点**
    - 面试官想确认应聘者对 Redis缓存相关的常见问题和解决方案的了解
    - 热点 Key 是指在缓存系统中被频繁访问的 Key
    - 这些 Key 可能是某个业务流程中的关键数据，也可能是一些常用的统计数据或固定数据
    - 可以简要解释热点 Key 的概念，并介绍发现和解决热点 Key 的一般方法和策略
    - 可以结合实际项目经验，举例说明如何根据具体情况发现热点 Key，以及采取哪些解决方案
- **难度【 * * * * 】**
- **答案**
    - 什么热点Key
        - 缓存中的某些Key对应的value存储在集群中一台机器，所有流量涌向同一机器，成为系统的瓶颈，无法通过增加机器容量来解决
        - 专业名词就是：【流量/数据倾斜】 Kafka、Mysql、Flink 等普遍都存在，简单说就是数据和访问分配不均
        - 请求集中，超过单Server的性能极限
            - 超过该主机网卡上限时
            - 热点 Key 的缓存过多，超过目前的缓存容量时，缓存分片服务被打垮
            - 当缓存服务崩溃后，请求击穿到后台 DB 上，进一步导致缓存雪崩
        - 比如：热卖商品、热点新闻、热点评论、大V明星结婚
        - 需要解决的两个问题
            - 如何发现热点key问题
            - 如何解决热点key问题
    - 如何发现Hot Key
        - 方式一：经验处理
            - 根据借业务经验，预估热 Key 出现，比如上架了茅台商品秒杀活动
                - 优点：方案简单，凭经验发现热 `Key`，提早发现处理
                - 缺点：没有办法预测所有热 Key 出现，比如某些突然火爆的商品，无法提前预测（特别短视频流行情况下）
        - 方案二：节点和solt流量监控

- 对于每个节点或者slot中的流量做监控，上报对每个slot的流量对比，热key出现时发现影响到的具体slot
  - 优点：监控方便，方案简单
  - 缺点：粒度过于粗，适用于前期集群监控方案，不适用于精准探测到热key



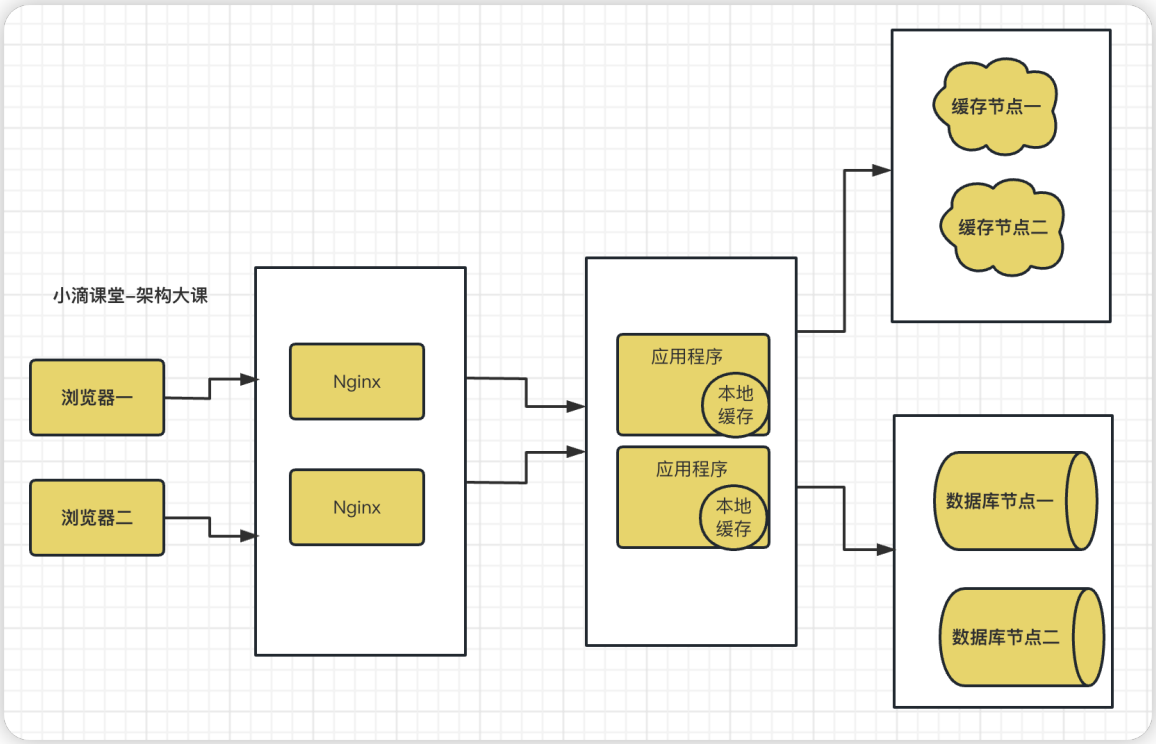
- 方式三：客户端统计
  - 客户端二次定制收集，应用程序连接redis都会使用客户端比如jedis等进行统计，基于这个客户端二次封装
  - 在发送请求前进行收集采集，定时把收集到的数据上报到统一的计算服务进行聚合统计
    - 优点：准确性高，实时行好
    - 缺点：需要对 SDK 工具进行二次开发，业务架构链路比较复杂

11-如何解决或避免Redis生产环境中的热点Key问题

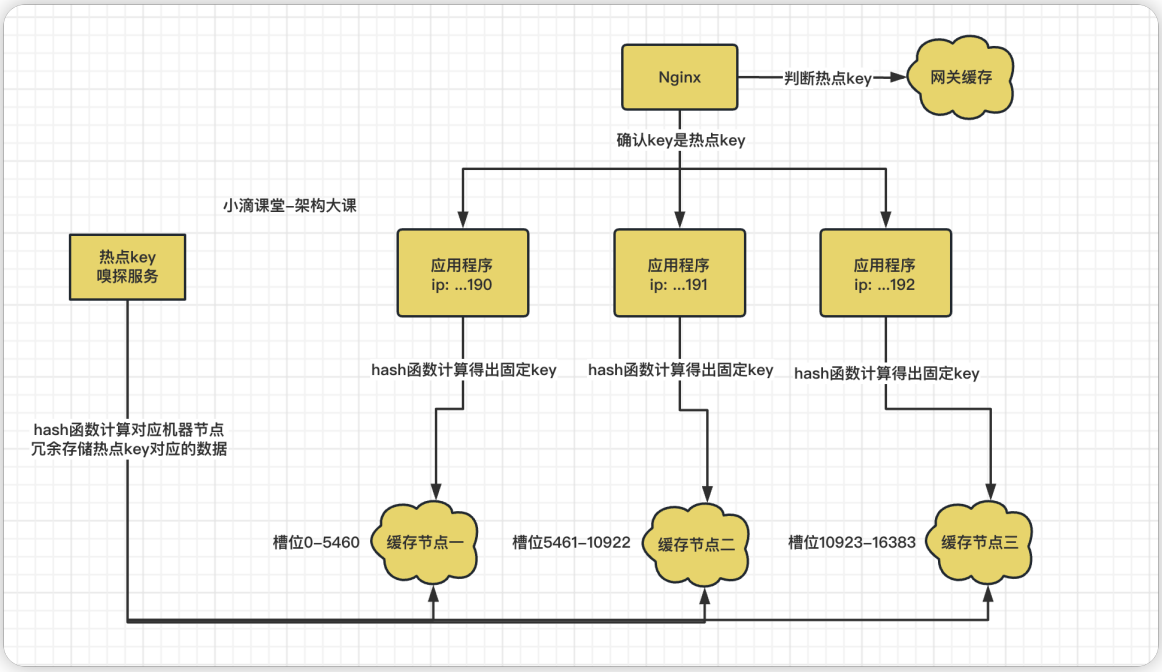
- **题目分析和考点**
  - 面试官想确认应聘者对 Redis缓存相关的常见问题和解决方案的了解
  - 解决热点 Key 的方法可以根据具体业务需求和系统特点采取不同的策略。
  - 常见的解决方法包括数据分片、缓存预热、使用更大的缓存服务器、增加缓存层级和使用分布式缓存等
  - 应聘者可以结合实际项目经验，举例说明如何根据具体情况发现热点 Key，以及采取哪些解决方案
- **难度【 * * * * 】**
- **答案**
  - 访问量大的时候才算热点key
    - 没固定值，但是依据个人经验，当某个key的访问频率很高，当一个key的qps到达1000的时候就需要关注

- Redis集群一般根据crc算法计算key归属哪个槽位，当某个key的qps过高，容易出现大量读请求落在某一个数据分片节点上
- 什么是Redis的哈希槽 slot
  - Redis集群预分好16384个槽，当需要在 Redis 集群中放置一个 key-value 时
  - 根据 CRC16(key) mod 16384的值，决定将一个key放到哪个桶中
  - 大体流程
    - 假设主节点的数量为3，将16384个槽位按照【用户自己的规则】去分配这3个节点，每个节点复制一部分槽位
      - 节点1的槽位区间范围为0-5460
      - 节点2的槽位区间范围为5461-10922
      - 节点3的槽位区间范围为10923-16383
      - 注意：从节点是没有槽位的，只有主节点才有
  - 存储查找
    - 对要存储查找的键进行crc16哈希运算,得到一个值，并取模16384，判断这个值在哪个节点的范围区间
    - 假设crc16("test_key")%16384=3000，就是节点一
    - crc16算法不是简单的hash算法，是一种校验算法

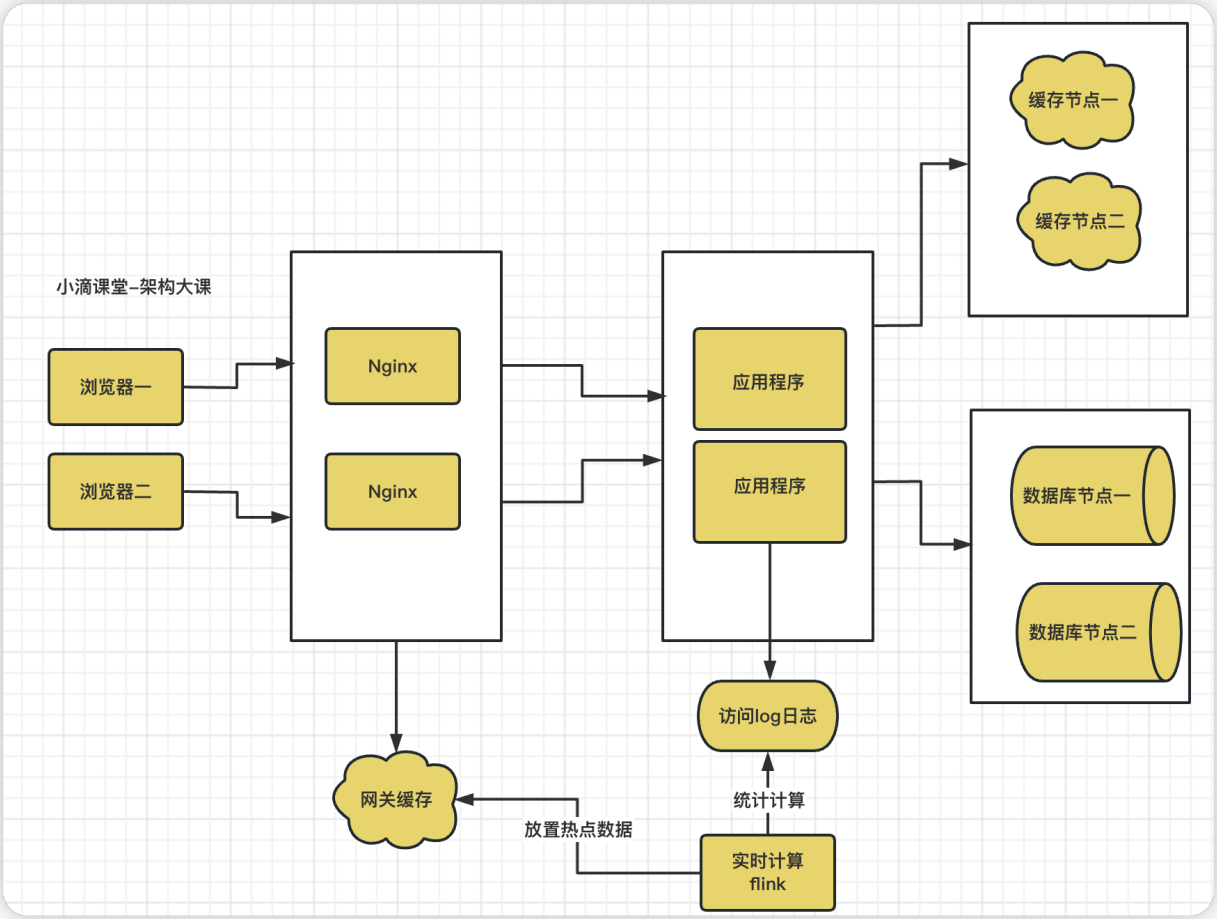
- 如何解决Hot Key
  - 热点key的解决方案一：本地缓存【推荐项目前期】
    - 避免带宽或者传输影响，本地缓存热点key数据，对于每次读请求
    - 将首先检查key是否存在于本地缓存中，如果存在则直接返回，如果不存在再去访问分布式缓存的机器



- 热点key的解决方案二：离散热点key
  - 保证不出现热key问题，分拆key也是一个好的解决方案
  - 常用方案，结合应用程序所在的机器ip或者mac网卡，得出一个固定值，拼接到key上面
  - 访问的时候也是根据业务本身的key，拼接到一样的算法，得到后端固定节点的redis上
  - 两步骤：拆key和组合key,
  - 缺点
    - 每次更新和新增时都需要去改动这N个key
    - 带来一些额外的开销，包括存储空间占用和应用程序修改复杂度等问题



- 热点key的解决方案三：网关缓存+实时计算【推荐项目后期】
  - 应用程序日志打印访问数据，结合流式计算，时间窗范围内统计
  - 计算出超过一定阈值的热点key，时间窗内访问增长快，预判会成为热点key，放置到业务网关缓存里面，前置热点缓存
  - 网关访问时先查找网关缓存是否存在对应的数据，如果存在则直接读取返回



12-说下Redis的BigKey，如何发现BigKey?

- **题目分析和考点**
  - 面试官想确认应聘者对 Redis缓存相关的常见问题和解决方案的了解
  - BigKey 是指占用较大内存空间的 Key, 这些 Key 可能存储着大量的数据，严重影响了 Redis 的性能和资源消耗
  - 重要的是理解 BigKey 的概念和对 Redis 性能的影响，并掌握发现 BigKey 的方法，以便进行适当的优化和处理
- **难度【* * * * 】**
- **答案**
  - 什么是BigKey

        - 在 Redis 中，当某个 Key 存储的值超过一定阈值时，就可能成为 BigKey
        - 占用比较大的 Key，通常超过了 10KB，甚至达到了 M 或者 G 级别，会导致 Redis 的内存使用率过高
        - Redis一个字符串最大512MB，一个二级数据结构(例如hash、list、set、zset)可以存储大约40亿个(2^32-1)个元素
        - 经验值
            - 当String类型的数据>10K
            - list、hash、set、sort set中元素个数超过1000个时
    - 问题点
        - 内存空间不均匀
            - 不利于充分利用各个节点的系统内存资源
        - IO和带宽阻塞
            - Redis单线程，操作bigkey的通常比较耗时，也就意味着阻塞Redis可能性越大
            - 假设一个bigkey为1MB，客户端每秒访问量为1000，那么每秒产生1000MB的流量，普通的千兆网卡128MB/s 直接不可用
        - 过期删除
            - 设置了过期时间，当它过期后，会被删除，这个时候如果使用主线程，则会阻塞redis，影响正常使用
    - 如何发现bigkey
        - **redis官方提供redis-cli --bigkeys 用来查找**
            - 给出每种数据结构的top 1 bigkey，同时给出了每种数据类型的键值个数以及平均大小
            - 注意事项
                - 由于bigkeys是调用scan，建议从节点执行
        - 客户端统计
            - 封装jedis客户端，存储前判断下对象的大小和元素个数，对应超过阈值的打印相关日志
            - 结合elk采集和可视化监控展示
    - 解决方案
        - 删除操作
            - big key删除不推荐使用 del key，会阻塞，推荐用lazy del，即unlink 命令
            - 异步延时释放key内存的功能，把key释放操作放在Background I/O单独的子线程处理，减少删除key对redis主线程的阻塞
        - 选择合适的数据结构
            - 多字段对象不采用String存储
            - 进行对象拆分，比如hash结构，mget获取需要的字段
        - 优化数据存储架构
            - 存储不同的地方，比如大V的粉丝列表，超过一定数值后，账号标识级别
            - 不同的数据源或集群存储查找，避免和业务redis混用

13-操作数据存取Redis为什么要序列化，常见的有哪些方式

- **题目分析和考点**
  - 面试官想确认应聘者对Redis 中数据序列化的重要性、序列化方式的了解，在设计和优化系统时对数据存储的考虑
  - 在将数据存储到 Redis 中时，需要将数据序列化为二进制格式进行存储。
  - 序列化的主要目的是将复杂的数据结构转化为可传输、可持久化存储的形式
  - 应聘者可以解释数据序列化的重要性，包括存储、传输和跨语言等方面的优势
- **难度【 * * 】**
- **答案**
  - 什么是序列化
    - 把对象转换为字节序列的过程称为对象的序列化。
    - 把字节序列恢复为对象的过程称为对象的反序列化。
    - 对象的序列化主要有两种用途
      - 把对象的字节序列永久地保存到硬盘上，通常存放在一个文件中
      - 在网络上传送对象的字节序列。
  - Redis为什么要序列化
    - 性能可以提高，不同的序列化方式性能不一样
    - 可视化工具更好查看
      - 采用默认的jdk方式会乱码（POJO类需要实现Serializable接口）
      - 采用JSON方式则不用，且可视化工具更好查看
  - 自定义redis序列化方式，提供了多种可选择策略
    - JdkSerializationRedisSerializer
      - POJO对象的存取场景，使用JDK本身序列化机制
      - 默认机制 ObjectInputStream/ObjectOutputStream进行序列化操作
    - StringRedisSerializer
      - Key或者value为字符串
    - Jackson2JsonRedisSerializer
      - 利用jackson-json工具，将pojo实例序列化成json格式存储
    - GenericFastJsonRedisSerializer
      - 另一种javabean与json之间的转换，同时也需要指定Class类型
    - Protocol Buffer
      - Google 提供的二进制序列化方案，具有高效和自描述的特点
    - JSON

        - 将数据序列化为 JSON 格式，具有易读性和跨语言的优势。

14-介绍下Redis持久化操作RDB和AOF的区别和联系

- **题目分析和考点**
    - 面试官想确认应聘者对对 Redis 持久化机制的理解，在设计和配置 Redis 持久化时的能力
    - RDB 通过创建快照的方式来保存 Redis 数据，而 AOF 则是通过追加写操作到日志文件中。
    - RDB 持久化生成的文件尺寸较小，恢复速度快；而 AOF 持久化生成的日志文件较大，恢复速度相对较慢
    - 两种持久化方式可以同时使用，即既可以启用 RDB 持久化又可以启用 AOF 持久化
    - 重要的是理解 RDB 和 AOF 持久化机制以及其特点，根据实际情况选择合适的持久化策略，以保证数据的持久性和可靠性
- **难度【 * * * 】**
- **答案**
    - Redis持久化介绍
        - Redis是一个内存数据库，如果没有配置持久化，redis重启后数据就全丢失
        - 因此开启redis的持久化功能，将数据保存到磁盘上，当redis重启后，可以从磁盘中恢复数据。
    - 两种持久化方式
        - RDB (Redis DataBase)
        - AOF (append only file)
    - RDB持久化介绍
        - 在指定的时间间隔内将内存中的数据集快照写入磁盘
        - 默认的文件名为dump.rdb
        - 优点
            - RDB文件紧凑，全量备份，适合用于进行备份和灾难恢复
            - 在恢复大数据集时的速度比 AOF 的恢复速度要快
            - 生成的是一个紧凑压缩的二进制文件
        - 缺点
            - 每次快照是一次全量备份，fork子进程进行后台操作，子进程存在开销
            - 在快照持久化期间修改的数据不会被保存，可能丢失数据
    - AOF持久化介绍
        - append only file，追加文件的方式，文件容易被人读懂
        - 以独立日志的方式记录每次写命令， 重启时再重新执行AOF文件中的命令达到恢复数据的目的
        - 写入过程宕机，也不影响之前的数据，可以通过 redis-check-aof检查修复问题
        - 提供了3种同步方式，在性能和安全性方面做出平衡
            - appendfsync always

            - 每次有数据修改发生时都会写入AOF文件，消耗性能多
        - appendfsync everysec
            - 每秒钟同步一次，该策略为AOF的缺省策略。
        - appendfsync no
            - 不主从同步，由操作系统自动调度刷磁盘，性能是最好的，但是最不安全
    - 优点
        - 数据更加安全
        - 当Redis AOF文件太大时，Redis能够在后台自动重写AOF
        - AOF以易于理解和解析的格式，一个接一个地包含所有操作的日志
    - 缺点：
        - AOF文件通常比同一数据集的等效RDB文件大
        - 根据确切的fsync策略，恢复的时候AOF可能比RDB慢

- 在线上我们到底该怎么做?
    - RDB持久化与AOF持久化一起使用
    - 如果Redis中的数据并不是特别敏感或者可以通过其它方式重写生成数据
    - 集群中可以关闭AOF持久化，靠集群的备份方式保证可用性
    - 自己制定策略定期检查Redis的情况，然后可以手动触发备份、重写数据；
    - 采用集群和主从同步

15-说下内存不足时-Redis的Key淘汰策略有哪些

- **题目分析和考点**
    - 面试官想确认应聘者对Redis的内存管理和数据淘汰策略的了解，在设计和优化 Redis 数据存储时的能力
    - Redis 是基于内存的缓存数据库，当内存不足时，Redis 需要根据策略决定如何释放或回收一些键值对
    - 可以简要解释 Redis 的内存管理问题，以及常见的 Key 淘汰策略，并提及适用场景和策略的选择原则
    - 分享在实际项目中如何设置和优化淘汰策略，以保证系统的性能和可用性
- **难度【 * * * 】**
- **答案**
    - 背景
        - redis在占用的内存超过指定的maxmemory之后，
        - 通过maxmemory_policy确定redis是否释放内存以及如何释放内存
        - 提供多种策略

- 策略
  - volatile-lru(least recently used)
    - 最近最少使用算法，从设置了过期时间的键中选择空转时间最长的键值对清除掉；
  - volatile-lfu(least frequently used)
    - 最近最不经常使用算法，从设置了过期时间的键中选择某段时间之内使用频次最小的键值对清除掉；
  - volatile-ttl
    - 从设置了过期时间的键中选择过期时间最早的键值对清除 (删除即将过期的)
  - volatile-random
    - 从设置了过期时间的键中，随机选择键进行清除；
  - allkeys-lru
    - 最近最少使用算法，从所有的键中选择空转时间最长的键值对清除；
  - allkeys-lfu
    - 最近最不经常使用算法，从所有的键中选择某段时间之内使用频次最少的键值对清除；
  - allkeys-random
    - 所有的键中，随机选择键进行删除；
  - noeviction
    - 不做任何的清理工作，在redis的内存超过限制之后，所有的写入操作都会返回错误；但是读操作都能正常的进行;

16-说下新版Redis主从复制同步的原理

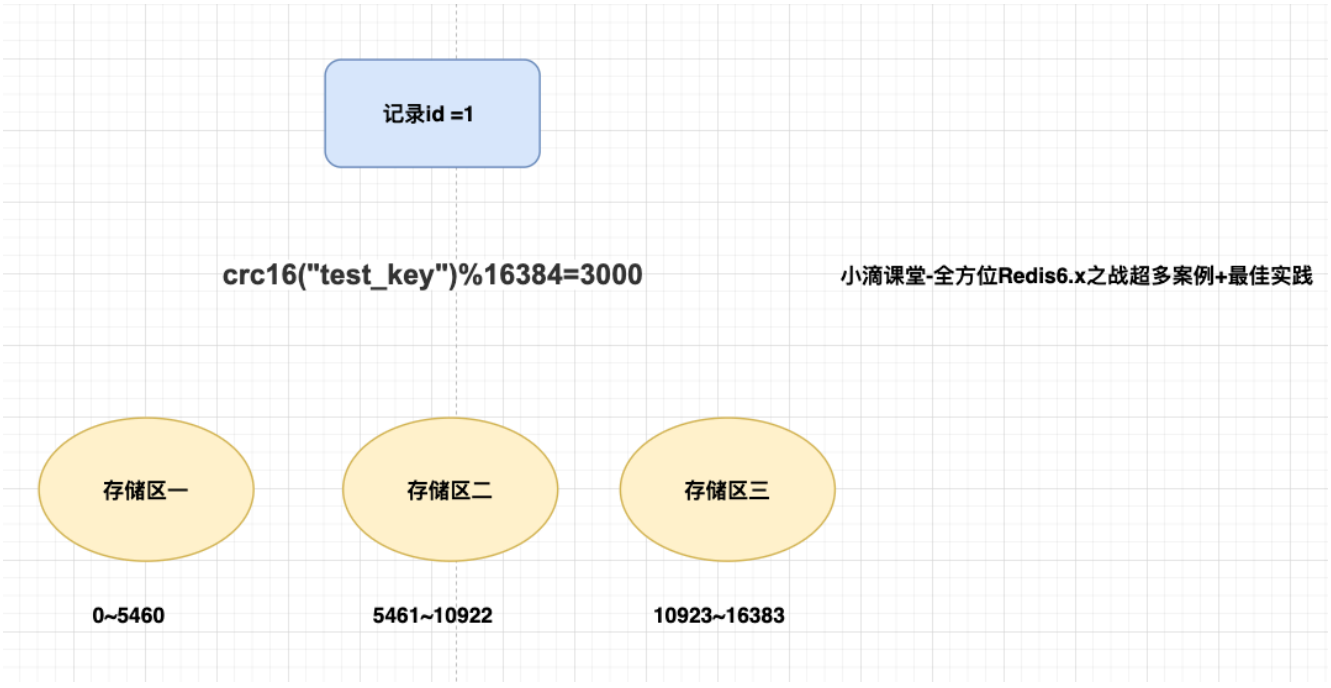
- **题目分析和考点**
  - 面试官想确认应聘者对 Redis 主从复制机制的理解
  - Redis 主从复制是一种数据同步机制，用于将一个 Redis 节点（主节点）上的数据复制到其他节点（从节点）。
  - 主节点负责接收写操作并将写操作同步给从节点，从节点则负责接收来自主节点的数据并进行复制，保持数据的一致性和可用性
  - 解释 Redis 主从复制的基本概念，重点介绍新版 Redis 主从复制的原理和流程，包括 SYNC 命令、全量复制和增量复制等步骤
- **难度【 * * * 】**
- **答案**
  - 主从复制分两种（主从刚连接的时候，进行全量同步；全同步结束后，进行增量同步）
    - 全量复制
      - master服务器会开启一个后台进程用于将redis中的数据生成一个rdb文件

            - 主服务器会缓存所有接收到的来自客户端的写命令，当后台保存进程 处理完毕后，会将该rdb文件传递给slave服务器
            - slave服务器会将rdb文件保存在磁盘并通过读取该文件将数据加载到内存
            - 在此之后master服务器会将在此期间缓存的命令通过redis传输协议发送给slave服务器
            - 然后slave服务器将这些命令依次作用于自己本地的数据集上最终达到数据的一致性
        - 增量复制
            - Slave初始化后开始正常工作时主服务器发生的写操作同步到从服务器的过程
            - 服务器每执行一个写命令就会向从服务器发送相同的写命令，从服务器接收并执行收到的写命令
    - 特点
        - 主从复制对于 主/从 redis服务器来说是非阻塞的，所以同步期间都可以正常处理外界请求
        - 一个主redis可以含有多个从redis，每个从redis可以接收来自其他从redis服务器的连接
        - 从节点不会让key过期，而是主节点的key过期删除后，成为del命令传输到从节点进行删除
            - 从节点开启 sync 看日志
    - 加速复制
        - 完全重新同步需要在磁盘上创建一个RDB文件，然后加载这个文件以便为从服务器发送数据
        - 在比较低速的磁盘，这种操作会给主服务器带来较大的压力
        - 新版支持无磁盘的复制，子进程直接将RDB通过网络发送给从服务器，不使用磁盘作为中间存储
        - repl-diskless-sync yes (默认是no)
    - 主从断开重连
        - 如果遭遇连接断开，重新连接之后可以从中断处继续进行复制，而不必重新同步
        - 2.8版本后 部分重新同步这个新特性内部使用PSYNC命令，旧的实现中使用SYNC命令

17-介绍下Redis集群Cluster的读写流程和原理

- **题目分析和考点**
    - 面试官想确认应聘者对 Redis集群Cluster的读写流程掌握情况
    - Redis 集群 Cluster 是 Redis 官方提供的分布式集群解决方案
    - 它通过将数据分散存储在多个节点上，提供了高可用性和可扩展性
    - Cluster 模式中有多个主节点，每个主节点负责存储和处理一部分数据，并与其他节点进行数据同步和负载均衡
- **难度【 * * * * * 】**
- **答案**
    - Redis集群模式介绍
        - Cluster模式是Redis3.0开始推出

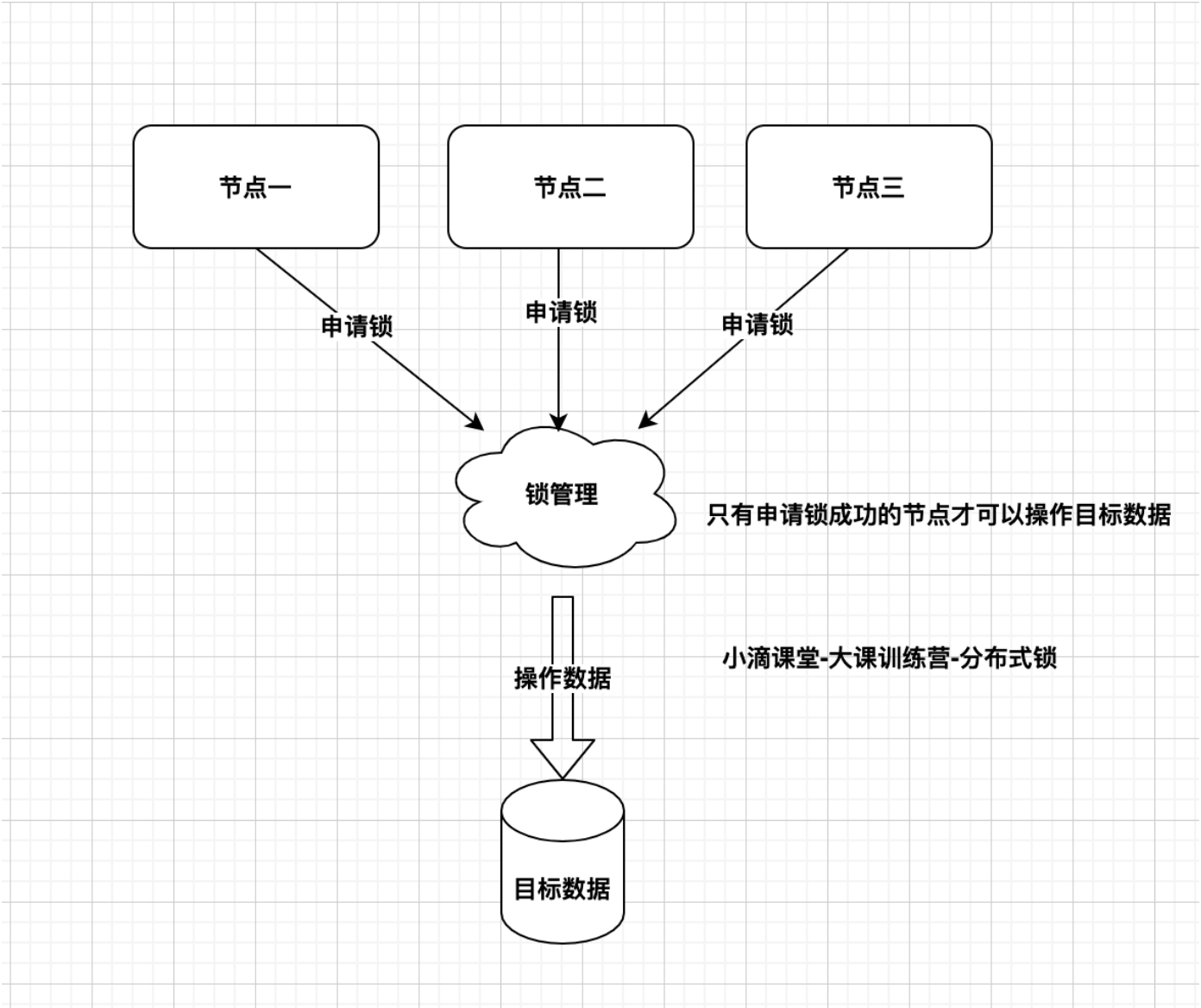
    - 采用无中心结构，每个节点保存数据和整个集群状态, 每个节点都和其他所有节点连接
    - 官方要求：至少6个节点才可以保证高可用，即3主3从；扩展性强、更好做到高可用
    - 各个节点会互相通信，采用gossip协议交换节点元数据信息
    - 数据分散存储到各个节点上
- 什么是Redis的哈希槽 slot
    - Redis集群预分好16384个槽，当需要在 Redis 集群中放置一个 key-value 时，根据 CRC16(key) mod 16384的值，决定将一个key放到哪个桶中
- 大体流程
    - 假设主节点的数量为3，将16384个槽位按照【用户自己的规则】去分配这3个节点，每个节点复制一部分槽位
        - 节点1的槽位区间范围为0-5460
        - 节点2的槽位区间范围为5461-10922
        - 节点3的槽位区间范围为10923-16383
        - 注意：从节点是没有槽位的，只有主节点才有
    - 存储查找
        - 对要存储查找的键进行crc16哈希运算,得到一个值，并取模16384，判断这个值在哪个节点的范围区间
        - 假设crc16("test_key")%16384=3000，就是节点一
        - crc16算法不是简单的hash算法，是一种校验算法



- 使用哈希槽的好处就在于可以方便的添加或移除节点。
    - 当需要增加节点时，只需要把其他节点的某些哈希槽挪到新节点就可以了；
    - 当需要移除节点时，只需要把移除节点上的哈希槽挪到其他节点就行了

18-什么是分布式锁，举例说下使用场景有哪些

- **题目分析和考点**
  - 面试官想确认应聘者对分布式锁的概念和使用场景的理解，在设计和开发分布式系统时处理并发访问的能力
  - 可以简要解释分布式锁的概念和作用，并列举一些常见的使用场景。
  - 可以结合自身项目经验，提供实际开发中使用分布式锁的案例，以及如何设计和实现分布式锁的思路和方案
- **难度【 * * 】**
- **答案**
  - 分布式锁
    - 分布式锁是一种用于协调分布式系统中并发访问的机制
    - 可以保证在分布式环境下，同一时刻只有一个进程或线程能够对共享资源进行操作，避免数据竞争和并发冲突。
  - 分布式锁的使用场景很广泛，常见的使用场景包括：
    - 防止多个进程或线程同时对共享资源进行写操作，避免数据不一致。
    - 控制并发访问，避免并发冲突和竞争条件。
    - 限制特定资源的访问频率，如限流操作
    - 案例：优惠券领券限制张数、商品库存超卖
  - 核心
    - 为了防止分布式系统中的多个进程之间相互干扰，我们需要一种分布式协调技术来对这些进程进行调度
    - 利用互斥机制来控制共享资源的访问，这就是分布式锁要解决的问题
  - 避免共享资源并发操作导致数据问题
    - 加锁
      - 本地锁：synchronize、lock等，锁在当前进程内，集群部署下依旧存在问题
      - 分布式锁：redis、zookeeper等实现，虽然还是锁，但是多个进程共用的锁标记，可以用Redis、Zookeeper、Mysql等都可以



搜索引擎

分布式事务

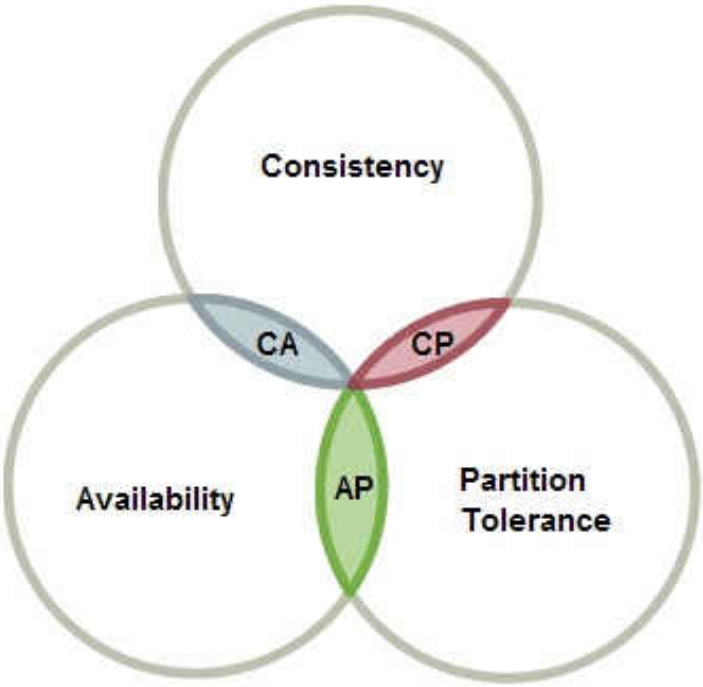
微服务和RPC

1-介绍下分布式系统下面的CAP定理和不同场景如何选择

- 题目分析和考点

- 面试官想确认应聘者是否对分布式系统CAP概念是否理解
- 是否知道哪些场景下选择CP和AP

- **难度【* * * * *】**
- **答案**
  - 一个分布式系统中，Consistency（一致性）、 Availability（可用性）、Partition tolerance（分区容错性），三者不可同时获得
    - Consistency（一致性C）
      - 指在分布式系统中的所有节点在同一时间点都具有相同的数据副本，**即对数据的新增/删除/更新 需要保证原子性**
      - 对一个系统进行数据【新增/修改/删除】成功后，其他读请求都需要获取到这个最新数据
      - 如果是【新增/修改/删除】失败，那全部读操作都不能读取到这个数据
    - Availability（可用性A）
      - 可用性是指系统必须在给定的时间内对请求进行响应
      - 每个请求都是可以得到响应的，不管请求是成功还是失败
      - 服务在任何时刻都必须是可用的，要求系统当中不应该有单点故障的存在
    - Partition tolerance（分区容错性P）
      - 指系统在遇到网络分区或节点故障时仍然能够继续运行
      - 除了全部整体网络故障，其他故障都不能导致整个系统不可用
  - 由于当前的网络硬件肯定会出现延迟丢包等问题，所以分区容忍性是必须需要实现的，只能在一致性和可用性之间进行权衡
  - 选择问题
    - 根据CAP定理，需要在一致性、可用性和分区容忍性中进行权衡选择，不同的应用场景对CAP定理的权衡选择有不同要求
    - **分区容忍性是必不可少的，因为网络分区是无法避免的**
    - **CP without A 放弃可以性**
      - 如果数据的一致性和正确性非常重要，那么可以选择保证一致性，但可能会牺牲一部分可用性；
      - 假如发生了分区，需要保证一致性，需要等待受影响的服务所在的节点数据一致，而等待期间无法对外提供服务
    - **AP wihtout C 放弃一致性**
      - 如果系统需要高可用性，可以选择牺牲一致性来获得更好的可用性
      - 假如发生了分区，节点之间失去联系，为了保证高可用，每个节点只能用本地数据提供服务，导致整体数据的不一致性



2- Nacos是CP还是AP模型，举例几个不同的业务场景下CAP选择?

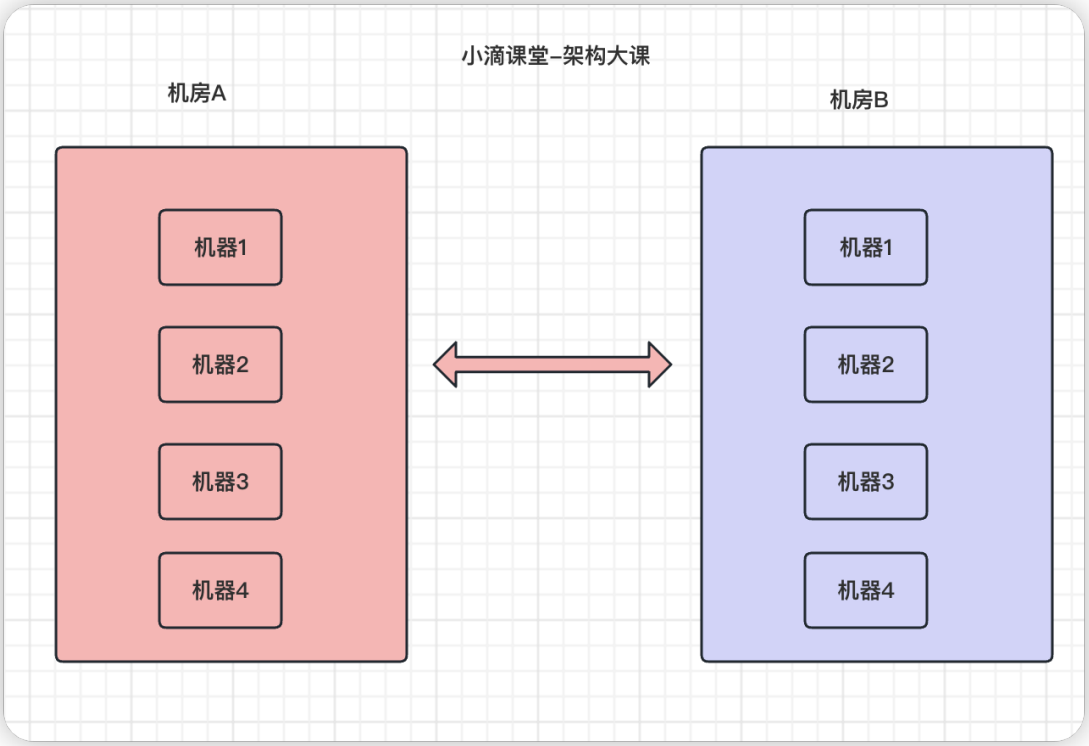
- 题目分析和考点
  - 面试官想确认应聘者是否对Nacos的底层原理是否掌握，支持CAP里面的哪个模型
  - 是否知道不同业务场景如何选择
- 难度【* * * * *】
- 答案
  - **AlibabaCloud微服务组件 Nacos的AP和还是CP模式?**
    - **注意：Nacos支持CP+AP模式，即Nacos可以根据配置识别为CP模式或AP模式，默认是AP模式**
    - Nacos的AP和CP模式是指在不同的场景下，Nacos会选择不同的一致性模式
    - AP模式：
      - 在分布式系统中，当发生网络分区时，为了保持可用性，Nacos会选择AP模式。
      - 在AP模式下，Nacos致力于保证服务的可用性，但可能导致数据的不一致性。
    - CP模式：
      - 当网络没有分区时，Nacos会选择CP模式。在CP模式下，Nacos优先保证数据的一致性，但可能导致服务的不可用
  - **举例几个不同业务场景下的选择**
    - **金融交易系统**

        - 在金融交易系统中，数据的一致性是非常关键的，因为不能有数据丢失或者数据不一致的情况发生。
        - 这种场景下需要选择一致性（Consistency）和分区容忍性（Partition tolerance）
        - 即保证数据的一致性，同时能够容忍网络分区。
    - **社交网络应用**
        - 对于社交网络应用来说，用户的可用性是非常重要的，系统需要能够实时响应用户的请求
        - 在这种情况下，可用性（Availability）是首要考虑因素，因此可能会选择牺牲一致性来获得更好的可用性。
    - **物联网系统**
        - 在物联网系统中，设备的连接和通信是关键，需要确保系统的可用性和分区容忍性。
        - 由于涉及到大规模设备的传感器数据，一致性的要求相对较低，因此可以选择保证可用性和分区容忍性
        - 而在一致性方面可能会放宽一些要求。
    - **实时在线游戏**
        - 在线游戏需要实时的响应和互动，因此可用性是最重要的
        - 一些实时游戏可以牺牲一致性来保证用户的游戏体验

3-分布式系统下有CAP选择问题，介绍下什么是网络分区

- 题目分析和考点
    - 面试官想确认应聘者是否对分布式系统CAP概念里面的网络分区是否理解
- 难度【* * *】
- 答案
    - 假设有一个分布式系统，该系统部署在两个不同的机房，而这两个机房之间通过内部独立的光纤链路进行通信。
    - 由于机房之间的光纤链路发生故障，导致两个机房之间的通信中断
    - 在这段时间内，两个机房内部的节点仍然可以相互通信
    - 但由于机房间的通信中断，整个分布式系统在此期间产生了两个独立的分区，形成了两个独立的系统
    - 可以发生在不同地区、不同数据中心、不同服务器之间，也可能是由于网络故障、硬件故障或其他原因引起的。

- 无论是哪种原因，当网络分区发生时，分区中的节点无法直接通信，可能导致数据不一致、延迟增加或无法提供某些服务
- 分布式系统的设计需要考虑网络的不可靠性和延迟，同时还要解决数据一致性的问题
- 在过去的研究中，发现在分布式系统中不可能同时保证一致性、可用性和分区容忍性，因为分布式系统的局限性



4-介绍下分布式系统下的BASE理论和应用场景

- 题目分析和考点
  - 面试官想确认应聘者是否对分布式系统的BASE理论是否掌握
  - 属于CAP的权衡结果，在业务开发中有大量的应用
- 难度【* * *】
- 答案
  - Base理论
    - 一种在分布式系统设计中常用的思想，它提供了一套设计原则，以应对分布式系统中的可用性和一致性的挑战
    - CAP 中的一致性和可用性进行一个权衡的结果，核心思想就是：
      - **无法做到强一致，但每个应用都可以根据自身的业务特点，采用适当方式来使系统达到最终一致性, 来自 ebay 的架构师提出**
    - **基本可用性（Basically Available）**

        - 系统在面对部分故障或异常情况时仍能保持基本的可用性，即系统能够处理和响应用户的请求
        - 尽管可能会有一些功能上的降级或性能上的损失。
    - **软状态（Soft State）**
        - 系统中的状态可以在一段时间内是不一致的，即系统中的数据存在延迟、异步或者临时性的不一致
        - 这是因为分布式系统往往面临网络延迟、并发写入等问题
    - **最终一致性（Eventual Consistency）**
        - 系统的数据最终会达到一致状态，即在一段时间内经过合适的修复和同步机制后，系统中的数据最终会达到一致的状态
        - 但不保证实时的一致性，而是保证在合理的时间内达到一致。
- 应用案例场景
    - 分布式数据库
        - 在分布式数据库中，采用BASE理论可以实现较高的可用性和灵活性。
        - 当网络分区或故障发生时，系统仍然可用，即保持基本的可用性；
        - 通过采用副本机制或异步同步机制，实现最终一致性。
    - 在线社交网络
        - 在在线社交网络中，由于用户数量庞大且活动频繁，系统需要具备较高的可用性和灵活性
        - 通过采用BASE理论，系统可以保持基本的可用性，即使在网络分区或故障的情况下，用户仍然能够继续使用系统。
    - 分布式事务或三方支付平台对接
        - 高并发项目中，比如电商里面的商品库存服务和订单服务数据一致性问题，保持最终一致性，可以提高性能
        - 订单的支付状态，对接微信支付或者支付宝支付，采用异步通知的方式，达到最终一致性，可以提高性能

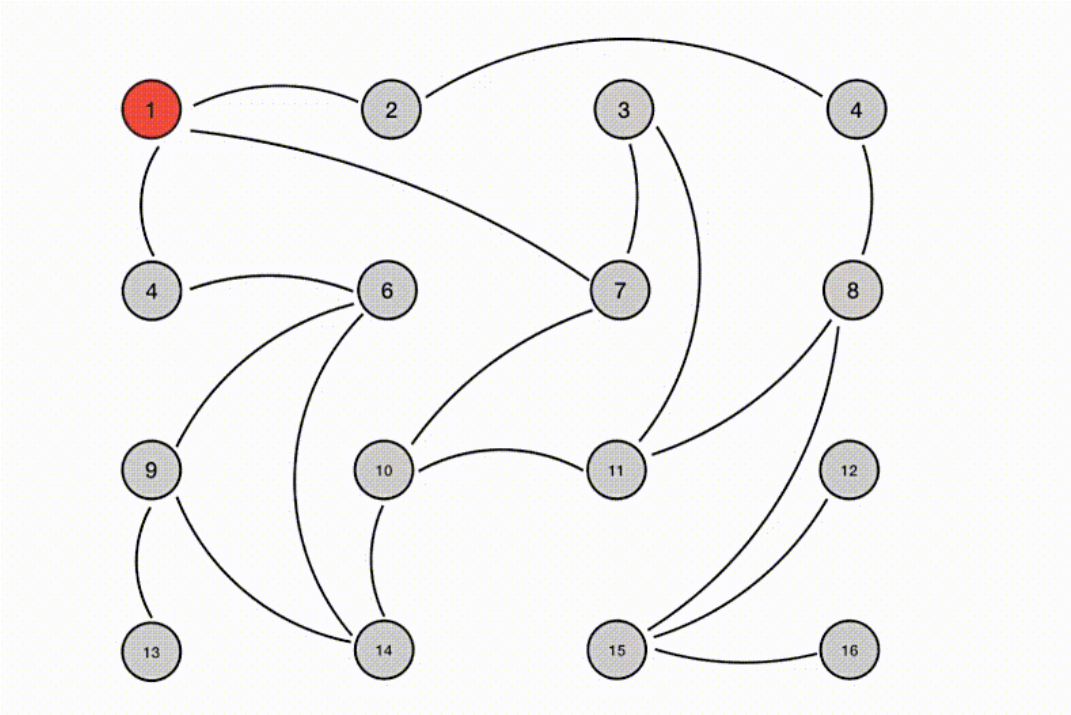
4-有没了解过一致性算法，说下Gossip算法的工作流程和特点

- 题目分析和考点
    - 面试官想确认应聘者是否对一致性算法有掌握
    - 有弱一致性算法和强一致性算法，而Gossip属于弱一致算法
- 难度【***】
- 答案
    - 什么是Gossip算法
        - 别名：“闲话算法”、“疫情传播算法”、“病毒感染算法”等

- 也被称为反熵（Anti-Entropy），熵是物理学上的一个概念，代表杂乱无章，而反熵就是在杂乱无章中寻求一致
- 是一种去中心化的信息传播和一致性维护算法，它通过节点之间的相互通信和随机化的信息交换来实现
- 属于弱最终一致性算法，仅能保证最终一致性而非强一致性，而且达到一致的时间不可控
- 通俗易懂的方式解释

> 想象一下你和一群朋友相遇在一个聚会上。你们每个人都有一些新鲜的八卦要分享给其他人。
> 开始时，你随机选择了一个朋友，将八卦告诉了他。接下来，你的朋友与他的朋友们分享这个八卦。
> 同时，其他朋友也开始用同样的方式分享他们知道的八卦。
> 随着时间的推移，每个人都与随机选择的朋友交谈，广泛传播和了解了一些消息

- 工作流程
  - 在分布式系统中，Gossip算法的工作方式类似于上述场景
  - 每个节点都与随机选择的其他节点进行通信，并将自己的信息传递给它们。
  - 这些节点在接收到新的信息后，将会以同样的方式继续广播给其他节点。
  - 这个过程不断重复，使得信息在整个系统中迅速传播
  - 参考下面的网图（图片来源网上-感谢绘图的博主）
    - 某个节点发生变化，在相邻节点随机选择N个节点(并非全部节点)，并同步相关信息



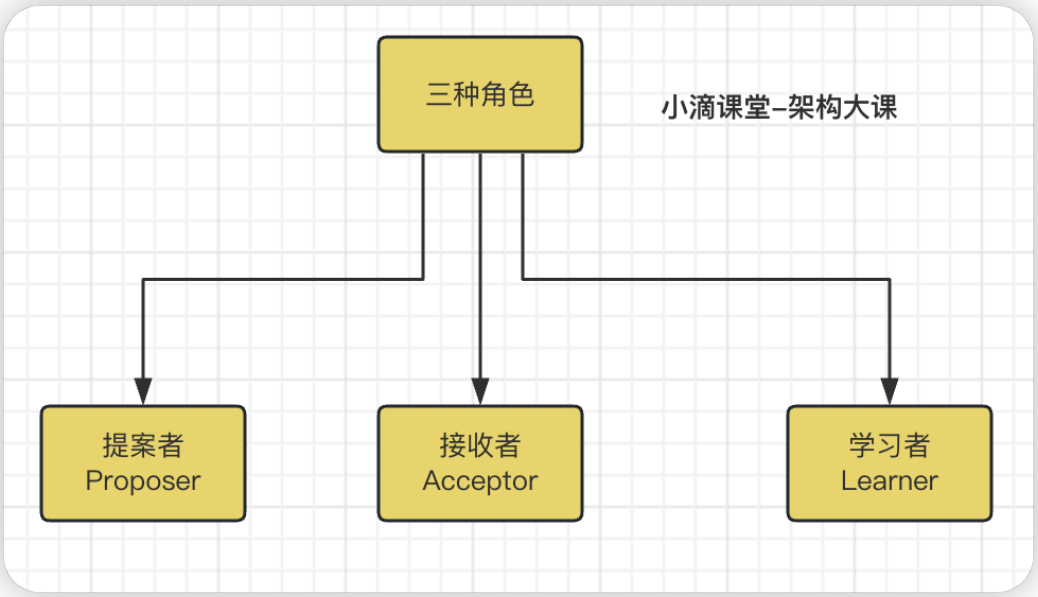
5-说下一致性算法Gossip优缺点和应用场景

- 题目分析和考点
  - 面试官想确认应聘者是否对一致性算法有掌握
  - 有弱一致性算法和强一致性算法，而Gossip属于弱一致算法
  - 任何一种算法存在都是有意义的， 不同业务场景使用不一样的算法
- 难度【 * * * 】
- 答案
  - Gossip算法，别名：“闲话算法”、“疫情传播算法”、“病毒感染算法”等，是一种去中心化的信息传播和一致性维护算法
  - 优点
    - 去中心化：Gossip算法不依赖于中心节点，每个节点都是对等的，使得系统更加分散和可扩展。
    - 容错性：由于节点之间的交流是随机的，当一些节点无法通信或者发生故障时，系统仍然可以继续运行和进行信息传播。
    - 高可靠性：通过随机化选择其他节点进行通信和信息传播，可以快速传播并检测到信息，从而提高系统的可靠性。
    - 迅速传播：通过节点之间的相互通信和交流，Gossip算法能够快速传播信息，并在系统中广泛传播和同步数据。
    - 自适应性：Gossip算法能够自动适应系统的变化，对新节点的加入或离开具有良好的适应能力。
  - 缺点
    - 最终一致性
      - Gossip算法在信息传播的过程中，可能会出现一段时间的不一致状态，但最终会达到一致状态。
      - 这可能导致系统中的节点在某些情况下无法立即获取最新的信息。
    - 通信开销
      - 由于每个节点都需要与其他节点进行通信和交流，Gossip算法可能会产生一定的通信开销和网络流量。
    - 配置复杂性
      - 在配置Gossip算法时，需要选择合适的参数和节点数量，以确保信息的快速传播和一致性。
      - 这可能对系统配置和管理造成一些复杂性
  - 使用场景
    - 分布式数据一致性
      - 在分布式数据库或存储系统中，使用Gossip算法可以确保数据在不同节点之间的一致性
      - 节点之间通过随机交换信息，传播自己的数据副本，并更新自己的数据状态。
    - 分布式故障检测与容错

        - 使用Gossip算法可以实现节点之间的故障检测和容错机制。
        - 节点通过定期与其他随机选择的节点通信，交换信息并检测对方的存活状态。
        - 如果某个节点长时间无法进行通信，其他节点将会感知到故障，并采取相应的容错措施。
    - 对等网络（P2P网络）
        - Gossip算法常用于对等网络中的信息传播和查询路由。
        - 每个节点都与邻居节点进行交流，分享自己所知道的信息，广播请求或查询，从而实现分布式的协作和资源共享。
    - Gossip 协议有名的应用包括 Redis-Cluster，Bitcoin，Cassandra 等
- 总结
    - Gossip算法是一种去中心化的信息传播和一致性维护算法
    - 具备随机化、传染性、最终一致性、容错性、可扩展性和自适应性等特点
    - 它适用于各种分布式系统中的信息传播、数据一致性、故障检测和资源共享等场景

6-有没了解过一致性算法Paxos，说下有什么缺点

- 题目分析和考点
    - 面试官想确认应聘者是否对一致性算法有掌握
    - 有弱一致性算法和强一致性算法，而Paxos属于强一致性算法
    - 任何一种算法存在都是有意义的， 不同业务场景使用不一样的算法
- 难度【* * * * *】
- 答案
    - **Paxos算法**
        - 是解决分布式一致性问题的经典算法，被广泛应用于分布式数据库、分布式存储和分布式一致性服务等领域
        - 通过引入一个协调者（也称为Proposer）和多个接受者（也称为Acceptor）之间的协商过程，来达成一致性
        - 算法的目标是要达成一个值的决议，即选择一个值作为系统的共识结果
        - 角色
            - **Proposer**: 提出提案 (Proposal)，包括提案编号 (Proposal ID) 和提议的值 (Value)。
            - **Acceptor**:参与决策响应Proposers的提案，收到Proposal后接受提案，若获得多数Acceptors的接受，则Proposal被批准。
            - **Learner**：不参与决策，从Proposers/Acceptors学习最新达成一致的提案（Value）



- 缺点
  - 复杂性
    - Paxos算法非常复杂，理解和实现都相对困难，协议步骤和消息传递非常繁琐，可能需要进行多次通信才能达成一致。
  - 性能开销
    - 算法在性能方面会有一些开销，在面对网络延迟或节点失效时。算法的多轮通信和多次消息传递会增加整体延迟
- **Paxos的最大特点就是难，不仅难以理解，更难以实现，而Raft算法是对Paxos算法的简化和改进，是重点需要掌握的**

7-有没了解过一致性算法Raft,说下原理流程

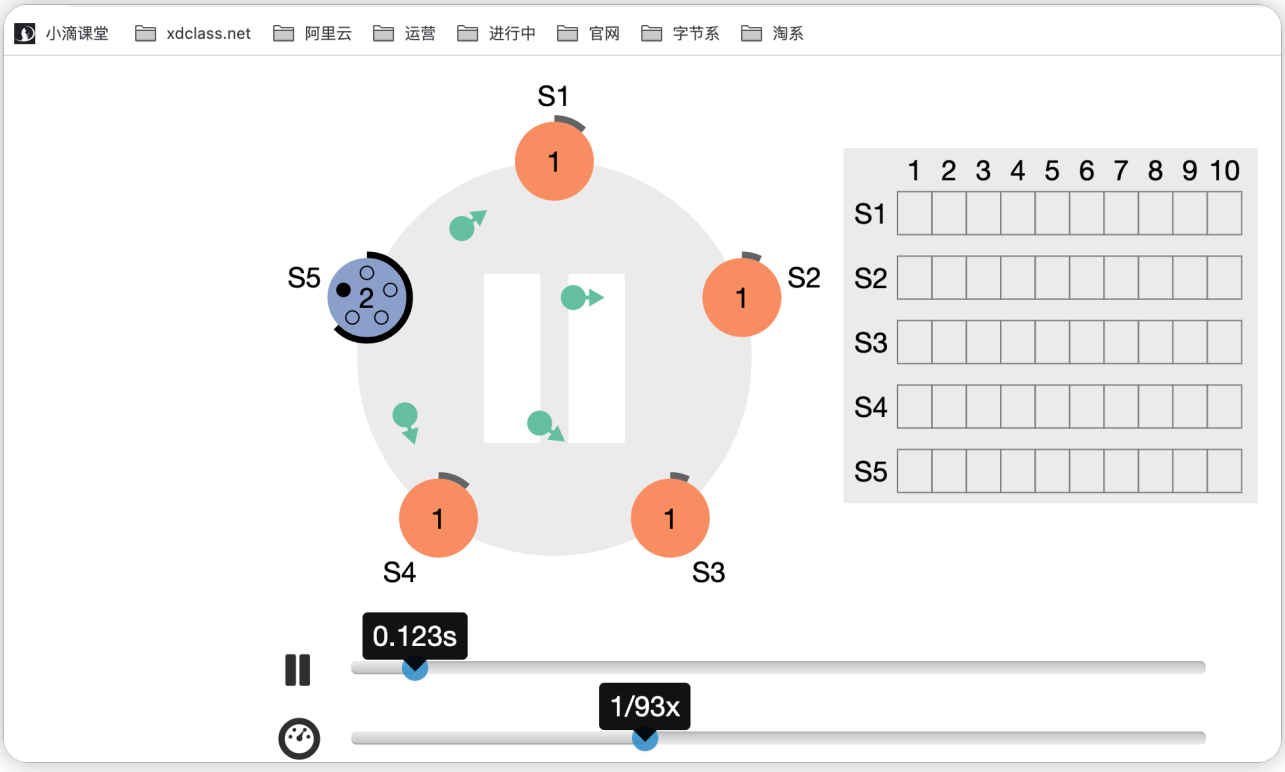
- 题目分析和考点
  - 面试官想确认应聘者是否对一致性算法有掌握
  - 有弱一致性算法和强一致性算法，而Raft属于强一致性算法
  - 与Paxos算法相比，Raft算法更易理解和实现，并且在分布式系统中得到了广泛应用
- 难度【* * * *】
- 答案
  - **Raft算法**
    - 是一种用于在分布式系统中实现一致性的算法

- 与Paxos算法相比，Raft算法更易理解和实现，并且在分布式系统中得到了广泛应用
- 基本概念
  - Raft算法通过引入领导者（Leader）、跟随者（Follower）和候选人（Candidate）三种角色，实现系统中的一致性。
  - 领导者负责处理客户端请求，并通过心跳机制保持与跟随者和候选人的联系。
  - 跟随者负责接收并复制领导者的操作，候选人则是在选举领导者的过程中起到中间角色
- 角色说明
  - 领导者（Leader）
    - 所有请求的处理者，并向Follower同步请求日志，当日志同步到大多数节点上后告诉Follower提交日志
  - 跟随者（Follower）
    - 被动更新者，接受并持久化Leader同步的日志，在Leader告之日志可以提交之后，提交日志
    - 如果客户端请求到Follower，则进行转发（重定向）给Leader
  - 候选人（Candidate）
    - Leader选举过程中的临时角色，如果Follower在一定时间内没收到Leader的心跳，则判断Leader发生了故障
    - 这个时候会重新发起Leader选举，当前节点则切换为Candidate候选者，直到选举成功
  - 注意
    - **每个follower都有一个随机的时钟表示的是follower等待成为leader的时间，谁的时钟先跑完，则发起leader选举**
    - Raft算法要求分布式系统在任何时刻只有一个领导者，而且在正常情况下，系统只包含一个领导者和多个跟随者。
    - Raft算法将时间分为一个个的任期（term），每一个term的开始都是Leader选举
    - 一旦成功选出领导者，该领导者将负责管理整个集群，直至当前任期结束。
    - 如果领导者选举失败，该任期将因缺乏领导者而终止。
- 重要流程
  - 领导选举（Leader Election）
    - 当系统中没有领导者存在时，候选人可以发起领导选举
    - 候选人会发送选举请求，并通过选举轮次和投票机制选择出一个新的领导者
  - 日志复制（Log Replication）
    - 领导者负责处理客户端请求，并将操作复制到跟随者的日志中
    - 一旦大多数节点成功复制该日志，领导者会通知跟随者应用日志操作
  - 安全性（Safety）：
    - Raft算法具有强的安全性特性，日志中只有经过多数派节点确认的操作才能被认为是已提交的
- 优点

- 相对容易理解和实现，降低了开发和维护的难度。
- 提供了拥有强安全性和一致性保证的分布式一致性解决方案。
- 具备领导者选举机制，使得系统能够快速响应客户端请求。

8-集群里面全部节点刚启动，Raft里面的选举流程是怎么的?

- 题目分析和考点
  - 面试官想确认应聘者是否对一致性算法有掌握
  - 有弱一致性算法和强一致性算法，而Raft属于强一致性算法
  - 与Paxos算法相比，Raft算法更易理解和实现，并且在分布式系统中得到了广泛应用
- 难度【* * * * *】
- 答案
  - 地址：https://raft.github.io/raftscope/index.html



  - 流程步骤
    - 节点状态初始
      - 当集群中的所有节点都刚启动时，它们都处于初始状态，并且没有领导者。
    - 发起选举

        - 初始状态的节点会成为候选者，并向其他节点发送选举请求（称为RequestVote请求）
        - 请求中包含候选者的任期号和候选者的信息。
    - 响应投票
        - 收到选举请求的节点会评估请求，并根据一定的条件进行投票
        - 节点只会为尚未投票或认为候选者任期号更高的候选者投票。
    - 获得多数投票
        - 如果候选者收到了来自集群中大多数节点的投票（包括自己），则它将成为领导者，并开始发送心跳信号以维持其领导者地位。
    - 选举失败
        - 如果候选者在选举过程中没有获得多数投票，它将重新开始新的选举过程
        - 候选者会增加自己的任期号，再次发送选举请求，以尝试获得多数选票。
    - 防止竞争者的选举
        - 当节点收到具有更高任期号的领导者的心跳信号时，它将放弃自己的候选者身份并转换为追随者，以避免出现多个领导者的情况。
- 注意
    - Raft选举是基于节点任期号及其比较的机制
    - 节点的任期号是一个递增的值，用于标识不同的选举周期
    - 节点只会投票给任期号更高的候选者，以确保只有最新的候选者成为领导者。

- 总结：集群里面全部节点刚启动，Raft里面的选举流程
    - 看哪个节点先超时，成为候选者
    - 候选者 先修改自己的trem任期+1，并投票给自己
    - 然后向其他节点发起request vote，进行拉选票
    - 由于只有1个候选人，所以其他节点都会把票投给这个节点，投票的节点都会把自己的term加1
    - 候选者收到了多数的投票后，状态就变成了Leader状态，并定时和其他节点保持心跳
        - 如果某个节点的得票数超过集群节点个数的一半以上，就会成为Leader节点

系统设计板块

1-互联网项目的三高设计是指哪些?

- 题目分析和考点
  - 面试官想知道应聘者对互联网项目的三高设计是否掌握
  - 是否知道互联网下的三高设计具体指哪些，包括 高并发、高可用、高性能
- 难度【 * * * 】
- 答案
  - 互联网下的三高设计：高并发、高可用、高性能
  - 高并发
    - QPS/TPS 来衡量系统的对任务的处理能力
      - TPS
        - Transactions Per Second 每秒事务数, 可以是一个接口、多个接口、一个业务流程, 包括增删改操作
      - QPS
        - Queries Per Second， 每秒查询数, 指一台服务器每秒能够响应的查询次数
        - QPS 只是一个简单查询的统计，不能描述增删改等操作
        - 如果只是查询操作 TPS = QPS
    - 大流量业务场景很多：FaceBook(Meta). 日活用户19.8 亿、抖音日活7亿
  - 高可用
    - SLA 衡量一个系统可用性有多高，目标系统 7 x 24 小时不间断服务，云厂商在宣传自己产品SLA时多少个9
    - 分类
      - 时间维度：系统可以正常使用时间与总时间之比（全年为例子）1年 = 365天 = 8760小时
        - 99.9 = 8760 * 0.1% = 8760 * 0.001 = 8.76小时
        - 99.99 = 8760 * 0.0001 = 0.876小时 = 0.876 * 60 = 52.6分钟
        - 99.999 = 8760 * 0.00001 = 0.0876小时 = 0.0876 * 60 = 5.26分钟
        - ...
      - 请求次数维度：请求总次数和失败的占比 (1000次请求为例子，相对简单)
        - 系统可用性99%：表示1000个请求中允许1000 * (1- 99%) = 10个请求出错
        - 系统可用性99.9%：表示1000个请求中允许1000 * (1- 99.9%) = 1个请求出错。
        - ...
    - 9越多代表全年服务可用时间越长服务更可靠，停机时间越短
    - 但往往存在网络/机房问题，应用更新发版导致服务不可用
    - 大厂多数业务4个9是刚需，5个9是目标，6个9是理想
  - 高性能
    - RT来衡量系统的响应速度，程序处理速度非常快延迟低Latency，所占内存少，cpu占用率低
    - 比如系统处理一个 HTTP 请求需要 100ms，这 100ms 就是系统的响应时间

- 三高不是孤立的，而是相互支撑的影响的
  - 高并发方面要求Throughput 大于 10万
  - 高性能方面要求请求延迟Latency小于 100 ms
  - 高可用方面要求系统可用性 SLA高于 99.99%
  - 但随着并发量上来，请求延迟肯定增大，处理不过来则可用性就会下降

2-介绍下架构设计里面的高并发-高性能技术方案有哪些?

- 题目分析和考点
  - 面试官想知道应聘者对互联网项目的三高设计是否掌握
  - 对高并发-高性能技术方案是否知道具体有哪些方式去实现
- 难度【 * * * 】
- 答案
  - 高并发技术有哪些（高并发和高性能是紧密相关）
    - 多大并发才算高并发？需要结合具体的场景和资源投入
      - 1万QPS的商品列表查看不属于高并发，稍微结合缓存即可
      - 5千TPS的下单接口属于高并发，链路不一样
    - 系统架构
      - 无状态业务-水平扩展（Scale Out），只要增加服务器数量，就能线性扩充系统性能
      - 架构的难点是难做到全链路的水平扩展
    - 【负载均衡】思想
      - 节点轮询、随机、加权轮询、节点固定hash
      - 哪里应用到?
        - 网络 DNS解析轮询、网关分发请求后端服务
        - 应用服务内部RPC负载均衡、数据存储-分库分表-负载分发
    - 【缓存】思想
      - 本地缓存/分布式缓存
      - 哪里应用到?
        - 前端浏览器缓存静态资源、网络DNS解析缓存
        - 应用程序 内存缓存/分布式缓存、数据存储Mysql Query Cache
    - 【池化复用】思想
      - 线程池/对象池/连接池/内存池

        - 哪里应用到
            - java线程池技术、Jdbc/Redis/HttpClient连接池、SpringIOC容器对象池
    - 【异步】思想
        - 多线程/消息队列
        - 哪里应用到?
            - 前端ajax异步请求、RocketMQ/Kafka 同步双写-异步刷盘、应用程序多线程异步处理
    - 【预处理-惰性更新】思想
        - 定时任务/懒加载
        - 哪里应用到?
            - 运营后台报表数据，定时任务提前计算好、Mybatis懒加载
    - 【分而治之】思想
        - Mater-worker
        - 哪里应用到
            - Hadoop中的MapReduce、JDK. Fork/Join Framework、消息队列的广播消息、归并排序算法

3-介绍下架构设计里面的高可用技术方案有哪些?

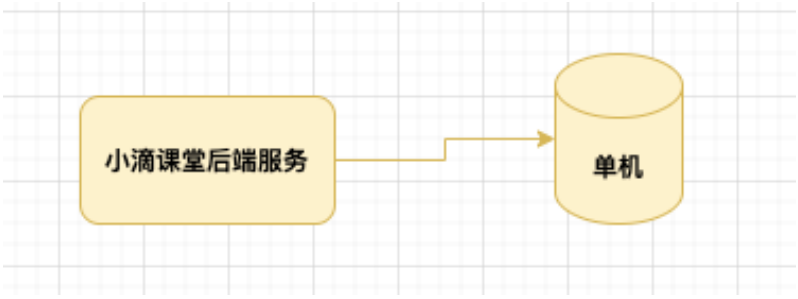
- 题目分析和考点
    - 面试官想知道应聘者对互联网项目的三高设计是否掌握
    - 对高可用技术方案是否知道具体有哪些方式去实现
- 难度【 * * * 】
- 答案
    - 高可用
        - SLA 衡量一个系统可用性有多高，目标系统 7 x 24 小时不间断服务，云厂商在宣传自己产品SLA时多少个9
        - 9越多全年服务可用时间越长服务更可靠，停机时间越短，大厂多数业务4个9是刚需-5个9是目标-6个9是理想
    - 技术方案（冗余集群化 + 自动故障转移failover）
        - 集群架构
            - 将多个相同的应用程序集中起来提供同一种服务，某个节点故障不影响系统
            - 可以横向扩展性增加节点提高并发处理能力
            - 实际应用
                - 微服务集群、Redis集群/Kafka集群/Nginx集群、Nacos集群/Mysql集群/ZK集群
        - 熔断降级

        - 保险丝，熔断服务，为了防止整个系统故障，抛弃一些非核心的接口和数据，返回兜底数据
    - 限流
        - 当访问频率或者并发请求超过其承受范围的时候，考虑限流来保证接口的可用性
        - 漏斗模型，不管流量多大均匀的流入容器，令牌桶算法，漏桶算法
    - 隔离
        - 服务和资源互相隔离，比如网络资源，机器资源，线程资源等，不会因为某个服务的资源不足而抢占其他服务的资源
    - 多活架构
        - 同城双活-双机房
            - 两个机房部署在同城，物理距离较近，两个机房用「专线」网络连接，比单个机房内延迟要大一些，但整体的延迟是可以接受的
        - 参考
            - 同机房：0.1ms
            - 同城双机房：1ms（100公里内）
            - 北京到广州：55ms
    - 异地多活-两地三中心
        - 两地是指 2 个城市，三中心是指有 3 个机房，其中 2 个机房在同一个城市
        - 同时提供服务，第 3 个机房部署在异地，只做数据灾备

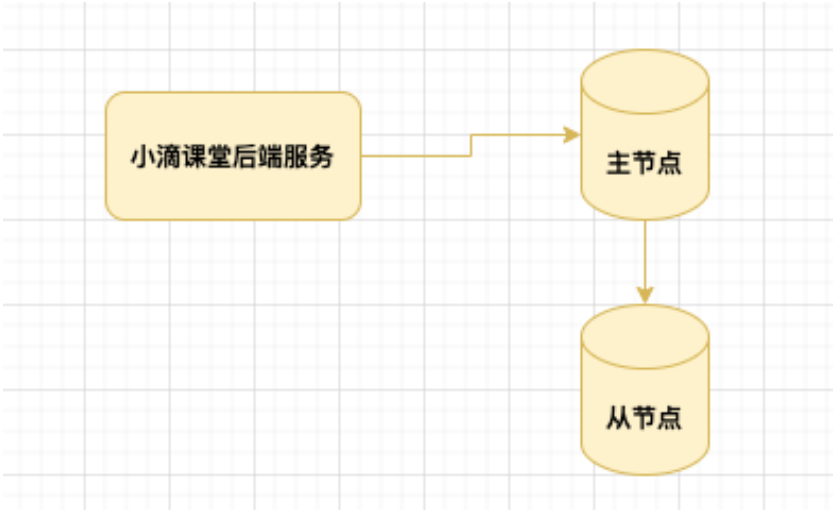
Mysql数据库板块

1-说下互联网架构下的Mysql主流架构方案概述和优化思路

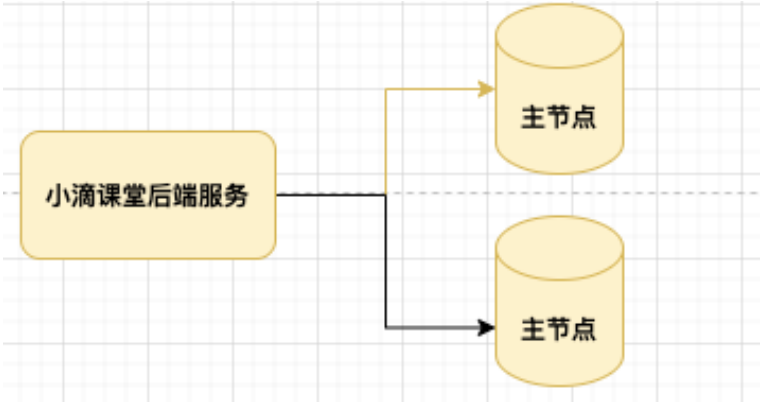
- 题目分析和考点
    - 面试官想要了解你对主流的 MySQL 架构方案有多少了解，包括优点、适用场景和限制
    - 对于互联网架构下的 MySQL，高可用性和容错性是非常重要的，面试官希望你能提到相关的方案和策略
    - 关注你是否能意识到数据库读写的性能瓶颈，并提出合理地的建议。
- 难度【* * * *】
- 答案
    - 单机
        - 请求量大查询慢
        - 单机故障导致业务不可用



- 主从（**推荐**）
  - 数据库主从同步，从库可以水平扩展，满足更大读需求
  - 单服务器TPS，内存，IO都是有限的
  - 索引选择
    - 主库只提供写，不建索引
    - 从库提供读，上面建立索引，提高读能力，可以一主多从，但是复制延迟会增大

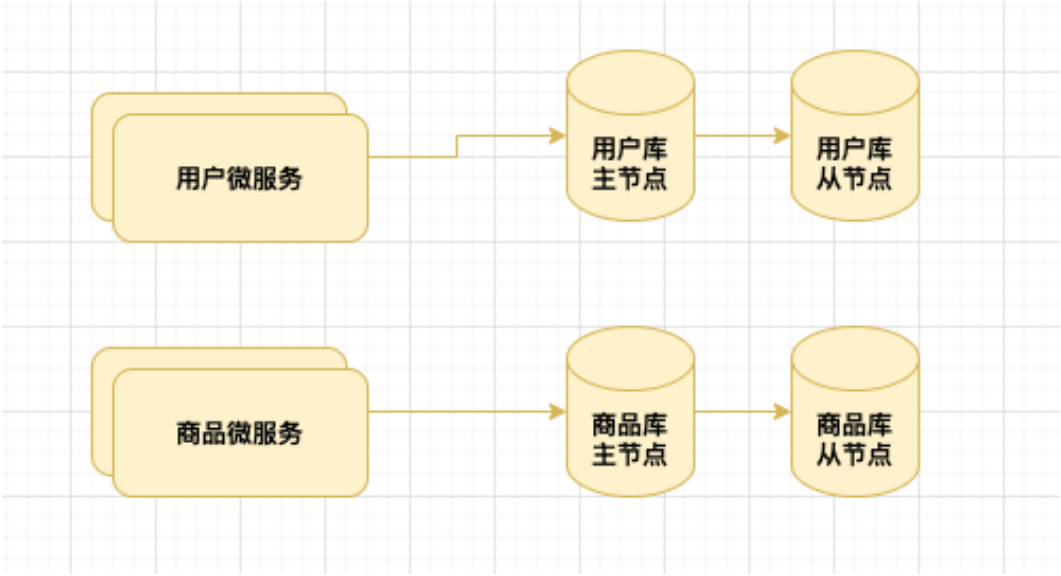


- 双主
  - 用户量级上来后，写请求越来越多
  - 一个Master是不能解决问题的，添加多了个主节点进行写入，
  - 多个主节点数据要保存一致性，写操作需要2个master之间同步更加复杂



- 分库和分表（**推荐**）
  - 垂直角度（表结构不一样）
    - 垂直分表: 将一个表字段拆分多个表，每个表存储部分字段

        - 好处: 避免IO时锁表的次数，分离热点字段和非热点字段，避免大字段IO导致性能下降
    - 垂直分库：根据业务将表分类，放到不同的数据库服务器上（微服务划分）
        - 好处：避免表之间竞争同个物理机的资源，比如CPU/内存/硬盘/网络IO
- 水平角度（表结构一样）
    - 水平分库：把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上
        - 好处: 多个数据库，降低了系统的IO和CPU压力
    - 水平分表：同个数据库内，把一个表的数据按照一定规则拆分到多个表中，对数据进行拆分，不影响表结构
        - 单个表的数据量少了，业务SQL执行效率高，降低了系统的IO和CPU压力
- 问题
    - 跨节点数据库Join关联查询、分库操作带来的分布式事务问题、数据库全局主键重复问题、执行的SQL排序、翻页问题



2-数据库单表1千万数据，1年会增长500万，性能比较慢说下优化思路

- 题目分析和考点
    - 面试官想要了解你对Mysql数据库性能优化是否有关注，慢查询诊断等
    - 性能优化不是个细节点，一定要看情况具体调整，不同的情况适合不同的思路
- 难度【* * * *】
- 答案
    - 数据库性能优化方法论
        - 千万不要一上来就说分库分表，这个是最忌讳的事项
        - 一定要根据实际情况分析，两个角度思考
            - 不分库分表
                - 软优化

        - 数据库参数调优
        - 分析慢查询SQL语句，分析执行计划，进行sql改写和程序改写
        - 优化数据库索引结构
        - 优化数据表结构优化
        - 引入NOSQL和程序架构调整
      - 硬优化
        - 提升系统硬件（更快的IO、更多的内存）：带宽、CPU、硬盘
    - 分库分表
      - 根据业务情况而定，选择合适的分库分表策略（没有通用的策略）
        - 外卖、物流、电商领域
      - 先看只分表是否满足业务的需求和未来增长
        - 数据库分表能够解决单表数据量很大的时,数据查询的效率问题，
        - 无法给数据库的并发操作带来效率上的提高，分表的实质还是在一个数据库上进行的操作，受数据库IO性能的限制
      - 如果单分表满足不了需求，再分库分表一起
  - 结论
    - 在数据量及访问压力不是特别大的情况，首先考虑缓存、读写分离、索引技术等方案
    - 缓存方案上个版本重点讲了，存在的问题和对应的解决方案
    - 如果数据量极大，且业务持续增长快，再考虑分库分表方案

3-说下数据库主流引擎Innodb和myisam存储内容分析

- 题目分析和考点
  - 面试官想要了解你对Mysql存储引擎的掌握，不同引擎的特性情况
- 难度【* *】
- 答案
  - 主流引擎Innodb和myisam对比（从MySQL 5.5版本开始默认使用InnoDB作为引擎）

	innodb	myisam
事务和外键	支持事务和外键,具有安全性和完整性	不支持事务和外键，提供高速存储和检索
锁	行级锁，锁定指定记录	表级锁，锁定整张表
索引结构	聚簇索引, 索引和数据在一起存储	非聚簇索引,索引和数据分开
并发处理能力	与隔离级别有关，采用多版本并发控制(MVCC)来支持高并发	表锁，会导致写操作并发率低
存储文件	一个.frm表结构文件，一个.ibd数据文件	一个.frm表结构文件，一个MYD表数据文件，一个.MYI索引文件
文件大小	最大支持64TB	最大支持256TB

- 数据文件
  - 存储引擎是myisam, 在data目录下会看到3类文件
    - frm
      - 每一个mysql表都使用.frm文件格式描述表结构定义，文件名和表名一致，.frm格式在所有平台上都是一样的
      - **在MySQL8后没有了frm文件，表结构存储在 表名.xxx.sdi**
    - myd
      - MyISAM 存储引擎专用，存放 MyISAM 表的数据，每一张表都会有一个 .MYD 文件
    - myi
      - MyISAM 存储引擎专用索引信息文件，是表数据文件中任何索引的数据树
  - 存储引擎是InnoDB, 在data目录下会看到2类文件
    - frm
      - 表结构的文件，在MySQL8.0后没有了frm文件，表结构存在/var/lib/mysql/ibdata1 目录中，是共享表空间文件
      - 此文件存储了所有使用InnoDB引擎的表的数据和元数据，包括InnoDB表结构、行数据等
      - 启用innodb_file_per_table选项时，InnoDB表会在磁盘上为每个表创建一个单独的.ibd文件来存储数据和索引
      - 如果创建表时明确禁用innodb_file_per_table选项，则表结构和数据都存储在共享表空间文件中（即ibdata1文件）
    - ibd
      - 表数据和索引的文件,表的索引(B+树)的每个非叶子节点存储索引，叶子节点存储索引和索引对应的数据。

        - 是InnoDB存储引擎中一个独立的文件，存放 InnoDB 的数据文件，它保存了表数据和索引的实际内容
        - 对于每个InnoDB表，都会有一个对应的.ibd文件

4-介绍Mysql慢查询日志作用和如何开启配置慢查询

- 题目分析和考点
    - 面试官想要了解你对Mysql慢查询的掌握情况
    - 数据库性能优化里面，慢查询是特别关键的知识点，多数sql性能都是可以通过慢查询发现出来
- 难度【* * * * 】
- 答案
    - MySQL慢查询日志
        - MySQL数据库自带的一个功能，用于记录执行时间超过指定阈值的SQL语句，以便于后续的性能优化工作
        - 帮助开发和DBA发现哪些SQL语句需要优化，在哪些地方需要修改，以提高数据库的性能
        - 默认情况下MySQL数据库是不开启慢查询日志的，long_query_time的默认值为10（即10秒，通常设置为1秒）
        - 慢查询日志记录 SQL 语句相关信息包括但不限于以下内容：
            - 执行时间、执行次数、告警时间、执行的 SQL 语句、使用的索引、扫描的行数、等待锁的时间

```
# Time: 2023-05-29T07:16:22.801248Z
# User@Host: root[root] @  [119.130.128.200]  Id:   706
# Query_time: 2.000534  Lock_time: 0.000000 Rows_sent: 1  Rows_examined: 1
SET timestamp=1685344580;
select * from product


Time: 表示日志记录的时间。格式为 ISO8601 标准的 UTC 时间。
User@Host: 表示执行该SQL语句的用户和主机名。
Id: 表示线程 ID，是一个整数表示。
Query_time: 表示 SQL 语句的执行时间，以秒为单位。
Lock_time: 表示在执行 SQL 语句期间等待锁定资源的时间。如果 SQL 语句没有使用锁，则为 0。
Rows_sent: 表示 SQL 语句返回的行数。
Rows_examined: 表示查询过程中扫描的行数。如果 SQL 语句没有执行查询操作，则该值为 0。
timestamp: 表示该 SQL 语句的执行时间戳，以秒为单位。
sql语句: 实际执行的SQL语句。
```

        - 通过分析慢查询日志，知道哪些 SQL 语句需要进行优化并进行相应的优化，以提高数据库的性能和效率
    - Mysql慢查询日志配置
        - 查询状态 `show variables like 'slow%'`
            - slow_query_log属性是OFF，处于关闭状态，也可以通过 0和1进行配置，1表示开启，0表示关闭
                - 开启慢查询日志 `set global slow_query_log = on`
            - slow_query_log_file表示慢查询日志文件的存放路径
                - `set global slow_query_log_file = '自定义路径'`

```
show variables like 'slow%'
```

Message

Variable_name	Value
slow_launch_time	2
slow_query_log	OFF
slow_query_log_file	/var/lib/mysql/e002648baa26-slow.log

        - 如何定义慢sql语句 `show variables like '%long%'`
            - long_query_time属性，值为10.000000，表示的只记录查询时间在10s以上的语句
            - 数据不多的情况，慢查询的临界值设置为0.02：`set long_query_time=0.02`

```
show variables like '%long%'
```

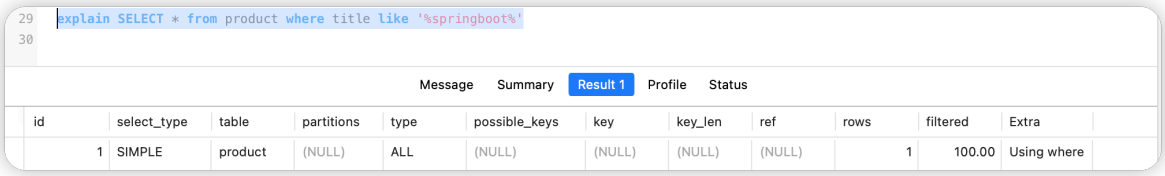
Variable_name	Value
long_query_time	10.000000
performance_schen	10000
performance_schen	10000
performance_schen	10000
performance_schen	10000

5-Mysql的执行计划分析，里面常用的有哪些关键字段?

- 题目分析和考点

- 面试官想要了解你对Mysql执行计划是否清楚，里面的查询类型，索引使用和排序是否掌握
- 属于常用的内容，因此对里面的各个字段都需要了解掌握好

- 难度【* * * * 】
- 答案
  - Mysql的执行计划
    - EXPLAIN 是用于分析并优化查询语句性能的工具，执行计划会解析查询语句并生成执行计划
    - 包括访问表和索引所需的策略、查询优化器的选择以及每个阶段的估计记录数等信息，深入SQL语句执行过程中的各个细节



  - 字段说明
    - id
      - 对于 SELECT 语句，每个查询都会被分配一个唯一的ID
      - 表示查询的标识符，数字越大越先执行
    - select_type
      - 表示查询类型或者子查询类型，使用不同的 select_type 来帮助评估查询性能，并确定可以采取哪些优化方法
      - 需要根据具体情况来进行相应的优化，例如尽量减少子查询的数量，避免使用不必要的 UNION 操作等等

类型	概述
SIMPLE	表示简单的 SELECT 查询，不包含子查询或 UNION 操作。
PRIMARY	表示外层查询的第一个 SELECT
UNION	表示 UNION 操作的第二个或后续的 SELECT 查询。
SUBQUERY	表示一个子查询，MySQL 会在子查询中先执行查询，比如where里面包括了子查询
DEPENDENT SUBQUERY	也表示一个子查询，但是外部 SQL 查询的结果会影响子查询的执行
DERIVED	表示派生表，MySQL 会在查询中创建一个新的临时表，这个临时表来自于 FROM 子句中的子查询
UNION RESULT	表示 UNION 操作的结果，MySQL 在创建结果集时使用临时表来存储数据

    - table
      - 表示查询涉及到哪些表，对于子查询等复杂查询可能涉及多张表

- partitions
  - 表示查询操作涉及到的分区表的分区情况
- type
  - 表示 MySQL 在表中找到所需行的方式，常见的类型包括 ALL, index，range, ref, eq_ref, const, system, NULL
  - 下面表格是性能由 低 到 高，all是最差的

Type	概述
all	全表扫描，MYSQL扫描全表来找到匹配的行
index	索引全扫描，MYSQL遍历整个索引来查找匹配的行；Extra 字段里面 出现 Using index，则是覆盖索引，不用二次回表查询
range	索引范围扫描，常见于<、<=、>、>=、between、in等操作符；相对于index的全索引扫描，它有范围限制，因此要优于index
ref	使用非唯一性索引或者唯一索引的前缀扫描，返回匹配某个单独值的记录行； 虽使用了索引但该索引列的值并不唯一,进行目标值附近的小范围扫描，不扫描全表
eq_ref	eq_ref 与 ref对比结果集只有一个，使用主键或者唯一索引进行查找,不用扫描更多行
const	最多只有一条匹配行，查询非常迅速，用到primary key 或者unique key，性能最高
system	表只有一行，基本不会出现，忽略
null	不访问数据库表，直接返回索引

  - **常规高频的sql推荐需要达到 range 级别以上，最好是ref以上**
- possible_key:
  - 表示 MySQL 可以使用哪些索引来优化查询
- key
  - 表示 MySQL 实际使用的索引，如果没有使用任何索引，则该值为 NULL
- key_len
  - 表示 MySQL 实际使用的索引的长度，该值与索引定义的长度有关
- ref
  - 表示 MySQL 使用哪个列或常量与索引列进行比较
- rows
  - 表示 MySQL 估计要扫描多少行才能找到所需记录，是一个估算值而不是确切值
- filtered
  - 查询条件过滤的效率，百分比形式表示， `Filtered` 越高，表示查询结果集中过滤数据所需要的开销越小，查询性能就越好

- Extra
  - 该字段包括一些额外的查询信息，包括使用何种排序方式、使用哪种 Join 操作等

类型	概述
Using index	选择使用了覆盖索引的特性，通过索引直接获取查询结果，而无需回表查询，提高了查询效率。
Using filesort	需要额外进行 一个文件排序操作来实现 ORDER BY 操作，可能会严重影响查询性能。
Using temporary	在执行查询时需要借助临时表来保存中间结果集，这常发生在排序、分组、子查询和 UNION 查询之中。
Using where	条件查询，在查询过程中需要进行表级别的条件过滤，即使共享了某些索引，也需要进行全表扫描查找符合条件的行。不是仅仅通过索引就可以获取所有需要的数据,则会出现 Using where
Range checked for each record	通过索引比较操作来过滤部分行，直到找到符合条件的行，这种操作常出现在使用 INDEX 和 ORDER BY 操作时。
Using join buffer (Block Nested Loop)	在执行连接操作时需要额外申请 join buffer 来存储中间结果，这种操作常发生在连接操作中。
Using index condition	利用了查找索引数据的过程中额外发现的过滤条件进行了优化，无需回表查询或查表，可以直接通过索引结果来返回查询的结果
Using sort_union() 和 Using union()	通过 UNION ALL 或 UNION DISTINCT 操作来合并查询结果集，使用了一些优化策略来提高查询效率。

6-Mysql性能优化-说下发现慢查询后的优化思路?

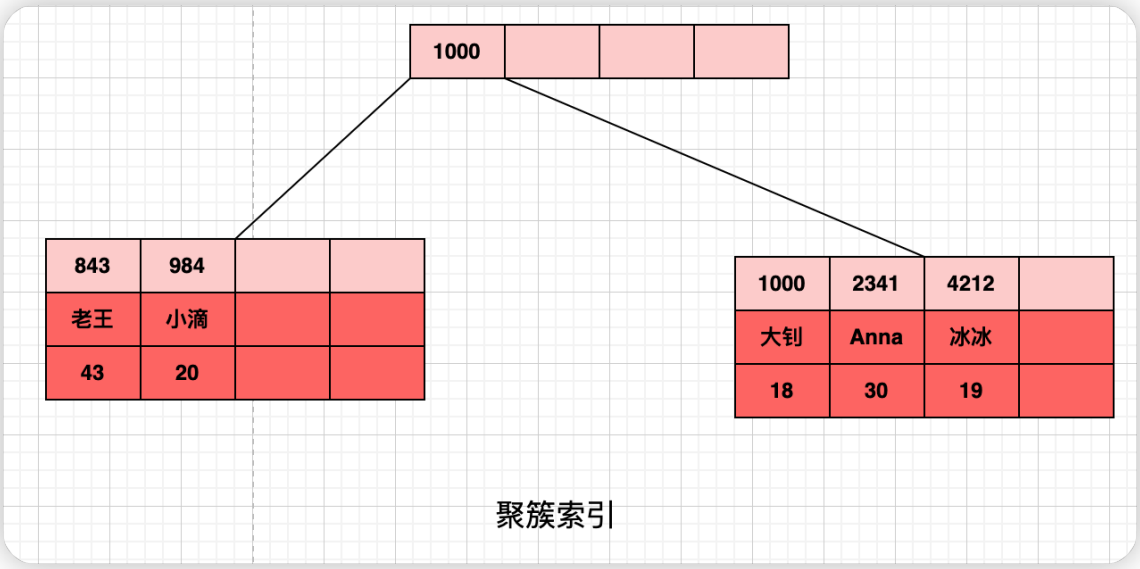
- 题目分析和考点
  - 面试官想要了解你对Mysql慢查询的掌握情况，针对慢查询是否有比较好的解决方案
  - 是否能掌握可以定位慢查询并针对性地进行优化，提升MySQL数据库的性能和响应效率
  - 优化的选择和策略应根据具体情况进行决定
- 难度【* * * *】
- 答案

- 使用性能分析工具：如EXPLAIN语句、SHOW PROFILES和SHOW STATUS等，查看查询执行计划、资源消耗和性能指标
- 根据对应的指标进行调整，根据慢查询的原因，常见优化思路有下面几个
- 优化索引
  - 确保表的列上有适当的索引：通过EXPLAIN分析查询计划，检查索引是否能够有效地支持查询，如果没有，可以创建或修改适当的索引。
  - 确保索引是高选择性的：索引的选择性代表索引列上不同值的比例，较高的选择性可以提高查询性能。
  - 避免过多或过长的索引：过多的索引会增加写操作的开销，过长的索引会增加存储和维护的开销。
- 优化查询语句
  - 简化复杂的查询：将复杂查询拆分为多个简单的查询，减少多表关联和子查询的使用。
  - 使用合适的查询操作符和函数：避免全表扫描，使用索引字段，优化WHERE条件和排序操作。
  - 避免过多的数据传输：限制返回结果集的行数和列数，使用合适的LIMIT和分页技术。
- 优化数据库配置
  - 调整缓冲区大小：如将innodb_buffer_pool_size设置为适当值，以减少磁盘I/O操作。
  - 调整线程池设置：适当配置线程池参数，控制并发连接数，避免消耗过多的资源。
  - 合理配置存储引擎：选择合适的存储引擎，如InnoDB或MyISAM，根据具体情况调整配置。
- 数据库结构优化
  - 合理划分表：如按功能、频繁访问的数据等进行划分，减小表的大小和查询的复杂度。
  - 垂直分割表：将大表拆分成多个小表，避免不必要的IO访问和冗余数据的存储。
  - 避免频繁的表结构修改：频繁的表结构修改会引起锁表，并影响数据库性能。

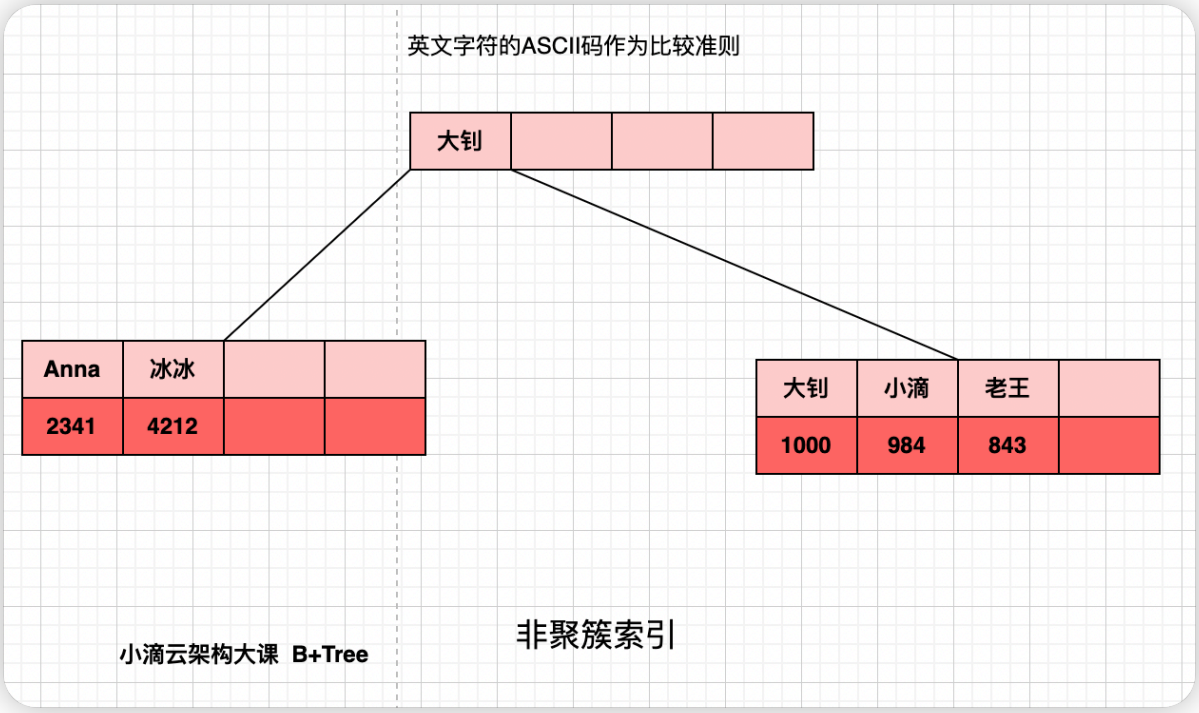
7-介绍下Mysql里面聚簇索引和非聚簇索引有什么区别?

- 题目分析和考点
  - 面试官想要了解你对Mysql索引分类的掌握
  - 聚簇索引和非聚簇索引的联系和区别，包括不限于底层数据结构
- 难度【* * *】
- 答案
  - 聚簇索引 clustered index
    - 表数据文件按B+Tree组织的，叶节点data域保存完整**行数据**, 树上的key就是主键, 以主键构建的B+树索引
    - 聚簇索引一般为主键索引，而主键一个表中只能有一个，**所以聚集索引一个表只能有一个**

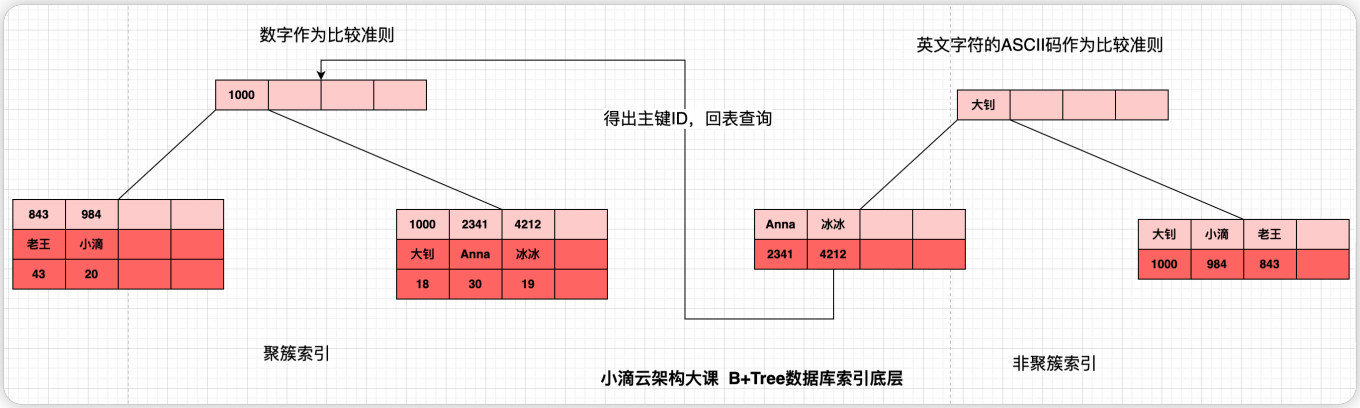
- 聚簇索引叶子节点存储的是行数据，而非聚簇索引叶子节点存储的是聚簇索引（通常是主键 ID）



- 非聚簇索引 nonclustered index
  - 索引结构的叶子节点放的是指向数据的主键（或者是地址）构建的B+树索引
  - 这种索引叫做非聚集索引、二级索引、辅助索引，非聚集索引一个表可以存在多个
  - 叶子节点中保存的不是实际数据，而是主键，获得主键值后去聚簇索引中获得数据行



- 注意
  - 非聚簇索引的叶子节点上存储的并不是真正的行数据，而是主键 ID或记录的地址
  - 当使用非聚簇索引进行查询时，会得到一个主键 ID，再使用主键 ID 去聚簇索引上找真正的行数据，这个过程称为**回表查询**
  - 所以聚簇索引查询效率更高，而非聚簇索引需要进行回表查询，性能不如聚簇索引



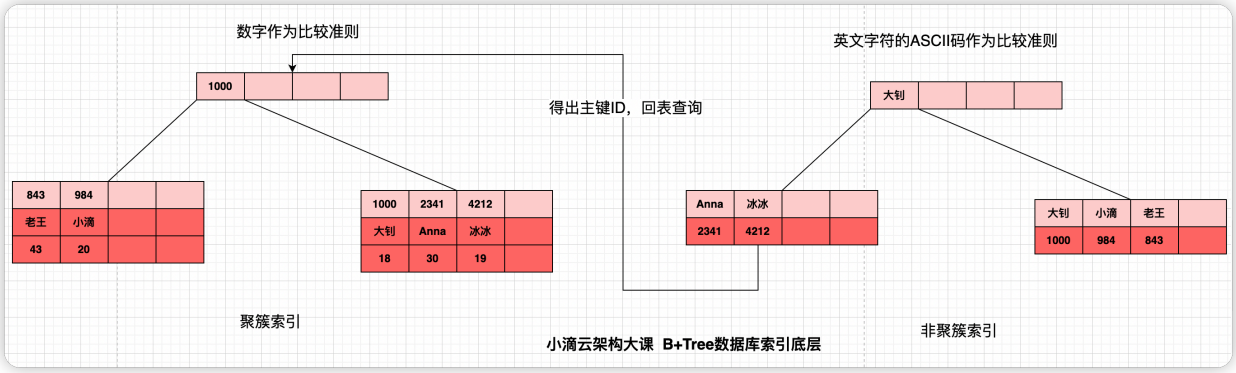
8-SQL性能优化里面索引很多，介绍下联合索引-覆盖索引-回表查询

- 题目分析和考点
  - 面试官想要了解你对Mysql索引分类的掌握
  - 是否索引很多同学容易混淆，如果只是简单背下来，没用充分理解好，则回答不上
- 难度【***】
- 答案
  - 什么是联合索引
    - 常规表中只会选择一个列作为索引字段，但是在某些特殊情况下，需要将多个列共同组成 一个索引字段，称之为联合索引
    - 语法

```
ALTER TABLE `table_name` ADD INDEX index_name ( `column1`, `column2`, `column3`,`column4` )
```

  - 案例
    - user表有个4个字段：id，name，age，phone四个列，id为主键，name普通索引
  - 回表查询
    - 在使用索引进行查询时，MySQL 执行的额外一次查询操作，用于从数据表中检索相关数据，并将其返回给用户
    - 语句 `select * from table where name = "冰冰"`
    - 当执行这个sql时发生回表，从 【非聚簇索引】的叶子结点中获取id值，根据id再去 【聚簇索引】中获取全部数据

- 在实际开发中，回表查询次数太多会影响查询性能，多使用包含查询到的所有列的索引（称为“覆盖索引”）来避免回表查询



  - 覆盖索引
    - 当一个查询请求从表中检索数据时，可以通过查询所需要的信息在索引中直接获得，不必回到数据表本身去访问数据表
    - 语句 `select id,name from user where name = "冰冰"`
    - 当执行这个sql，发生【索引覆盖】从索引的叶子结点能获取到全量查询的列，因为id和name都在【二级索引】中
    - **简单来说：如果一个索引可以覆盖一个查询，那么它就是一个覆盖索引**
    - 可以避免在数据表中进行回表查询，从而减少对磁盘 IO 操作的需求，提高查询性能和执行效率

9-覆盖索引一定是联合索引吗，联合索引一定是覆盖索引吗?

- 题目分析和考点
  - 面试官想要了解你对Mysql索引分类的掌握
  - 是否索引很多同学容易混淆，如果只是简单背下来，没用充分理解好，则回答不上
- 难度【***】
- 答案
  - 覆盖索引不一定是联合索引，它是指仅通过索引本身就能够获取到查询结果所需的所有列，而不必回到主表中去查询的情况。
  - 覆盖索引
    - 当一个索引覆盖了一个查询时，这个索引包含了查询时所需的所有列
    - 如果查询语句只需要从表中检索少量列数据，而索引正好包含了这些列，那么它就可以被认为是一个覆盖索引
    - 这个索引可能由单个字段或者多个联合字段构成
  - 联合索引
    - 联合索引可以包含多个字段，但并非所有联合索引都是覆盖索引

- 当查询语句需要使用联合索引来加速查询时，如果索引中包含了查询所需的所有列信息，那么这个联合索引就是覆盖索引
- 否则，它不是覆盖索引，可能会发生回表查询操作
- 案例
  - 建一个联合索引 ,`idx_name_age_phone` 为索引名称，`name`、`age` 和 `phone` 分别为需要建立索引的列

```
CREATE INDEX idx_name_age_phone ON user(name, age, phone);
```

  - 使用联合索引，并且是覆盖索引的 SQL 语句如下

```
SELECT name, age, phone FROM user WHERE name = 'xxx' AND age = xx and phone
= 'xxx';
```

  - 注意
    - 由于使用了联合索引来覆盖查询，语句中只查询了索引中涉及到的列，而未查询其他列，也就避免了回表查询。
    - 如果这个联合索引只包含 `name` 和 `age` 两列，则该查询无法使用一个覆盖索引，MySQL 仍将需要进行回表查询

10-Mysql创建多个字段联合索引，如何判断sql采用了哪些字段

- 题目分析和考点
  - 面试官想要了解你对Mysql索引是否掌握
  - 日常数据库性能优化中，索引是最关键的优化，原则很多人都能背，但是真正去操作的是少数，停留表面是不行的，
  - 虽然很多人知道索引好处，明明用了索引却还是慢，是加错了还是没生效?
  - 一个表有多个索引，如何判断sql执行是哪个索引？这些核心技能都是高级工程师需要具备的
- 难度【****】
- 答案
  - 可以通过Mysql执行计划里面的字段 **key_len**（字节数）
    - 表示 MySQL 实际使用的索引的长度，该值与索引定义的长度有关，索引长度越短越好，非精确统计
    - 如何计算索引长度，联合索引是多个字段列的bytes相加
      - 比如varchar(N)里的N是字符数，如果是utf8字符集时，1个字符占用3个字节

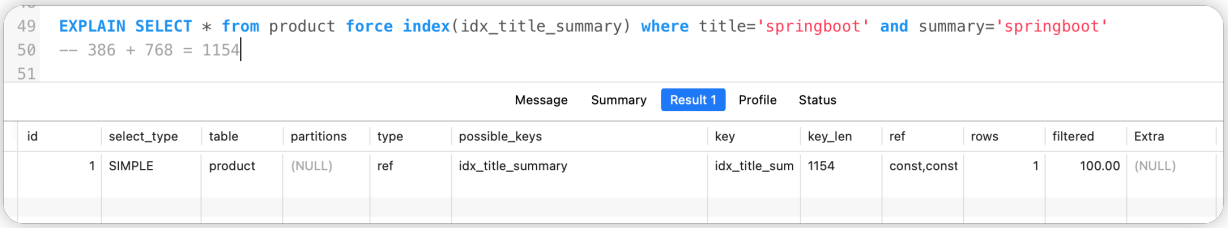
        - Mysql 5.0版本以上 varchar是存储字符，之前是字节数
    - 后面加的2Bytes是用来表示字段长度的，可算出该索引占用多少bytes
    - 字符串（utf8编码）
        - char(n)：n字节长度
        - varchar(n) 长度是3n+2 ， 需要2字节存储字符串长度，
    - 数值
        - tinyint 1字节
        - int 4字节
        - bigint 8字节
    - 时间
        - date 3字节
        - timestamp 4字节
        - datetime 8字节
- 案例：下面的语句

```
CREATE TABLE `product` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `title` varchar(128) NOT NULL,
  `cover_img` varchar(255) DEFAULT NULL COMMENT '封面图',
  `amount` decimal(10,2) DEFAULT NULL COMMENT '现价',
  `summary` varchar(255) DEFAULT NULL COMMENT '概要',
  `detail` longtext COMMENT '详情',
  `gmt_modified` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  `gmt_create` datetime DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb3 ROW_FORMAT=DYNAMIC;


#查看表的相关索引
show INDEX from product

CREATE INDEX idx_title_summary on product(title,summary)

CREATE INDEX idx_summary on product(summary)

CREATE INDEX idx_title on product(title)
```

- 如果字段可以是null，需要1个字节记录是否可以为null，如果允许为null，则需要 +1 个字节存储

```
EXPLAIN SELECT * from product where title='springboot'
--不为null (3 * 128+2 = 386)
--为null (3 * 128+2 + 1 = 387)

EXPLAIN SELECT * from product where  summary="springcloud"
--可以为null (3 * 255 + 2 + 1 = 768)


EXPLAIN SELECT * from product force index(idx_title_summary) where
title='springboot' and summary='springboot'
-- 386 + 768 = 1154
```

- 索引可以通过key_len计算采用联合索引里面的哪几个字段



- 备注

```
utf-8编码    (uft-8也称之为utf-8mb3，修改表结构为utf8mb3)
  1个字节=8个位
  1个中文字符=3个字节
  一个英文字母字符存储需要1个字节


LENGTH(str) 返回字符串str的长度，以字节为单位
CHAR_LENGTH(str)  返回字符串str的长度，以字符为单位
```

11-建表语句定义字段类型 int（10）和int(1) 区别是啥?

- 题目分析和考点
  - 面试官想要了解你对Mysql字段类型是否掌握
- 难度【***】
- 答案
  - **如果不使用 zerofill 是没有任何区别的**
  - 在mysql中 int占4个字节，那么对于无符号的int，最大值是2^32-1 = 4294967295，将近40亿
  - int后面的数字不能表示字段的长度，是一样大小，不会限制值的合法范围
  - int(3)、int(4)、int(8) 在磁盘上都是占用4 bytes的存储空间

- 作用
  - 定义字段的时候，定义了zerofill，那int(1)，你存入10显示就是10
  - 而int(4)，存入10显示就是0010，也就是前面补0
  - 定义一定长度的编号，不足补0的情况，比如卡号、学生编号等
- 案例

```
#创建表
CREATE TABLE `user` (
  `id` int(4) unsigned zerofill NOT NULL AUTO_INCREMENT,
   PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8mb4;


#插入数据
INSERT INTO `user` (`id`) VALUES (1),(10),(100),(1000);


#查询
select * from user;
```

12-说下Mysql索引失效的常见情况有哪些?

- 题目分析和考点
  - 面试官想要了解你对Mysql索引失效是否掌握

    - 只有知道哪些情况下索引会失效，才会编写索引生效的sql
    - 一般面试的时候面试官也会给出案例语句，基于给的建表语句进行分析是否索引失效
- 难度【*****】
- 答案
    - 准备

```
#建表
CREATE TABLE `account` (
  `id` bigint NOT NULL AUTO_INCREMENT,
  `phone` varchar(64) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL,
  `pwd` varchar(128) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL,
  `province` varchar(80) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL,
  `city` varchar(60) CHARACTER SET utf8mb3 COLLATE utf8mb3_general_ci NOT NULL,
  `status` int NOT NULL,
  `gmt_create` datetime DEFAULT NULL,
  PRIMARY KEY (`id`)
  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;

#插入数据
INSERT INTO `shop`.`account` (`id`, `phone`, `pwd`, `province`, `city`, `status`,
`gmt_create`) VALUES (1, '13113', '1234', '广东省', '广州市', 1, '2024-01-09
00:00:00');
INSERT INTO `shop`.`account` (`id`, `phone`, `pwd`, `province`, `city`, `status`,
`gmt_create`) VALUES (2, '887323', '1421', '浙江省', '杭州市', 0, '2025-10-23
00:00:00');
INSERT INTO `shop`.`account` (`id`, `phone`, `pwd`, `province`, `city`, `status`,
`gmt_create`) VALUES (3, '4312', '992323', '广东省', '深圳市', 1, '2023-10-13
00:00:00');

#创建索引
CREATE INDEX idx_provice_status_city on account(province,status,city)
```

    - 联合索引
        - 最左匹配原则，最左边的列一定需要存在，否则索引会不生效
        - 范围查询右边的列，不走索引
        - 语句一（3个字段联合索引命中）

```
explain SELECT * from account where province='广东省' and status=1 and city
= '广州市'
-- 428 = (3*80+2) + 4 +(3*60+2)
```

        - 语句二

```
explain SELECT * from account where province='广东省' and  status=1
-- 246 = (3*80+2) + 4
```

- 语句三

```
explain SELECT * from account where province='广东省'
-- 242  = (3*80+2)
```

- 语句四（最左边的列不存在，索引失效）

```
explain SELECT * from account where   status=1 and city = '广州市'
```

- 语句五（跳过中间字段则失效，只有最左的列生效）

```
explain SELECT * from account where province='广东省'  and city = '广州市'
-- 242
```

- 语句六（范围查询右边的列，不走索引）

```
explain SELECT * from account where province='广东省' and  status>0 and city
= '广州市'
--246 = (3*80+2) + 4
```

- Or连接索引失效
  - or连接索引，or前面字段有索引，但是or后面字段没索引，那涉及的索引都失效）
  - 编写sql如果用到or连接，记得检查下or左右的字段是否都有索引，不然则索引失效

```
explain SELECT * from account where province='广东省' and  gmt_create = '2024-
01-01'
-- 242

explain SELECT * from account where province='广东省' or  gmt_create = '2024-01-
01'
-- 没有采用索引

# 给 gmt_create 字段加索引
explain SELECT * from account where province='广东省' or  gmt_create = '2024-01-
01'
-- 依旧没有采用索引

# 需要插入多条数据，如果数据过少，优化器不会走索引
```

- like查询索引是否一定失效
  - 左边精确查询索引生效

```
explain SELECT * from account where province like '广%'
-- 242
```

        - 左边模糊查询，索引失效（左右都加也失效）

```
explain SELECT * from account where province like '%省'
-- 没采用索引
```

    - 字段重复性高导致索引失效（插入多条数据，某个字段内容重复性高）
        - 索引生效

```
explain SELECT * from account where province = '广西省'
```

        - 索引失效（**索引是用来查询少量数据，mysql优化器会检查如果全表扫描比找索引快，则会不走索引**）

```
explain SELECT * from account where province = '广东省'
```

    - 函数操作
        - 索引失效

```
explain SELECT * from account where substring(province,1,1)='广'
```

13-说下你自己Mysql索引设计常见原则有哪些

- 题目分析和考点
    - 面试官想要了解你对Mysql索引是否掌握
    - 设计数据库表有很多原则，特别是索引设计这个一定要考虑充分
- 难度【****】
- 答案
    - 索引失效常见案例总结
        - 最左匹配原则，最左边的列一定需要存在，否则索引会不生效
        - 联合索引范围查询右边的列，不走索引
        - 联合索引跳过中间字段，则只有左侧存在的索引的才生效
        - or连接索引，or前面字段有索引，但是or后面字段没索引，那涉及的索引都失效
        - like查询索引，左边精确查询索引生效；左边模糊查询，索引失效
        - 字段重复性高导致索引失效，mysql优化器会检查如果全表扫描比找索引快，则会不走索引
        - 字符串没加引号导致索引失效

```
explain SELECT * from account where province = 888
-- 索引失效


explain SELECT * from account where province = '888'
-- 索引生效
```

- 索引字段上进行函数操作，运算符操作 导致索引失效
- IS NULL不走索引，IS NOT NULL走索引，设计字段的时候，如果没有要求必须为NULL，那最好给个默认值空字符串

```
explain SELECT id from account where province is  NULL
-- 不走索引


explain SELECT id from account where province is NOT NULL
-- 走索引
```

- 索引设计常见案例原则
  - 高频次查询且数据量大的表建立索引
  - 经常需要排序、分组和联合操作的字段建立索引
  - 短索引可以提升访问的IO效率，对于BLOB、TEXT或很长的varchar列使用前缀索引
  - 删除无用索引，同列上创建多个索引，越多索引维护成本越高，优化器在优化查询时也需要逐个考虑，会影响性能
  - 根据业务需求，设计好联合索引，业务使用的时候尽量用到联合索引，避免回表查询
  - 尽量选择区分度高的列作为索引，区分度越高性能越好，比如唯一索引
  - 索引列不参与计算，带函数的查询不建议做为索引列
  - 尽量扩展利用现有索引，联合索引的查询效率比多个独立索引高
  - 尽量避免NULL，应该指定列为NOT NULL，含有空值的列很难进行查询优化，可以用0或一个空串代替NULL
  - 唯一索引与普通索引
    - 唯一索引和普通索引在性能上没有本质的区别，但在数据的唯一性方面
    - 唯一索引在数据插入和更新时需要更多计算，因此略微慢一些。
  - 聚簇索引与非聚簇索引
    - 聚簇索引在性能上优于非聚簇索引，因为聚簇索引是将数据存储在一起的
    - 这样检索数据时可以最大程度地减少磁盘 IO 操作。但如果经常更新表中的数据，则聚簇索引的维护成本相对较高。
  - 覆盖索引与非覆盖索引
    - 覆盖索引可以直接从索引中获取数据，无需回表查询，因此执行速度更快

        - 但是如果查询需要取出的数据列不在索引中，则无法使用覆盖索引，需要进行回表查询，效率较低。

    - 个人多年经验观点
        - 性能好坏取决于具体的应用场景和查询需求，单独看某个指标可能会忽略一些重要的因素
        - 综合考虑多种因素，如查询类型、表结构、数据量、访问模式等等，重新设计表结构和索引，达到最优的数据存储和查询效率

14-Mysql数据库日常使用有没积累自己的工具库?

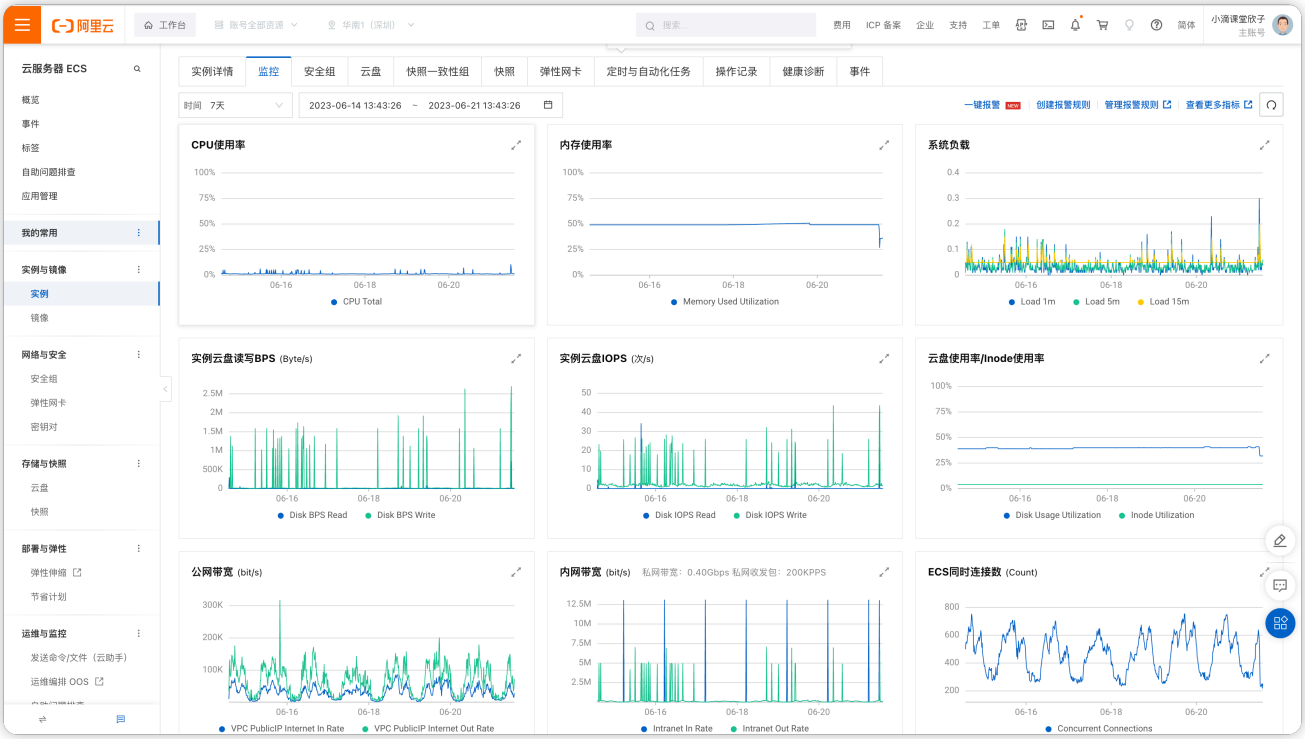
- 题目分析和考点
    - 面试官想要了解你对Mysql是否足够了解，有没积累自己的工具集
    - 比如慢查询分析、主从复制问题监控、数据库备份、索引性能优化的开源工具
- 难度【* * * * *】
- 答案
    - 这边会用些开源工具，比如Percona Toolkit，是Percona公司出品的，是一个功能强大、稳定、易用的MySQL工具集
        - 用于管理MySQL数据库，可以提供很多实用工具，简化MySQL的管理，提高MySQL的性能，并能够解决很多常见的MySQL问题
        - Percona Toolkit的开源特性和丰富的社区资源，为用户提供了很多灵活的解决方案，非常适合用于高性能、高可用的MySQL环境
        - 官网：https://docs.percona.com/percona-toolkit/index.html
    - 日常使用mysql可能会遇到很多问题
        - 重复索引检测
            - pt-duplicate-key-checker可以用于检查某张表在给定的索引中是否存在重复的键值
        - 慢查询分析
            - Percona Toolkit中的pt-query-digest工具可以分析MySQL的慢查询日志，找出慢查询语句，通过分析排除MySQL性能瓶颈和优化查询
        - 复制监控
            - Percona Toolkit的pt-table-checksum和pt-table-sync工具可用于验证MySQL主从复制是否一致，检测数据同步的延迟或故障，并保持主从数据的一致性。
        - 表和索引优化
            - Percona Toolkit的pt-online-schema-change工具通过在线方式修改MySQL表结构，避免了长时间锁表带来的影响，从而避免对业务造成影响。

    - 数据备份
        - Percona Toolkit的xtrabackup工具可以实现MySQL的在线备份，无需停止数据库，保障业务连续性。
    - 状态监控
        - Percona Toolkit的pt-mysql-summary和pt-summary等工具用于定期生成汇总MySQL服务器统计信息的报告，包括MySQL运行时间、活跃线程状态、连接统计信息等，方便管理员快速掌握数据库的情况。

业务稳定性运行-监控告警体系

1-说下为什么互联网公司必备监控，主要解决哪些问题

- 题目分析和考点
    - 面试官可以区分培训机构做的项目，还是自己企业中实际使用
    - 多数练习项目都不会讲监控告警，而企业项目中99%都会用到
- 难度【***】
- 答案
    - 互联网公司产品通常是通过软件、网站、App或其他数字化方式提供服务的
    - 这类产品在使用过程中可能会面临一系列风险和挑战
    - 比如
        - 网络故障或稳定性问题
            - 由于网络故障、硬件故障或配置错误等原因，可能会导致访问不稳定或宕机，进而影响用户的体验。
        - 性能瓶颈和延迟
            - 当用户访问量增加时，可能会对服务器产生超过负荷或达到带宽上限的压力，导致网页或其他平台响应速度变慢，影响用户体验和满意度。
        - 数据安全性问题
            - 不法分子可能通过攻击防火墙、病毒、恶意软件等途径获取数据或进行恶意操作，在一定程度上破坏了数据的安全性
    - 所以互联网公司需要在开发、测试、发布、运维等不同阶段对产品进行监控，以便及时发现问题并采取相应措施



- 监控能帮助互联网公司完成以下任务
  - 实时监控系统性能和服务质量，确保系统稳定性，及时发现和解决故障。
  - 监控用户行为和数据流量，进一步优化服务质量，提高用户体验。
  - 提前发现并预测端到端的IT操作风险，避免数据安全性问题和资金损失。
  - 监控系统的架构、代码和服务，提高系统构建和部署的质量，优化资源和运维成本

2-你们公司项目的系统可用性SLA有多高? 4个9表示什么意思

- 题目分析和考点
  - 面试官可以区分培训机构做的项目，还是自己企业中实际使用
  - 多数练习项目都不会讲监控告警，而企业项目中99%都会用到
  - 监控的目标也是可以衡量技术团队的绩效，很多公司会用SLA衡量团队的技术水平
  - 特别是高级岗位，务必对这个掌握；往往多数人员可能连SLA都不知道什么意思
- 难度【****】
- 答案
  - 系统可用性指标
    - SLA 衡量一个系统可用性有多高，目标系统 7 x 24 小时不间断服务，云厂商在宣传自己产品SLA时多少个9
      - 分类
        - 时间维度：系统可以正常使用时间与总时间之比（全年为例子）1年 = 365天 = 8760小时
          - 99.9 = 8760 * 0.1% = 8760 * 0.001 = 8.76小时

                - 99.99 = 8760 * 0.0001 = 0.876小时 = 0.876 * 60 = 52.6分钟
                - 99.999 = 8760 * 0.00001 = 0.0876小时 = 0.0876 * 60 = 5.26分钟
                - ...
            - 请求次数维度：请求总次数和失败的占比 (1000次请求为例子，相对简单)
                - 系统可用性99%：表示1000个请求中允许1000 * (1- 99%) = 10个请求出错
                - 系统可用性99.9%：表示1000个请求中允许1000 * (1- 99.9%) = 1个请求出错。
                - ...
    - 9越多代表全年服务可用时间越长服务更可靠，停机时间越短，但往往存在网络/机房问题，应用更新发版导致服务不可用
    - 大厂多数业务4个9是刚需，5个9是目标，6个9是理想
    - 业界用多少个9衡量系统的可用性
        - 基本可用：2个9，一年中大概停机时间小于88小时
        - 较高可用：3个9，一年中大概停机时间小于9小时
        - 高可用：4个9 即 99.99%，一年中大概停机时间小于53分钟
        - 极高可用：5个9，一年中大概停机时间小于5分钟

3-说下你们团队如何保障公司业务项目的稳定运行

- 题目分析和考点
    - 如果有带团队经验，肯定是需要知道这个事情，保障团队项目的稳定运行
    - 有基本的方法论和思路，特别是高级岗位，务必对这个掌握；
- 难度【 * * * * 】
- 答案
    - 如何保证业务稳定运行
        - 作为技术leader，肯定需要保证团队所负责的业务稳定运行
            - 团队灌输思想
                - 稳定性是技术人员的生死线，是最重要的技术kpi
            - 如何落地
                - 预防
                    - 90%的事故都是发布引起的
                    - 变更三板斧：可监控、可灰度、可回滚，不允许"三无发布"

        - 面向失败编程、定期CodeReview
    - 应急响应
        - 故障发现和分类
            - 技术指标监控
            - 业务指标监控
        - 故障排查
            - 直接负责人、链路负责人
            - 看日志直接定位、排除法
        - 故障恢复
            - 最快方式恢复正常运行
            - 禁止不思考重启回滚-防止二次故障引入
    - 定级和复盘（任何公司都不能100%避免）
        - 事故级别和责任人
        - 发生事故的原因：测试没发现、发布不规范、不熟悉配置、流量评估没做好、限流降级没做好?
        - 改进措施、避免下次再犯
- 组织文化建设
    - 安全生产月报周报
    - 事故纳入KPI考核、奖金、晋升
    - 分享和沉淀稳定性建设的例子

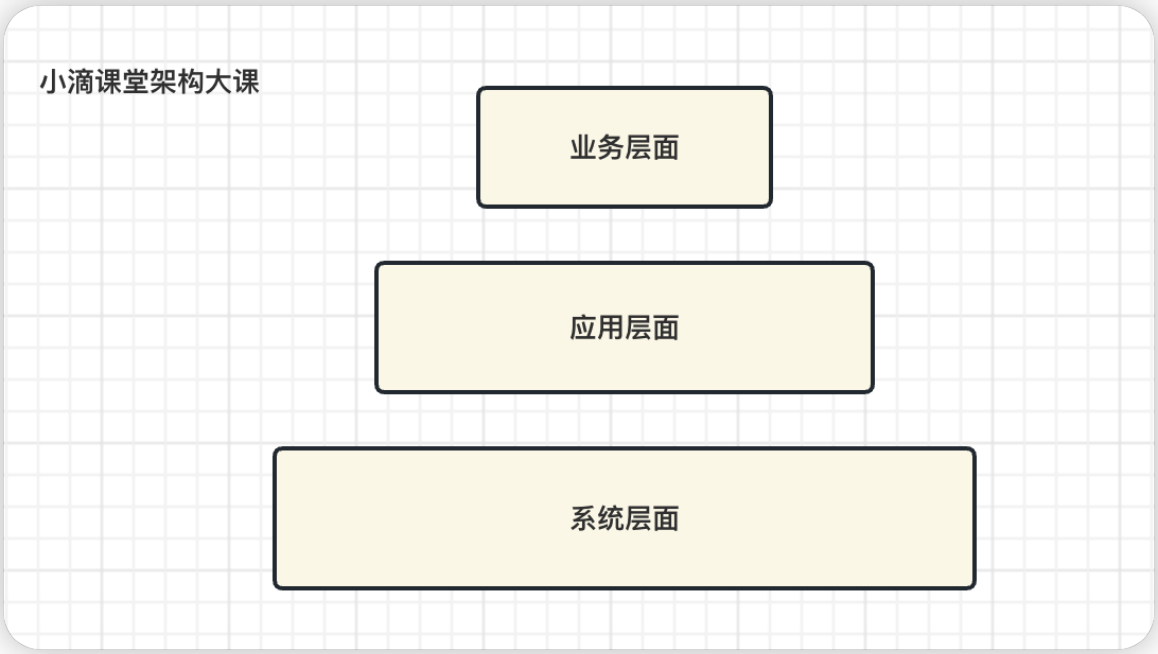
4-是否经历过发生产环境问题，如何进行生产事故定级

- 题目分析和考点
    - 面试官可以区分培训机构做的项目，还是自己企业中实际使用
    - 基本技术人员都有经历过生产环境的事故，那如何进行事故定级是一个必备的技能点
    - 技术团队奖惩制度，大家有这个意识，则更加注重代码质量，业务也可以更好的发展
- 难度【* * * *】
- 答案
    - 故障等级定义
        - 一个完整的故障等级定义一般由业务场景的功能模块+影响面+对应等级组成。
            - 功能模块
                - 从功能受损后对用户实际受影响的程度可以简单将模块分为核心功能、次核心功能和非核心功能等模块
                - 核心功能模块主要是直接影响用户使用服务的

        - 比如视频播放、用户创建订单购买
    - 非核心模块影响到 用户体验，但是对主路径功能没有重大影响的
        - 比如查询类、管理后台数据订单查询
- 影响面
    - 描述某个功能模块受损后的现象和结果，
    - 最常使用的指标是成功量、成功率、耗时、影响用户数、失败量、影响时长等指标
    - 根据业务层面对影响面的判断，对不同级别的影响面匹配不同的故障等级 (P1-P4)
- 等级 (P1-P4)
    - P1为最高等级
        - 一般代表了公司业务的核心系统或其它极为重要的系统出现了严重问题
        - 导致公司业务受阻、用户流失、财务损失等严重后果，对公司声誉和市场地位造成重大打击
        - 比如：
            - **大规模数据泄露、严重的网络攻击导致系统瘫痪、关键业务服务中断**
            - 2017年XX公司一次网关异常导致多个业务系统出现异常，平台业务正常订单的下单和支付操作都受到了影响，造成了5000万的业务损失。
    - P2为重要等级
        - 代表了业务系统中重要的一部分出现了故障或问题，需要紧急处理
        - 可能导致公司的业务受限、用户不满、合规风险增加等
        - 比如
            - **中小规模数据泄露、严重的网络攻击导致系统瘫痪、关键业务服务中断**
            - 2016年XX某电商页面出现闪退、商品详情页打不开、购物车不能正常结算等问题，因服务器压力大导致，并在两个小时内解决问题
    - P3为一般等级
        - 代表出现了某些问题或故障，但对整体业务影响较小，仍需采取及时措施进行处理。
        - 比如
            - 个别用户数据泄露、局部系统故障、产品安全漏洞
            - 2018年XX宝账户登陆出现一定程度问题，但非常规操作仍可登陆，影响不大，在48小时内解决了问题
    - P4为最低等级
        - 代表问题或故障对业务影响较小，但仍需要监控和处理，以确保业务系统能够正常运行。
        - 比如
            - 个别用户投诉、产品功能异常
            - 系统后台人员误删除了部分用户数据，数据量不大且数据已建立全面备份，能够快速恢复，影响不大

5-生产环境项目进行监控，说下你们监控哪些指标?

- 题目分析和考点
  - 面试官可以区分培训机构做的项目，还是自己企业中实际使用
  - 互联网公司监控和指标很多，按照顺序分层归类
  - 这些指标不是绝对的，应该根据企业的业务和系统对应合适的选择
  - 对于不同的监视重点，还应根据需求包括不同级别的指标
- 难度【* * * *】
- 答案



  - 系统和网络层
    - 主要关注底层基础设施的状态和事件，以确保服务器、网络、CPU和存储设备等运行正常，提供稳定的资源支持。
    - 操心系统常见指标
      - CPU利用率：服务器上CPU主要的核心使用率情况。
      - 实时负载：系统上所有进程的数目，计算干活的进程数，以及等待队列的进程数，也就是当前的机器的实时压力情况。
      - 内存使用率：服务器内存使用情况，包括已使用、空闲等情况。
      - 网络带宽利用率：服务器网络使用度，包括网卡、负载均衡、网络连接等的带宽使用情况。
      - 硬盘I/O读写速度：磁盘读写速率。

        - 硬盘容量：服务器硬盘容量使用情况，包括已使用、空闲等情况。
        - 进程使用率：检查系统进程情况，包括进程执行状态、占用集群资源等情况。
        - 端口连接状态：检查系统端口连接的状态。
        - 错误日志记录：记录系统产生的错误日志，包括错误类型、时间、处理结果等情况。
        - 响应时间：服务器响应请求的时间
    - 网络常见指标
        - 带宽利用率：网络带宽利用率评估，包括上传和下载比率。
        - 包丢失率：测量包在传输中丢失的数量和百分比。
        - 延迟时间：从发送请求到得到响应且完成处理信息所需的时间。
        - 网络流量：流经网络的实时数据量和数据流量。
        - 网络错误率：网络传输中发生的错误数量和百分比。
        - 连接数：网络连接总数。
        - 网络响应时间：网络请求响应时间。
        - 网络拥塞状态：系统可用资源数量和使用率。
        - 传输速度：网络传输速率，包括平均速率和实时速率。
- 应用层（业务应用程序、中间件应用程序等）
    - 关注整体服务的状态和运行质量，能够及时预测系统运行瓶颈，保证产品的高效和用户体验
    - 常见指标（java为例）
        - 请求响应时间：从请求到获得响应的整个时间。
        - 错误率：应用程序产生错误的请求占总数的百分比。
        - CPU使用率：应用程序当前使用的处理器资源百分比。
        - 线程实例数：当前在应用程序中运行的线程实例数量。
        - 平均程序执行时间：应用程序各模块的平均执行时间。
        - 堆内存使用率：应用程序中Java虚拟机（JVM）分配的内存占用的百分比。
        - 平均延迟时间：从请求到响应开始的时间差。
        - 垃圾回收时间：在JVM中收集不再使用的内存对象所需的时间。
        - 响应代码：HTTP请求成功或失败代码。
- 业务层
    - 重点关注业务运营的分析和结果，通过监控平台运营情况和各种配置，获取更多业务数据
    - 及时发现行业发展趋势，指引业务方向，实现全方位监控、预测和干预
    - 指标
        - GMV销售额：项目特定时间内总的销售额。
        - 日活、月活：日活跃用户数、月活跃用户数
        - 客单价：平均每个订单的金额
        - 支付成功率：成功支付订单和总订单的比率

        - 订单量：每次交易中的订单数量。
        - 购物车转化率：加入购物车商品与实际付款之间的比率。
        - 转化率：网站访客转化为潜在客户（注册、订阅、购买等）的百分比。
        - 用户留存率：每月活跃用户数量与上月活跃用户数量的比率。
        - 退款率：以退货所得的金额与总交易金额之间的比率。
        - 每次交易的平均时间：从访问网站到交易结束的总时间。
        - 每个访问的平均时间：用户在网站上花费的总时间除以有效付款数量。
        - 启动错误率：应用程序在第一次启动时无法正确启动的次数。
        - 平均执行时间：任务执行所需的平均时间。
        - 异常数量：系统产生的错误、崩溃、死锁等数量。
        - 页面加载时间和速度：网页从请求到加载的时间和速度。
        - 点击率：网站广告点击率。
        - ...

- 人工监控：客服和产品运营接收用户反馈（大厂里面的小部门挺多的）

- 个人经验
    - 技术人员一定要关注技术+业务 两个指标，所以我习惯这样分类，用于早报、周报和绩效KPI考核上面
        - 技术指标（技术人员关注）
            - 上述的系统+网络指标、应用指标
        - 业务指标（产品和运营人员关注）
            - 上述的业务指标就是

6-有没搭建过监控系统，说下用过哪些监控告警框架?

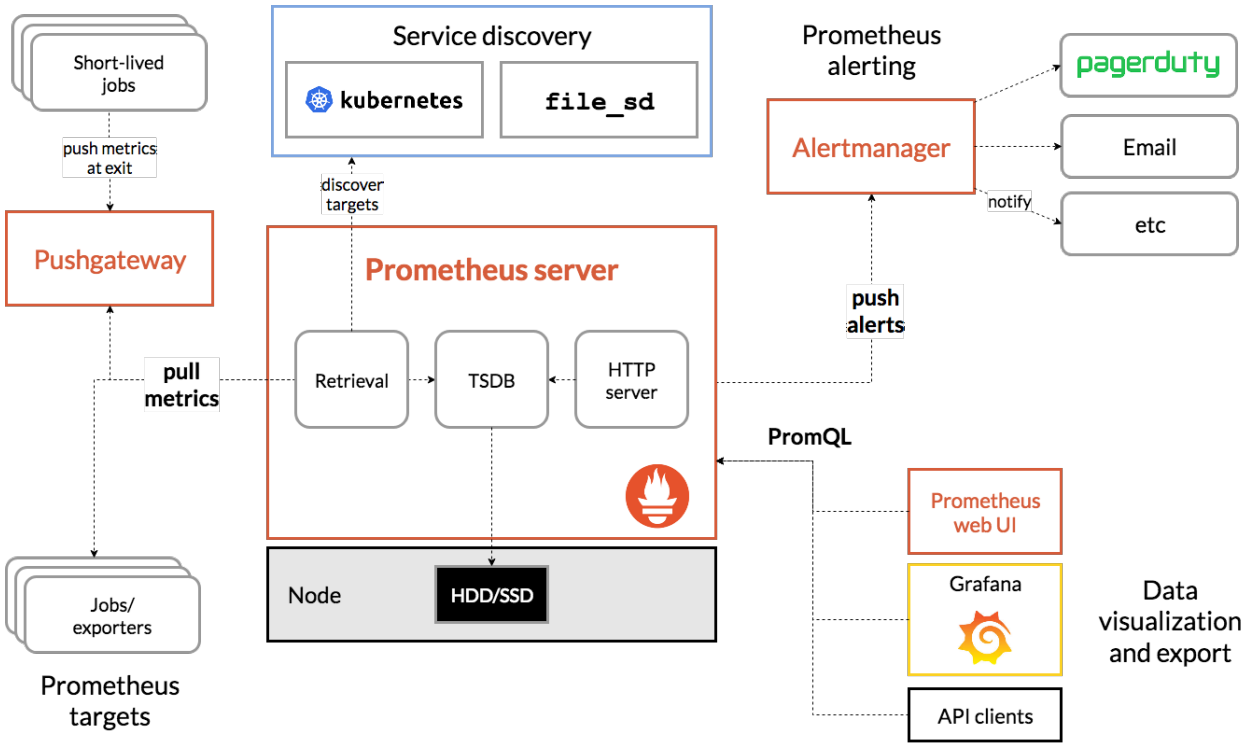
- 题目分析和考点
  - 如果是作为高级工程师，独立负责过项目的话
  - 项目肯定有上线，项目有上线肯定有监控
  - 基本会采用开源的监控框架，那对于不同监控框架需要有了解对比，知道什么场景下哪个框架适合
- 难度【* * *】
- 答案
  - 主流监控告警框架对比
    - Zabbix（支持 pull/push 两种模式）
      - 基于C语言开发, 是一种基于服务器-客户端体系结构开发的企业级监控软件。它允许监控各种网络服务，服务器资源和硬件
      - 优点：
        - 用户友好，易于部署和设置
        - 具有插件式框架，可以灵活地满足不同的监控需求
        - 提供了丰富的图形化监控数据
        - 具有高扩展性，用户可以轻松添加自定义功能。
      - 缺点：
        - 功能强大，使用比较笨重，Zabbix的安装和部署需要更多的时间和资源
        - 对于复杂的IT架构，Zabbix的配置难度较大,
        - 响应时间不够实时，对于实时性要求较高的应用场景可能不够理想
    - Open-Falcon（push模式）
      - 是一种分布式、高性能的监控解决方案，它被用于大型的高可用性系统和广泛的云计算环境
      - 优点：
        - 轻量级，可以部署在物理机和虚拟机上，部署和配置足够简单
        - 支持查询优化，支持多种数据源输入
        - 具有简单和易于使用的图形化界面
      - 缺点：
        - 国内使用量较小，社区不够活跃
        - 对于某些复杂的IT架构，Open-Falcon在准备和安装方面可能存在困难
        - 针对不同需求经验丰富的开发人员需代码调整
    - Prometheus（Pull模式）

        - go语言开发, 是可扩展的开源监控系统，用于收集存储多维度的时间序列数据
        - 支持PromQL查询语言和提供的图形化展示工具
        - 可视化和告警这些功能交由Grafana和Alertmanager等第三方产品来实现，拓展性强
        - 功能上的简洁，作为一个轻量级的后起之秀，在性能和展示方面优势比较明显，对容器监控支持的非常好
        - 优点：
            - 可扩展性强，支持水平扩展，满足大规模监控需求
            - 支持灵活的查询和查询语言，提供了丰富的查询函数和操作符
            - 可以从不同的数据源收集数据，支持多维度数据聚合
            - 可以轻松集成Alertmanager实现告警
        - 缺点
            - 存储机制不够灵活，大规模数据的存储和处理存在压力
            - 对运维工程师的技术水平要求较高

7-有没了解过Prometheus，说下用途和主要组成部分

- 题目分析和考点
    - Prometheus当下主流的系统监控和警报工具，多数Prometheus组件是Go语言写的
    - 面试官想考查Prometheus相关技术的掌握情况，现在已经成为一个独立于企业级的开源项目
    - 对于Prometheus的日常使用，适合哪些场景需要知道
- 难度【 * * * 】
- 答案
    - 用途
        - Kubernetes集群监控
            - 使用Prometheus可以收集和监控Kubernetes集群的指标数据，例如CPU、内存、网络等。
            - 使用Prometheus Operator部署Prometheus，然后通过Grafana可视化工具展示监控指标的仪表板
        - 网络监控
            - Prometheus可以监控网络的状态和性能，例如TCP连接数、网络延迟和带宽利用率等
            - 使用Prometheus的Blackbox Exporter插件来执行网络探测，检查网络服务是否可用
        - 应用程序性能监控
            - 通过Prometheus的客户端库可以在应用程序中嵌入指标收集代码，并收集应用程序的性能指标数据
            - 例如请求数、响应时间、错误率等，帮助开发人员监控应用程序的性能，并进行调试和优化
        - 数据库监控

        - 可以使用Prometheus的Exporter插件监控各种类型的数据库，例如MySQL、PostgreSQL、Redis和MongoDB
        - Exporter可以将数据库的指标数据转换为Prometheus可以处理的格式，并将其发送到Prometheus进行监控和警报
    - 服务器监控
        - 使用Prometheus可以监控服务器的CPU、内存、磁盘和网络使用情况等指标，服务器上运行的各种服务的状态和性能
        - 能够实时地存储和查询系统和服务的各种指标，如性能、CPU利用率、内存使用和请求计数等。
- 核心组成部分

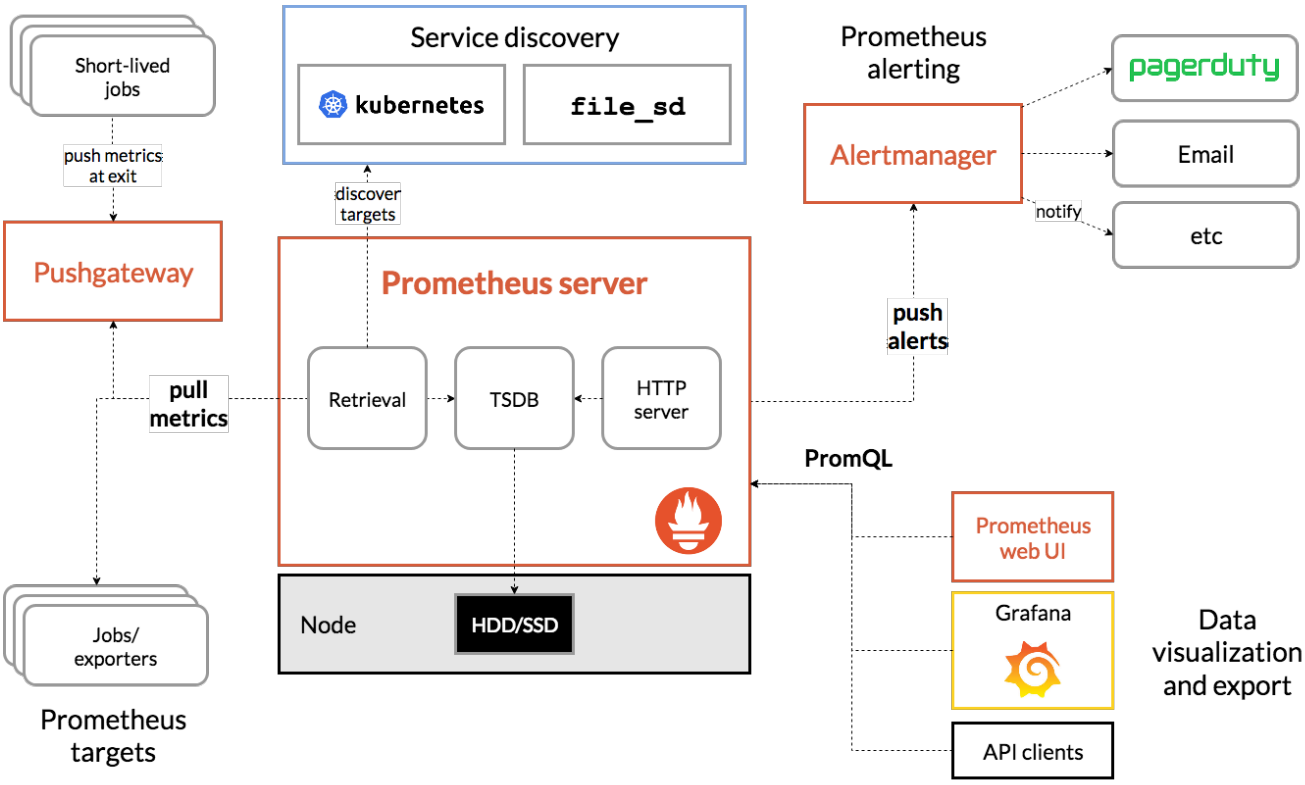


    - Prometheus server
        - 核心组件，负责抓取、存储和查询指标数据，提供API以供访问
        - Prometheus Server本身就是一个时序数据库，将采集到的监控数据按照时间序列的方式存储在本地磁盘当中
        - 内置的UI界面，通过这个UI可以直接通过PromQL实现数据的查询以及可视化
    - Exporter
        - Prometheus插件或独立组件，负责抓取指定服务或系统的性能指标数据
        - Prometheus原理是通过 HTTP 协议周期性抓取被监控组件的状态，输出这些被监控的组件的 Http 接口为 Exporter
        - Exporter将监控数据采集的端点通过HTTP服务的形式暴露给Prometheus Server，将其公开为HTTP端点或指定的格式
        - Prometheus server通过轮询或指定的抓取器从Exporter提供的Endpoint端点中提取数据
    - Alertmanager

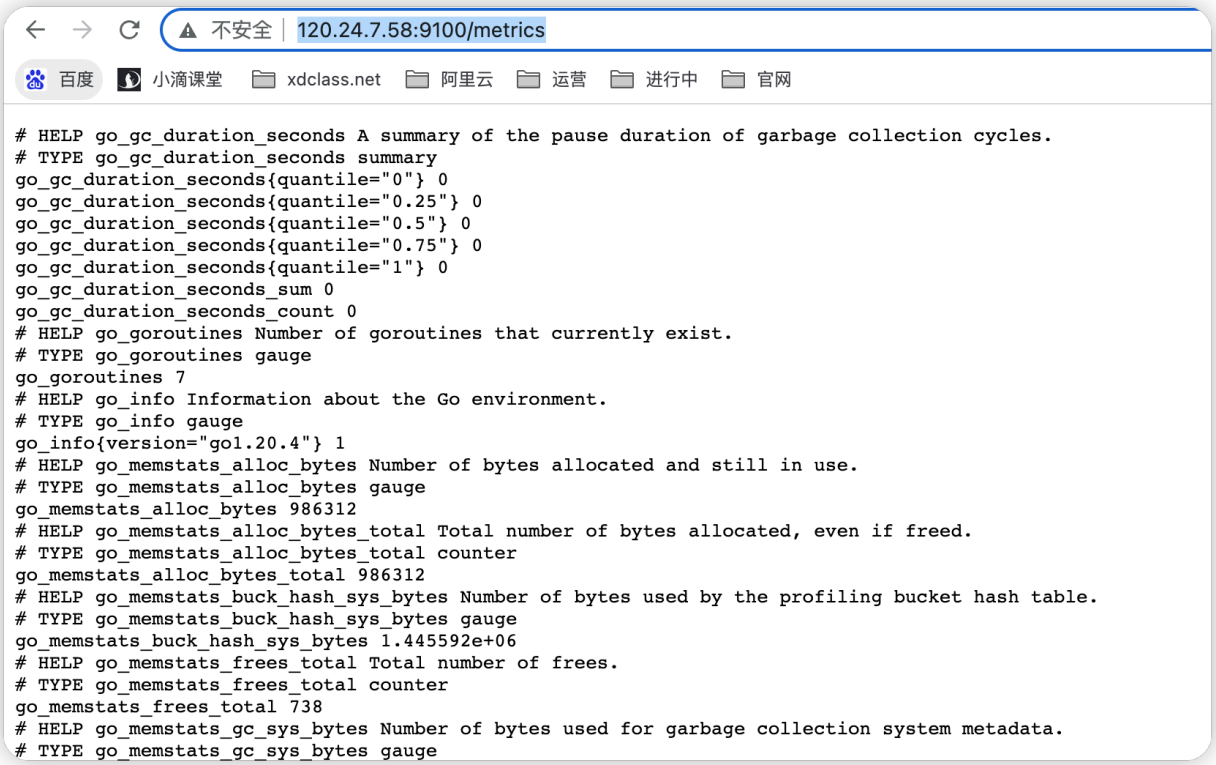
        - 在Prometheus Server中支持基于PromQL创建告警规则，如果满足PromQL定义的规则，就会产生一条告警
        - Prometheus告警管理器组件，负责管理告警规则、通知和报警策略的设置，提供第一类和第二类警报的分类管理服务
    - PushGateway
        - Prometheus数据采集基于Pull模型进行设计，在网络环境必须要让Prometheus Server能够直接与Exporter进行通信
        - 当这种网络需求无法直接满足时，就可以利用PushGateway来进行中转
        - 通过PushGateway将内部网络的监控数据主动Push到Gateway当中
        - Prometheus Server则可以采用同样Pull的方式从PushGateway中获取到监控数据
    - Service Discovery
        - 服务发现功能，动态发现待监控的Target，完成监控配置的重要组件

8-Prometheus里面Exporter的作用，用过哪些Exporter

- 题目分析和考点
    - 面试官想知道应聘者对Prometheus的掌握情况，当下主流的系统监控和警报工具，支出多种监控
    - 向Prometheus提供监控样本数据的程序都可以被称为一个Exporter
- 难度【* * *】
- 答案
    - 什么是Prometheus的Exporter
        - 向Prometheus提供监控样本数据的程序都可以被称为一个Exporter
        - 是一种用于将不同数据源的指标提供给Prometheus进行收集和监控的工具。
        - 运行在应用程序、计算机、网络设备或者其他系统上的代码，它可以将系统的指标信息以一种标准格式公开
        - 将指标数据公开为HTTP端点或者指定的格式（如Redis、JMX等），Prometheus然后可以通过轮询或指定的抓取器
        - 总结
            - Exporter是Prometheus的指标数据收集组件，负责从目标Jobs收集数据
            - 并把收集到的数据转换为Prometheus支持的时序数据格式
            - 只负责收集，并不向Server端发送数据，而是等待Prometheus Server 主动抓取



- Prometheus社区以及其他团队开发了大量的Exporter，覆盖了许多不同类型的系统和服务
  - Node Exporter、MySQL Exporter、Redis Exporter、MongoDB Exporter、Nginx Exporter...
  - 使用方式
    - 在主机上安装了一个 Exporter程序，该程序对外暴露了一个用于获取当前监控样本数据的HTTP访问地址
    - Prometheus通过轮询的方式定时从这些Target中获取监控数据样本，并且存储在数据库当中
- 所有的Exporter程序都需要按照Prometheus的规范，返回监控的样本数据
  - 比如接下去的Node Exporter为例，当访问/metrics地址时会返回内容和本身Prometheus协议保持一致即可
  - 主要由三个部分组成，Prometheus会对Exporter响应的内容逐行解析
    - 样本的一般注释信息（HELP）
    - 样本的类型注释信息（TYPE）
    - 样本

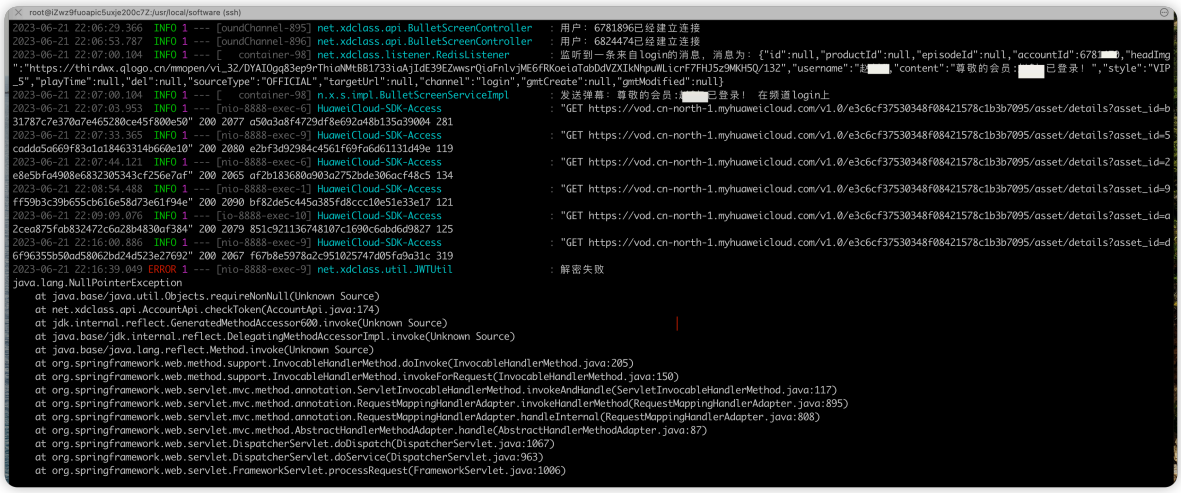


- 常用的Exporter
  - Node Exporter：用于监控操作系统节点（例如服务器）的各种指标，如CPU使用率、内存使用率、磁盘使用量、网络流量等。
  - Redis Exporter：用于监控Redis数据库的性能指标，包括连接数、内存使用量、命令执行次数等。
  - Nginx Exporter：用于监控Nginx服务器的指标，包括请求数、连接数、响应时间等。
  - Kafka Exporter：用于监控Apache Kafka消息队列的指标，如主题的消息数、分区的偏移量、消费者组的消费速率等。
  - Docker Exporter：用于监控Docker容器和宿主机的指标，包括CPU使用率、内存使用量、网络流量等。

9-说下日常开发项目里面有哪些日志和使用场景?

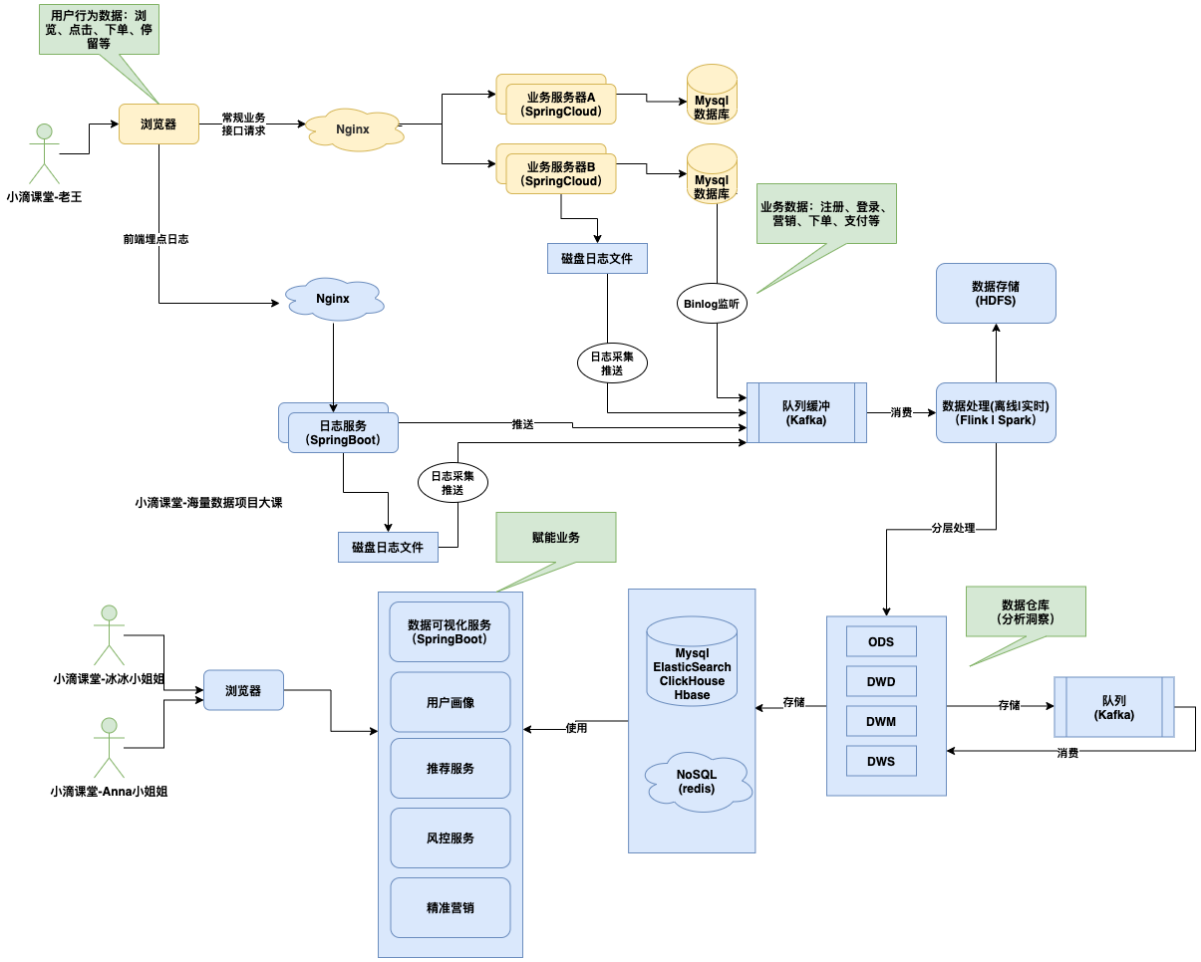
- 题目分析和考点
  - 面试官想知道应聘者对开发里面的多种类型日志是否有关注
  - 除了常规的代码日志，还有多种日志类别，需要知道对应的类型
- 难度【***】
- 答案
  - 日志归类
    - 应用程序日志：
      - 调试日志（Debug Log）：用于在开发过程中输出调试信息，帮助开发人员进行代码调试和错误排查。
      - 信息日志（Info Log）：记录应用程序运行的关键信息，用于查看应用程序的正常运行情况。

        - 警告日志（Warning Log）：记录可能导致问题的潜在警告，但不会中断应用程序的正常运行。
        - 错误日志（Error Log）：记录错误信息、异常情况和故障信息，需要进行及时修复才能保证应用程序的正常运行。
    - 系统日志
        - 系统性能日志：记录系统资源使用情况、响应时间、内存占用等性能指标。
        - 安全日志：记录系统的安全事件，如登录尝试、访问控制等，用于监测和分析系统的安全状况。
        - 系统错误日志：记录系统级别的错误信息，如硬件故障、软件崩溃等。
    - 用户操作和行为日志
        - 记录用户相关的日志，用于跟踪用户操作和行为，也就是【数据埋点】
            - 对网站、App、进行业务数据采集，包括用户行为数据及其他实际需要的数据进行采集上报
            - 方便分析用户行为、提高用户体验，通过这些采集的数据就可以进行分析并辅助公司企业做决策
        - 例如：
            - 用户登录日志：记录用户的登录信息，包括登录时间、IP地址等。
            - 用户操作日志：记录用户在应用程序中的操作，包括点击、搜索、提交表单等。
            - 错误日志：记录用户在应用程序中遇到的错误和异常，用于分析和解决问题
- 日志的用途
    - 系统和程序运行监控和错误排查
        - 应用程序日志



        - nginx的访问日志

- 业务使用，比如用户画像构建、推荐服务、数据仓库建设



10-生产环境业务程序打印日志，说下有哪些注意事项?

- 题目分析和考点
  - 面试官想知道应聘者对开发里面的日志是否了解，如果只是初级开发，一般没权限接触应用发布上线
  - 应用程序上线，基本都会配置日志，方便排查问题和日志埋点采集利用，这个也是区分培训机构和有实际开发的经验的关键点
- 难度【* * * *】

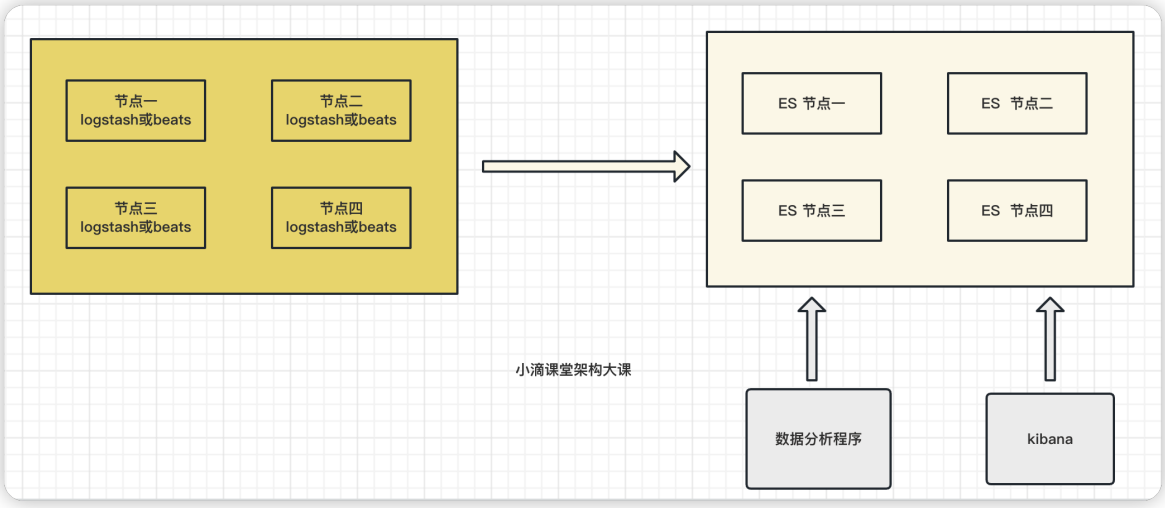
- 答案
  - 日志数据的注意事项
    - 日志级别选择
      - 根据日志信息的重要程度，选择合适的日志级别，例如：DEBUG、INFO、WARNING、ERROR等
      - 确保日志信息的粒度和输出级别符合需求。
    - 日志格式化
      - 为了方便阅读和分析，日志应该包含有用的信息，如时间戳、日志级别、请求ID、模块名称、具体的错误信息等
      - 格式化日志输出可以使用通用的工具库如Log4j、logback等。
    - 异常处理
      - 在捕获异常时，记录相关的错误信息、堆栈跟踪等，以便后续排查和分析问题
      - 可以使用try-catch块来捕获异常，并在catch块中记录相关的日志信息。
    - 日志存储和管理
      - 确定日志存储的位置和方式，可以选择将日志输出到文件、数据库、日志服务器等
      - 确保合理的日志轮转和备份策略，以及有效的日志管理和监控机制。
    - **敏感信息保护**
      - 在日志输出时，注意敏感信息的处理，如密码、个人身份信息等，应进行脱敏处理或者在合适的级别下去除。
    - 日志审计和安全
      - 对于敏感操作或重要日志事件，可以考虑加入审计日志，记录相关的操作细节和安全事件，以便追踪和监控系统的安全性。
    - 性能影响：
      - 日志的输出操作可能对系统的性能产生一定的影响，要谨慎选择日志输出的频率和内容，避免频繁的输出大量冗余信息。
    - **异步日志**
      - 日志输出可以考虑使用异步方式进行，以避免阻塞主线程，可以利用线程池等机制来异步处理日志消息。
      - 注意
        - 常用的Logback默认情况下是同步输出日志，即在日志发生时直接输出到目标位置（如文件、控制台等）
        - 然后再进行下一步操作，同步输出会阻塞主线程，可能会对系统的性能产生一定的影响
        - 可以通过Logback的AsyncAppender组件来实现异步日志输出
        - 当日志事件发生时，它会将日志事件放入一个队列中，然后由单独的线程异步地处理和输出日志

```
<appender name="ASYNC" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="CONSOLE" />
</appender>

<appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
  <!-- 其他配置项 -->
</appender>

<root level="INFO">
  <appender-ref ref="ASYNC" />
</root>
```

定义了一个名为ASYNC的AsyncAppender，并指定其内部的真正执行日志输出的<appender>为名为CONSOLE的ConsoleAppender。然后将ASYNC添加到根logger的appender列表中

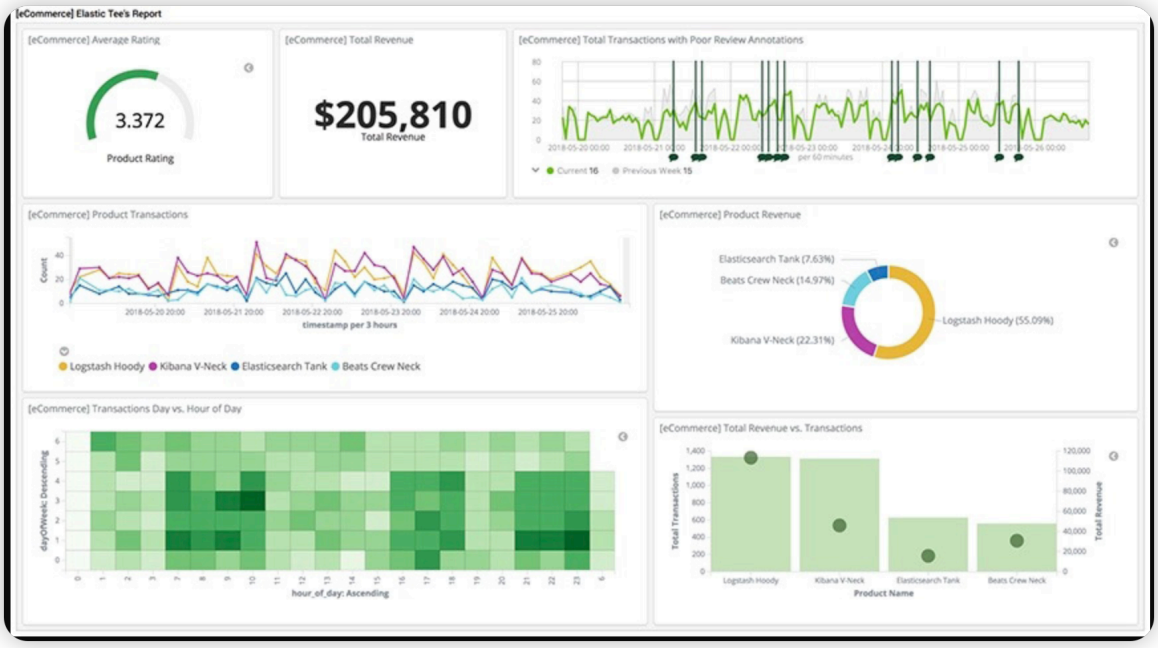
11-有没了解日志可视化工具ElasticStack 说下用途

- 题目分析和考点
  - 面试官想知道应聘者对开发里面的日志相关分析工具是否掌握
  - 操作系统、网关、应用程序产生了很多日志，有单机有分布式部署，如何查看方便
  - 海量日志需要分析、聚合统计应该怎样操作比较方便
- 难度【***】
- 答案
  - 什么Elastic Stack
    - 先看日志链路架构图

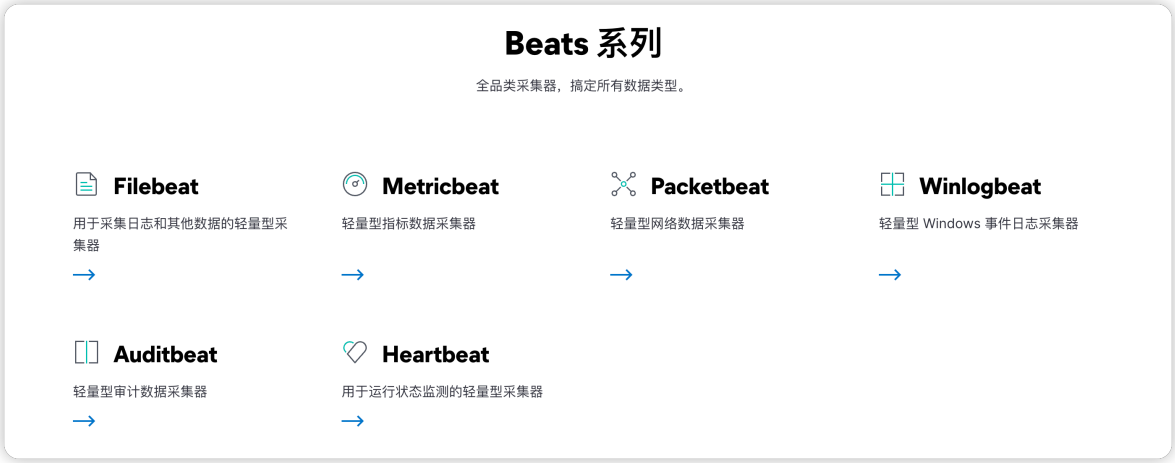


  - 是一个开源的数据分析和可视化平台，由Elasticsearch、Logstash、Kibana和Beats组成

- 它在各种应用场景中被广泛用于实时数据分析、搜索和可视化
  - **很多同学听说过ELK，实际上ELK是三款软件的简称，分别是Elasticsearch、 Logstash、Kibana组成**
  - **在发展的过程中有新成员Beats的加入，所以就形成了Elastic Stack，ELK是旧的称呼，Elastic Stack是新的名字**
- Elasticsearch
  - 一个分布式实时搜索和分析引擎，高性能和可伸缩性闻名，能够快速地存储、搜索和分析大量结构化和非结构化数据
  - 基于java开发，它是Elastic Stack的核心组件，提供分布式数据存储和搜索能力。
- Logstash
  - 是一个用于数据收集、转换和传输的数据处理引擎，支持从各种来源（如文件、日志、数据库等）收集数据
  - 基于java开发，并对数据进行结构化、过滤和转换，然后将数据发送到Elasticsearch等目标存储或分析系统。
- Kibana
  - 基于node.js开发，数据可视化和仪表盘工具，连接到Elasticsearch，通过简单易用的用户界面创建各种图表、图形和仪表盘
  - 帮助用户快速探索和理解数据，并进行强大的数据分析和可视化



- Beats
  - 是轻量级的数据收集器，用于收集和发送各种类型的数据到Elasticsearch或Logstash
  - Beats提供了多种插件和模块，用于收集和传输日志、指标数据、网络数据和安全数据等



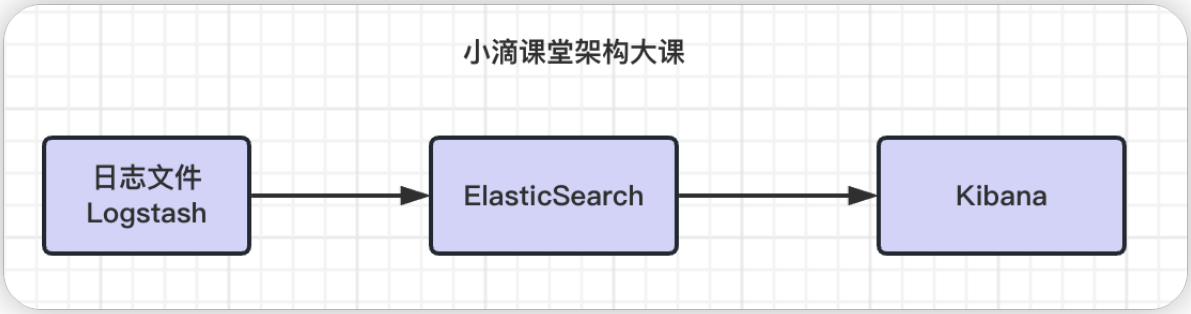
- 种类
  - Filebeat：用于收集和传输日志文件的数据收集器。Filebeat能够监控指定的日志文件或目录，并将数据发送到Elasticsearch或Logstash进行进一步的处理和分析。
  - Metricbeat：用于收集和传输指标数据的数据收集器。Metricbeat可以收集系统级别的指标数据，如CPU利用率、内存使用情况、网络流量等，并将数据发送到Elasticsearch或Logstash等目标存储或分析系统。
  - Packetbeat：用于实时网络数据分析的数据收集器。Packetbeat可以监控网络流量，对网络中的数据包进行解析和分析，提取关键信息，并将数据发送到Elasticsearch或Logstash进行进一步的处理和可视化。
  - Heartbeat：用于监测应用程序、服务和网络的数据收集器。Heartbeat可以定期发送请求并检测目标服务器的可用性和响应时间，以便进行故障排查和性能监测。
  - Auditbeat：一个用于安全审计和日志收集的数据收集器。Auditbeat能够监控系统的安全审计事件，如文件操作、用户登录、特权访问等，并将数据发送到Elasticsearch或Logstash进行安全分析和监测。
  - Winlogbeat：专门用于Windows环境下日志收集和传输的数据收集器。Winlogbeat可以收集Windows事件日志，并将数据发送到Elasticsearch或Logstash进行进一步的处理和分析。
- 注意
  - beats是用来优化logstash的，因为logstash消耗的性能比较多
  - 如果只是单纯的为了收集日志，使用logstash就有点大材小用了，另外有点浪费资源
  - 而beats是轻量级的用来收集日志的， 而logstash更加专注一件事，那就是数据转换，格式化，等处理工作

- 总结
  - Elastic Stack的架构主要是将各个组件连接起来形成数据处理和分析的流程
  - 数据可以通过Logstash的输入插件进行收集和处理，然后通过输出插件将数据发送到Elasticsearch进行存储和索引
  - Kibana可与Elasticsearch进行连接，提供灵活的可视化界面，将数据分析结果以图表、仪表盘等形式展示出来
  - Beats作为数据收集器，可以直接收集数据并将其发送到Elasticsearch或Logstash。
  - 整个Elastic Stack的架构是高度可扩展和灵活的，可以根据特定需求进行定制和扩展

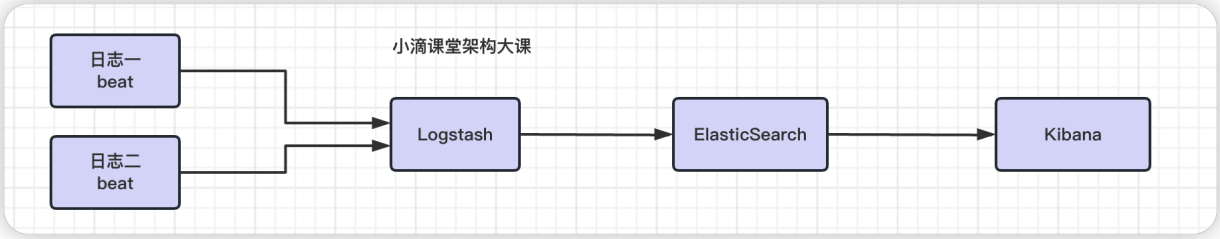
- 它被广泛应用于日志分析、实时监控、安全分析、商业智能和搜索等领域，为用户提供强大的数据处理和分析能力

12-说下你们公司日志数据采集的链路和解释Beats和Logstash关系

- 题目分析和考点
  - 面试官想知道应聘者对开发里面的日志相关分析工具是否掌握
  - 对数据的采集链路各个节点的作用是否清楚
  - 包括数据收集、转换和传输的数据处理引擎，支持从各种来源（如文件、日志、数据库等）收集数据
- 难度【* * * *】
- 答案
  - 常见的数据采集流程
    - **Logstash 采集 –> Elasticsearch –> Kibana**



    - Beats 采集（Filebeat/Metricbeat）–> Elasticsearch –> Kibana
    - Beats 采集（Filebeat/Metricbeat）–> Logstash –> Elasticsearch –> Kibana



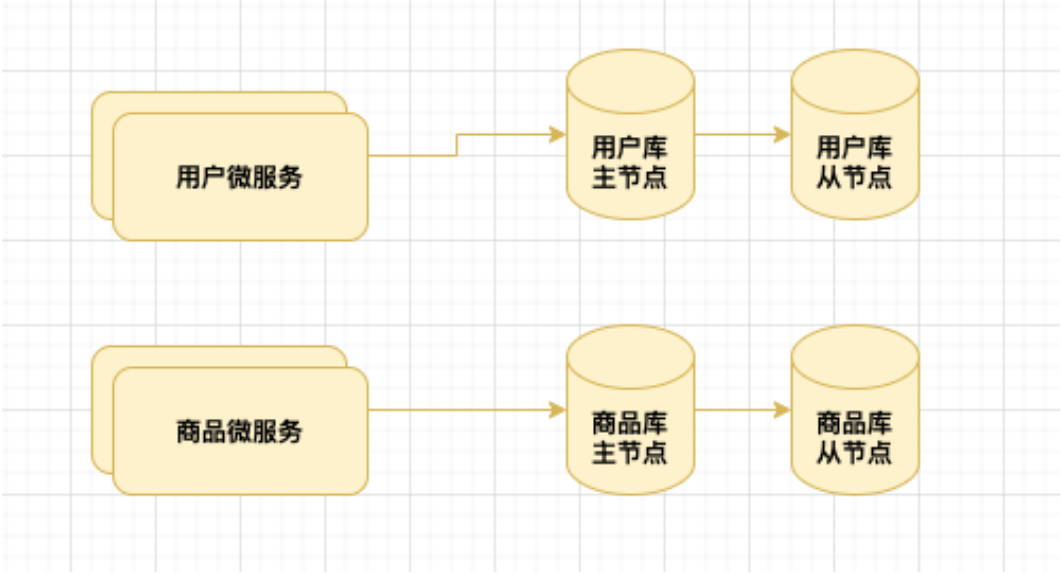
  - 很多人的疑惑点，Beats 可以直接把数据直接写入到 Elasticsearch 之中，为什么还需要 Logstash 呢?
    - 这主要的原因是因为 Logstash 有丰富的 filter 供我们使用，可以帮加工数据，并最终把数据转为对应要求的格式
    - Beats 在 ELK 中是一个非常轻量级的应用，是用 go 语言写的，它不需要有很强大的运行环境来运行，通常运行于客户端中
    - 总结
      - beats是用来优化logstash的，因为logstash消耗的性能比较多

        - 如果只是单纯的为了收集日志，使用logstash就有点大材小用了，另外有点浪费资源
        - 而beats是轻量级的用来收集日志的， 而logstash更加专注一件事，那就是数据转换，格式化，等处理工作

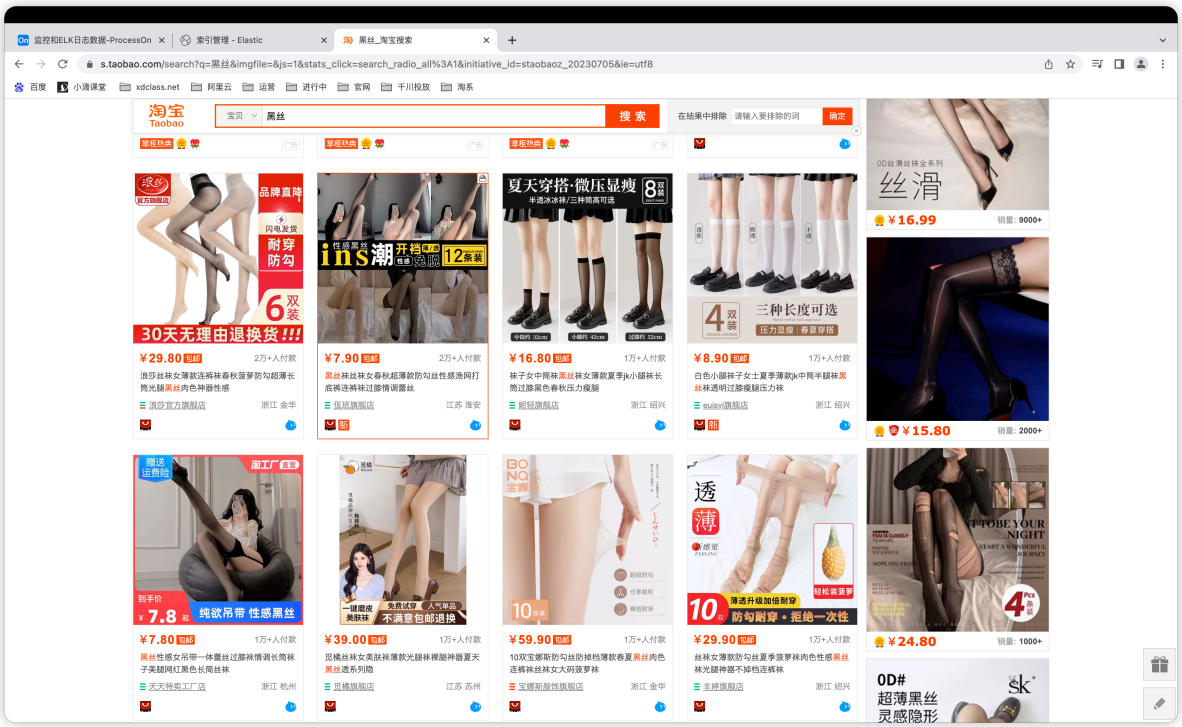
海量数据处理

1-说下业务里面涉及海量数据里面的存储和搜索方案

- 题目分析和考点
    - 面试官想知道应聘者对海量数据处理是否知道有哪些解决方方案
    - 海量数据如何存储，上亿，十亿，百亿 数据量?
    - 互联网大厂平台，比如百度，谷歌，信息流平台 ，如何解决【搜索】
    - 电商平台：搜索商品如何解决，关键词评分，中文分词，高量提示怎么做
- 难度【 * * *】
- 答案
    - 解决方案一：分库和分表
        - 垂直角度（表结构不一样）
            - 垂直分表: 将一个表字段拆分多个表，每个表存储部分字段
            - 垂直分库：根据业务将表分类，放到不同的数据库服务器上（微服务划分）
        - 水平角度（表结构一样）
            - 水平分库：把同个表的数据按照一定规则分到不同的数据库中，数据库在不同的服务器上
            - 水平分表：同个数据库内，把一个表的数据按照一定规则拆分到多个表中，对数据进行拆分，不影响表结构



- 问题
  - 跨节点数据库Join关联查询、分库操作带来的分布式事务问题、全局主键重复问题、执行的SQL排序、翻页问题
  - 分库分表解决部分问题，但是不能分词，比如 电商【黑丝】，只能搜索关键词一样的标题商品，不能模糊检索



- 解决方案二：非关系性数据库 ElasticSearch
  - 是一个开源的全文搜索和分析引擎，专注于快速的数据存储和高性能检索。
  - 可以扩展到大规模的数据集，同时提供快速的查询和分析能力
  - 无论是搜索引擎、日志分析、数据挖掘还是实时监控等场景，Elasticsearch都是一个强大而灵活的选择
  - 核心特点：
    - 分布式存储和处理

        - Elasticsearch将数据分布在多个节点上，实现了数据的并行存储和处理
        - 这使得它能够处理大规模的数据集，并提供高吞吐量和低延迟的查询响应。
    - 实时性能
        - 提供实时的查询和分析，它使用倒排索引的方式来存储数据，使得数据能够快速被检索和过滤
        - 它使用近实时（near real-time）的索引刷新机制，数据的变化几乎可以立即被搜索到
    - 多样的查询功能
        - Elasticsearch支持多种查询和过滤方式，包括全文搜索、精确匹配、模糊搜索、范围搜索、聚合查询等
        - 支持多种分词方式，比如英文，中文，自定义分词规则
        - 这使得它能够满足各种查询需求，从简单的关键字搜索到复杂的统计分析。
    - 强大的分析能力
        - Elasticsearch集成了强大的分析功能，包括聚合查询、分面查询（faceted search）、统计分析、机器学习等
        - 这些能力使得用户可以从海量数据中提取有价值的信息和洞察。
    - 可扩展性和容错性
        - Elasticsearch可以轻松水平扩展，通过增加节点来增加存储和处理能力
        - 它使用分片和复制机制来实现数据的分散存储和冗余备份，提高了系统的可用性和容错性。

2-说下Elastic Search的核心概念，介绍副本和分片的作用

- 题目分析和考点
    - 面试官想知道应聘者对Elastic Search是否掌握，对于里面的副本和分片是否清楚用途
- 难度【* * *】
- 答案
    - 核心概念
        - 在新版Elasticsearch中，文档document就是一行记录(json)，而这些记录存在于索引库(index)中, 索引名称必须是小写

Mysql数据库	**Elastic Search**
Table（水平分表）	Index（不同分片）
Row	Document
Column	Field

- 索引index
  - 在新版本中的Elasticsearh中，索引的概念等同于Mysql中表的概念，Elasticsearch中的索引可以直接存储数据。
- 文档document
  - 真正的数据，存储一条数据就是一份文档，存储格式为JOSN，等同于mysql中的一条数据
- 分片shards
  - 数据量特大，没有足够大的硬盘空间来一次性存储，且一次性搜索那么多的数据，响应跟不上
  - ES提供把数据进行分片存储，这样方便进行拓展和提高吞吐
- 副本replicas
  - 分片的拷贝，当主分片不可用的时候，副本就充当主分片进行使用
  - 索引分片的备份，shard和replica一般存储在不同的节点上，用来提高高可靠性
  - 案例
    - 假如Elasticsearch中的每个索引分配5个主分片和1个副本
    - 如果集群中至少有两个节点，索引将会有5个主分片和另外5个复制分片（1个完全拷贝）这样每个索引总共有10个分片
- 总结
  - 数据切分为多个分片是为了提高可处理数据的容量和方便后续水平扩展
  - 分片做副本是为了提高集群的稳定性，提高容灾特性和吞吐量。
  - 副本越多 数据越高可用，但是消耗的资源就越多
  - 分片越多数据就会越分散，避免数据过于集中导致某个节点压力过大
  - 副本和分片的数量不一定是越多越好
    - 过多的分片，会占用大量的CPU以及内存等资源
    - 过多的副本，副本之间数据同步的会占用大量网络资源等
  - ES默认为一个索引创建1个主分片和1个副本，在创建索引的时候使用settings属性指定，每个分片必须有零到多个副本
  - **注意：索引一旦创建成功，主分片primary shard数量不可以变(只能重建索引)，副本分片replica shard可以改变**

2-ElasticSearch如果主分片和副本分片分配不合理 有什么处理方案?

- 题目分析和考点
  - 面试官想知道应聘者对Elastic Search是否掌握，对于里面的副本和分片是否清楚用途

- 业务一开始规划的索引分片分配不合理，例如下面场景应该怎么办?
  - 原有数据量太大、分片数太少
  - 原有数据量太小、分片数太多
  - ES索引分片，一旦创建，原索引是不能修改分片数量

- 难度【*****】
- 答案
  - 业务一开始规划的索引分片分配不合理，比如 原有数据量太大、分片数太少；原有数据量太小、分片数太多
  - 利用Elastic Search重建索引reindex解决问题
    - 创建一个全新的索引来替换现有索引,在索引重建过程中，会创建一个新的索引，并定义新的映射（mapping）和设置
    - 可以使用一些技术手段将数据从旧索引中复制到新索引中，最终实现从旧索引到新索引的无缝迁移
    - 索引重建的目的
      - 修复数据错误，如果旧索引中存在错误或数据不一致，通过索引重建可以创建一个全新的索引副本，并确保数据的一致性和正确性。
      - 更改索引设置和映射
        - 索引重建可以让更改索引的设置、调整分片和副本配置以及修改字段映射
        - 通过创建新索引并将数据复制到其中，可以应用新的索引设置和映射。
      - 数据迁移和升级
        - 当需要升级Elasticsearch版本、迁移集群或更改硬件时，索引重建可以无缝地迁移数据到新的索引中
        - 以便在升级或迁移过程中不中断服务

  - 单索引数据量比较大，数据同步速度比较慢时，如何处理?
    - 索引重建过程可能需要较长时间，并且会占用一定的资源，在进行索引重建之前，最好在测试环境中进行测试
    - 索引重建还需要谨慎评估索引大小、硬件配置和可用存储等因素，以确保成功完成重建并满足性能需求。
    - 增加资源，通过增加Elasticsearch的资源（如内存、CPU等），可以提高reindex操作的效率。
    - 避免磁盘IO瓶颈，在进行reindex操作时，可以通过将源索引和目标索引放在不同的磁盘上，或者使用更快的SSD提升速度
    - 在进行reindex操作时，可以将 目标索引的刷新间隔设置为-1，从而避免不必要的刷新操作，提高reindex操作的效率。
    - 如果单索引数据量比较大，在迁移前将目标索引的副本数设置为0，以加快数据同步速度，待数据迁移完成后，再修改回来

3-说下ElasticSearch搜索的文档排序的评分机制

- 题目分析和考点
  - 面试官想知道应聘者对Elastic Search是否掌握，对于里面搜索评分机制是否掌握
  - 文档评分机制主要基于TF-IDF（词频-逆向文档频率）算法和向量空间模型
- 难度【****】
- 答案
  - Elasticsearch中搜索的时候，文档排序里面有个评分，表示相关性
  - 输入搜索语句到结果显示，展现的是一个按score得分从高到底排好序的结果集
  - 在Elasticsearch 8.X版本中，文档评分机制主要基于TF-IDF（词频-逆向文档频率）算法和向量空间模型
  - 概念
    - 词频（Term Frequency，TF）
      - 指的是一个查询中的【关键词】在【文档】中出现的频率
      - 出现次数越高，TF值越高，表示关键词在文档中的重要性越高。
    - 逆向文档频率（Inverse Document Frequency，IDF）
      - 用于衡量一个词的普遍重要性，查询【关键词】在整个索引文档中出现了多少次，次数越多表示越不重要，评分越低
      - IDF值越高，表示该词在整个文档集合中较为罕见，因此具有较高的重要性
  - 查看评分计算规则

```
GET /xdclass_shop/_search?explain=true
{
  "query": {
    "match": {
      "summary": "java"
    }
  }
}
```

```
{
  "value": 1.4226694,
  "description": "score(freq=1.0), computed as boost * idf * tf from:",
  "details": [
    {
      "value": 2.2,
      "description": "boost",
      "details": []
    },
    {
      "value": 1.3862944,
      "description": "idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:",
      "details": [
        {
          "value": 1,
          "description": "n, number of documents containing term",
          "details": []
        },
        {
          "value": 5,
          "description": "N, total number of documents with field",
          "details": []
        }
      ]
    },
    {
      "value": 0.46647233,
      "description": "tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:",
      "details": [
        {
          "value": 1,
          "description": "freq, occurrences of term within document",
          "details": []
        },
        {
          "value": 1.2,
          "description": "k1, term saturation parameter",
          "details": []
        },
        {
          "value": 0.75,
          "description": "b, length normalization parameter",
          "details": []
        },
        {
          "value": 6,
          "description": "dl, length of field",
          "details": []
        },
        {
          "value": 6.4,
          "description": "avgdl, average length of field",
          "details": []
        }
      ]
    }
  ]
}
```

- 查看评分机制
  - 总计算公式

```
boost * idf * tf
字段的权重 * 逆向文档频率的值 * 关键词在文档中出现的频率
```

- idf计算公式

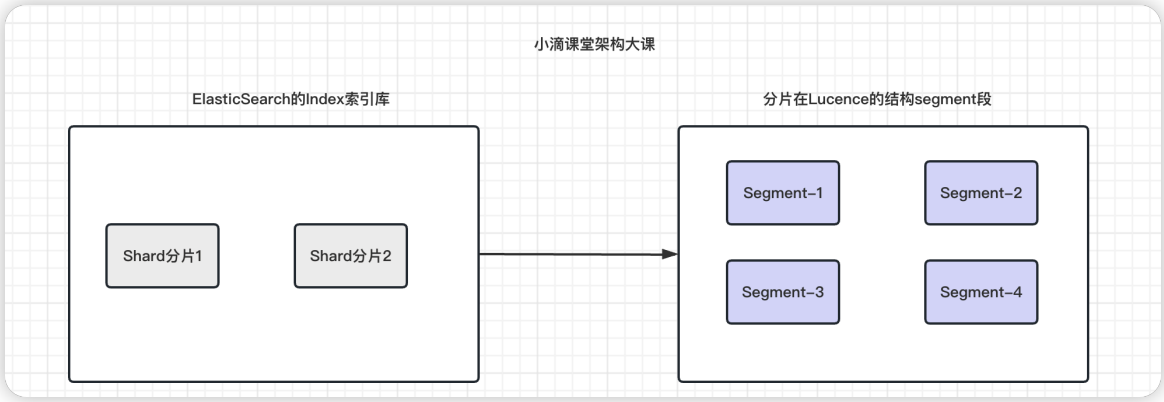
```
log(1 + (N - n + 0.5) / (n + 0.5))
N：包含这个查询字段的总文档数
n：包含该词的总文档数
```

- tf计算公式

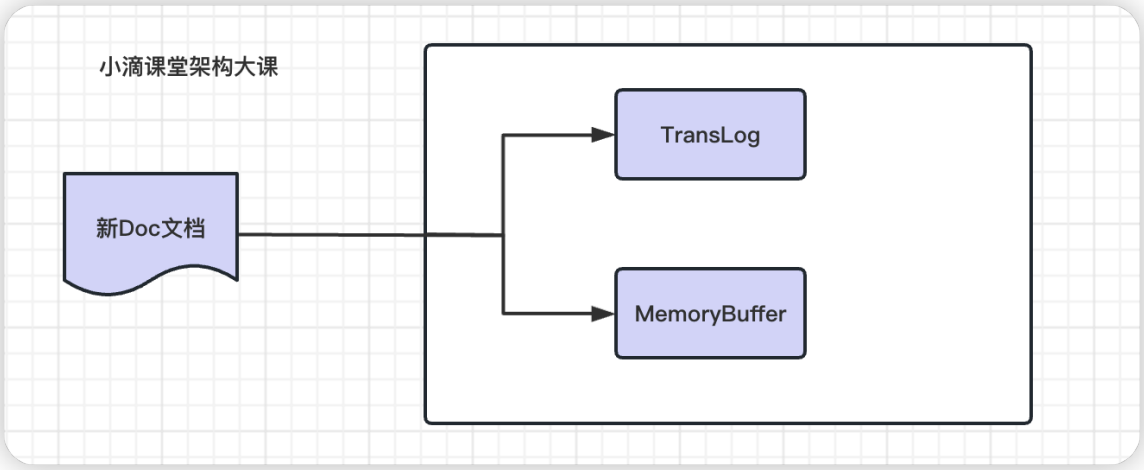
```
freq / (freq + k1 * (1 - b + b * dl / avgdl))


freq：关键词在当前文档中出现的次数
k1：调节参数，默认1.2
b：调节参数，默认0.75
dl：分词字段个数
avgdl：平均分词字段个数
```

4-说下Elastic Search8索引分片写入原理是怎样的?

- 题目分析和考点
  - 面试官想知道应聘者对Elastic Search是否掌握，对于里面搜索评分机制是否掌握
  - 文档评分机制主要基于TF-IDF（词频-逆向文档频率）算法和向量空间模型
- 难度【***】
- 答案
  - Elasticsearch索引文档写入原理
    - 概念
      - 分片（shard）
        - 分片是将索引数据分割成更小的、可分布式存储和处理的单元
        - 每个索引都由一个或多个分片组成，每个分片都是一个独立的Lucene索引
      - 段（segment）
        - 段是Lucene索引的基本单元，每个分片都包含一个或多个段
        - 段是不可变的，表示一定范围内的文档数据和相关的倒排索引，每一个Segment本质上就是一个倒排索引
        - **总结：每个shard分片是一个lucene实例，每个分片由多个segment组成，每个segment占用内存，文件句柄**



- 事务日志（Translog）
  - 防止缓存中的Segment还没有来得及被commit到磁盘当中，就发生了一些意外，比如断电等情况，导致数据丢失
  - 引入translog文件来保存这些记录，新文档先写入内存 buffer 和 translog 文件，每个 shard 都对应一个 translog 文件
  - 当translog大到一定程度，将会发生一个commit操作也就是全量提交
  - translog 也是先写入 os cache 的，默认每隔 5 秒刷一次到磁盘中去
  - 所以默认情况下，可能有 5 秒的数据会仅仅停留在 buffer 或者 translog 文件的 os cache 中
  - 如果此时机器挂了，会丢失 5 秒钟的数据。但是这样性能比较好，最多丢 5 秒的数据
  - 也可以将 translog 设置成每次写操作必须是直接 fsync 到磁盘，但是性能会差很多



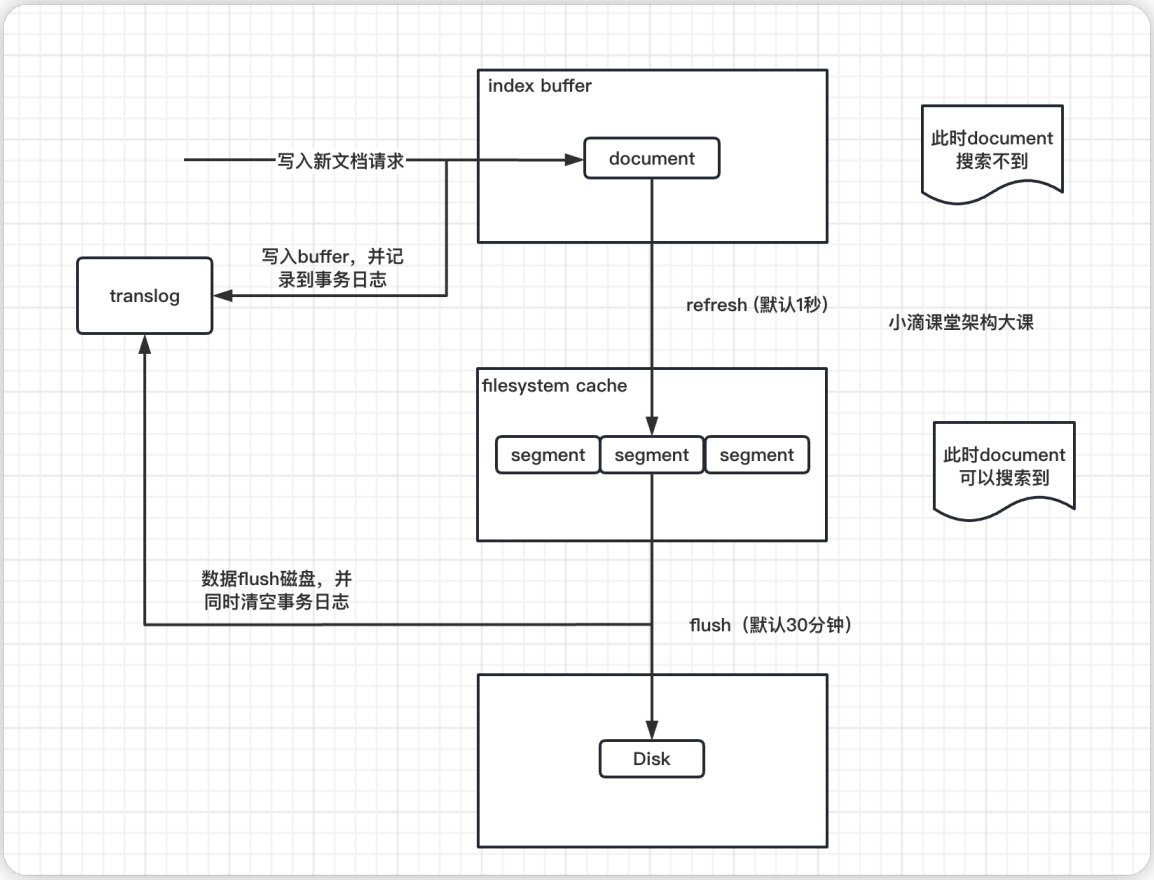
- 刷新（Refresh）
  - 将缓存中的文档写入Segment，并打开Segment，让他可以被搜索的过程
  - Refresh操作默认每秒执行一次， 将内存 buffer 的数据写入到一个新 的 Segment 中
  - **因此新加了一条数据，在下一秒就可以被搜索，也因此ES被称为可以实时的搜索NRT**
  - 这个时候索引变成可被检索的，写入新Segment后 会清空内存buffer
  - 经过了refresh间隔，将该时间段写入的全部数据refresh成一个segment
  - refresh过程是很消耗性能的，如果你的系统对实时性要求不高，可以通过API控制refresh的时间间隔

```
PUT /xdclass_shop
{
  "settings": {
    "refresh_interval": "30s"
  }
}
```

- 合并（merge）
  - 每秒都会有新的segment生成，用不了多久segment的数量暴涨，每个段都将十分消耗文件句柄、内存、和cpu资源
  - 系统无法忍受的，所以将零散的segment进行合并，ES通过后台合并段解决这个问题。
  - ES因为会定期的把一些小的Segment或者没用的Segment进行合并，减少存储占用空间
  - 小段被合并成大段，再合并成更大的段，然后将新的segment打开供搜索，旧的segment删除
  - 影响
    - 每个segment的占据内存是随着gc不会释放掉的，导致系统内存不够，进一步导致查询超时等问题
    - 查询时会遍历每个segment，过多的segment会导致查询速度下降

- 刷盘（Flush）
  - 实现文档数据从文件系统缓存刷到磁盘的过程
  - 会定期触发，也可以当translog的数据达到某个上限的时候会进行一次flush操作
  - 默认条件是，每 30 分钟主动进行一次 flush，或者当 translog 文件大小大于 512MB主动进行一次 flush
    - 配置`index.translog.flush_threshold_period`和`index.translog.flush_threshold_size`
  - 在ES中， 要保证被索引的文档能够立即被搜索到， 有两种方法：_refresh 或者 _ flush。

- 索引写入流程



- 当一个写请求发送到ES后，ES会将数据写入内存缓冲区（memory buffer）并添加事务日志
- 为避免频繁地将每条数据直接写入硬盘文件，导致硬盘进行随机写入，而随机写入的效率较低，会对性能造成严重影响
- 在设计时，ES引入了Linux的高速缓存（File system cache）来提高写入效率
- 当写入请求发送到ES后，数据会被暂时写入内存缓冲区，此时写入的数据还不能立即被查询到
- 默认ES会每秒将内存缓冲区中的数据刷新到Linux的文件系统缓存中，并清空内存缓冲区，写入的数据就可以被查询到。
- 这样的设计可以避免频繁进行随机硬盘写入，通过利用Linux的文件系统缓存，提高了Elasticsearch的写入效率

5-说下你用Elastic Search8.X常见性能优化最佳实践

- 题目分析和考点
  - 面试官想知道应聘者对Elastic Search是否掌握
  - ES数量常规是亿级别为起点，很多技术人员达不到官方的数据，多数是团队现有技术水平不够和业务场景不一样
  - 那是否各个条件都有进行分析，知道常用的最佳实践
- 难度【****】
- 答案

- 官方数据Elastic Search最高的性能可以达到，PB级别数据秒内相应
  - 1PB=1024TB = 1024GB * 1024GB
  - 但是很多同学公司的Elastic Search集群，里面存储了几百万或者几千万数据，但是ES查询就很慢了
  - 记住，ES数量常规是亿级别为起点，之所以达不到官方的数据，多数是团队现有技术水平不够和业务场景不一样
  - 海量数据检索领域榜单：https://db-engines.com/en/ranking/search+engine
- Elastic Search8.X常见性能优化最佳实践
  - 硬件资源优化：
    - 内存分配
      - 将足够的堆内存分配给Elasticsearch进程，以减少垃圾回收的频率
      - ElasticSearch推荐的最大JVM堆空间是30~32G, 所以分片最大容量推荐限制为30GB
      - 30G heap 大概能处理的数据量 10 T，如果内存很大如128G，可在一台机器上运行多个ES节点
      - 比如业务的数据能达到200GB, 推荐最多分配7到8个分片
    - 存储器选择
      - 使用高性能的存储器，如SSD，以提高索引和检索速度
      - SSD的读写速度更快，适合高吞吐量的应用场景。
    - CPU和网络资源
      - 根据预期的负载需求，配置合适的CPU和网络资源，以确保能够处理高并发和大数据量的请求。
  - 分片和副本优化：
    - 合理设置分片数量
      - 过多的分片会增加CPU和内存的开销，因此要根据数据量、节点数量和性能需求来确定分片的数量。
      - 一般建议每个节点上不超过20个分片
    - 考虑副本数量
      - 根据可用资源、数据可靠性和负载均衡等因素，设置合适的副本数量
      - 至少应设置一个副本，以提高数据的冗余和可用性。
      - 不是副本越多，检索性能越高，增加副本数量会消耗额外的存储空间和计算资源，
  - 索引和搜索优化：
    - 映射和数据类型
      - 根据实际需求，选择合适的数据类型和映射设置
      - 避免不必要的字段索引，尽可能减少数据在硬盘上的存储空间。
    - 分词和分析器
      - 根据实际需求，选择合适的分词器和分析器，以优化搜索结果。
      - 了解不同分析器的性能特点，根据业务需求进行选择

        - 查询和过滤器
            - 使用合适的查询类型和过滤器，以减少不必要的计算和数据传输
            - 尽量避免全文搜索和正则表达式等开销较大的查询操作。
- 缓存和缓冲区优化：
    - 缓存大小
        - 在Elasticsearch的JVM堆内存中配置合适的缓存大小，以加速热数据的访问
        - 可以根据节点的角色和负载需求来调整缓存设置。
    - 索引排序字段
        - 选择合适的索引排序字段，以提高排序操作的性能
        - 对于经常需要排序的字段，可以为其创建索引，或者选择合适的字段数据类型。
- 监控和日志优化
    - 监控集群性能
        - 使用Elasticsearch提供的监控工具如Elastic Stack的Elasticsearch监控、X-Pack或其他第三方监控工具
        - 实时监控集群的健康状态、吞吐量、查询延迟和磁盘使用情况等关键指标。
    - 配置日志级别和轮转策略
        - 根据需求配置Elasticsearch的日志级别和轮转策略，以避免日志文件过大影响磁盘空间
        - 并能方便故障排查和性能分析。
- 集群规划和部署：
    - 多节点集群
        - 使用多个节点组成集群，以提高数据的冗余和可用性。多节点集群还可以分布负载和增加横向扩展的能力。
    - 节点类型和角色
        - 根据节点的硬件配置和功能需求，将节点设置为合适的类型和角色
        - 如数据节点、主节点、协调节点等，以实现负载均衡和高可用性。
- 数据备份和恢复
    - 定期备份数据
        - 根据数据重要性和业务需求，定期备份Elasticsearch数据
        - 可以使用snapshot和restore功能、在线备份工具或者文件系统级别的备份工具。
- 性能测试和优化：
    - 压力测试
        - 使用性能测试工具模拟真实的负载，评估集群的性能极限和瓶颈
        - 根据测试结果，优化硬件资源、配置参数和查询操作等。
    - 日常性能调优
        - 通过监控指标和日志分析，定期评估集群的性能表现，及时调整和优化配置，以满足不断变化的需求。

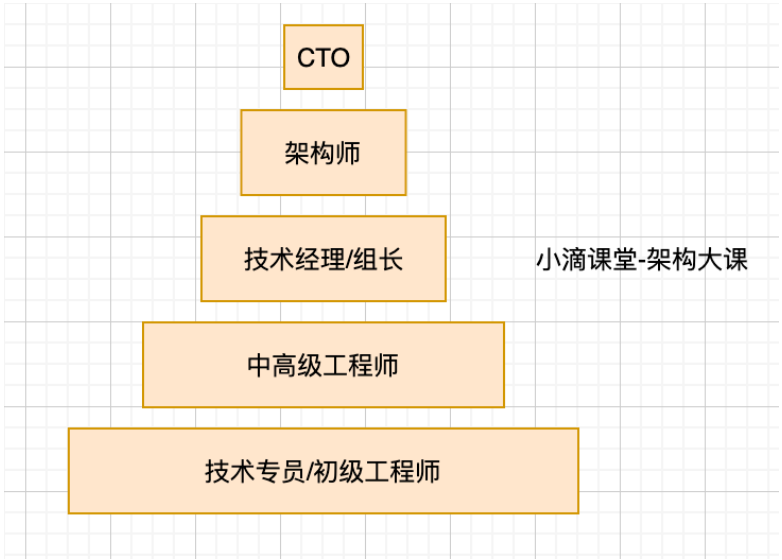
        - 安全性和权限：
            - 启用安全功能
                - 在生产环境中，为了确保数据的安全性，启用适当的安全功能，如访问控制、身份验证和传输加密等。
            - 权限控制
                - 根据用户角色和权限需求，设置合适的访问控制策略。限制敏感数据的访问权限，并定期审计用户权限。
        - 升级和版本管理：
            - 计划升级
                - 定期考虑升级Elasticsearch版本，以获取新功能、性能改进和安全修复。
                - 在升级过程中，确保备份数据并进行合理的测试。
            - 版本管理
                - 跟踪Elasticsearch的发行说明和文档，了解新版本的特性和已知问题，并根据实际需求选择合适的版本。

高级技术管理岗

1-你认为的技术组长或架构师的关键能力是哪些

- 题目分析和考点
    - 面试官想知道应聘者对技术管理岗的认知，如果是应聘相关岗位，则需要知道市面上多数岗位的内容
    - 对于大部分程序员来说，架构师算是职业最终目标
        - 不是每一位程序员经过5~8年后都可以成长为架构师的
        - 除了架构师工作所必需的基本理论和技能后
        - 需要有机会利用所掌握的技能, 在对应的系统和条件下做出的合理架构
        - 不断的复盘自己的架构模式，也在于机遇、个人的努力和天赋
- 难度【* * * * *】
- 答案
    - 技术管理岗关键的能力：技术选型+难点攻克+沟通管理
    - 多数人的成长方向就是 应用架构师，方向多种，但不要局限到语言上
        - java架构师、go架构师、C++架构师 等
        - 对标
            - 阿里P7技术专家/P8高级技术专家

        - 腾讯10或者11级
        - 字节2-2或3-1
- 具体做啥：产品PRD设计->架构师设计->方案评审->-开发->测试->上线->版本迭代更新
- 要求
    - 强技术要求+广度+深度+自驱力；
        - 比如
            - 要做搜索，是采用ElasticSearch 、Solr 还是Mysql like查询?
            - 项目是否要加入MQ解耦，要的话是选择Kafka、RocketMQ 还是RabbitMQ、ActiveMQ?
            - 业务涉及到实时计算应该采用Flink、还是Spark、还是KafkaStreaming?
            - 复杂链路的线上业务突然RT超时严重，怎么排查和解决问题?
            - 线上事故频繁，消息队列堆积两天没人发现，怎么没人发现，接下去怎么处理?
            - 团队技术脱节严重，开发不规范，工期频繁延误，怎么解决?
            - ...
            - 这些【点和线】都是可以让中高级工程师解决，多数人停留在中间的地方，要更强大的人来把控全局
            - 架构师的能力就是【面 】我们就需要从【点->线->面】持续积累，升级打怪



    - 懂业务+抽象+理解能力强
        - 具有多个领域业务经验，业务抽象能力强，化繁为简，逐个击破
        - 比如
            - 阿里的Lazada 里面 优惠券系统：满减券、无门槛、新人卷、活动卷...如何设计更加高内聚，低耦合
            - 电商公司的支付系统：对接微信、支付宝、京东、抖音、各大银行... 如何设计更加高内聚，低耦合
            - 公司业务发展快速，有个新的业务让你接手负责 ， 需要知道怎么处理
    - 懂管理+带团队+沟通汇报+影响力

        - 项目取得好成果,老板想提拔你，掌握晋升和向上汇报PPT的能力
        - 知道如何让其他团队高效配合，项目需要跨多个部门一起才可以推动
        - 知道如何管理带好团队，团队人员变动大，老油条多如何带好

2-刚入职新公司，如何融入和快速带好团队

- 题目分析和考点
    - 面试官想知道应聘者对技术管理岗的认知和是否有自己带团队的方法论
    - 尤其是新入职公司的技术管理岗
- 难度【* * * *】
- 答案
    - 架构师兼技术负责人，我会根据下面几个方式进行
    - 前期
        - 学习公司文化
            - 了解新公司价值观、工作方式和团队文化，尽快融入公司文化，与团队成员保持良好的沟通，并适应公司的工作氛围。
        - 寻求帮助和指导
            - 与同事建立积极的合作关系，主动寻求帮助和指导，与资深员工或团队领导建立良好的沟通渠道
            - 了解公司的技术栈、流程和开发规范；技术管理岗也需要放下架子
        - 积极参与团队活动
            - 参与团队内部的活动和项目，积极与团队成员合作。展示自己的技术能力和贡献，建立信任和认可。
        - 与团队成员建立良好的人际关系
            - 与团队成员建立良好的人际关系，包括与同事进行正式或非正式的聊天，了解他们的兴趣爱好和经验分享。
        - 提出建设性的意见和想法
            - 在团队讨论或会议中，积极提出自己的观点和建议。展示自己的专业能力和创造力，共同推动团队的发展和进步。
    - 后期
        - 立规矩：找所有研发沟通，定好准则和底线，团队的红线意识要强。
            - 变更三板斧:可监控、可灰度、可回滚，不允 许"三无发布"
            - 重要业务： 面向失败编程、定期CodeReview，90%的事故都是发布引起的
            - 看过程，重结果，奖惩分明
        - 组团队：招聘新人，培养自己的人，合适的人做合适的事情
            - 技术团队有两种人

        - 创新能力强，活跃性高
        - 话少相对不怎么沟通
    - 做业务
        - 做好新业务 团队要有竞争力（新人，有冲劲，加班多）
        - 维护老业务 保证业务的正常运行（熟悉系统的老人，安分守己）
    - 定节奏：
        - 打破原有的工作节奏，温水煮青蛙，则容易被淘汰
        - 定新的节奏做事，加班制度、早会/周会制度

3-说下如何成为架构师，哪些人适合当架构师

- 题目分析和考点
    - 面试官想知道应聘者对技术管理岗的认知和是否有自己的职业规划
    - 不想当将军的士兵不是好兵，技术人员需要有自己的规划，也是面试里面潜力股的体现
- 难度【 * * * 】
- 答案
    - 如何成为架构师，哪些人适合当架构师，总结起来3个词的工作做好就行
        - 技术
            - 技术解决项目业务的问题，通过业务驱动技术发展，技术反哺业务，场景案例如下
                - 运营每天都需要去应用商店看，评论找差评进行反馈整理
                - 通过技术爬取各大应用商店的产品的评论，语义识别归类
            - 业务起飞是体现技术最大的价值，才能证明技术的真正价值
                - 技术是手段而不是目的；不要喜新厌旧，没价值或者低ROI的重构、升级
        - 管理
            - 沟通讲人话，你的业务方不指技术团队，还有产品、运营、客服团队很多
            - 带团队 做好项目
            - 对上汇报，对下管理
        - 业务
            - 识别重点+时间线
            - 每个团队项目不少， 把握主链路 （每次考试说重点的划线，结果TM的全部划线是重点）
            - 业务部门最需要的技术人员，口碑和技术最好的则这人的方式值得你去学习

        - 上面3个词做好，能打造你的【影响力】团队内部，部门之间，公司领导层面
            - 从而推动很多事情的发展，需求、排期、资源、人脉人际关系

        - 走到聚光灯下、勇于承担、换位思考

    - 我个人职业成长学习规划
        - 学习做事方法，目标明确，推进项目落地，拿结果
        - 沉下心来做事，最终的结果是水到渠成
        - 不要给自己设限，学习的过程中要常去思考、总结复盘
        - 遇到技术问题不慌乱，有自己标准的排查思路，解决后要能总结复盘
        - 专业技能任然是立身之本，努力让自己超过同层级的技术能力，成为那20%
        - 不要求整个架构课100%掌握，掌握80%，可以理论上认为当下超过80%程序员

4-你认为高级技术岗的技术广度和深度是怎样的?

- 题目分析和考点
    - 面试官想知道应聘者对技术管理岗的认知和是否有自己的职业规划
    - 对技术广度和深度的理解情况
- 难度【 * * * 】
- 答案
    - 架构师的技术广度和深度
        - 技术人员对技术要精还是要广，这个问题没有明确标准的答案
        - **个人看法**：**精于基础，广于工具，熟于业务，保持持续学习的心态**
        - **找到最适合自己的技术成长路径**，持续学习
    - 架构师是不是就没有不会的问题?
        - 架构师分很多不同领域，也存在N多不会的，包括工作了那么年也是
        - 优势
            - 对比多少工程师来讲，架构师全局观更强，解决问题方式更快
            - 做事具备方法论，不同领域的经验积累了更多，学习力更强
        - 【技术决定生产力】
            - 没有秘籍和诀窍，多数人都是 八小时内求生存八小时外求发展
            - 只要愿意花业余时间去学,再加上长时间的坚持，就可以达到理想的水平
            - 最简单的道理：同样的需求工作项，为啥我可能1天完成，而你需要5天还不一定?

        - 场景
            - 经常需要有不同需求的项目，难点等， 也包括你跳槽去不同领域的公司，如何快速得出设计
                - 参考大厂的解决方案和架构，然后做个低配版本
                - 电商、直播、在线教育、社交
            - 一个三方包怎么快速知道的使用技巧和API
                - 搜博文，看官方文档 是多数人的选择
                - 第二种：git找开源地址，里面有test、sample案例

    - 架构师能力的评判标准
        - 厉害的架构师，团队感觉不到他的存在
            - 当老王在手术室里面，主任医生很轻松就没问题；
            - 老王感觉不到他的存在，但是当主任医生慌起来的时候，啥问题?

        - 阿里一位大神架构师说过的话，特别认可
            - 先选一个好的挑战，为了创造价值，这个挑战不是技术挑战，而是为了解决什么样的用户痛点/问题，要对业务有很好的了解
            - 把简单的东西想复杂，为了控制好风险和准备好未来
            - 把复杂的东西做简单，为了做好产品且控制好成本
            - 最后把复杂的东西讲简单，为了做好传承

        - 常规的架构师，团队尊敬他，差的架构师，团队吐槽他
        - 看系统适当评估，评估发展1~3年，不是考虑10年
        - 主流技术多，但避免乱用，越简单的架构越完美

5-有没了解过中台，能否解释下对中台理解

- 题目分析和考点
    - 面试官想知道应聘者对中台的概念是否掌握
    - 避免盲目做中台图，不要神化中台，是对系统的复用，避免重复建设功能，一般业务发展到适合的时候才适合建设中台
- 难度【* * * *】
- 答案
    - 需求背景

        - 公司业务发展快，需求方太多，人力又是有限的，怎么样用有限的人力去维护无限的需求
        - 中台的出现多数都是历史原因
            - 业务基本不太可能按照理想路线走，所以各种需求变动，导致架构、代码会越来越乱
            - 毕竟不太可能一开始就有大量时间做详细规划和完美设计
            - **多套代码，产品提出一个功能在一个版本上线之后，另一个版本想执行想用得重新写一遍**
            - **功能复用性比较差，系统组件维护也比较复杂**
    - 例子
        - 小滴课堂公司业务发展
            - 一开始是卖课程的里面涉及到课程产品支付
            - 后来推出了小滴严选，做了实物产品支付
            - 后来开发了营销功能，拼团、秒杀，也需要用到支付功能
            - 后来开发了技术社区，里面积分购买和兑换，也需要有支付功能
            - 后来做了收购融资XX公司，需要打通业务数据共享
        - 产品经理老王来了
            - 需要加个支付分期免息功能
            - 需要加个D币积分抵扣功能和分账
            - ....
            - 每加个功能，在N多个平台都需要重复写代码逻辑

    - **什么是中台（不是单纯技术层面、系统层面，而是企业的组织架构，一种理念）**
        - 大家其实一直有做过中台的理念，只不过没人告诉你这个是中台理念
            - 方法复用、类要复用、模块复用
            - 系统复用 是更大层级上的复用，即中台
        - 中台就是以共享系统服务的形式存在，提炼各个业务域的共性需求，打造成组件化的系统
        - 核心业务能力以服务的方式进行沉淀，实现服务在不同场景中的业务能力重用
        - 以接口的形式提供给前台使用, 最大限度地减少系统建设中的“重复造轮子”的问题
        - 比如：业务中台是加快系统研发上线，提高效率，但不是完全不用写任何代码，只是更少而已
    - 分类

        - 业务中台
            - 业务微服务：商品中心、用户中心、支付中心、物流中心、营销中心等
        - 技术中台
            - 基础设施：基础服务、分布式缓存|队列|文件|调度
            - 运维告警：监控中心、配置中心、DevOps平台
            - 自动化平台：自动化测试平台
        - 数据中台
            - 数仓建设、用户画像、推荐系统、数据大屏...
        - AI中台

    - 是否要建设中台
        - 企业IT生态系统庞大大
        - 大量低水平重复建设的系统
        - 各项业务数据互通
    - 中台建设要求
        - 要求对业务流程有一个【标准的抽象】，相关的业务能在这个标准流程里面进行
        - 作为中台，需求肯定会很多， 系统设计时要让系统可扩展性强
        - 对系统设计抽象有比较强的品控，不要因为多人共建让系统代码迅速的腐化膨胀
        - 突出关键能力-【抽象能力】

6-开发中有没画过架构图，画架构图是为了什么

- 题目分析和考点
    - 面试官想知道应聘者是否独立负责过项目和带团队
    - 架构图是给人看的，这些人我们习惯称为【业务方、客户】
    - 对架构图的理解也是区别是否有实际经验和包装简历的关键点
- 难度【 * * * 】
- 答案
    - 什么是架构图?
        - 架构图 = 架构 + 图
        - 用图的形式把系统架构展示出来，配上简单的文案

- 画架构图是为了什么
  - **一图胜千言，解决沟通障碍，给不同的【业务方】看懂**
  - 业务方很多，不同人看到角度不一样，你让【产品经理】看 【物理部署视图】他看得懂?
- 架构图是给人看的，这些人我们习惯称为【业务方、客户】
  - 人员
    - 上级：你的公司Leader(晋升汇报)、老板、外部投资人
    - 团队内：产品、运营、测试、技术、运维同学
    - 外部：最终系统使用的用户
  - 好比阿里这边评定绩效，有一项就是业务方评分
    - 你做的外部用户的活动系统，测试同学会进行测试，太多bug肯定就不行
    - 你做的给运营同学使用的系统，不能提升她运营的效率，业务方是否满意?

7-常规项目的架构图有多个,用一个图不行吗?

- 题目分析和考点
  - 面试官想知道应聘者是否独立负责过项目和带团队
  - 架构图是给人看的，这些人我们习惯称为【业务方、客户】
  - 对架构图的理解也是区别是否有实际经验和包装简历的关键点
- 难度【 * * * 】
- 答案
  - 为什么要搞出这么多个架构图？用一个图不行吗?

        - 一开始确实是一个图表示系统架构设计
        - 但是业务方很多，不同人看到角度不一样，你让软件用户看物理部署视图？他看得懂？
            - 要明确沟通交流面向的客户
            - 开发人员、运维人员、项目经理、软件最终用户、客户
        - 架构图包括 功能需求、部署需求、流程需求等太复杂，所以还是要单一职责模式
        - 不同架构视图承载不同的架构设计决策，支持不同的目标和用途
        - 架构图也不能太多（过度文档化）维护更新起来成本大

8-如何判断架构图的好和坏，有没合适的画图理论

- 题目分析和考点
    - 面试官想知道应聘者是否独立负责过项目和带团队
    - 架构图是给人看的，这些人我们习惯称为【业务方、客户】
    - 对架构图的理解也是区别是否有实际经验和包装简历的关键点
- 难度【* * *】
- 答案
    - 画架构图有哪些理论，有没最合适的架构图画法
        - 架构图本质上是从不同的视角，不同的抽象角度去看，业界存在多个划分理论
        - 那画架构图肯定有一定的【架构设计理论】主流的有多个 **TOGAF**、**RUP 4+1** 等
        - 没有最合适的的架构图画法，只有最适合的
        - 脱离实际业务场景的架构图，基本都是不合格的

    - 如何判断架构图的好和坏?
        - 业务抽象设计的合理性，是否满足高内聚、低耦合的要求，不能太宽泛，也不能太细粒度
        - 层级划分目标系统边界，自下而上 或 由上而下，一般包括 基础设施、数据层、应用层、用户层四个层次
        - 纵向分层 上层依赖于下层越底层，越是基础服务；横向并列关系，级别相同
        - 理清架构图类型、业务要全面、专业术语一致、图形清晰美观、颜色类型划分合理
        - 最重要是：**你的业务方能 满意+看懂！！！**

业务中台基础能力架构

- 不同架构图应该使用哪种方法来画?
  - 可以用的表示法和工具很多，没有太多的限制，把握对应的视图关注点才是关键
    - Xmind、EdrawMax、PPT、PowerDesigner
    - OmniGraffle、Visio、Process On
  - 开始阶段不要陷入过度设计中，没那么多需求不一定要那么多图（你是否有那么多客户）
  - 关于UML
    - 没有使用标准的架构描述语言，要注明架构图元素的用意（比如方框、形状、边框、线条、颜色等等）
    - 使用标准的架构描述语言，只要在图例里添加关键性的架构描述，但是UML不太灵活
    - 所以可以不用UML构图，但架构图的组成元素，要保证符合一贯理解

9-说下你日常画业务架构图的思路和方法论

- 题目分析和考点
  - 面试官想知道应聘者是否独立负责过项目和带团队
  - 架构图是给人看的，这些人我们习惯称为【业务方、客户】
  - 对架构图的理解也是区别是否有实际经验和包装简历的关键点
  - 在画架构图之前，想清楚3个问题，架构图想表达什么？有什么用？给谁看?
- 难度【****】
- 答案
  - 在画架构图之前，想清楚3个问题，架构图想表达什么？有什么用？给谁看?
    - 表达是业务系统之间的关系，梳理业务结构
    - 将复杂的业务逻辑简单化，降低理解难度，更方便业务方理解

- 给业务方查看，业务相关干系人

- 业务架构图
  - 表达业务是如何开展的，服务于业务目标，通过描绘业务上下层关系，简单的业务视图降低业务系统的复杂度，提高客户理解度
  - 图中【尽量不出现技术】的字眼，不同架构图的读者是不同的，确保能看懂。
  - 架构图中模块的划分粒度，一定要合适，既不能太宽泛，也不能太细粒度
  - 无技术背景人员可参与实现的讨论，向技术人员描述解决方案核心要做什么，必须实现的关键是什么
  - 明白一个点
    - 先有业务，再有系统，微服务/系统/中心 是类似概念
    - 系统是来实现业务的，比如电商业务里面A系统、B系统
- 业务架构类型
  - 上中下结构：用户展现层-业务平台层-公共能力层-数据存储层-基础资源层
    - 案例一（图片来源-阿里云数字政府）

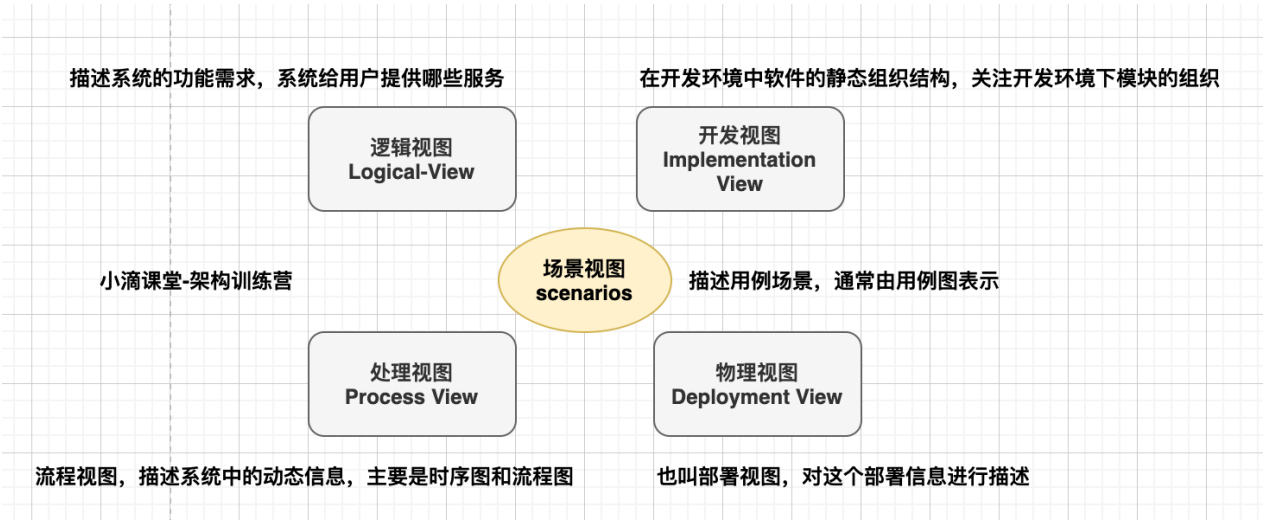
业务中台基础能力架构

  - 左中右结构：上游产业 - 业务平台- 下游产业
    - 相对较少用，就是倒置过去
- 画图三步走
  - 分层
    - 业务按照层级进行划分，各个层级属于独立的版块
    - 下层为上层提供服务能力支撑
    - 比如：IaaS / PaaS / SaaS
  - 分模块
    - 同层级中进行小归类
    - 属于平行关系，可以独立存在
    - 比如：能力开放平台 有运营能力平台、服务总线

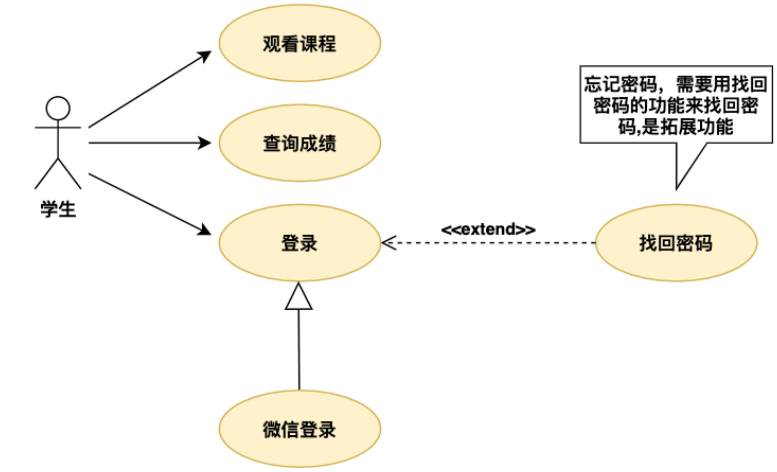
        - 分功能
            - 独立功能划分出来，即业务入口
            - 业务方重点关注的功能点，可以认为是微服务划分
            - 比如
                - 能力开放平台 有运营能力平台 里面有 服务管控、SLA承诺、计量服务

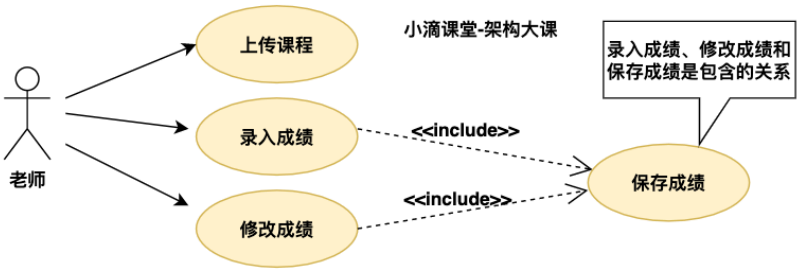
10-有没了解过RUP4+1视图,分别解释下用途

- 题目分析和考点
    - 面试官想知道应聘者是否独立负责过项目和带团队，主流的画图理论是否掌握
    - 架构图是给人看的，这些人我们习惯称为【业务方、客户】
    - 对架构图的理解也是区别是否有实际经验和包装简历的关键点
- 难度【* * * * *】
- 答案
    - 什么是RUP 4+1
        - 全称 Rational Unified Process，中文统一软件开发过程，是一个面向对象画图的方法论
        - 在RUP中采用“4+1”视图模型来描述软件系统的体系结构
        - 包括 5 种视图：场景视图、开发视图、逻辑视图、物理视图、处理视图



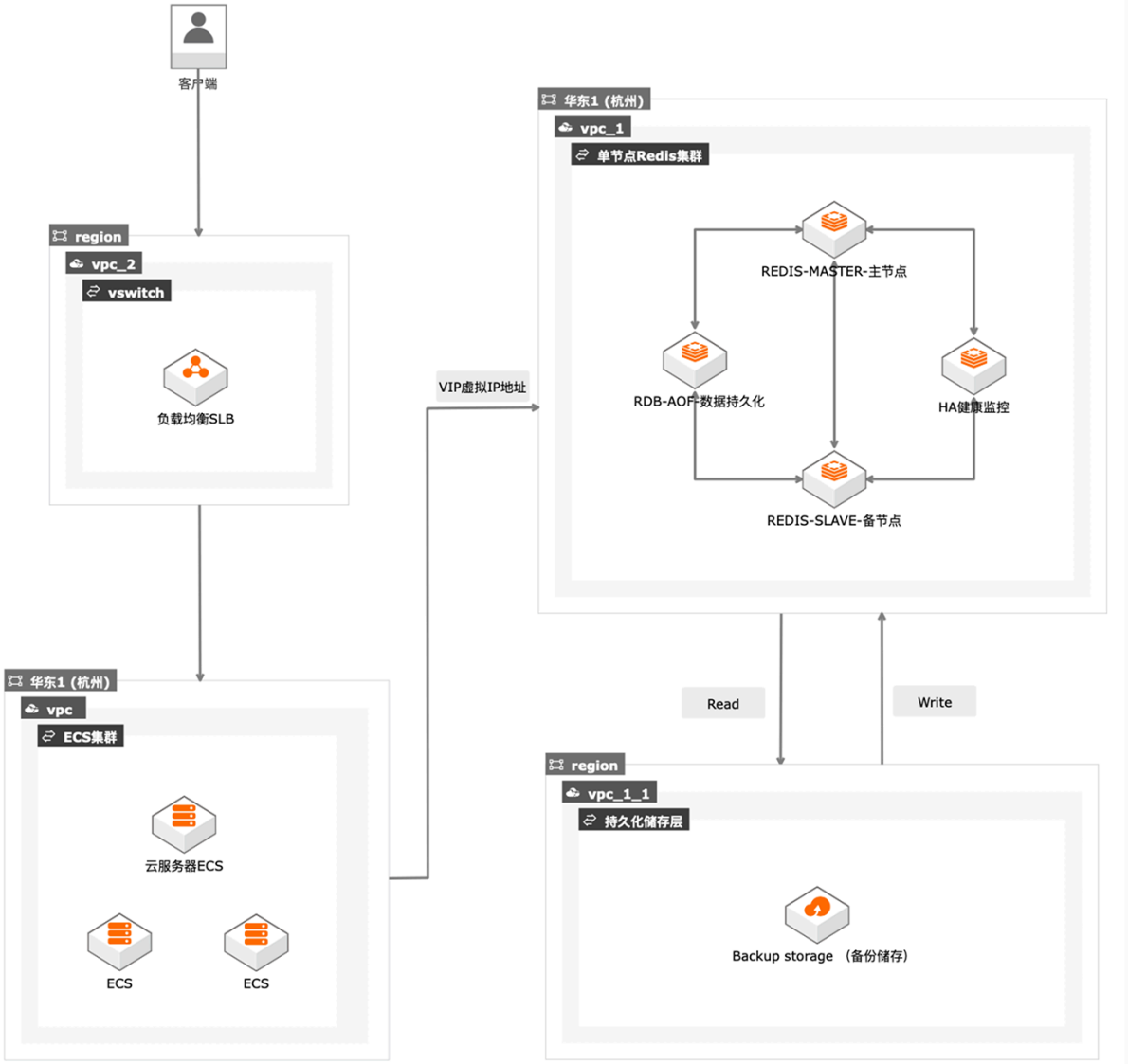
    - 分类
        - 场景视图
            - 也叫用例视图，4+1中的1，是最核心的是视图
            - 在UML中通常由用例图表示并不限于使用UML的用例图进行设计,可以使用其它方法



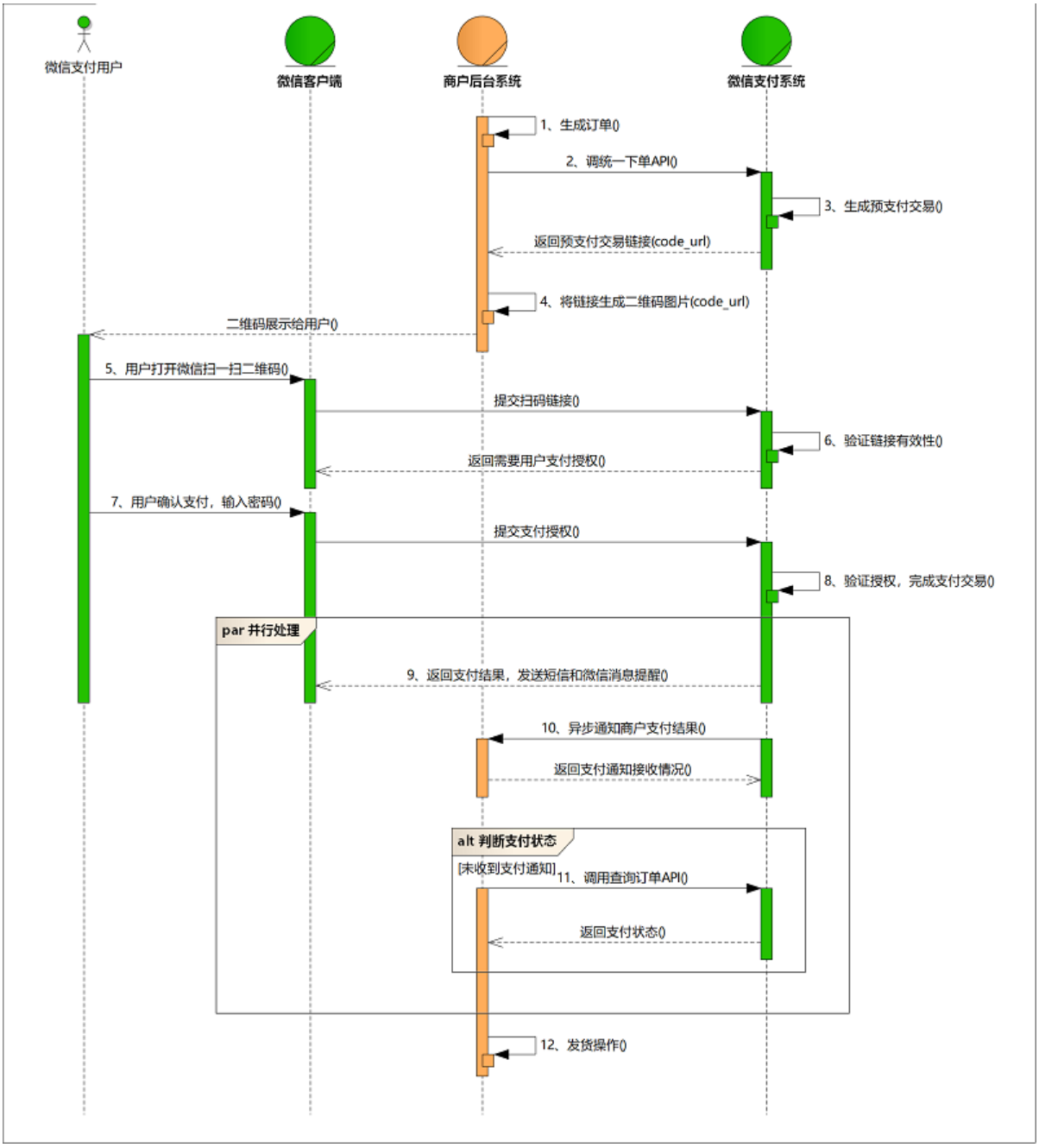


- 逻辑视图
  - 描述系统的功能需求，系统给用户提供哪些服务，比如扫码、登录、支付
  - 对系统职责的的逐级划分，理清功能需求和系统边界，也叫系统架构、应用架构、后端架构、技术架构
  - 注意
    - 一般会分颜色表示不同状态，比如 已经开发好的，开发中的，规划的
    - 方框长短没啥特殊，对齐美观

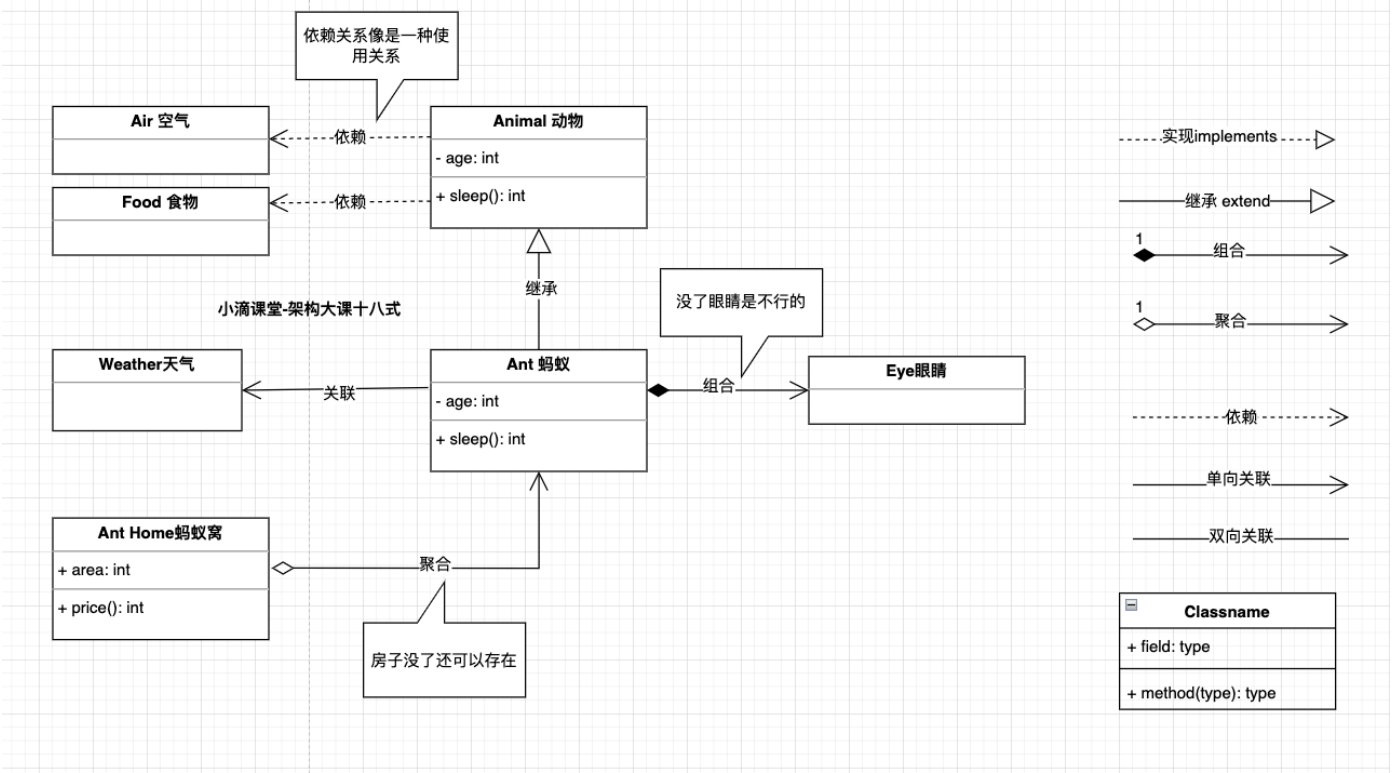
- 物理视图
  - 也叫部署视图，部署架构，对这个部署信息进行描述，包括服务器、虚拟机、终端设备分布等



- 处理视图
  - 也叫流程视图，描述系统中的动态信息，最常见的是逻辑架构元素之间的交互关系
  - 比如 微信扫码支付下单功能是如何实现的
  - 在UML中通常由时序图和流程图表示



- 开发视图
  - 描述在开发环境中软件的静态组织结构，关注软件开发环境下实际模块的组织
  - 主要是程序员角度看系统的逻辑组成



11-入职后/晋升/接手老项目/重构怎么快速上手项目代码?

- 题目分析和考点
  - 面试官想知道应聘者是否独立负责过项目和带团队，
  - 对熟悉新项目有没比较好的思路和解决方案
  - 包括团队里入职了新同事，是否有能力可以更好的让新人上手对应的项目
- 难度【* * * *】
- 答案
  - 常见的问题
    - 新公司的项目分层和之前不一样，怎么办?
    - 新公司编码规范和之前不一样，怎么办?
    - 新公司技术栈和之前用的不一样，怎么办?
    - 业务复杂-逻辑混乱-看不懂逻辑，怎么快速看懂代码?
  - 这个都是很正常现象
    - 阿里这边有个俗话，入职阿里的前3个月，天天想离职的冲动
    - 业务复杂、数据量大、中间件多、自研平台又多

        - 如HSF、鹰眼、Aone，MetaQ、Amazon、TDDL、ScheduleX、PandoraBoot等，外部也没资料可学
        - 其实都是换汤不换药，应用场景都类似，宏观去认知做啥的，然后跑个Demo就知道了

- 常见的问题
    - 新公司的项目分层和之前不一样
        - 每个公司都有不一样的分层，取决于认知情况
        - 有些人认为这样、有些人认为那样，这个只要常规遵循团队规范即可
    - 新公司编码规范和之前不一样
        - 目前多数公司Java开发都是遵循阿里巴巴编码规范
        - 老项目则另说，有些技术债务是需要自己去承担的

    - 新公司技术栈和之前用的不一样
        - 大厂基本都会有自研的中间件或者基于开源中间件进行二次开发的组件
        - 只要封装的合理，能加快开发速度；但也有些大厂的架构组封装不合理或者历史原因遗留的坑
        - 理想看待，只要把大课技术栈掌握就行，每个技术栈都是有应用场景，可以水平替换过去。
            - 比如阿里这边分布式调度是ScheduleX，外部可以用XXL-Job
            - 内部RPC是用HSF，外部是用SpringCloud、Dubbo等
            - 内部分库分表中间件是TDDL，外部是SharingJDBC、MyCat等

    - 刚入职或接时手项目，业务复杂-逻辑混乱-看不懂逻辑
        - 多数情况都是进入项目组开发，比较少机会是从0开始的
        - 怎样快速上手代码-多年经验分享
            - 切记、切记、切记不要直接看代码（下面不懂的则问公司老人）
            - 第一步：看产品，体验整体功能，思考背后大体逻辑，App、PC、H5等
            - 第二步：看产品需求文档，历史需求文档和现在进行的需求文档
                - 业务架构图-应用架构图
            - 第三步：看项目技术栈-补充单独不会的技术框架，跑起个demo案例
            - 第四步：看项目技术架构图和部署环境
                - 技术架构图-部署视图
            - 第五步：看数据库ER设计和关联关系、是否有分库分表
                - 数据架构图
            - 第六步：看团队负责的功能、模块、服务、包作用
            - 第七步：调下团队负责的某个功能链路，从发起请求到响应结果，经过的后端调用链和代码逻辑

        - 第八步： 按照上述步骤，再看其他的模块业务逻辑和调用链，不要在意细节点或者没用到的组件
            - 流程图-时序图-类图

    - 记住：不管新老项目，从顶层设计开始，由宏观到细节，这个是上手和重构的最佳选择

12-如何看待纠正-面试造飞机进去拧螺丝

- 题目分析和考点
    - 面试官想知道应聘者是否正确认识高级技术岗
    - 包括如果应聘者是带团队，后续招聘选人的时候，是否有自己的选人标准
    - 对人员的综合能力评定是否有比较好的思路
- 难度【* * * * *】
- 答案
    - 很多技术人员的反馈【面试问了一大堆，但是进去后是拧螺丝】
        - 这个不是这样的理解的，面试是面试官为了考察个人综合能力，有多个考查点，算法、数据库、并发、性能优化，架构，面试官会给每个点打分，得出这个人的综合能力。
        - 我们学会了画饼那我们需要具备这个能力去做，不然给你一堆面粉，你知道怎么把面粉做成面包吗？能保证做出来的吃了不拉肚子?
        - **在很多大厂里面的项目里面，就是在造飞机，只是造飞机上必修要拧螺丝，没螺丝的飞机你敢上?**
        - 面试的一道题【java创建线程池的底层是怎样？】
            - 你可能都没去看过，或者只是停留表面，使用java自带的线程池创建工具Executors创建，一直也没遇到啥问题
            - 但是你知道高并发场景下，在线程任务堆积处理不过来时，你系统就很容易OOM直接宕机
            - 是因为默认底层的队列长度是整数的最大值2147483647, 即20多亿，你单机系统内存怎么存储？所以就内存溢出挂了。
            - 这个是因为你没用心去看去学，或公司业务量没到达那个层级，但你记住这个就是螺丝，可能因为你没拧好导致严重事故

    - 跳槽的看法
        - 跳槽是我们互联网行业人人都会经历的，从个人职业发展，到公司营业不佳倒闭多种原因。跳槽好处也多，工资容易涨，厉害的还能比上个公司翻几倍，涨的少也能多几个好几千
        - 不建议频繁跳槽，这个会成为技术领域深造的瓶颈，特别是工作1~5年的同学，但如果公司学不到技术或对于自己未来发展不好，自己综合评估决定
        - 一旦选择跳槽，【良禽择木而栖，不要将就】, 刚工作几年的同学，如果工资还可以尽量先去能学到 技术的公司，前面可能会辛苦点

- 技术学到后面的就不是一点点的涨工资了，主流互联网公司年薪超百万的有很多人，阿里、腾讯、华为、京东、头条等
- 所以前面几年的积累很重要，能去大厂尽量先去，不管是实习还是工资低，因为【镀金+开阔眼界】真很重要。
- 不但是为了面试，更为了自己的技术能力的提升，平时多积累多留一个心眼，知其然知其所以然，知道这个点更要知道使用场景或者原理

设计模式

容器和K8S

1-有没了解过三种云服务模型IaaS- PaaS-SaaS

- 题目分析和考点
  - 面试官想知道应聘者是否掌握云服务模型，对IaaS- PaaS-SaaS的理解
- 难度【****】
- 答案
  - 三种云服务模型, 也称为三层架构
    - IaaS 基础设施即服务 Infrastructure as a service

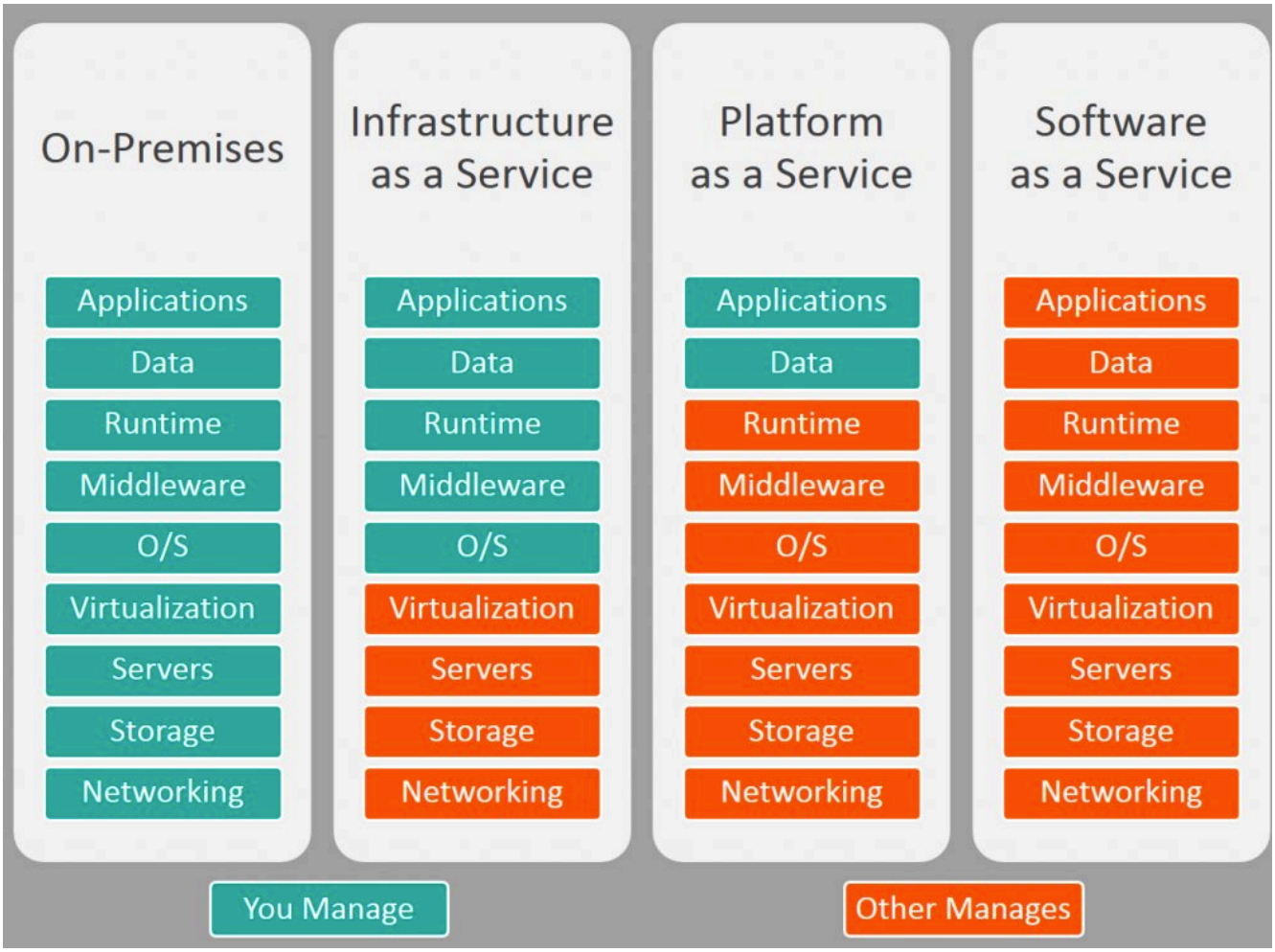
      通即虚拟的硬件资源，如虚拟的主机、存储、网络、安全等资源
      用户无需购买服务器、网络设备和存储设备，只需要通过网络租赁即可搭建自己的应用系统
      将硬件外包到别的地方去，IaaS公司会提供场外服务器，存储和网络硬件，你可以租用，节省了维护成本和办公场地
      公司产品 Amazon, Microsoft,阿里云，腾讯云，各自的产品
      最熟悉的例子：阿里云ECS主机的带宽、磁盘空间、GPU等

    - PaaS 平台即服务 platform as a service

为开发人员提供了一个框架，使他们可以基于它创建自定义应用程序
所有服务器，存储和网络都可以由企业或第三方提供商进行管理
使开发人员可以自由地专注于创建软件，同时不必担心操作系统、软件更新，存储或基础架构
公司产品：Google App Engine，Apache Stratos，OpenShift，阿里云，腾讯云等
最熟悉的例子：阿里云OSS、RDB、短信服务、日志服务

- SaaS 软件即服务 software as a service

云应用程序服务,利用互联网向其用户提供应用程序，这些应用程序由第三方供应商管理
大多数SaaS应用程序直接通过Web浏览器运行，不需要在客户端进行任何下载或安装
具有多租户的特性，不需要额外加工，按需付费，因此成本低高效快捷
公司产品：淘宝、小鹅通、亚马逊，CRM（客户关系管理）、HRM（人力资源管理）、SCM（供应链）



- 最通俗的例子
  - 小滴课堂老王-【建房子娶老婆】和【在线教育平台】
    - 传统方式：设计、找地皮、造砖头、水泥、钢筋；消耗人力1000人
      - 买主机、建机房、找运营商弄带宽
      - 开发在线教育微服务平台
      - 开发部署-监控-发布平台
      - 消耗程序员：100人

        - IaaS 服务：直接花钱买厂商做好的 砖头、水泥、钢筋：消耗人力 500人
            - 直接租用阿里云的主机、选CPU、硬盘、内存、带宽等
            - 开发在线教育微服务平台
            - 开发部署-监控-发布平台
            - 消耗程序员：50人

        - PaaS服务：直接买厂商搭建好的墙面，自己组装；消耗人力100人
            - 开发在线教育微服务平台
                - 阿里云的数据库、缓存、MQ、分布式事务GTS、日志服务 等厂商直接提供的中间件服务
            - 开发部署-监控-发布平台
                - Rancher、KubeSphere、
            - 消耗程序员：20人

        - SAAS服务：直接买厂商做好的房子；消耗人力10人
            - 直接租用厂商提供的在线教育平台
            - 消耗程序员：1人
    - 总结
        - 从Iaas到SaaS的过程，企业的云化的程度也越来越高，内部IT所需要做的事越来越少。
        - 可以这么看，运维同学经常打交道的是IaaS，开发经常打交道的是PaaS，运营和客户经常打交道的是SaaS